

ビジネス課題を 解決する技術

数理モデルの力を引き出す3ステップフレームワーク

森下光之助 著



How to Solve Business Issues
with Data Science

データサイエンスを価値創出に つなげたいすべての人へ

数理最適化 × 数理モデル × データの力で
最適な意思決定を導出する

技術評論社

[ご注意]

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた運用は、必ずお客様自身の責任と判断によって行ってください。これらの情報の運用の結果について、技術評論社および著者はいかなる責任も負いません。

本書記載の情報は、2025 年 5 月時点のものを掲載していますので、ご利用時には、変更されている場合もあります。

また、ソフトウェアに関する記述は、特に断わりのないかぎり、2025 年 5 月現在での最新バージョンをもとにしています。ソフトウェアはバージョンアップされる場合があります、本書での説明とは機能内容や画面図などが異なってしまうこともあり得ます。本書ご購入の前に、必ずバージョン番号をご確認ください。

以上の注意事項をご承諾いただいた上で、本書をご利用願います。これらの注意事項をお読みいただくずに、お問い合わせいただいても、技術評論社および著者は対処しかねます。あらかじめ、ご承知おきください。

本文中に記載されている会社名、製品名等は、一般に、関係各社／団体の商標または登録商標です。

本文中では ®, ©, ™ などのマークは特に明記していません。

0.1 ビジネス課題とデータサイエンス

今やデータサイエンスは特別な企業だけのものではありません。あらゆる業界で必須のビジネスツールとなっています。企業は日々蓄積される大量のデータを活用し、競争力を高めようとしています。

製品開発、マーケティング、サプライチェーン、顧客サービスなど、ビジネスのあらゆる場面でデータに基づく意思決定が行われています。このトレンドを受け、多くの企業がデータの専門家であるデータサイエンティストを採用し、ビジネス課題を解決するためのデータ活用体制を強化しています。

従来のデータ分析ツールの進化に加えて、近年のデータサイエンスを変えた最大の革新は ChatGPT、Gemini、Claude などの生成 AI の登場です。これらの AI 技術により、データサイエンティストの作業環境は一変しました。複雑なプログラムコードを AI が自動生成してくれたり、対話形式で指示するだけで分析が進められるようになりました。以前なら何時間もかかっていた作業が数分で完了し、データサイエンティストは技術的な細部より本質的な分析に集中できるようになったのです。このような効率化により、企業はより速く、より多くのデータ分析プロジェクトに取り組めるようになり、データサイエンスへの期待は増し、投資が加速しています。

AI ツールの普及により、データサイエンティストに求められる役割は大きく変わりました。高度な分析ツールが誰でも使える時代だからこそ、「ビジネス課題をデータサイエンスで解ける問題に変換する能力」が重要になります。今日のデータサイエンティストには、技術的なスキルだけでなく、ビジネスの本質を理解し、適切なデータと分析手法を選び、実用的な解決策を導き出せる能力が求められています。

本書では、ビジネス課題をデータサイエンスの問題として定式化するプロセスを段階的に解説します。数理モデリング、数理最適化、機械学習などのデータサイエンスの技術を実際のビジネス課題にどう適用するかを、具体例を交えて紹介します。

データサイエンスは単に知的好奇心を満たすだけでなく、実際のビジネスで成果を出し、利益を生み出すための実用的な道具です。理論と実践をつなぐこの本が、データサイエンスを駆使してビジネスに新しい価値をもたらすための実践的なガイドとなれば幸いです。

0.2 本書のアプローチ

本書では、データサイエンスを用いてビジネス現場で課題解決を行う考え方を紹介します。数理モデリング、数理最適化、機械学習といったデータサイエンスの道具を利用しますが、理論面に深入りせず、問題解決の急所となる知識を厳選して取り上げます。筆者の経験では、ビジネス課題を解決するために高度な数学が必要なケースは必ずしも多くはありません。むしろ簡単な数学を自在に扱えることが重要です。

本書は、日々実務でデータ分析を行っているデータサイエンティスト、データアナリスト、機械学習エンジニア、そしてデータサイエンスをビジネス成果につなげたいと考えるすべての人に、実践的な技術を解説することに努めます。また、本書はデータサイエンスを学ぶ学生にも向けて執筆しています。日々学んでいる学問知識を実務と結びつける手助けになれば幸いです。

データサイエンスの実践には分析コードをなめらかに書くスキルも不可欠です。そこで、本書では数学的な解説だけでなく Python による実装も行います。数式だけではイメージがつかみにくい部分でも、実装を通じて理解を深めることができるでしょう。

本書では、線形回帰や Random Forest といった具体的な統計手法・機械学習手法を詳しく解説することはありません。同様に、マーケティングにおける RFM 分析や MMM（マーケティングミックスモデリング）のような、分析フレームワークも取り上げません。その代わりに、ビジネス課題にデータサイエンスでアプローチするための「考え方」を紹介します。

ビジネス課題は企業が直面する状況によって千差万別であり、すべての問題に適用できる汎用的な解法は存在しません。だからと言って、「この種類の問題ならこの分析手法」という一対一の対応関係を網羅的に覚えようとするアプローチには限界があります。なぜなら、ビジネスの現場で遭遇する問題の種類は無数にあり、それらをすべて分類して対応方法を準備することは現実的ではないからです。さらに、このようなアプローチでは、これまでに経験したことのない新しい問題に直面した場合、適切な対応方法を見つけることができません。

より効果的なアプローチは、特定の手法ではなく、考え方の枠組み、つまり「フレームワーク」を身につけることです。適切なフレームワークを習得すれば、問題の種類や状況にかかわらず、一貫した思考プロセスで課題に取り組むことができます。これにより、未知の問題に対しても解決策を導き出せるようになります。

そこで本書では、個別の分析手法やビジネス現場でよく使われる特定のモデルを列挙する代わりに、まず汎用性の高い包括的なフレームワークを紹介します。その後、具体的なビジネス課題の解決プロセスを通して、そのフレームワークを実際にどう適用するかを学びます。このアプローチにより、読者のみなさんは単なる手法の暗記ではなく、さまざまなビジネスシーンで応用可能な思考法を身につけることができるでしょう。

フレームワークを体得するためには、教科書的な例題ではなく、実際のビジネス現場で直面する複雑で多面的な課題に取り組む経験が必要です。そうした生きた事例を通してこそ、読者のみなさんが直面した問題にフレームワークを応用する力が身につきます。

この考え方に基づき、本書では筆者自身が実務で取り組んできた具体的なビジネス課題を題材として採用しています。筆者は REVISIO 株式会社*1に所属しており、テレビ CM の効果測定を専門としているため、テレビ CM を用いたマーケティングを事例として取り上げました。

しかし、ここで重要なのは、テレビ CM を用いたマーケティングそのものを学ぶことが本書の目的ではないという点です。これらのケーススタディはあくまで、フレームワークの適用方法を具体的に示すための実例として機能しています。本書で紹介するフレームワークの本質は、業界や分野を超えた普遍的な思考法にあります。E コマース、製造業、金融、医療など、どのような業界のビジネス課題にも応用できるでしょう。

本書を読み終えた後は、ぜひご自身が直面している固有のビジネス課題にこのフレームワークを適用してみてください。理論を実践に移す過程で、さらに深い理解と洞察が得られるはずです。そして、その経験を通じて、ご自身のビジネス文脈に合わせた課題解決が可能になるでしょう。

0.3 本書の構成

本書は全5章で構成されています。

- 1 章：ビジネス課題を解決する技術
- 2 章：マーケティングにデータサイエンスを導入する

* 1 <https://revisio.co.jp/>

はじめに

- 3 章：認知形成を数理モデリングする
- 4 章：連続最適化で広告予算を配分する
- 5 章：離散最適化で広告出稿番組を選択する

まず、1 章では、ビジネスの現場でデータサイエンスを用いて価値を生み出すための考え方を紹介します。データサイエンスを効果的に活用するためには、ビジネス課題をデータサイエンスで解決可能な問題に変換する必要があります。ビジネス課題をデータサイエンスの問題に落とし込み、解決するための統一的方法として 3 ステップのフレームワークを提案し、具体例を通じてその適用方法を解説します。

1 章で本書の根本的な考え方を確立したあと、続く 2 章から 5 章では、筆者が実務で直面した実際のビジネス課題に対して、3 ステップフレームワークを使って统一的にアプローチします。複数の実践的なビジネス事例に取り組むことで、読者のみなさんが汎用的な課題解決能力を身につけることを目指しています。

2 章では、テレビマーケティングを題材に数理モデルの構築方法を解説します。特に、データの背後に存在するメカニズムを理解し、その洞察を数理モデルに反映する方法を解説します。利用可能なデータだけでは情報が不足している状況において、分析者の仮説を効果的に組み込む手段として、数理モデルがいかに有効であるかを示します。

3 章では、2 章で構築した数理モデルをさらに発展させていきます。広告接触と認知形成といった複雑な関係性に対しても、分析者の仮説を数理モデルに取り入れることで、より妥当な推論が可能になることを示します。

4 章では、テレビ広告とデジタル広告のような複数の広告出稿媒体に対する予算配分の最適化に取り組めます。いずれかの広告に少なくとも一度接触した人の割合（ユニークリーチ）を最大化するという具体的な課題に対し、数理モデルと数理最適化を組み合わせたアプローチ方法を解説します*2。

最後に、5 章では「多数の番組候補の中から広告を出稿する番組を選定する」という実務的な課題に取り組めます。選択肢の組み合わせが膨大になる状況で、効率的な広告出稿を実現するための手法を解説します。

* 2 いわゆる MMM（マーケティングミックスモデル）とは少し毛色が異なります。MMM は売上など最終的な KPI をターゲットにした時系列モデルを考えることが多いですが、4 章ではユニークリーチを最大化するという、より広告施策と連動しやすい KPI を設定します。また、時系列要素も考慮しません。

0.4 本書で扱わないトピックと関連書籍

本書はビジネス課題をデータサイエンスの問題に落とし込み、データを活用してそれを解決する一連のフレームワークについて解説しています。データサイエンスとプログラミングの基礎知識があると内容を理解しやすいでしょう。

もちろん、内容の理解に必要な統計学の概念やプログラミングの知識は都度説明するようにしています。各章の解説と実際のコード例を順に追っていけば、データサイエンスの専門家でなくても、フレームワークの考え方と応用方法を自然に理解できるでしょう。

統計学

統計学については、確率分布や期待値などの基本的な知識があることを前提としています。必要であれば、統計学の入門書として、例えば久保川 (2024) を参照してください。

本書では、数理統計学・ベイズ統計学・機械学習・数理最適化といった分野の考え方を取り入れています。それぞれの専門的な理論や証明には立ち入らず、実践的な応用に焦点を当てています。各分野についてさらに深く学びたい読者は、数理統計学については久保川 (2017) を、ベイズ統計学については Lambert (2018) を、機械学習については Hastie et al. (2009) を、数理最適化に関しては梅谷 (2020) を参考にしてください。

プログラミング

本書のコード例は Python で書かれており、NumPy、Polars^{*3}、SciPy、Matplotlib、scikit-learn などのデータ分析系ライブラリを使用しています。本書はこれらのツールの基本的な使い方を理解していることを前提としていますが、コード例には解説を加えています。もし特定のライブラリの使い方に不安がある場合は、各ライブラリの入門書や公式ドキュメントを参照するとよいでしょう。また、本書のサンプルコードを動かしながら学ぶことも効果的な学習方法です。

* 3 本書ではデータフレーム処理ライブラリとして Pandas ではなく、実行速度やメモリ効率で勝る Polars を採用しました。Polars の入門書としては富山他 (2024) を参考にしてください。

マーケティング

本書ではテレビマーケティングを具体的な題材として用います。分析に必要なテレビ広告の基礎知識は解説しますが、これはあくまでフレームワークの応用を理解するための必要最低限の知識に留めています。マーケティング理論全般の解説は本書の目的ではありません。数学を用いたマーケティング分析について詳しく学びたい方は、片平 (1987)、上田・生田目 (2017)、古川他 (2011) を参考にしてください。

因果推論

本書は因果推論について扱っていません。データ分析では「相関関係」と「因果関係」を区別することが重要です。例えば、「テレビ CM の出稿量と売上の増加に相関がある」という分析結果が得られても、それだけでは「CM が売上を増加させた」という因果関係を証明できません。

本書のフレームワークを理解し活用するために因果推論の知識は必須ではありませんが、データ分析の結果を正しく解釈し、ビジネス意思決定に活かすためには、因果推論についても学ぶことを勧めます。因果推論の考え方を学べる入門書としては、中室・津川 (2017) と安井 (2020) が優れています。

機械学習のテクニック

本書では機械学習モデルを一部活用していますが、機械学習モデルの予測精度を向上させるための専門的なテクニックについては解説対象外としています。例えば、特徴量エンジニアリング、ハイパーパラメータのチューニング、アンサンブル学習、交差検証の設計などの詳細は解説していません。

本書の主眼はビジネス課題の定式化とフレームワークの応用にあり、機械学習モデルはあくまでその手段として用いています。予測精度の追求自体が目的ではないため、これらの詳細には立ち入りません。

機械学習の実践的なテクニックを深く学びたい方は、門脇他 (2019) や Thakur (2021) を参考にしてください。

関連書籍

データサイエンスを用いたビジネス課題解決に関する書籍は他にも多数出版され

していますが、それらの多くは特定の分析手法やアルゴリズムの解説に重点を置いています。一方で、ビジネスの現場でデータサイエンスを真に活かすための「考え方」や「問題設計のアプローチ」に焦点を当てた書籍は比較的少数です。本書と同様に「考え方」を重視している文献としては、例えば高柳・長田 (2023)、Vaughan (2021)、Meerschaert (2015)、Provost and Fawcett (2014)、齋藤・安井 (2021)、齋藤 (2024) が挙げられます。これらの文献は、具体的な分析手法だけでなく、問題の捉え方や解決アプローチの設計など、本書が重視する「考え方」についてさまざまな角度から論じています。本書と併せて参照することで、データサイエンスをビジネス価値につなげるための思考法をより深く理解することができるでしょう。

本書では、ビジネス課題解決のための数理モデリングを深く掘り下げます。数理モデリングに関しては、対話形式で書かれた浜田 (2018) とその続編である浜田 (2020) が読みやすい入門書です。

また、本書では数理モデルによる予測を用いて最適な意思決定を行い、ビジネス課題を解決する方法を解説します。このような予測を活用した意思決定分析についてさらに知識を深めたい場合は、馬場 (2021) が優れた手引きとなります。

0.5 本書で用いる数式の記法

本書で用いる数式の記法を以下にまとめます。最初から通して読む必要はなく、本書を読み進める中で不明な点が出てきた際に、あらためて本節を参照するようにしてください。

確率変数と実現値

確率変数には大文字を使い、その**実現値**（観測値、実測値）に小文字を使います。例えば、確率変数を X とした場合、その実現値を x で表します。また、定数と最適化問題の決定変数にも小文字を使います。

ベクトルは $\mathbf{x} = (x_1, x_2, x_3)$ のように太字で表記します。

パラメータ

確率分布のパラメータや**回帰係数**など、データから推定したい対象にはギリシャ

はじめに

文字を利用します。例えば、ベルヌーイ分布のパラメータである成功確率を θ で表します。

実際にデータから計算した推定値には $\hat{\theta}$ のようにハットを付けて区別します**。

確率分布

確率変数 X が**離散確率分布**に従うとき、 X を**離散確率変数**と呼び、**確率関数**（確率質量関数）を $\Pr(X = x)$ で表します。**連続確率分布**に従うときは X を**連続確率変数**と呼び、**確率密度関数**を $p(x)$ で表します。確率変数が従う確率分布を明示する場合は、それぞれ

$$\Pr(X = x) = \text{Bernoulli}(x \mid \theta)$$

$$p(x) = \text{Beta}(x \mid \alpha, \beta)$$

のように、分布の名称とパラメータを記載します**。

離散確率変数 X と Y に対して、**同時確率関数**を $\Pr(X = x, Y = y)$ とし、 $Y = y$ で条件付けたときの X の**条件付き確率関数**を $\Pr(X = x \mid Y = y)$ とします（ $Y = y$ を省略して $\Pr(X = x \mid y)$ と記載することもあります）。連続確率変数の場合はそれぞれ $p(x, y)$ 、 $p(x \mid y)$ です。

本書で利用する具体的な確率分布を以下にまとめます。

ベータ分布

本書では、CM 接触確率や CM 認知確率の分布として**ベータ分布**を用います。

ベータ分布は連続確率分布で、確率密度関数は以下で定義されます。

$$\begin{aligned} p(x) &= \text{Beta}(x \mid \alpha, \beta) \\ &= \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)} \end{aligned}$$

確率変数のとりうる範囲は $x \in [0, 1]$ 、パラメータのとりうる範囲は $\alpha, \beta > 0$ です。

本書では、パラメータを $\mu = \alpha/(\alpha + \beta)$ 、 $\nu = \alpha + \beta$ と変換した

$$f(\theta \mid \mu, \nu) = \text{Beta}(\theta \mid \mu\nu, (1 - \mu)\nu)$$

* 4 本書では、 $\hat{\theta}$ は実測値から計算された推定値（estimate）を表します。パラメータの推定方法自体を表す推定量（estimator）とは異なることに注意してください

* 5 ベルヌーイ分布とベータ分布については後述します。

$$= \frac{\theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)}$$

を利用します。ここで、パラメータ $\mu \in (0, 1)$ は期待値を表します。パラメータ $\nu > 0$ は分布の密集度を表します。 ν が大きいほど分布が期待値の周りに密集します。

ガンマ分布

本書では、CM 接触回数の期待値の分布として**ガンマ分布**を用います。ガンマ分布は連続確率分布で、確率密度関数は以下で定義されます。

$$\begin{aligned} p(x) &= \text{Gamma}(x \mid \alpha, \beta) \\ &= \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x} \end{aligned}$$

確率変数のとりうる範囲は $x > 0$ 、パラメータのとりうる範囲は $\alpha, \beta > 0$ です。

本書では、パラメータを $\mu = \alpha/\beta$ と変換した

$$\begin{aligned} p(\theta \mid \mu, \alpha) &= \text{Gamma}\left(\theta \mid \alpha, \frac{\alpha}{\mu}\right) \\ &= \frac{\left(\frac{\alpha}{\mu}\right)^\alpha}{\Gamma(\alpha)} \theta^{\alpha-1} e^{-\frac{\alpha}{\mu}\theta} \end{aligned}$$

を利用します。ここで、パラメータ $\mu > 0$ は期待値を表します。パラメータ α は分布の密集度を表します。 α が大きいほど分布が期待値の周りに密集します。

ベルヌーイ分布

本書では、視聴者が CM に接触したかどうか、CM を認知したかどうか、などの二値の事象を表現する分布として**ベルヌーイ分布**を用います。

ベルヌーイ分布は離散確率分布で、確率関数は以下で定義されます。

$$\begin{aligned} \Pr(X = x) &= \text{Bernoulli}(x \mid \theta) \\ &= \theta^x (1 - \theta)^{1-x} \end{aligned}$$

確率変数のとりうる範囲は $x \in \{0, 1\}$ 、パラメータのとりうる範囲は $\theta \in [0, 1]$ です。ここで、パラメータ θ は確率変数が 1 をとる確率を表します。よって、このベルヌーイ分布の確率関数は、CM 接触確率が θ の CM を 1 回だけ放送したときに、CM に x 回接触する確率を表します。

はじめに

なお、ベルヌーイ分布に従う確率変数の期待値は、確率変数が 1 をとる確率、つまり θ と等しくなります。

$$\begin{aligned}\mathbb{E}[X] &= 1 \cdot \Pr(X = 1) + 0 \cdot \Pr(X = 0) \\ &= \Pr(X = 1) \\ &= \theta\end{aligned}$$

二項分布

本書では、CM 接触回数を表す分布の一つとして**二項分布**を用います。

二項分布は離散確率分布で、確率関数は以下で定義されます。

$$\begin{aligned}\Pr(X = x) &= \text{Binomial}(x \mid n, \theta) \\ &= \binom{n}{x} \theta^x (1 - \theta)^{n-x}\end{aligned}$$

これは、CM 接触確率が θ の CM を n 回 CM を放送したときに、ちょうど x 回だけ CM に接触する確率を表します。確率変数のとりうる範囲は $x \in \{0, 1, 2, \dots\}$ 、パラメータのとりうる範囲は $n \in \{1, 2, \dots\}$ 、 $\theta \in [0, 1]$ です。

ベータ二項分布

本書では、CM 接触回数を表す分布のひとつとして**ベータ二項分布**を用います。

ベータ二項分布は離散確率分布で、確率関数は以下で定義されます。

$$\begin{aligned}\Pr(X = x) &= \text{BetaBinom}(x \mid n, \mu, \nu) \\ &= \binom{n}{x} \frac{B(\mu\nu + x, (1 - \mu)\nu + n - x)}{B(\mu\nu, (1 - \mu)\nu)}\end{aligned}$$

確率変数のとりうる範囲は $x \in \{0, 1, 2, \dots\}$ 、パラメータのとりうる範囲は $n \in \{1, 2, \dots\}$ 、 $\mu \in (0, 1)$ 、 $\nu > 0$ です。

ベータ二項分布の確率関数は、二項分布の CM 接触確率 θ がベータ分布 $\text{Beta}(\theta \mid \mu\nu, (1 - \mu)\nu)$ に従うと仮定して、CM を n 回放送したときにちょうど x 回だけ CM に接触する確率を表します。よって、パラメータ μ は仮定したベータ分布の期待値を、 ν は密集度を表します。

ポアソン分布

本書では、CM 接触回数を表す分布の一つとして**ポアソン分布**を用います。

ポアソン分布は離散確率分布で、確率関数は以下で定義されます。

$$\begin{aligned}\Pr(X = x) &= \text{Poisson}(x \mid \theta) \\ &= \frac{\theta^x e^{-\theta}}{\Gamma(x+1)}\end{aligned}$$

これは、CM 接触回数の期待値を θ とした場合に、ちょうど x 回だけ CM に接触する確率を表します。確率変数のとりうる範囲は $x \in \{0, 1, 2, \dots\}$ 、パラメータのとりうる範囲は $\theta > 0$ です。

負の二項分布

本書では、CM 接触回数を表す分布のひとつとして**負の二項分布**を用います。

負の二項分布は離散確率分布で、確率関数は以下で定義されます。

$$\begin{aligned}\Pr(X = x) &= \text{NBinom}(x \mid \mu, \alpha) \\ &= \frac{\Gamma(\alpha + x)}{\Gamma(x+1)\Gamma(\alpha)} \left(\frac{\alpha}{\mu + \alpha}\right)^\alpha \left(\frac{\mu}{\mu + \alpha}\right)^x\end{aligned}$$

確率変数のとりうる範囲は $x \in \{0, 1, 2, \dots\}$ 、パラメータのとりうる範囲は $\mu, \alpha > 0$ です。

負の二項分布の確率関数は、ポアソン分布の CM 接触回数の期待値 θ がガンマ分布 $\text{Gamma}\left(\theta \mid \alpha, \frac{\alpha}{\mu}\right)$ に従うと仮定した場合に、ちょうど x 回だけ CM に接触する確率を表します。よって、パラメータ $\mu > 0$ は仮定したガンマ分布の期待値を、パラメータ α は密集度を表します。

期待値と分散

確率変数 X の**期待値**を $\mathbb{E}[X]$ 、**分散**を $\text{Var}[X]$ 、確率変数 (X, Y) の**共分散**を $\text{Cov}[X, Y]$ とします。

また、 $Y = y$ で条件付けた X の**条件付き期待値**を $\mathbb{E}[X \mid Y = y]$ とし、離散確率変数の場合はそれぞれ以下で定義します。

$$\begin{aligned}\mathbb{E}[X] &= \sum_{x \in \mathcal{X}} x \Pr(X = x) \\ \text{Var}[X] &= \sum_{x \in \mathcal{X}} (x - \mathbb{E}[X])^2 \Pr(X = x) \\ \text{Cov}[X, Y] &= \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} (x - \mathbb{E}[X]) (y - \mathbb{E}[Y]) \Pr(X = x, Y = y)\end{aligned}$$

はじめに

$$\mathbb{E}[X | Y = y] = \sum_{x \in \mathcal{X}} x \Pr(X = x | Y = y)$$

ここで、 $\mathcal{X}$ と $\mathcal{Y}$ はそれぞれ確率変数 X と Y がとりうる値の集合を表しています。

なお、連続確率変数の場合は、確率関数が確率密度関数に、総和が積分になります。

$$\mathbb{E}[X] = \int_{x \in \mathcal{X}} xp(x)dx$$

$$\text{Var}[X] = \int_{x \in \mathcal{X}} (x - \mathbb{E}[X])^2 p(x)dx$$

$$\text{Cov}[X, Y] = \int_{x \in \mathcal{X}} \int_{y \in \mathcal{Y}} (x - \mathbb{E}[X]) (y - \mathbb{E}[Y]) p(x, y) dx dy$$

$$\mathbb{E}[X | Y = y] = \int_{x \in \mathcal{X}} xp(x | y)dx$$

集合

集合は $\mathcal{X}$ のように筆記体（カリグラフィー）で表し、**要素** x が集合 $\mathcal{X}$ に含まれることを $x \in \mathcal{X}$ で表します。

集合に含まれる要素の個数は $|\mathcal{X}|$ で表します。例えば、 $\mathcal{X} = \{1, 2, 3\}$ の場合は $|\mathcal{X}| = 3$ です。**空集合**は $\emptyset$ で表します。なお、 $|\emptyset| = 0$ です。**和集合**は $\cup$ で、**積集合**は $\cap$ で表します。

論理記号

論理和「または」を $\vee$ で、**論理積**「かつ」を $\wedge$ で表します。

指示関数

指示関数を $\mathbf{1}\{\cdot\}$ で表します。ここで、指示関数とは、条件を満たせば 1 を返し、満たさなければ 0 を返す関数です。例えば、 x が 0 より大きければ 1 を返し、そうでなければ 0 を返す指示関数を、

$$\mathbf{1}\{x > 0\} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

と記載します。

数理最適化

最大化または**最小化**の両方を指す包括的な用語として**最適化**を用います。

制約付き最大化問題は以下のように表記します。

$$\begin{aligned} \max_x \quad & f(x) \\ \text{s.t.} \quad & x \in \mathcal{X} \end{aligned}$$

で表します。ここで、 x は**決定変数**、 $f(x)$ は**目的関数**です。s.t. は such that または subject to の略で、以降に満たすべき**制約条件**を記載します。最大化ではなく最小化問題の場合は $\max$ の代わりに $\min$ と記載します。

目的関数を最大化する x の値は x^* と表記します。 x^* がどの最適化問題から得られたかを明示する際は

$$\begin{aligned} x^* = \arg \max_x \quad & f(x) \\ \text{s.t.} \quad & x \in \mathcal{X} \end{aligned}$$

と記載します。ここで、 $\arg \max$ は「最大化問題の解となる値」を意味します。

なお、パラメータ θ の推定値は最小二乗法や最尤法などの数理最適化で得られた値ですが、 θ^* ではなく $\hat{\theta}$ と表記します。

記法のまとめ

本書で用いる数式の記法とその説明を表 0.1 にまとめます。

▼ 表 0.1: 本書で用いる記号・記法の意味

記号・記法	記号・記法の意味
X	確率変数
x	実現値、定数、決定変数
$\boldsymbol{x}$	ベクトル
$\Pr(X = x), p(x)$	確率関数、確率密度関数
$\Pr(X = x, Y = y), p(x, y)$	同時確率関数、同時確率密度関数
$\Pr(X = x \mid Y = y), p(x \mid y)$	条件付き確率関数、条件付き確率密度関数
$\mathbb{E}[X]$	期待値
$\text{Var}[X]$	分散
$\text{Cov}[X, Y]$	共分散
$\mathbb{E}[X = x \mid Y = y]$	条件付き期待値
θ	パラメータ
$\hat{\theta}$	パラメータの推定値
$\mathcal{X}$	集合
$ \mathcal{X} $	集合に含まれる要素の個数
$\cup, \cap$	和集合、積集合
$\vee, \wedge$	論理和、論理積
$\mathbf{1}\{x \in \mathcal{X}\}$	指示関数
$\max, \min$	最大化、最小化
$\arg \max, \arg \min$	最適化問題の解となる値
s.t.	制約条件

0.6 本書で用いる Python コード

本書に掲載している Python コードは、GitHub 上に BSD 3.0 ライセンスで公開しています。

https://github.com/ghmagazine/mathematical_modeling_book

Python コードの実行環境には Jupyter Notebook を利用しています。データサイエンスの進歩は極めて早く、ライブラリの開発も日進月歩で進んでいます。本書

で利用しているライブラリの使用方法が本書執筆時点と異なる場合がありますので、ご注意ください。

目 次

はじめに	iii
0.1 ビジネス課題とデータサイエンス	iii
0.2 本書のアプローチ	iv
0.3 本書の構成	v
0.4 本書で扱わないトピックと関連書籍	vii
0.5 本書で用いる数式の記法	ix
0.6 本書で用いる Python コード	xvi

1 章	
ビジネス課題を解決する技術	1
1.1 ビジネス課題とデータサイエンスの問題	2
1.1.1 データサイエンスの問題と数学の文章題	2
1.1.2 ビジネス課題と文章題のギャップ	3
1.2 ビジネス課題を解決するためのフレームワーク	6
1.3 ステップ 1：ビジネス課題を数理最適化問題として定式化する	10
1.3.1 数理最適化問題としての定式化	10
1.3.2 なぜ数理最適化問題として定式化するのか	14
1.3.3 条件付き期待値の導入	15
1.3.4 なぜ条件付き期待値を利用するのか	18
1.4 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する	19
1.4.1 単純集計	19
1.4.2 数理モデルの導入	20
1.5 ステップ 3：数理最適化問題を解いて最適なアクションを導出する	36
1.6 本章のまとめ	39

2 章	
マーケティングにデータサイエンスを導入する	41
2.1 マーケティング活動としてのテレビ CM	42
2.1.1 マーケティングにおけるテレビ CM の役割	42
2.1.2 広告指標	44
2.1.3 指標の測定	46

2.1.4	テレビ CM の出稿形態	48
2.1.5	スポットにおけるグロスリーチとユニークリーチの関係	51
2.1.6	実務における課題	53
2.2	ステップ 1：ビジネス課題を数理最適化問題として定式化する	54
2.3	ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する	57
2.3.1	データの準備と可視化	57
2.3.2	機械学習モデルによる推定	62
2.3.3	線形回帰モデルによる推定	65
2.3.4	対数変換による推定	67
2.3.5	既存の予測モデルの限界	69
2.3.6	CM 接触の数理モデルの構築	70
2.3.7	ポアソン分布によるリーチカーブの推定	73
2.3.8	負の二項分布によるリーチカーブの推定	82
2.4	ステップ 3：数理最適化問題を解いて最適なアクションを導出する	100
2.5	本章のまとめ	103
	コラム：シミュレーションによる数理モデルの検証	106

3 章

認知形成を数理モデリングする 111

3.1	ビジネス課題：製品認知率の改善	112
3.2	ステップ 1：ビジネス課題を数理最適化問題として定式化する	112
3.3	ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する	114
3.3.1	求められるデータ	114
3.3.2	データの読み込みと可視化	115
3.3.3	機械学習モデルによる推定	120
3.3.4	線形回帰モデルによる推定	122
3.3.5	数理モデルによる問題の分解	124
3.3.6	CM 接触確率と製品認知率の関係	125
3.3.7	個人差を考慮したモデルの拡張	129
3.3.8	パラメータの推定方法	135
3.3.9	数理モデルの実装	139
3.4	ステップ 3：数理最適化問題を解いて最適なアクションを導出する	153
3.5	本章のまとめ	155

4 章

連続最適化で広告予算を配分する

159

4.1	ビジネス課題：広告予算配分の最適化	160
4.2	ステップ 1：ビジネス課題を数理最適化問題として定式化する	161
4.3	ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する	162
4.3.1	必要なデータ	162
4.3.2	データの読み込みと可視化	163
4.3.3	機械学習モデルによる予測	167
4.3.4	線形回帰モデルによる予測	171
4.3.5	数理モデルの構築	172
4.3.6	重複リーチのモデリング	175
4.3.7	重複リーチの補正	181
4.3.8	各媒体のユニークリーチの予測	190
4.3.9	予測モデルの統合	196
4.4	ステップ 3：数理最適化問題を解いて最適なアクションを導出する	202
4.4.1	最適解の算出	202
4.4.2	最適化のインパクト	204
4.5	本章のまとめ	208

5 章

離散最適化で広告出稿番組を選択する

211

5.1	ビジネス課題：番組選択の最適化	212
5.2	ステップ 1：ビジネス課題を数理最適化問題として定式化する	212
5.3	ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する	214
5.3.1	必要なデータ	214
5.3.2	データの読み込みと可視化	215
5.3.3	線形回帰モデルによる予測	220
5.3.4	リッジ回帰による予測	224
5.3.5	機械学習モデルによる予測	226
5.3.6	数理モデルの構築	228
5.3.7	数理モデルの推定	237
5.3.8	数理モデルの実装	241
5.3.9	データを用いた予測精度の検証	243

5.4 ステップ 3：数理最適化問題を解いて最適なアクションを導出する 244

5.4.1 離散最適化問題 244

5.4.2 貪欲法による近似解の探索 245

5.5 本章のまとめ 250

コラム：フリークエンシー分布の推定 252

コラム：合算のユニークリーチを各番組の貢献度に分解する 267

 おわりに 287

 参考文献 289

 索引 292

 著者プロフィール 297

1章

ビジネス課題を解決する技術

1章では、ビジネスの現場でデータサイエンスを用いて価値を生み出すための考え方を紹介します。あいまいなビジネス課題を具体的なデータサイエンスの問題に落とし込んで解決する手法として、3ステップフレームワークを提案します。数理最適化、数理モデル、そしてデータの力を借りて、課題に対する最適なアクションを導出する方法を解説します。

1.1 ビジネス課題とデータサイエンスの問題

本書では、ビジネス上のさまざまな課題を解決するために、ビジネス課題をデータサイエンスの問題へと落とし込み、データに基づいて意思決定を行う方法を解説します*1。さて、「ビジネス課題をデータサイエンスの問題に落とし込む」とは、具体的に何を指すのでしょうか。それは、ビジネス課題を数式を用いて定式化し、データサイエンスで解決できる問題に変換することです。

「数式で定式化」と言われても、具体的なイメージがつかみにくいかもしれません。そこで、具体的なイメージをつかむ第一歩として、まずは簡単な数学の文章題を扱います。文章題の例を通じて、ビジネスの課題をどのように数理的な問題へ変換するのか、その基本的な考え方を説明します。

1.1.1 データサイエンスの問題と数学の文章題

「ビジネス課題をデータサイエンスの問題に落とし込む」ことは、数学の文章題を例にとって考えるとわかりやすいと思います。

太郎くんは、10 分前に出発した花子さんに追いつきたいとします。50 分以内に追いつくためには、太郎くんは分速何メートルで移動すればよいでしょうか。ただし、花子さんの移動速度は分速 100 メートルとします。

数学のおさらいにこの問題を解いてみましょう。まず、この文章題で求めたいものは太郎くんの移動速度です。これを分速 x メートルとおきます。次に、花子さんの移動速度は分速 100 メートルで、花子さんは 10 分前に出発しているので、現時点ですでに $100 \times 10 = 1000$ メートル先に進んでいます。そして、太郎くんは分速 100 メートルより速く移動することで、毎分 $x - 100$ メートルだけ花子さんとの差を縮めることができます。

これらの情報を用いると、太郎くんが花子さんに 50 分以内に追いつくためには

$$\frac{100 \times 10}{x - 100} \leq 50$$

を満たすような移動速度 x が必要であることがわかります。

* 1 本書では、「課題」と「問題」を次のように区別します。「課題」とは、目指す方向は決まっているものの、解決への具体的な道のりや必要な要素がまだ見えていない状態を指します。それに対し「問題」とは、解決に必要な要素が明確に洗い出され、具体的な解決策の検討に移れる状態を指します。

この不等式を x について解くことで、

$$x \geq 120$$

が求まります。つまり、分速 120 メートル以上で移動すれば、太郎くんは 50 分以内に花子さんに追いつくことができるとわかりました。

1.1.2 ビジネス課題と文章題のギャップ

さて、本書は文章題の解き方を解説する本ではないので、話をビジネス課題に戻します。数学の文章題では課題を解くために必要な情報がすべて与えられますが、ビジネス課題はそうではありません。基本的に、ビジネス課題は以下のように大雑把な目標だけが決まっている状態で分析者に提示されます。

先に出発した花子さんに追いつきたい

これは、答えを出すための条件や道筋があらかじめ決まっておらず、自分で考える範囲が広い、自由度の高い課題です。では、この課題にどう取り組めばよいのでしょうか？

先ほどの文章題は簡単に回答を得られました。ということは、文章題と今回のビジネス課題を比較し、何が足りないのか、何が明確でないのかを洗い出せば、文章題と同じように回答を得ることができそうです。

具体的には、以下の点を自分で考え、設定し、情報を集める必要があります。

- 最終的に何をを目指すのか、具体的な目標の水準を明確化する
- どうやってその目標を達成するのかという手段を決める
- 課題を解くために必要な情報は何かを考え、情報を集める
- 情報が不足している点については、できるだけ妥当な仮定を置く

このように、文章題でははじめから与えられていた要素を、ビジネス課題では分析者自身で能動的に考え、設定していくことが求められます。

具体的に、太郎くんの立場でこの課題を解くためにはどうすればよいかを考えてみましょう。まず、目標の水準を明確化する必要があります。1 分で追いつかなければならないのか、それとも 1 時間かけて追いつけばよいのかをステークホルダーと議論することで明らかにします。議論の結果、ここでは「50 分で追いつく」という目標に落ち着いたとします。

1章 ビジネス課題を解決する技術

次に、どうやってこの目標を達成するのか、手段を決める必要があります。ここでは、花子さんよりも高速に移動することで追いつくという手段をとることにしましょう。他の手段として、花子さんに戻ってきてもらう、花子さんの移動を妨害する、などの手段も考えられますが、それらの選択肢はとらないことにします。

さらに、課題を解くために必要な情報を特定します。幸い、先ほど情報が完備されている文章題を解いたおかげで、必要な情報はわかっています。花子さんはどのくらい前に出発しているのか、そしてどのくらいの速度で移動しているのかを知ることが必要です。

最後に、情報が不足している点に仮定を置きます。花子さんはいままで同じ速度で移動し続けていたのか、これからも同じ速度で移動し続けるのかはわからないわけですが、ここでは移動速度は一定だと仮定を置くことにします。例えば、花子さんが疲労から徐々に移動速度を落としていくと想定することもできますが、今回は考えないことにします*2。

ここまでたどり着けば、あいまいだったビジネス課題を数式で明確に表現することができます。現時点で不明な情報である花子さんの移動速度を分速 α メートルとし、花子さんが何分前に出発したかを β 分前とします。この設定のもとで、太郎くんが 50 分以内に花子さんに追いつくために必要な移動速度を求めるには

$$\frac{\alpha\beta}{x - \alpha} \leq 50$$

を満たすような x を計算すればよさそうです。太郎くんの移動速度は花子さんの移動速度以上でないと追いつけないので、 $x - \alpha > 0$ を仮定して、上の不等式を x について解くことで

$$x \geq \frac{\alpha(\beta + 50)}{50}$$

を得ます。この条件を満たす x が太郎くんに求められる移動速度です。

* 2 基本的に、強い仮定ほど強い推論を引き出すことができますが、仮定が妥当でなく間違った推論をしてしまう危険性も高まります。一方で、弱い仮定を置くと（または仮定を置かないと）、データから得られる情報が少ない場合は弱い推論しか行えなかったり、まったく結論が出ないこともあります。これは、データサイエンスの文脈ではバイアスとバリエーションのトレードオフとして解釈できます。例えば、線形回帰モデルは、目的変数と説明変数の間に「線形性」という強い構造的仮定を置きます。この仮定が現実のデータ生成プロセスと合致していれば、少ないデータからでもパラメータ推定が安定し、未知データに対しても高い汎化性能を示します。しかし、もし関係性が非線形であれば、この強い仮定がバイアスを生み出し、系統的に誤った予測を出力してしまいます。同様のトレードオフは、ベイズ統計学における事前分布の設定にも見られます。強い信念を反映した情報のある事前分布を用いると、データからの情報が少なくても、事後分布の不確実性は小さくなります。しかし、その事前情報が真の状態から大きく乖離していた場合、たとえデータが異なる情報を示唆していても、事後分布はその誤った事前情報に引きずられ、系統的に歪んでしまう可能性があります。対照的に、無情報事前分布や弱い事前分布を用いると、データから得られた情報が事後分布により強く反映されますが、データが少ない場合には事後分布の不確実性は大きいままとなり、明確な結論を得にくいこともあります。

1.1 ビジネス課題とデータサイエンスの問題

さて、上記の不等式には、現時点では具体的な値がわからない未知のパラメータ^{*3} (α, β) が含まれているので、このままでは具体的に必要な移動速度を計算できません。計算のためには、 (α, β) に具体的な数字を代入する必要があります。

ここでデータが力を発揮します。いま知りたいことに直接的に関連していて、信頼性の高いデータをもっていればパラメータの推定精度が高まります。一方で、間接的で信頼性の低いデータしかもっていなければ、パラメータの推定精度が低くなります。

例えば、花子さんの移動速度 α に関して、過去の移動速度を測定したデータがあれば、そこから花子さんの平均的な移動速度を推定できます。ただし、常識的に考えるとそのようなデータは測定していないので、一般的な徒歩の分速である分速 80 メートルを利用するしかないかもしれません。花子さんの移動速度が一般的な移動速度からかけ離れている可能性は低いと思われるので、これでも悪くない推論ができそうです。よって、花子さんの移動速度は分速 80 メートルと推測することになります。

同様に、何分前に出発したかの β の推定に関しても、花子さんが毎日何時に出発するかを知っていれば、今日も同じくらいの時間に出発したと想定してよいかもしれません。ここでは、普段の出発時刻から 20 分前に出発したと推測できたとします。

データから $(\alpha, \beta) = (80, 20)$ と推測できたので、これを前述の必要速度を求める式に代入することで

$$x \geq \frac{80(20 + 50)}{50}$$

$$\iff x \geq 112$$

を得ます。つまり、50 分以内に花子さんに追いつくために、太郎くんは分速 112 メートル以上の速度で移動する必要があることがわかりました。課題の解決法はわかったので、あとは「分速 112 メートル以上の移動をする」と意思決定し、実行することで、目的を達成することができます^{*4}。

*** 3** 本書では、母集団の特性を表す値だけでなく、データを用いて推測する対象を広くパラメータと呼びます。もちろん、線形回帰モデルの回帰係数や、確率分布の母数もパラメータです。

*** 4** ところで、ここまでビジネス課題を数式を用いて定式化してきましたが、こんなことを考えている間に花子さんはどんどん先に進んでしまうと考える読者もいるかもしれません。まったくもってその通りです！花子さんに追いつきたければ、太郎くんは何も考えずに全力疾走すればいいのです。今回の具体例がそうであるように、ビジネスの現場では常にスピードが求められます。数ヶ月かけて 100 点の分析を行うより、80 点の分析を 1 週間で出して意思決定した方がよいケースは多くあります。データサイエンティストは、目の前の分析のことだけを考えるのではなく、ビジネス全体に対する広い視野をもち、正確性とスピードのトレードオフの中でちょうどよいバランスを見つける必要があると言えるでしょう。

1章 ビジネス課題を解決する技術

以上の文章題の例のように、あいまいだったビジネス課題を数式を用いて定式化し、データサイエンスで解決可能な問題に変換することを、本書では「ビジネス課題をデータサイエンスの問題に落とし込む」と呼びます。データサイエンスの問題として定式化できれば、あとは不明な情報をデータを活用して推測し、具体的な意思決定を導くことで、ビジネス課題の解決につなげることができます。今回は「花子さんに追いつく」を例にとりましたが、実際のビジネス課題である「売上の増加」や「顧客の離脱の防止」も同様にして考えることができます。

1.2

ビジネス課題を解決するための フレームワーク

前節では、数学の文章題の例を用いて、ビジネス課題をデータサイエンスの問題に落とし込んで、適切な意思決定を行う方法について説明しました。それでは、文章題の例を超えて、実務で遭遇するビジネス課題をどうやってデータサイエンスの問題に落とし込んで解決するのでしょうか。本書では、データサイエンスを用いてビジネス課題を解決する統一的なフレームワークとして、以下の3ステップを提案します。

ステップ1 ビジネス課題を数理最適化問題として定式化する

ステップ2 数理モデルを構築し、未知のパラメータをデータから推定する

ステップ3 数理最適化問題を解いて最適なアクションを導出する

ステップ1では、解くべきビジネス課題を数式を用いて定式化することで、データサイエンスの問題として再定義します。このとき、定式化はできる限り数理最適化問題にすることを目指します。数理最適化問題としての定式化を通じて、目的、目的を達成するための手段、実務的な制約条件を明確にできるからです。

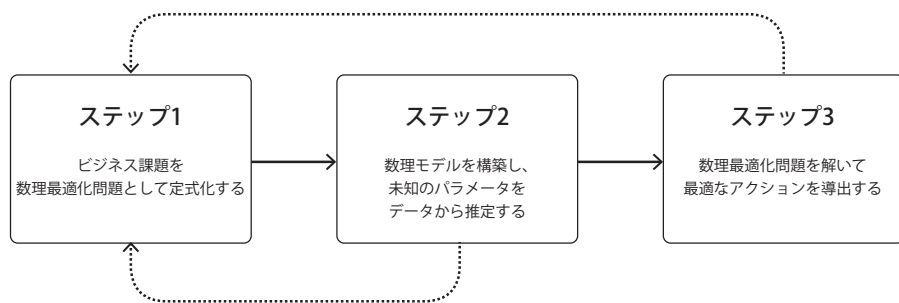
データサイエンティストが実務で遭遇するような問題において、定式化した数理最適化問題をそのまま解けるケースは多くありません。というのも、多くの場合、最適化問題におけるアクション^{*5}と成果の関係性が明らかではないからです。そこで、ステップ2では、アクションと成果の関係性に対する数理モデルを構築し、モデルのパラメータをデータを用いて推定することで、ステップ1で定式化した数理最適化問題を解けるようにします。もしデータが不足している場合は、ドメイン知識を総動員してできるだけ妥当な仮定を置くことが求められます。

***5** 本書では、アクションという用語を文脈に応じて、行動、施策、選択、手段という意味で用います。

1.2 ビジネス課題を解決するためのフレームワーク

最後に、ステップ3では、未知の関係性がなくなった数理最適化問題を解くことで、最適なアクションを導出します。このステップの難易度は、定式化した数理最適化問題の複雑さに依存します。必要に応じて、数理最適化問題を解くための効率的なアルゴリズムを利用します。

この3ステップは必ずしも順番通りに進むわけではなく、試行錯誤を繰り返して行ったり来たりする必要があります（図 1.1）。ステップ2で適切なデータが入手できない場合や、ステップ3で数理最適化問題の解を見つけるのが困難な場合は、ステップ1に戻って問題の定式化を見直すことが有効です。



■ 図 1.1: 3 ステップフレームワーク

それでは、3 ステップフレームワークのイメージをつかむため、前節の文章題に、3 ステップフレームワークを適用してみましょう。

ステップ1：ビジネス課題を数理最適化問題として定式化する

まず、ステップ1ではビジネス課題を数理最適化問題として定式化します*6。

文章題の例は、太郎くんは先に出発してしまった花子さんに50分以内に追いつきたいという設定でした。数理最適化問題を定式化するために、まずは決定変数、つまり目的を達成する手段を考えましょう。ここでは走って追いつくという手段をとるので、太郎くんの移動速度である分速 x メートルが決定変数になります。

次に、目的関数、つまり最適化したい指標を考えます。走って移動するのは大変なので、移動速度はできるだけ遅くしたいと考えているとします。つまり、目的関数は $f(x) = x$ で、これを最小化します。

* 6 数理最適化問題の記法に慣れていない読者は、この例は流し読みして問題ありません。次節で数理最適化問題の基本的な概念と記法を解説します。

1章 ビジネス課題を解決する技術

最後に、制約条件を考えましょう*7。太郎くん君は、できるだけ移動速度を落としながら、「先に出発してしまった花子さんに 50 分以内に追いつく必要がある」という条件を満たす必要があります。花子さんの移動速度を分速 α メートルとし、花子さんが何分前に出発したかを β 分前とすると、以下の制約条件を満たす必要があります。

$$\frac{\alpha\beta}{x - \alpha} \leq 50$$

さらに、太郎くんの移動速度 x が花子さんの移動速度 α より遅いと永久に追いつけないので、 $x > \alpha$ という制約を追加します。

上記の設定のもとで、このビジネス課題は以下の移動速度最小化問題として定式化できます。

$$\begin{aligned} \min_x \quad & x \\ \text{s.t.} \quad & \frac{\alpha\beta}{x - \alpha} \leq 50 \\ & x > \alpha \end{aligned}$$

ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ステップ 1 で数理最適化問題を定式化できました。すでに未知のパラメータ α と β が特定されているので、数理モデルの構築はスキップして、ステップ 2 ではこれらのパラメータを推定します。ここでは、 α は一般的な徒歩の分速から分速 80 メートル、 β は花子さんの普段の出発時刻から 20 分前と推測できたとします。未知のパラメータに推測した具体的な値を代入することで、数理最適化問題の具体的な解が求められるようになります。

$$\begin{aligned} \min_x \quad & x \\ \text{s.t.} \quad & \frac{80 \times 20}{x - 80} \leq 50 \\ & x > 80 \end{aligned}$$

ステップ 3：数理最適化問題を解いて最適なアクションを導出する

最後に、ステップ 3 では、具体化した数理最適化問題を解きます。この数理最適

* 7 記号を定義する都合上、本書では決定変数 → 目的関数 → 制約条件の順に記載することが多いです。実際はこの順番通りに考えるわけではなく、まず確定している部分を定式化し、その後に足りない部分を埋めていきます。

1.2 ビジネス課題を解決するためのフレームワーク

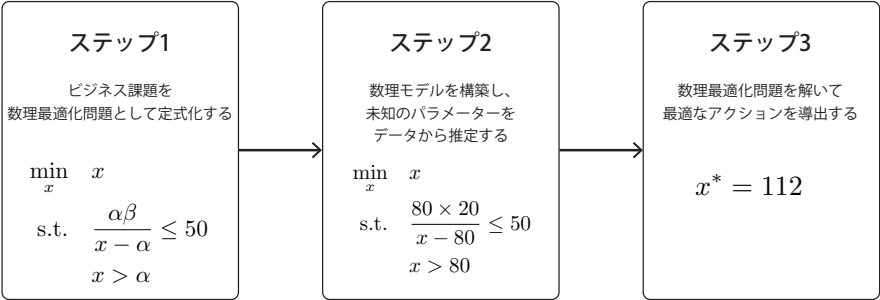
化問題は単純で、できるだけ移動速度 x を小さくしたいので、制約条件を満たす範囲で最も小さい x を求めます。

二つ目の制約条件 $x > 80$ に注意しながら、一つ目の制約条件を x について解くと

$$\begin{aligned}\frac{80 \times 20}{x - 80} &\leq 50 \\ \iff x &\geq \frac{80(20 + 50)}{50} \\ \iff x &\geq 112\end{aligned}$$

であり、これを満たす最小の値は $x^* = 112$ です*8。よって、太郎くんは分速 112 メートルで移動することが最適なアクションであることがわかりました。

なお、ここで取り上げた数理最適化問題は単純なので解析的に解くことができましたが、問題が複雑な場合は数理最適化問題を解くための適切なアルゴリズムを用いることになります。



■ 図 1.2: 文章題に 3 ステップフレームワークを適用

ここまでのステップを図 1.2 にまとめました。このように、3 ステップフレームワークを用いることで、ビジネス課題に統一的な方法でアプローチすることができます。フレームワーク全体のイメージがつかめたところで、以降の節では、各ステップについて詳しく解説していきます。

* 8 本書では、最適な値を表すために文字の右上に * を付けます

1.3

ステップ1：ビジネス課題を数理最適化問題として定式化する

ステップ1は、ビジネス課題をデータサイエンスの問題に変換する最初のステップです。ビジネスの現場でデータサイエンスを用いて価値を出すためには、「どうやってデータサイエンスの問題を解決するか」以上に、「ビジネス課題をどんなデータサイエンスの問題に落とし込むか」が重要です^{*9}。データサイエンスをビジネス価値につなげるためには、当たり前ですが、データサイエンスの問題を解くことがビジネス課題を解くことに直結するように問題を変換する必要があります。そのためには、ビジネス課題を数理最適化問題として定式化することが有効です。

1.3.1 数理最適化問題としての定式化

ビジネス上の意思決定問題は、多くの場合、数式を用いて数理最適化問題として定式化することができます。数理最適化問題というと仰々しいかもしれませんが、これは「選択肢の中から最も良いものを選ぶ」という意思決定としては当たり前のプロセスを数式を用いて表現したものです。ビジネス課題を数理最適化問題として定式化するためには、目的関数、制約条件、決定変数と定数を数式を用いて明確に定義する必要があります。

- 目的関数
 - 最大化、または最小化したい指標
 - 例えば、利潤、売上、一人あたり製品購入数、認知率、Web サイト訪問者数、費用など
- 制約条件
 - 最適化問題を解く上で守らなければいけない条件
 - 例えば、予算、時間、品質、人員など
- 決定変数と定数
 - 最適化問題を解く上で自分が変化させることができる要素を決定変数（または単に変数）、固定の要素を定数と呼ぶ。置かれているビジネス状況に応じ

^{* 9} もちろん、データサイエンスによる課題解決でインパクトを出すためには、「どのビジネス課題を解決するべきか」というもう一段上流の観点も重要です。ただし、本書はデータサイエンスで課題を解決する方法に焦点を当てるので、より大きなビジネスインパクトを生み出すビジネス課題を見つける方法についてはふれません。同様に、分析の結果決まったことをやり切る方法も重要ですが、こちらも本書では割愛します。興味のある読者は、例えば安宅 (2024) や河本 (2022) を参考にしてください。

1.3 ステップ 1：ビジネス課題を数理最適化問題として定式化する

て、何を変数と考え、何を定数と考えるべきかを判断する

。例えば、「製品の製造数は変数で、製品一つあたりの価格は定数」など

「選択肢の中から最も良いものを選ぶ」という意思決定は、数理最適化の言葉では「制約条件のもとで目的関数の値を最大化（または最小化）させる決定変数の値を見つけ出す」ことだと言えます。

それでは、実際に数式を用いて数理最適化問題を定義します。まずは一般的な状況を考えましょう。基本的に、数理最適化問題は以下の形式をとります。

$$\begin{aligned} \max_x \quad & f(x) \\ \text{s.t.} \quad & x \in \mathcal{X} \end{aligned}$$

定式化した要素を一つひとつ説明します。

- 数理最適化問題には最大化問題と最小化問題があるが、ここでは最大化問題を定式化している。最小化を考える場合は $\max$ ではなく $\min$ と表記する
- x は変化させることができる決定変数。変数であることを明示して定数と区別するために、 $\max_x$ と表記する
- $f(x)$ は最大化したい目的関数。目的関数 $f(x)$ は、 x の値に応じて値が変わるので、 $f(x)$ の値を最大化する x の値を見つけることが目的となる
- $\mathcal{X}$ は制約条件を満たす x の集合。制約条件であることを示すために s.t. と表記する^{*10}

簡単な数値例でイメージをつかんでおきましょう。決定変数を x 、目的関数を $f(x) = 2x$ とし、制約条件を満たす決定変数の集合を $\mathcal{X} = [0, 2]$ とした場合（つまり $0 \leq x \leq 2$ ）、最大化問題は以下のように表現できます。

$$\begin{aligned} \max_x \quad & 2x \\ \text{s.t.} \quad & 0 \leq x \leq 2 \end{aligned}$$

このとき、制約条件を満たしつつ目的関数の値を最大化する決定変数の値は $x^* = 2$ で、目的関数の値は $f(2) = 2 \times 2 = 4$ です。

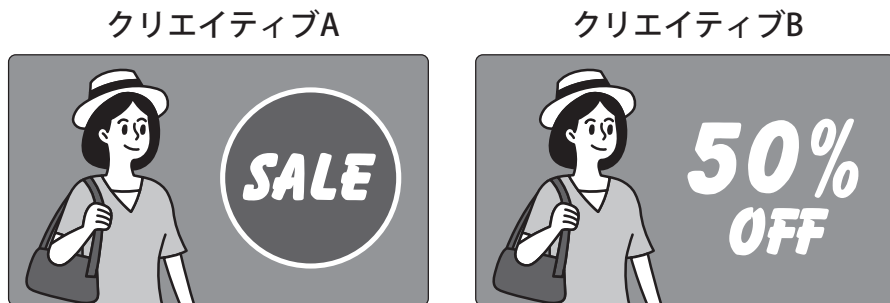
次は、もう少し現実的な例で、目的関数、制約条件、決定変数と定数を用いた数理最適化問題の定式化プロセスを見ていきましょう。

* 10 s.t. は such that または subject to の略です。

1 章 ビジネス課題を解決する技術

例：AB テスト

Web バナー広告として、クリエイティブ A^{*11}とクリエイティブ B のどちらを掲載するかを決定したいというビジネス課題を考えます（図 1.3）。



■ 図 1.3: バナー広告の AB テストの例

まずは決定変数を考えましょう。とれるアクションはクリエイティブ A かクリエイティブ B のどちらかを選ぶことなので、決定変数を $x \in \{a, b\}$ とします。

次に、目的関数、つまり最大化したい指標を考えます。どちらのクリエイティブが優れているかは、ステークホルダーとの議論の結果、バナー広告のクリック率が高いかどうかで判断することになりました。クリック率を $c(x)$ とすると、クリエイティブ A のクリック率は $c(a)$ 、クリエイティブ B のクリック率は $c(b)$ で表せます。

最後に、その他の制約条件は特にないとします。

上記の設定のもとで、このビジネス課題は以下のクリック率最大化問題として定式化できます。

$$\begin{aligned} \max_x \quad & c(x) \\ \text{s.t.} \quad & x \in \{a, b\} \end{aligned}$$

最適なアクションとしてクリック率を最大化する x^* を導出することで、最適な意思決定を行うことができます。

例：営業チームの最適人員数

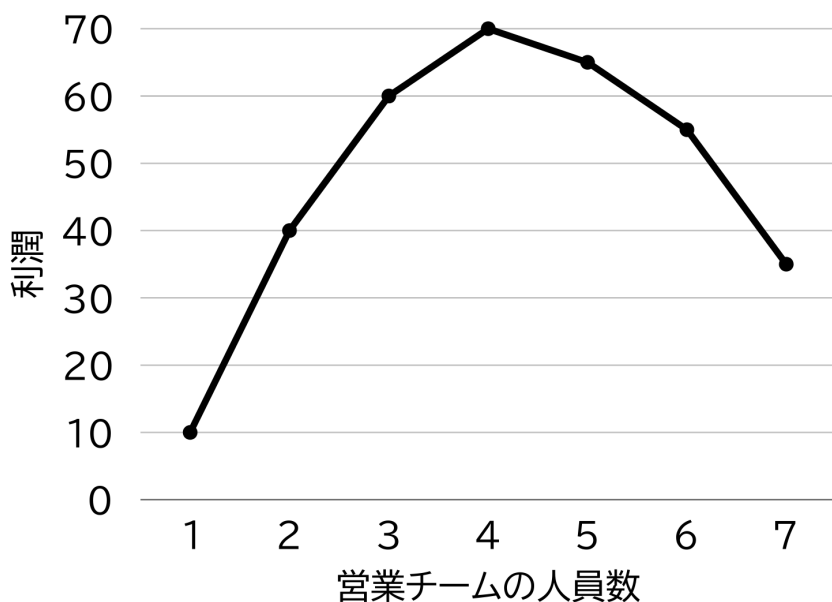
最適な営業チームの人員数を決定したいというビジネス課題を考えます。

* 11 広告業界では、広告の制作物のことをクリエイティブと呼びます。

1.3 ステップ 1：ビジネス課題を数理最適化問題として定式化する

まずは決定変数を考えましょう。とれるアクションは営業チームの採用を行うことによる人員の増加で、営業チームの人員数を決定変数 $x \geq u$ とします。採用はできても解雇はできないとして、とりうる値を現状の人員数 u 以上に限定しています。

次に、どのくらいの人員数が「最適」であるかを判断するために、最適化したい指標を決める必要があります。今回は利潤 $p(x)$ を最大化したいとします。ここで、利潤 $p(x)$ は営業の人数に対してどのくらいの利潤が得られるかを表す関数です。人員を増加させると売上は伸びますが、人件費も増加するので、利潤を最大化するには売上と費用のトレードオフを考慮する必要があります（図 1.4）。



■ 図 1.4: 人員数と利潤の関係（例）

最後に、制約条件として、営業チームの採用にかけられる費用は最大でも b であるとします。話を単純にするために、営業一人あたりの人件費は定数だとします。これを c と置きましょう。

上記の設定のもとで、このビジネス課題は、以下の利潤最大化問題として定式化できます。

1 章 ビジネス課題を解決する技術

$$\begin{aligned} \max_x \quad & p(x) \\ \text{s.t.} \quad & cx \leq b \\ & x \geq u \end{aligned}$$

最適なアクションとして利潤を最大化する人員数 x^* を導出することで、最適な意思決定を行うことができます。

例：広告出稿の最適水準

どのくらい多くの広告を出稿するのか、つまり広告の最適な出稿量を決定したいというビジネス課題を考えます。

まずは決定変数を考えます。とれるアクションは広告の出稿量を変化させることで、広告の出稿量を変数 $x \geq 0$ とします。マイナスの広告出稿はできないので、とりうる値を 0 以上に限定しています。

次に、どの出稿量が「最適」であるかを判断するために、最大化したい指標を決める必要があります。今回は製品の認知率 $a(x)$ とします。ここで、 $a(x)$ はどのくらい広告を出稿すればどのくらいの認知が得られるかを表す関数です。

最後に、制約条件として、広告にかけられる費用は最大でも b であるとしします。広告出稿 1 単位あたりの費用は定数で、これを c とします。

上記の設定のもとで、このビジネス課題は以下の認知率最大化問題として定式化できます。

$$\begin{aligned} \max_x \quad & a(x) \\ \text{s.t.} \quad & cx \leq b \\ & x \geq 0 \end{aligned}$$

最適なアクションとして認知率を最大化する x^* を導出することで、最適な意思決定を行うことができます。

1.3.2 なぜ数理最適化問題として定式化するのか

なぜビジネス課題を数理最適化問題として定式化するのでしょうか。その答えは、定式化を通じて目的、アクション、制約条件を明確に定義でき、ビジネス価値の創出に直結する意思決定を行えるからです。

1.3 ステップ 1：ビジネス課題を数理最適化問題として定式化する

まず、最適化問題の目的関数を定義することで、何をどう改善すれば「良い」状態と言えるのかを明確にすることが求められます。単に売上を伸ばすことが目的なのか、それとも利潤を最大化することなのか。顧客の離脱率を抑えるべきか、それとも新規顧客の獲得を優先すべきか。最大化または最小化したい指標を明確に定義することで、目的が具体化され、そこで初めて目的を達成するためのアクションを検討できます。

次に、決定変数を定義することで、どのような手段で目的を達成しようとしているのか、つまりアクションの候補が明確になります。製品の認知度を上げるために広告出稿量を増やすのか、それともインパクトの強いクリエイティブを制作するのか。テレビ広告だけでなく、YouTube への出稿も検討するのか。アクションの候補を具体化することで、無数の手段を漠然と考えるのではなく、有効な手段に焦点を当てて検討を進めることができます。

最後に、制約条件を定義することで、アクションを実行する上で考慮すべき現実的な制限を明確にできます。ビジネスには、予算の上限、利用可能な人員のような明示的な制約だけでなく、組織間の力関係や慣習といったさまざまな「暗黙の前提」が存在します。これらを制約条件として定義することで、実行可能な範囲内で最善の策を探ることができます。同時に、「暗黙の前提」そのものを見直し、改善するきっかけにもなります。

データ分析そのものは価値を生み出さず、分析結果に基づく意思決定とその実行があって初めてビジネス価値が生まれます。数理最適化問題としての定式化を経ずに分析を進めると、「何を目的に分析したのか」や「分析結果を受けて具体的にどう動くのか」があいまいなままになりがちです。これでは、せっかくデータ分析を行っても、実際のビジネス価値向上につながらないという事態に陥りかねません。

数理最適化問題を定式化するプロセスは、ビジネス課題「目的・アクション・制約」という構造で捉え、具体的な意思決定問題として定義する作業に他なりません。したがって、数理最適化問題の定式化は、データに基づいた合理的な意思決定を促し、ビジネス価値の創出へとつながる強力なアプローチであると言えます。

1.3.3 条件付き期待値の導入

ここまでの議論で、解くべきビジネス課題を数理最適化問題として定式化できました。しかし、この段階ではまだ最適なアクションを決定することはできません。なぜなら、データサイエンティストが直面する大半の実務課題において、アクシヨ

1章 ビジネス課題を解決する技術

ンと成果の関係 $f(x)$ は未知であり、この関係性を特定しなければ最適なアクションを求めることができないからです。

例えば、先ほどの AB テストの例では各クリエイティブのクリック率 $c(a)$ と $c(b)$ が未知のため、最適なクリエイティブを選ぶことができません。同様に、営業チームの人員数の例では利潤 $p(x)$ が未知のため、最適な人員数を選ぶことができません。したがって、定式化した数理最適化問題を解くためには、データを用いてアクションと成果の関係を推定する必要があります。

データを用いて関係性を推定する上で、考慮しなければならない重要な要素があります。それは、多くの場合、選択に対する成果は確定的には決まらず、確率的に変動するということです。例えば、AB テストの例でクリエイティブ A を表示した場合、必ずクリックされるわけでも、まったくクリックされないわけでもありません。当たり前ですが、クリックされることもあればクリックされないこともあります。これは、集団レベルではクリエイティブに興味がある人と興味がない人がいるためと考えられますし、個人レベルではその瞬間の興味や気分、時間的な余裕によってクリックされるかどうかが変わるためと考えられます。いずれにせよ、選択に対する成果はあたかも確率的に決まっているかのようにデータ上は観測されます。

では、このように成果が確率的に変動する場合、どうやって「より良い」アクションを評価すればよいのでしょうか？ 毎回結果が異なるため、一度の結果だけで判断するのは適切ではありません。このような状況では、平均的にどれくらいの成果が期待できるか、つまり成果の平均値（期待値）に注目するのが一般的なアプローチになります。より良い成果をもたらす可能性の高いアクションを評価するためです。実際、AB テストの例において、それぞれのクリエイティブの「クリック率」を比較するのは、まさにこの「平均」に注目するアプローチです。

ある選択肢を選んだときの平均的な成果を数式で表現するために、**条件付き期待値**を用います。アクションを x で表し、確率的に決まる成果を確率変数 Y で表すとします。このとき、アクション x を選んだときの成果 Y の期待値は、条件付き期待値 $\mathbb{E}[Y \mid x]$ で表現できます。条件付き期待値を用いることで、最適化問題は以下のように定式化できます。

$$\begin{aligned} \max_x \quad & \mathbb{E}[Y \mid x] \\ \text{s.t.} \quad & x \in \mathcal{X} \end{aligned}$$

上記の条件付き期待値を用いた定式化について、前述の例を用いてイメージをつかみましょう。

1.3 ステップ 1：ビジネス課題を数理最適化問題として定式化する

例：AB テスト

AB テストの例を再度考えます。成果であるクリックされたかどうかを確率変数 $C \in \{0, 1\}$ で表します。ここで、確率変数 C は 1 か 0 のどちらかの値をとり、 $C = 1$ はクリックされたことを、 $C = 0$ はクリックされなかったことを表すします。このとき、選択 x に対するクリックの条件付き期待値は $\mathbb{E}[C | x]$ で表現できます。なお、1 か 0 をとる確率変数の期待値は、その確率変数が 1 をとる確率と等しいため、条件付き期待値 $\mathbb{E}[C | x]$ は、 x を選んだ場合のクリックされる確率として $\Pr(C = 1 | x)$ と表現できます^{*12}。

まとめると、クリック率最大化問題は条件付き期待値を用いて以下のように表現できます。

$$\begin{aligned} \max_x \quad & \mathbb{E}[C | x] = \Pr(C = 1 | x) \\ \text{s.t.} \quad & x \in \{a, b\} \end{aligned}$$

条件付き期待値 $\mathbb{E}[C | x] = \Pr(C = 1 | x)$ をステップ 2 でデータから推定し、ステップ 3 で最適化問題を解くことで、最適な意思決定を導くことができます。

例：営業チームの最適人員数

営業チームの最適人員数の例を再度考えます。成果である利潤を確率変数 P で表します。このとき、人員数 x に対する利潤の条件付き期待値は $\mathbb{E}[P | x]$ で表現できます。利潤最大化問題は条件付き期待値を用いて以下のように表現できます。

$$\begin{aligned} \max_x \quad & \mathbb{E}[P | x] \\ \text{s.t.} \quad & cx \leq b \\ & x \geq u \end{aligned}$$

条件付き期待値 $\mathbb{E}[P | x]$ をステップ 2 でデータから推定し、ステップ 3 で最適化問題を解くことで、最適な意思決定を導くことができます。

例：広告出稿の最適水準

広告出稿の例を再度考えます。成果である認知されたかどうかを確率変数 $A \in \{0, 1\}$ で表します。このとき、選択 x に対する認知率の条件付き期待値は

* 12 確率変数が 1 か 0 のみをとる場合、期待値の定義から

$$\mathbb{E}[C | x] = 1 \cdot \Pr(C = 1 | x) + 0 \cdot \Pr(C = 0 | x) = \Pr(C = 1 | x)$$

となり、期待値と確率変数が 1 をとる確率は等しくなります。本書ではこの性質を繰り返し利用します。

1 章 ビジネス課題を解決する技術

$\mathbb{E}[A \mid x]$ で表現できます。AB テストの例と同様に、条件付き期待値 $\mathbb{E}[A \mid x]$ は、 x を選んだ場合の認知される確率として $\Pr(A = 1 \mid x)$ とも表現できます。

製品認知率最大化問題は条件付き期待値を用いて以下のように表現できます。

$$\begin{aligned} \max_x \quad & \mathbb{E}[A \mid x] = \Pr(A = 1 \mid x) \\ \text{s.t.} \quad & c \times x \leq b \\ & x \geq 0 \end{aligned}$$

認知の条件付き期待値 $\mathbb{E}[A \mid x] = \Pr(A = 1 \mid x)$ をステップ 2 でデータから推定し、ステップ 3 で最適化問題を解くことで、最適な意思決定を導くことができます。

1.3.4 なぜ条件付き期待値を利用するのか

前節では、成果が確率的に決まる状況で数理最適化問題の定式化に条件付き期待値を利用しましたが、なぜ条件付き期待値を利用するのでしょうか。条件付き期待値である必然性はあるのでしょうか。

結論を言うと、条件付き期待値である必要はありません。例えば、平均的な成果を高めるだけでなく成果のばらつきも抑えたいのであれば、期待値だけでなく分散も考慮する、(特に下振れをなくしたいのであれば) 期待値ではなく 10%点などの条件付き分位点を考慮するなど、条件付き期待値以外の指標を用いることも考えられます。

ただし、成果が確率的に変動する状況では、成果の平均値に焦点を当てることが多く、また条件付き期待値は数学的な取り回しが相対的に容易なので、(少なくとも初期の段階では) 条件付き期待値を用いて定式化を行うのが実践的だと筆者は考えています。

1.4 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

1.4

ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ステップ 1 では、ビジネス課題を数理最適化問題として定式化しました。特に、成果が確率的に決まる状況を取り上げ、条件付き期待値を用いて平均的な成果を最大化する問題として落とし込みました。

定式化された最適化問題を解くためには、アクションと成果の関係を明らかにする必要があります。そこで、ステップ 2 では、アクションと成果の関係、つまり条件付き期待値をデータから推定します。ステップ 2 を行うことで、実際に最適化問題を解くステップ 3 に進むことができます。

条件付き期待値をデータから推定する難易度は、設定した問題と保有しているデータによって大きく異なります。一般に、アクションと成果の関係が間接的になるほど、条件付き期待値を推定する難易度は高くなります。

1.4.1 単純集計

まず、最も単純な方法は、アクションごとに成果の平均値を計算することです。これを単純集計と呼びます。例えば、AB テストの例では、ユーザに対してクリエイティブをランダムに出し分けて、クリックされたかどうかを直接確認することができるので、条件付き期待値を単純集計で推定できます。

例：AB テスト

クリック率最大化問題

$$\begin{aligned} \max_x \quad & \mathbb{E}[C \mid x] = \Pr(C = 1 \mid x) \\ \text{s.t.} \quad & x \in \{a, b\} \end{aligned}$$

を解くために、条件付き期待値 $\mathbb{E}[C \mid x] = \Pr(C = 1 \mid x)$ をデータから推定します。

推定には単純集計を用います。具体的には、AB テストを行い、クリエイティブ A とクリエイティブ B のクリック率を計算します。各クリエイティブをランダムにそれぞれ n 人のユーザに出し分け、各クリエイティブごとにクリックされた人数 (n_a, n_b) を測定することで、単純集計を行うことができます。

$$\widehat{\mathbb{E}}[C \mid a] = \widehat{\Pr}(C = 1 \mid a) = \frac{n_a}{n}$$

1 章 ビジネス課題を解決する技術

$$\widehat{\mathbb{E}}[C \mid b] = \widehat{\Pr}(C = 1 \mid b) = \frac{n_b}{n}$$

これが単純集計による条件付き期待値の推定です。

1.4.2 数理モデルの導入

AB テストの例では、とりうるアクションが少数で、アクションと成果の関係が直接的であり、各アクションに対する成果のデータを得られる状況でした。このような条件が揃っていれば、単純にアクションを実行した結果を集計し、その平均値を求めることで、条件付き期待値を推定できます。

しかし、現実のビジネス課題は、AB テストのような単純な状況だけとは限りません。例えば、以下のような状況では、すべてのアクションについて十分にデータを収集することは困難であり、単純集計以外の方法を検討する必要があります。

- とりうるアクションが多数ある、または連続的である（例：広告予算の配分）
- アクションと成果の関係が複雑で、直接は見えづらい（例：売上、利潤）

このような状況で力を発揮するのが、アクションと成果の関係性を数理モデルで表現すること、つまり、条件付き期待値の関数形を具体的に特定するアプローチです。関数形を仮定する利点は、たとえ手元にあるデータが限られていても、その仮定されたパターンに沿って、データがない点の成果を推測できる点にあります。これにより、データが不足の問題を補いながら条件付き期待値を推定することが可能となります。

例として、最も単純な数理モデルの一つである線形回帰モデルを見てみましょう。これは、アクションと成果の関係を以下のような直線的な関係を仮定するモデルです。

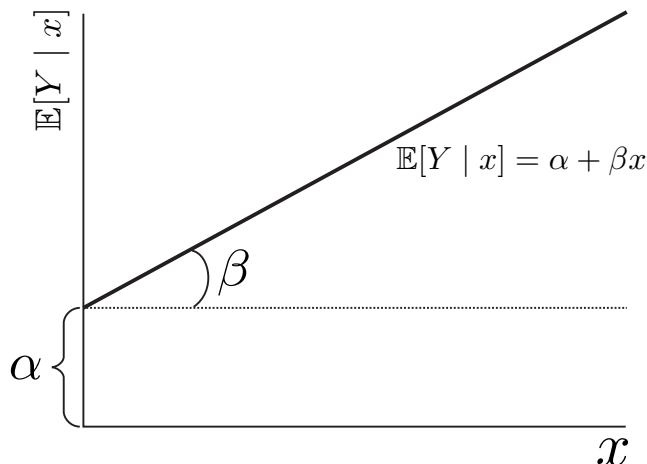
$$\mathbb{E}[Y \mid x] = \alpha + \beta x$$

このモデルは、横軸に x 、縦軸に条件付き期待値をとると、 α は切片、 β は直線の傾きを表していると解釈できます (図 1.5)。

このモデルにおいて、未知のパラメータは (α, β) の二つだけです。よって、理論上は、たった二つのデータポイントがあれば、パラメータを推定し、データを観測していない領域に対しても予測を行うことができます^{*13}。

* 13 もちろん、これはあくまで理論上の最低限の話で、データポイントは 2 点より多いに越したことはありません。データポイントが多いほど推定の精度が上がります。

1.4 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 1.5: 線形回帰モデル

この線形回帰モデルの例が示すように、一般に手元にあるデータだけでは情報が不足している場合は、**データ生成過程**^{*14}に対する「仮定」を分析に導入することで、観測されたデータそのものからは得ることのできない情報を補うアプローチが有効です。線形モデルの例は、「アクションと成果の関係性は線形になっているはずだ（少なくとも近似できるはずだ）」という分析者の信念を反映することで、データにはない情報を付与していると考えられます。

ただし、このアプローチを用いる際には注意が必要です。仮定が現実と大きく異なっていた場合は、誤った仮定に基づいて行われた条件付き期待値の推定もまた、現実からかけ離れた誤まったものになってしまうリスクがあります。したがって、数理モデルを用いる際には、以下の点に気を付けてください。

- 分析に利用できるデータの質と量に応じて仮定の強弱を調整する。例えば、データが少なければシンプルな線形回帰モデル、データが豊富ならより複雑な関係性を学習できる非線形モデル、のように使い分ける
- データ生成過程に対するドメイン知識を駆使する。対象となるビジネスや現象に対する深い理解に基づき、できるだけ現実を反映した、妥当性の高い数理モデルを構築する

* 14 データ生成過程（Data Generating Process, DGP）とは、観測されるデータがどのような仕組みや法則に基づいて生み出されているかを示す、分析対象の現象の背後に存在するメカニズムのことを指します。

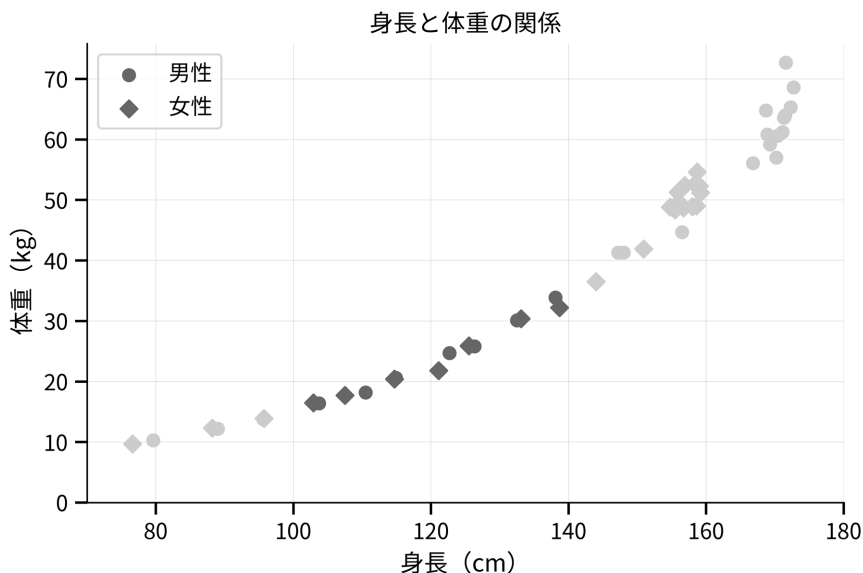
1 章 ビジネス課題を解決する技術

データ生成過程に対するドメイン知識を駆使して数理モデルを考えることが重要だと述べましたが、実際にどのようにしてドメイン知識を数理モデルの構築に活かすのでしょうか？ 具体的なイメージをつかむために、簡単な例を見てみましょう。

例：身長から体重を予測する

この具体例では、身長から体重を予測するという簡単な例を通じて、特にデータが限られた範囲にしかない場合に、ドメイン知識を使ってより妥当な数理モデルを構築するプロセスと効果を見ていきます^{*15}。

まずは、分析に使うデータを見てみましょう。図 1.6 は、身長と体重の関係を示した散布図です。これは、2019 年度の「国民健康・栄養調査」^{*16}の公開データを元に作成したもので、性別・年齢層ごとの身長と体重の平均値^{*17}がプロットされています。このデータを使って、身長 x から体重 Y を予測するモデル、すなわち条件付き期待値 $\mathbb{E}[Y | x]$ を推定するモデルを構築することを目指します。



■ 図 1.6: 身長と体重の関係

* 15 同様の例に対する分析は McElreath (2020) にも記載があります。

* 16 <https://www.e-stat.go.jp/dbview?sid=0003224177>

* 17 一つひとつの点は、性別ごとに 26 歳 -29 歳、30 歳 -39 歳、40 歳 -49 歳、50 歳 -59 歳、60 歳 -69 歳、70 歳以上の身長と体重の平均値です。

1.4 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

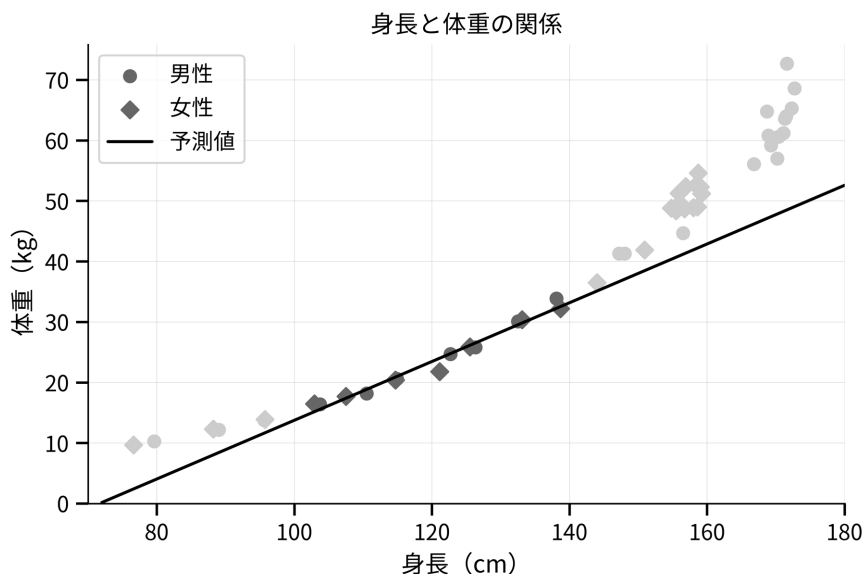
ここで、あえて次のような問題設定を考えます。手元にあるのは身長が 100cm から 140cm の人に関するデータ（図の濃いグレーの点）のみであり、この限られたデータを使って、範囲外である身長 100cm 以下や 140cm 以上の人の体重（薄いグレーの点）も予測したい、という状況です。つまり、パラメータを推定するための学習データが存在しない領域への予測、いわゆる外挿（extrapolation）が必要となります。

データが存在しない領域を予測する外挿では、データがない部分の関係性がどうなっているかを「仮定」する必要がある、分析者のドメイン知識に基づいて数理モデルを構築することが不可欠となります。散布図を見ると、身長が高いほど体重も重くなる傾向があり、その関係は非線形性があることが見てとれます。

まず、最も単純なモデルとして、身長 x と体重 Y を以下のような線形関数でモデリングしてみましょう。

$$\mathbb{E}[Y | x] = \alpha + \beta x$$

このモデルで学習と予測を行った結果が図 1.7 です。



■ 図 1.7: 線形モデルによる予測結果

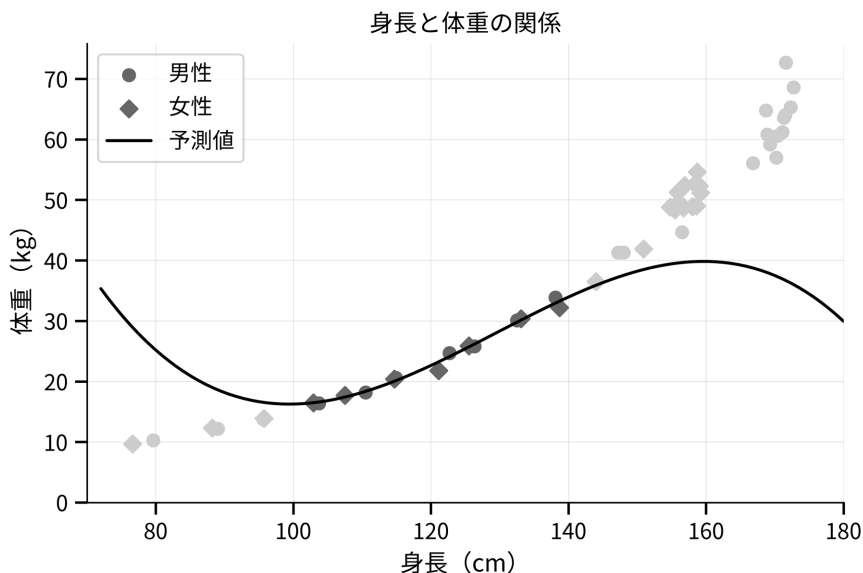
1章 ビジネス課題を解決する技術

図 1.7 を見ると、線形モデルでは、学習データが示す非線形な関係をうまく捉えられていないことがわかります。これは、「関係は線形である」という仮定が妥当ではなかったために、予測結果に大きな誤差が生じていると考えられます。

そこで、身長と体重の非線形な関係を表現するために、条件付き期待値を以下のような 3 乗項まで含めた関数形でモデリングしてみましょう。

$$E[Y | x] = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

先ほどの線形モデルとは違い、2 乗項 $\beta_2 x^2$ と 3 乗項 $\beta_3 x^3$ を追加したので、このモデルは身長と体重の非線形な関係を表現することができます。このモデルで学習と予測を行った結果は図 1.8 です。

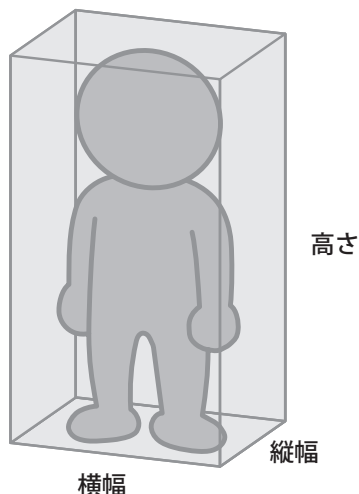


■ 図 1.8: 3 乗項までを含めたモデルの予測結果

図 1.8 を見ると、学習データが存在する領域（濃いグレーの点）では、データによくフィットし、非線形性をうまく捉えて身長から体重を予測できているように見えます。しかし、学習データが存在しない領域（薄いグレーの点）に目を向けると、問題が生じています。身長が低くなるほど体重が軽くなる、身長が高くなるほど体重が重くなるといった、直感に反する不自然な予測をしてしまっています。

このように、データに柔軟にフィットする複雑なモデルであっても、データがない領域での予測（外挿）が妥当になるとは限りません。ここで重要になるのが、デー

1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 1.9: 人間の体積を直方で近似

タ生成過程、つまり「身長と体重の関係が物理的にどのように決まるのか」に対するドメイン知識を活用することです。「現象の背後にあるメカニズムはこうなっているはずだ」という知識や考察を、数理モデルの関数形に反映させるを試みます。

まず、人の体の密度が大まかに一定だと考えれば、体重はその人の体積に比例すると考えるのが自然です。もちろん、同じ体積でも頭は他の部位より重いかもしれませんが、部位によって重さが違う可能性はいったん無視して、密度は均一だと仮定しましょう。

そうすると、次は身長から体積を求める方法を考える必要があります。大雑把に考えると、人間の体積は四角い箱、つまり直方で近似できそうです（図 1.9）。直方の体積は「縦幅×横幅×高さ」で求められます。

「高さ」は身長そのものです。さらに、「縦幅」と「横幅」は身長に比例するとみなせそうです。すると、人間の体積は

$$\text{体積} \approx \underbrace{x}_{\text{身長}} \cdot \underbrace{\alpha x}_{\text{縦幅}} \cdot \underbrace{\beta x}_{\text{横幅}} = \alpha\beta x^3$$

で近似できると考えられます。ここで、 α は身長を「縦幅」に、 β は身長を「横幅」に変換する係数です。

体重は体積に比例すると仮定したので、その係数を γ とすると、人間の重さは

1 章 ビジネス課題を解決する技術

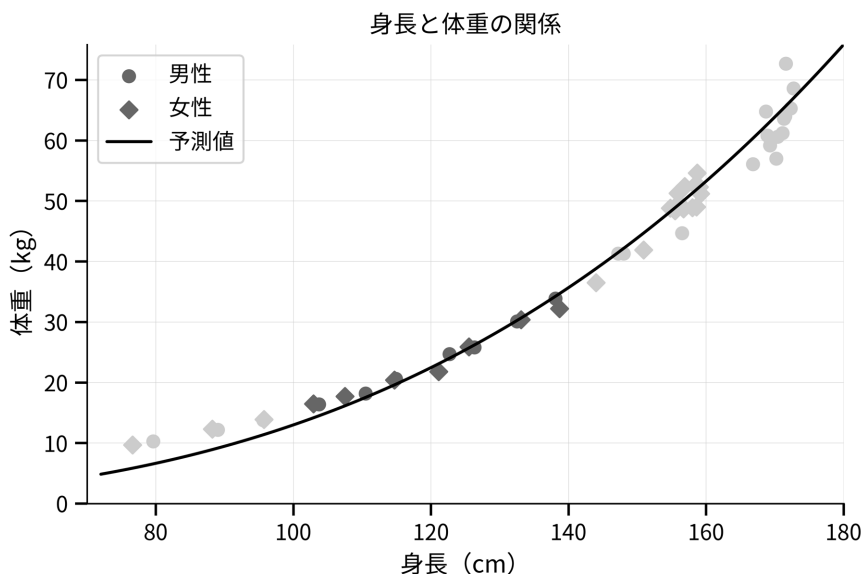
$$\text{体重} \approx \gamma \text{体積} \approx \gamma \alpha \beta x^3$$

で近似できます。つまり、体重は身長³に比例すると考えていることになります。身長から体重を予測する上では、パラメータ $\gamma\alpha\beta$ を分けて識別する必要はないので、まとめて θ とします。よって、身長で条件付けた体重の条件付き期待値を

$$\mathbb{E}[Y | x] = \theta x^3$$

という関数形でモデリングします。

この数理モデルを用いた予測結果が図 1.10 です。



■ 図 1.10: 3 乗項「のみ」モデルの予測結果

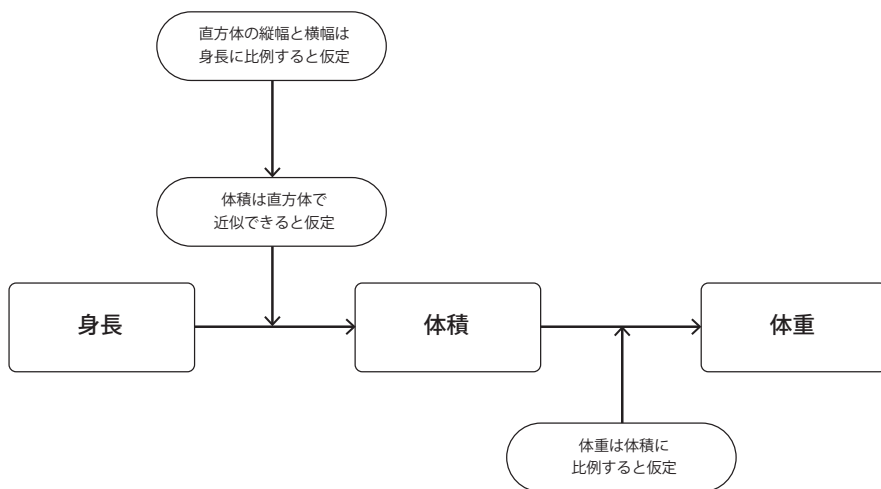
注目すべきはデータの（薄いグレーの点）です。ここまでのモデルと比較すると、多少のズレがあるとはいえ、データが存在しない領域でも直感的で妥当な予測ができていることがわかります。

これが、データ生成過程に対するドメイン知識を数理モデリングに反映させることの効果です。ドメイン知識に基づいて、関係性の本質を捉えたシンプルな仮定をモデルの関数形に組み込むことで、特にデータが存在しない領域において、より信頼性の高い予測を行うことができました。具体的には、ドメイン知識から

1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

- 人間の体重は体積に比例する
- 人間の体積は直方体で近似できる
- 人間の縦幅と横幅は身長に比例する

という三つの仮定を導き、数理モデルに反映しました（図 1.11）。結果として、 $\mathbb{E}[Y | x] = \theta x^3$ というパラメータが一つのシンプルなモデルを構築し、データの存在しない領域においても高い予測性能を発揮することができました。



■ 図 1.11: 体重から身長を予測するモデル

もちろん、今回構築した数理モデルはあくまで単純化された近似です。人間の体重と身長の関係性はより複雑なメカニズムで決まっていると考えられます。実際には、体の密度は均一ではありませんし、縦幅と横幅も身長に比例するとは限りません。さらに予測精度を高めるためには、年齢、性別、人種のような他の要因も含めて、数理モデルを構築していくことが求められるでしょう。

例：営業チームの最適人員数

ドメイン知識を数理モデルに反映するイメージをつかむため、さらに営業チームの最適人員数の例を見ていきましょう。この例では、ドメイン知識を数理モデルに反映することで、解くべき問題を簡単化できることを示します。

利潤最大化問題

1 章 ビジネス課題を解決する技術

$$\begin{aligned} \max_x \quad & \mathbb{E}[P \mid x] \\ \text{s.t.} \quad & cx \leq b \\ & x \geq u \end{aligned}$$

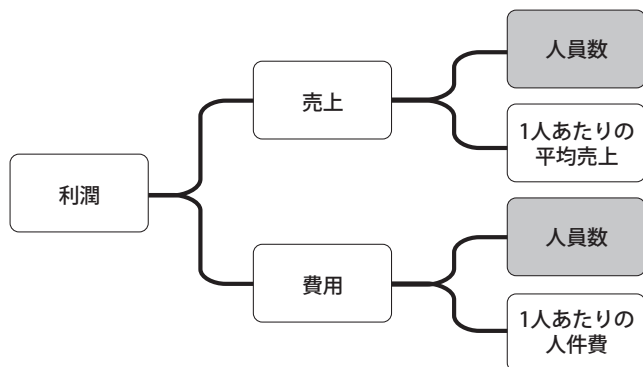
を解くためには、条件付き期待値 $\mathbb{E}[P \mid x]$ をデータから推定する必要があります。

その際、営業チームの人員数とその成果として現れる利潤の関係を表すデータが十分に揃わないことが問題になります。AB テストの例とは違い、人員を一人ずつ増やして利潤の増分をみていくような実験を行うことは現実的ではありません。実験が行えない以上は、過去のデータから人員と利潤の関係性を把握する必要がありますが、単純集計では過去の前例のない人員数の場合の利潤を推定することができません（外挿ができません）。また、利潤は売上から費用を引いたものです。人員数が売上と費用の二つの要素に影響することも話を複雑にしています。

そこで、利潤の条件付き期待値を直接データから推定しようとするのではなく、まずはドメイン知識を利用して問題を分解することを目指します。問題を分解する際は、図 1.12 のような KPI ツリーを思考の補助に用いることが有効です。

KPI ツリーとは、最終的な目標を達成するために、その目標を構成する要素を分解し、具体的な行動レベルの指標まで落とし込んで、樹形図で可視化したものです。ここでは、最終的な目標は利潤、具体的な行動指標としては人員数を想定しています。人員数は直接的に操作できる変数なので、他の要素と区別するためにグレーで色付けしています。

まず、「利潤は売上 R から費用 C を引いたものである」というドメイン知識（定義）を利用して、問題を少し簡単にします。



■ 図 1.12: 営業チームの人員数と利潤の関係を分解

1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

$$\begin{aligned}\mathbb{E}[P \mid x] &= \mathbb{E}[R - C \mid x] \\ &= \mathbb{E}[R \mid x] - \mathbb{E}[C \mid x] \\ &= \mathbb{E}[R \mid x] - cx\end{aligned}$$

ここで、2行目の式変形には期待値の線形性^{*18}を、3行目の式変形には一人あたりの人件費は c で固定されているという問題設定を利用しています。

この変形で、利潤の条件付き期待値を推定する問題は、営業チームの人員数 x に対する売上の条件付き期待値 $\mathbb{E}[R \mid x]$ を推定する問題に変換されました。利潤についてのドメイン知識を明示的にモデルに組み込んだことで、利潤を直接推定するのではなく、その前段階の売上を推定する問題に簡単化することができました。売上を推定する問題は、人員から費用を推定する問題が切り離された分、利潤を推定する問題よりも難易度が下がっています。

それでは、売上の条件付き期待値 $\mathbb{E}[R \mid x]$ はどのように推定すればよいでしょうか。単純な方法は、売上を「一人あたりの平均売上 $(\bar{r}) \times$ 人員数 (x) 」に分解し、人員数が増えても一人あたりの平均売上は変わらないと仮定することです。この場合、過去の営業チームの人員数とそのときの売上のデータから、一人あたりの平均売上 $\bar{r}$ を推定できます。

このとき、利潤の条件付き期待値は

$$\hat{\mathbb{E}}[P \mid x] = \bar{r}x - cx$$

で推定できます。

もしかすると、「人員数を増やしても一人あたりの売上が変わらない」という仮定は現実とはそぐわないかもしれません。その場合は、人員数と売上の関係に線形ではない関係を仮定することもできます。例えば、人員数が増えると売上が増えるが、徐々に人員増加の売上への影響は小さくなっていく、という関係を、以下のような関数形で表現することもできます。

$$\mathbb{E}[R \mid x] = \alpha x^\beta$$

ここで、売上は正の数字になるように $\alpha > 0$ を、人員数が増加にすると売上は増加するが、その増加は徐々に小さくなるように $0 < \beta < 1$ を仮定します。推定すべきパラメータは (α, β) の二つなので、最低限、過去の営業チームの人員数とそのときの売上のデータが2点あれば、この条件付き期待値を推定することができます。

^{* 18} a と b を定数、 X と Y を確率変数とします。このとき、 $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ が成り立ちます。この「確率変数の和の期待値は、確率変数の期待値の和になる」という性質は期待値の線形性と呼ばれています。

1 章 ビジネス課題を解決する技術

この場合、推定された人員数と売上との関係を使って、利潤の条件付き期待値は以下で推定できます。

$$\hat{\mathbb{E}}[P \mid x] = \hat{\alpha}x^{\hat{\beta}} - cx$$

例：広告出稿の最適水準

さらに、広告出稿の最適水準についても、ドメイン知識を数理モデルに反映させることで、より現実に即したモデルを構築することができます^{*19}。

製品認知率最大化問題

$$\begin{aligned} \max_x \quad & \mathbb{E}[A \mid x] = \Pr(A = 1 \mid x) \\ \text{s.t.} \quad & cx \leq b \\ & x \geq 0 \end{aligned}$$

を解くためには、条件付き期待値 $\mathbb{E}[A \mid x] = \Pr(A = 1 \mid x)$ をデータから推定する必要があります。

例えば、広告の出稿量と製品認知率の関係に対して、「広告を出稿するほど製品認知率が上がるが、その関係は線形ではなく、増加率は徐々に小さくなる」というドメイン知識をもっているとします。このような関係性を表す一例として、以下のような対数を使ったモデリングが考えられます。

$$\Pr(A = 1 \mid x) = \alpha + \beta \log(x + 1)$$

ここで、 $\log$ は自然対数を意味します。 $\log(x)$ だと出稿量が 0 のとき負の無限大になってしまうので、 $\log(x + 1)$ としています。このとき、パラメータ α は出稿量が 0 のときの製品認知率（つまり、広告とは関係なく、そもそも製品を知っていた人の割合）、 β は出稿量が増えるとどれだけ製品認知率が増加するかを表しています。

ただし、この対数を使ったモデリングは、広告出稿量 x を増加させていくと製品認知率の予測値がいずれ 100% を超えてしまうという問題があります。この問題を解決するために、確率の予測値が 0 から 1 の間に収まるように以下のようなロジスティック関数を使ったモデリングを考えることもできます。

* 19 この例は、他の例よりも複雑なモデリングが要求されます。2 章で広告出稿に関するドメイン知識を、3 章で同様の認知に関するモデリングを詳細に取り上げるので、この段階ではすべてを理解する必要はありません。

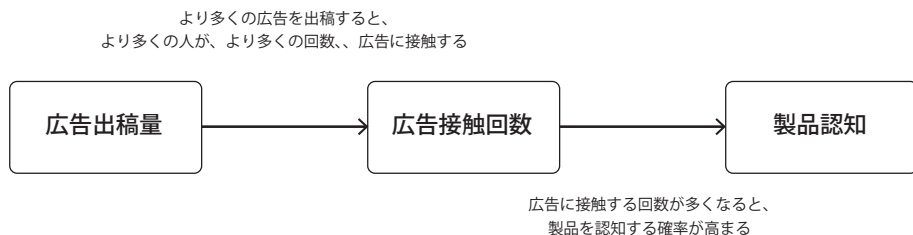
1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

$$\Pr(A = 1 | x) = \text{logit}^{-1}(\alpha + \beta x) = \frac{1}{1 + e^{-(\alpha + \beta x)}}$$

対数モデルやロジスティック関数は、広く用いられている標準的なモデリング手法であり、広告効果が通減する傾向や、予測される確率値が0から1の間に収まるといった、観測される現象のパターンを捉える上で有効です。なので、これらを用いること自体に問題があるわけではありません。しかし、これらの関数形は、「なぜ」広告出稿が製品認知につながるのか、その背後にあるメカニズムを明示的に表現しているわけではありません。よって、分析者がもっているドメイン知識を直接モデルの構造に反映できれば、より現実に即した、なぜ広告出稿が製品認知につながるのかを説明しやすい数理モデルを構築できる可能性があります。

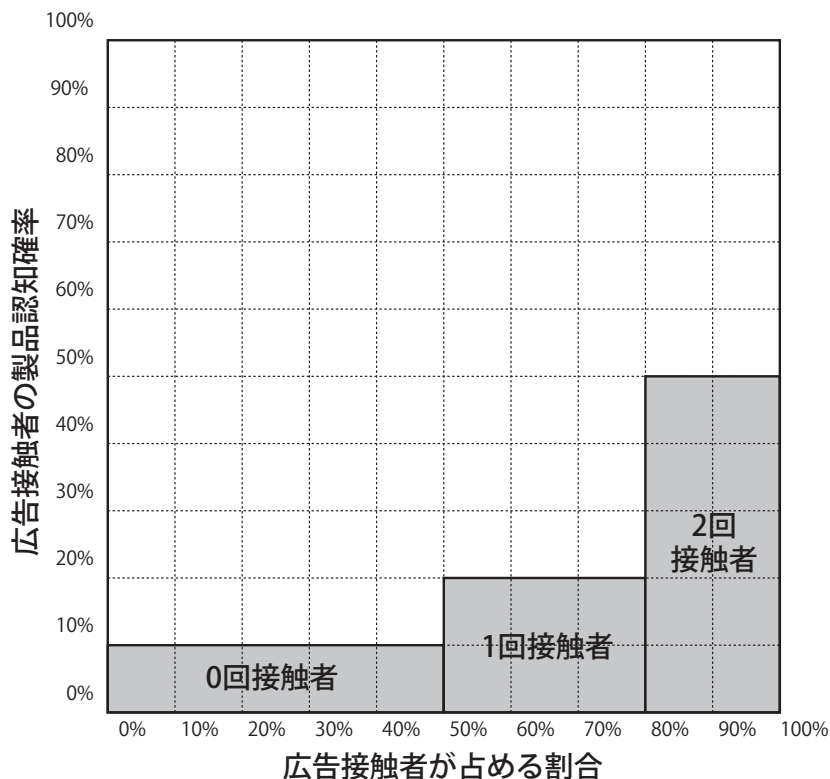
そのようなドメイン知識を反映させた数理モデルを構築するために、広告出稿量と製品認知率の関係をあらためて考えてみましょう。広告出稿はどのようなプロセスを経て認知率に影響を与えるのでしょうか。自然な発想として、「広告出稿を増やすと広告を見る人数／見る回数が増え、広告を見る人数／見る回数が増えたと製品を認知する人が増える」といった2段階のプロセスがあると考えることができそうです。この仮説を数理モデルに落とし込みましょう。「広告出稿量→製品認知」といった直接的な関係を考えるのではなく、間に広告接触回数を挟んで「広告出稿量→広告接触回数→製品認知」という2段階のモデルを考えます(図1.13)。

最初から一般的な状態を考えるのは難しいので、話を簡単にするために、広告接触回数が0回、1回、2回の3パターンしかない状況を考えましょう。広告接触回数が0回の人(広告に接触していない人)が集団全体に占める割合が50%、広告接触回数が1回の人(広告に接触している人)の割合が30%、広告接触回数が2回の人(広告に接触している人)の割合が20%だとします。そして、広告接触回数が0回の人(広告に接触していない人)が製品を認知する確率が10%、広告接触回数が1回の人(広告に接触している人)が製品を認知する確率が30%、広告接触回数が2回の人(広告に接触している人)が製品を認知する確率が50%だとします(図1.14)。



■ 図 1.13: 製品認知のモデリング例

1章 ビジネス課題を解決する技術



■ 図 1.14: 広告接触者の割合と製品認知確率

集団全体の製品認知率は、広告接触回数ごとの製品認知率を、広告接触回数の分布で重みづけて平均することで計算できます。

$$\begin{aligned}
 \text{集団全体の製品認知率} &= \text{0回接触者の割合} \times \text{0回接触者の製品認知確率} \\
 &\quad + \text{1回接触者の割合} \times \text{1回接触者の製品認知確率} \\
 &\quad + \text{2回接触者の割合} \times \text{2回接触者の製品認知確率}
 \end{aligned}$$

よって、数値例における集団全体の製品認知率は、

$$0.5 \times 0.1 + 0.3 \times 0.3 + 0.2 \times 0.5 = 0.29$$

より、29%となります。

上記の単純な例で得られた知見を一般化しましょう。広告接触回数を確率変数 $F \in \{0, 1, 2, \dots\}$ とします。すると、製品認知率は以下で表すことができます。

1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

$$\begin{aligned}
 \Pr(A = 1 \mid x) &= \Pr(A = 1 \mid F = 0) \Pr(F = 0 \mid x) \\
 &\quad + \Pr(A = 1 \mid F = 1) \Pr(F = 1 \mid x) \\
 &\quad + \Pr(A = 1 \mid F = 2) \Pr(F = 2 \mid x) \\
 &\quad + \dots \\
 &= \sum_{f=0}^{\infty} \Pr(A = 1 \mid f) \Pr(F = f \mid x)
 \end{aligned}$$

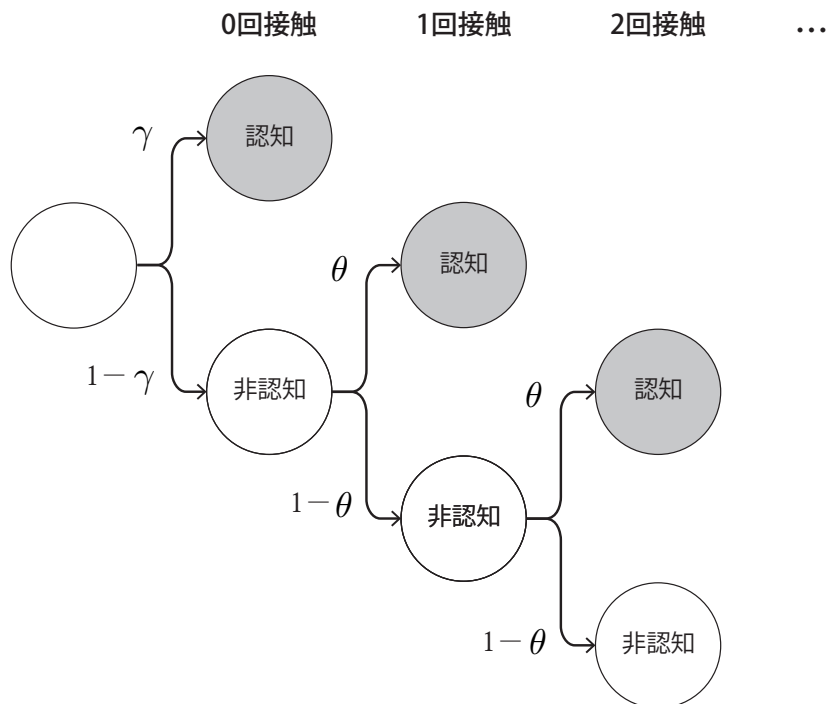
まず、第一項の $\Pr(A = 1 \mid f)$ は広告接触回数 f で条件付けた製品を認知する確率です。どのくらい広告に接触すればどのくらいの確率で製品を認知するのかを表しています。次に、第二項の $\Pr(F = f \mid x)$ は広告出稿量 x で条件付けた、広告接触回数が f 回になる確率です。つまり、広告を x だけ出稿した際に、広告に f 回接触する人がどのくらいの割合で出現するのかを表しています。このように、広告接触回数 F を間に挟むことで、広告出稿量 x で条件付けた製品を認知する確率を、二つのパートに分解できます。

さて、これまでの議論により、広告出稿量から製品認知率を予測する問題を、広告出稿量から広告接触回数（の分布）を予測する問題 ($\Pr(A = 1 \mid f)$) と、広告接触回数から製品認知率を予測する問題 ($\Pr(F = f \mid x)$) に分解できました。このように問題を分解するメリットは、より単純で扱いやすい個別の関係性に焦点を当て、それぞれのメカニズムを深く考察できる点にあります。これにより、より現実 に即した、解釈性の高いモデル構築が可能となります。

それでは、分解された個別の問題について考えていきましょう。まずは、広告接触回数から製品認知率を予測する問題、つまり第一項の $\Pr(A = 1 \mid f)$ の推定について考えます。直感的には、広告接触回数 f が増えるほど製品を認知する確率が上がるが、その関係は線形ではなく、増加率は徐々に小さくなると予想されます。この仮説に即したモデルとして、広告出稿量と製品認知率の関係を直接モデリングしたときと同様に、対数やロジスティック関数を使ったモデルも考えられます。ただし、広告接触回数と製品認知率の間に「対数」や「ロジスティック関数」の関係性があるという必然性はありません。そこで、より説得力のあるモデルを目指し、ドメイン知識に基づいて「広告接触がどのように認知につながるか」というメカニズム自体をモデルに反映させることを試みます。具体的に、図 1.15 のような認知メカニズムを考えてみましょう。

このメカニズムは、二つのパラメータ (γ, θ) をもっています。

1章 ビジネス課題を解決する技術



■ 図 1.15: 広告接触と製品認知のメカニズム

まず、「広告接触とは無関係に製品を認知している確率」を $\gamma \in [0, 1]$ とします。製品が元々有名な場合、広告に接触しなくても製品を認知している確率が高いと考えられるため、 γ は高い値になるでしょう。一方で、新規の製品であれば、 γ は低い値になります。広告に接触していない人（0 回接触の人）が製品を認知している確率が γ なので、逆に製品を認知していない確率は $1 - \gamma$ です。

次に、「広告に接触した際に製品を認知する確率」を $\theta \in [0, 1]$ とします。 θ は「広告接触が製品を認知させる力」を表していると解釈できます。製品を強烈に印象付ける広告であれば θ は大きな値をとり、逆に印象に残らないよくある広告であれば θ は小さな値をとります。

この設定のもとで、まずは広告に 1 回だけ接触した場合の製品認知率を考えましょう。この場合の製品を認知する確率は、「そもそも製品を知っていた、または、1 回の広告接触によって製品を認知した」確率です。この確率は、「そもそも製品を知らない、かつ、1 回の広告接触によって製品を認知しなかった」確率を 100%か

1.4 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

ら引くことで計算できます*20。また、「そもそも製品を知らない」という事象と、「1回の広告接触によって製品を認知しなかった」という事象が独立であると仮定すると、「そもそも製品を知らない、かつ、1回の広告接触によって製品を認知しなかった」確率は「そもそも製品を知らない」確率と「1回の広告接触によって製品を認知しなかった」確率の積になります*21。よって、

$$\Pr(A = 1 \mid F = 1) = 1 - (1 - \gamma)(1 - \theta)$$

となります。

次に、広告に2回だけ接触した場合を考えましょう。広告に1回だけ接触した場合と同様に考えると、製品を認知する確率は、「そもそも製品を知らない」確率と「1回目の広告接触で製品を認知しなかった」確率と、「2回目の広告接触でも製品を認知しなかった」確率の積を100%から引けば求まります。つまり、

$$\Pr(A = 1 \mid F = f) = 1 - (1 - \gamma)(1 - \theta)^f$$

です。

これを一般化して広告に f 回接触した場合を考えると、製品認知率を以下の数式で表現できます。

$$\Pr(A = 1 \mid f) = 1 - (1 - \gamma)(1 - \theta)^f$$

このモデルは、対数やロジスティック関数のような「よくある」関数形を使ったモデルとは異なり、認知に至るプロセスに関する具体的なメカニズムが数理モデルに反映されています。メカニズムをモデルに反映させることによって、「広告なしでの認知率は100 γ %」、「広告1回接触あたりの新規認知率は100 θ %」といったように、パラメータが具体的な意味をもつため、モデルの解釈性や納得性が格段に高まっています。

ここまで、広告出稿量から製品認知率を予測する問題を、広告出稿量から広告接触回数（の分布）を予測する問題と、広告接触回数から製品認知率を予測する問題に分解し、分解した問題について、ドメイン知識に基づいた数理モデルを構築して

* 20 離散確率変数 (X, Y) に対して、

$$\Pr(X = x \vee Y = y) = 1 - \Pr(X \neq x \wedge Y \neq y)$$

が成り立つことを利用しています。ここで、 $\vee$ は「または」を、 $\wedge$ は「かつ」を意味します。

* 21 離散確率変数 (X, Y) に対して、 X と Y が独立とは、

$$\Pr(X = x \wedge Y = y) = \Pr(X = x) \Pr(Y = y)$$

であることを意味します。

1 章 ビジネス課題を解決する技術

きました。この数理モデルをどうやってデータから推定するのか、そして分解した問題の残り半分である「広告出稿量で条件付けた広告接触回数が f 回になる確率」の数理モデルをどのように構築するのかという二つの問題が残されていますが、イントロダクションの具体例としては長くなってしまったので、これらの詳細は以降の章で詳しく解説していくことにします。

1.5

ステップ3：数理最適化問題を解いて最適なアクションを導出する

ステップ3：数理最適化問題を解いて最適なアクションを導出する

ステップ2では、ステップ1で定式化した数理最適化問題で未知の部分だったアクションと成果の関係に対して数理モデルを構築し、データから推定する方法を検討しました。ステップ3では、アクションと成果の関係性が明らかになった最適化問題を解いて、最適なアクションを導出しましょう。

例：ABテスト

クリック率最大化問題は、クリエイティブAとクリエイティブBのクリック率をデータから推定し、最適なクリエイティブを選ぶ問題でした。

$$\max_{x \in \{a, b\}} \widehat{\mathbb{E}}[C | x] = \widehat{\Pr}(C = 1 | x)$$

この問題は、選択肢が二つだけなので、クリエイティブAとクリエイティブBのクリック率を推定し、クリック率が高いクリエイティブを選べば解決します。例えば、推定の結果 $\widehat{\mathbb{E}}[C | a] = 0.1$ 、 $\widehat{\mathbb{E}}[C | b] = 0.2$ が得られた場合、クリック率の高いクリエイティブBを採用します^{*22}。

例：営業チームの最適人員数

営業チームの最適人員数を知るためには、利潤最大化問題

$$\begin{aligned} \max_x \quad & \widehat{\mathbb{E}}[P | x] \\ \text{s.t.} \quad & cx \leq b \end{aligned}$$

* 22 この例ではクリエイティブBのクリック率が10%高いと推定されたことをもってクリエイティブBを採用しました。もちろん、この差が「統計的に有意かどうか」を検証することもできますが、本書では統計的有意性についてはふれません。統計的有意性や仮説検定について学びたい読者は久保川(2017)を参照してください。また、ABテストについて深く学びたい読者はKohavi et al. (2021)をご確認ください。

1.5 ステップ 3：数理最適化問題を解いて最適なアクションを導出する

$$x \geq u$$

を解く必要がありました。

人員数が増減しても一人あたりの平均売上 $\bar{r}$ が変わらないと仮定した場合、つまり、人員数と売上に線形の関係を保定した場合、最適化問題は以下になります。

$$\begin{aligned} \max_x \quad & \bar{r}x - cx \\ \text{s.t.} \quad & cx \leq b \\ & x \geq u \end{aligned}$$

この最適化問題を解くことで、最適な人員数 x^* を求めることができます。最適解としては、以下の 3 パターンが考えられます。

- $\bar{r} > c$ の場合
 - 人員数を増やすことで増える売上が採用費用を上回るので、予算が許す限り採用で人員を増やす。よって、 $x^* = b/c$ とする（人員数は整数なので、小数部分は切り捨てる）
- $\bar{r} < c$ の場合
 - 人員数を増やすことで増える売上よりも採用費用のほうが高く、利潤が減少する。よって、採用は行わない（ $x^* = u$ とする）
- $\bar{r} = c$ の場合
 - 人員数を増やすことによる売上の増加と採用費用の増加がちょうど等しいため、採用はしてもしなくてもかまわない（ $u \leq x^* \leq b/c$ とする）

このように、人員数と売上の関係に線形の関係を保定した場合、最適な人員数は「まったく採用しない」か「予算上限まで採用する」のどちらかになります（またはどちらでもよい）。

このような極端な状況はあまり現実的とは言えないので、もう一つの定式化である $\mathbb{E}[P \mid x] = \alpha x^\beta - cx$ を用いた場合の最適化問題を考えてみましょう。

$$\begin{aligned} \max_x \quad & \hat{\alpha}x^{\hat{\beta}} - cx \\ \text{s.t.} \quad & cx \leq b \\ & x \geq u \end{aligned}$$

1章 ビジネス課題を解決する技術

人員数 x は人数なので実際には整数ですが、簡単のため以降では実数として扱います。まず、人員数を一人追加したときの利潤の変化量は、目的関数を x で微分して求めることができます。

$$\frac{d}{dx} (\hat{\alpha}x^{\hat{\beta}} - cx) = \hat{\alpha}\hat{\beta}x^{\hat{\beta}-1} - c$$

これが0より大きいうちは、人員数を増やすことで利潤が増加するので、人員数を増やすことが正解です。一方で、これが0より小さい場合は、人員数を減らすことで利潤が増加するので、人員数を減らすことが正解です。「人員数が増加すると売上は増加するが、その増加は徐々に小さくなる」と仮定していたことも合わせて考えると、最適な人員数は、 $\hat{\alpha}\hat{\beta}x^{\hat{\beta}-1} - c = 0$ を満たす x になることがわかります。よって、式変形から

$$x^* = \left(\frac{c}{\hat{\alpha}\hat{\beta}} \right)^{\frac{1}{\hat{\beta}-1}}$$

が最適な人員数とわかります。

ただし、人員数には既存人員数と予算上限から $u \leq x \leq b/c$ という制約があります。よって、もし x^* が下限より小さい場合は $x^* = u$ が、上限より大きい場合は $x^* = b/c$ がとりうる中での最適解であることに注意してください。

今回は最適解を解析的に求めましたが、より複雑な問題では数値計算による解法を用いることが一般的です。数値計算による解法については、2章で詳しく説明します。

例：広告出稿の最適水準

製品認知率最大化問題は、広告出稿量から広告接触回数（の分布）を予測する問題と、広告接触回数から製品認知率を予測する問題に分解することができました。

$$\begin{aligned} \max_x \quad & \mathbb{E}[A \mid x] = \sum_{f=0}^{\infty} \widehat{\Pr}(A = 1 \mid f) \widehat{\Pr}(F = f \mid x) \\ \text{s.t.} \quad & cx \leq b \\ & x \geq 0 \end{aligned}$$

第一項の広告接触回数と製品認知率の関係は以下のようにモデル化しましたが、第二項はまだモデル化していませんでした。

$$\widehat{\Pr}(A = 1 \mid f) = 1 - (1 - \hat{\gamma}) \left(1 - \hat{\theta} \right)^f$$

なので、ここではこの最適化問題を解くことはできません。この具体例の最適解に関しては、次章以降で詳しく議論することにします。

1.6 本章のまとめ

本章では、データサイエンスを用いてビジネス課題を解決する統一的な方法として、以下の3ステップフレームワークを提案し、具体例を通じて理解を深めました。

ステップ1 ビジネス課題を数理最適化問題として定式化する

ステップ2 数理モデルを構築し、未知のパラメータをデータから推定する

ステップ3 数理最適化問題を解いて最適なアクションを導出する

まず、ステップ1では、ビジネス課題を数理最適化問題として定式化することで、データサイエンスの問題に落とし込みます。次に、ステップ2では、アクションと成果の関係性に対して数理モデルを構築し、モデルのパラメータをデータから推定することで、数理最適化問題を解けるようにします。最後に、ステップ3では、アクションと成果の関係性が明らかになった数理最適化問題を解くことで、最適なアクションを導きます。

本章では、簡単な具体例に対して3ステップフレームワークを適用しましたが、このフレームワークはより現実的なビジネス課題にも適用することができます。次章以降では、より複雑なビジネス課題に対して3ステップフレームワークを適用し、データサイエンスを用いてビジネス課題を解決する方法を学んでいきます。

2章

マーケティングに データサイエンスを導入する

2章以降では、3ステップフレームワークを用いて筆者が実務で直面した実際のビジネス課題に取り組んでいきます。2章では、ビジネス課題として、実際に筆者が直面している「テレビCMの出稿量の最適化」を取り上げます。1章で紹介した3ステップフレームワークを適用し、あいまいなビジネス課題を具体的なデータサイエンスの問題に落とし込み、データを用いて最適なアクションの提案を行います。

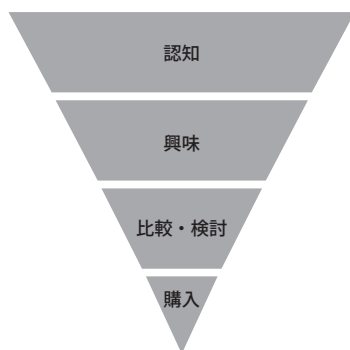
2.1 マーケティング活動としてのテレビCM

本書では、ビジネス課題の具体的な例として、テレビCMを用いたマーケティング活動を取り上げます。ビジネス課題をデータサイエンスの問題に落とし込むためには、対象分野の実務的な知識が必要不可欠です。対象分野の実務的な知識があるからこそ、課題解決に有効な数理最適化や数理モデルを構築できます。

そこで本節では、マーケティング活動としてのテレビCMに関して、本書の内容を理解するために必要最低限な範囲に絞って解説します。ここですべてを理解する必要はありません。本書を読み進める中で不明な点が出てきた際に、あらためて本節を参照するようにしてください。

2.1.1 マーケティングにおけるテレビCMの役割

マーケティングにおけるテレビCMの役割を考えるために、まずは消費者の購買心理プロセスについて説明します。一般的には、消費者はある製品やサービスをまったく知らない状態から最終的に購入や利用に至るまで、いくつかの心理的なステップを経ると考えます。図2.1はマーケティングファネルと呼ばれる購買心理プロセスのモデルです。製品（またはサービス）を「認知」し、それに「興味」をもち、他の選択肢と「比較・検討」を行い、最終的に「購入」に至るという一連の流れを示しています*1。

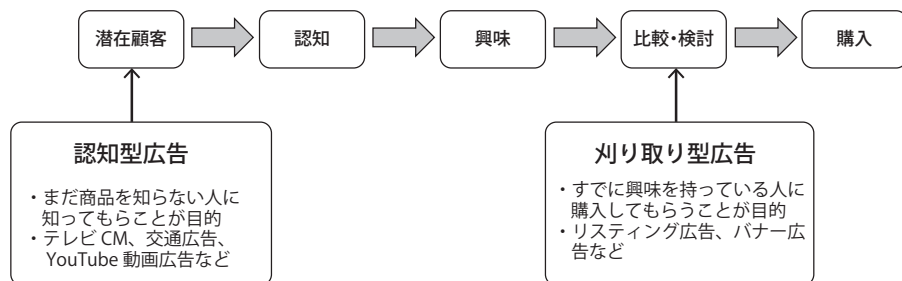


■ 図 2.1: マーケティングファネル

* 1 このプロセスはあくまで一例です。他に有名なモデルとしては、AIDA (Attention, Interest, Desire, Action) モデルが挙げられます。

2.1 マーケティング活動としてのテレビ CM

では、このマーケティングファネルの中で、テレビ CM はどのような役割を果たすのでしょうか？ 広告は、マーケティングファネルのどの段階をターゲットにするかに応じて、「刈り取り型広告」と「認知型広告」に分類されます（2.2）。



■ 図 2.2: 広告の分類

「刈り取り型広告」は、購入に近い段階である「比較・検討」段階の顧客にアプローチする広告です。製品を買うかどうか迷っている人や、すでに興味をもっている人の背中を押して、「購入してもらう」ことを目指します。例えば、インターネット検索の結果に表示される広告（リスティング広告）や、ウェブサイト上のバナー広告などがこれにあたります。

一方で、「認知型広告」は、まだ製品のことを知らない潜在顧客に「知ってもらう」ことを目的としています。幅広い層にアプローチして、製品の存在を知らせるのが役割です。テレビ CMをはじめ、新聞や雑誌の広告、街中の交通広告、YouTubeなどの動画広告がこのタイプに含まれます。

テレビ CM は、特に「認知型広告」として重要な役割を果たします。なぜなら、テレビは多くの家庭に普及しており^{*2}、一度にたくさんの潜在顧客に情報を届ける力があるからです。「まずは広く知ってもらう」という目的において、テレビ CM は効果的な手段の一つと言えるでしょう。

このように、マーケティング活動におけるテレビ CM の主な役割は、製品やサービスの認知を高めること、そのためにより多くの潜在顧客に広告を届けることです。この役割がどの程度果たされているかを評価するためには、「認知の量」や「広告が届いた量」を測定する必要があります。次節では、そのための具体的な指標について詳しく見ていきましょう。

* 2 消費動向調査の令和 7（2025）年 3 月分調査によると、世帯のカラーテレビの普及率は 90.3%です。
<https://www.esri.cao.go.jp/jp/stat/shouhi/shouhi.html>

2 章 マーケティングにデータサイエンスを導入する

2.1.2 広告指標

認知の指標

前述のように、テレビ CM の大きな役割は、より多くの認知を獲得することです。認知の広さ・強さを表す指標として広く使われているものに助成想起と純粋想起があります。

助成想起は、ブランド・製品・サービスの名前や画像、ブランドのロゴ、CM などを提示した上で、それを知っているかを確認します。例えば、「この CM を見たことがありますか」という質問をして、「見たことがある」と答えた人の割合を測定する指標です。

一方で、純粋想起は、助成想起のような手がかりを提示せずに、そのブランド・製品・サービスを知っているかを確認します。例えば、「あなたが知っているシャンプーの製品名をすべて挙げてください」という質問をして、各製品名が挙げられた割合を測定する指標です。回答者は手がかりなしで製品名を挙げる必要があるため、純粋想起された製品は助成想起よりも強く記憶に残っていると考えられます。

なお、助成想起のことを「認知」と呼び、純粋想起のことを単に「想起」と呼ぶこともあります。本書では、認知を測定する指標として助成想起を用います。よって、以降で「認知」という言葉が出てきた場合、それは助成想起のことを指します。

広告接触の指標

より多くの認知を獲得するためには、「より多くの人に」「より多くの回数」CM を見てもらうことが重要です。この「広告が届いた量」を測る指標として、**グロスリーチ**、**ユニークリーチ**、**平均フリークエンシー**という三つの指標があります。

まず、グロスリーチは「CM への延べ接触人数」を指します。「延べ」という言葉が示すように、同じ人が複数回 CM に接触した場合は、その回数分だけ接触としてカウントします。

一方、ユニークリーチは「一度でも CM に接触した人数」です。同じ人が複数回 CM に接触しても、2 回目以降の接触はカウントしません。ユニークリーチは単に「リーチ」と呼ばれることもあります。

最後に、平均フリークエンシーは「一度でも CM に接触した人における、CM 接触回数の平均値」です。この値は、定義からグロスリーチをユニークリーチで割ることで求められます。なお、CM 接触回数のことをフリークエンシーと呼ぶこともあります。本書では、これら二つの用語を同じ意味で用います。

2.1 マーケティング活動としてのテレビ CM

これらの指標について、具体的な例を通して理解を深めましょう（表 2.1）。この表は、5 人の集団に対して CM を 5 回放送した際に、グロスリーチ、ユニークリーチ、平均フリークエンシーがそれぞれどのように計算されるかを示しています。なお、デジタル広告とは異なり、テレビ CM では 1 回の放送で複数人が接触しうる点に注意してください。

▼ 表 2.1: グロスリーチとユニークリーチの計算例

CM 放送回	1 回目 放送	2 回目 放送	3 回目 放送	4 回目 放送	5 回目 放送
A さん	1	0	1	0	1
B さん	0	1	0	1	0
C さん	1	1	1	0	1
D さん	0	0	1	0	0
E さん	0	0	0	0	0
グロスリーチ（累積）	2 人 (40%)	4 人 (80%)	7 人 (140%)	8 人 (160%)	10 人 (200%)
ユニークリーチ（累積）	2 人 (40%)	3 人 (60%)	4 人 (80%)	4 人 (80%)	4 人 (80%)
平均フリークエンシー （累積）	1 回	1.33 回	1.75 回	2 回	2.5 回

CM 放送回ごとに、順を追って見ていきましょう。まず、1 回目の放送では A さんと C さんの 2 人が CM に接触しました。この時点では、グロスリーチもユニークリーチも 2 人となります。5 人中 2 人が接触したため、割合で表すとどちらも 40%です。接触者はそれぞれ 1 回ずつの接触なので、平均フリークエンシーは 1 回です。

次に、2 回目の放送では B さんと C さんが接触しました。B さんはこれが初めての接触ですが、C さんにとっては 2 回目の接触です。したがって、C さんの接触はグロスリーチにはカウントされますが、ユニークリーチにはカウントされません。結果として、2 回目の放送終了時点での累積値は、グロスリーチが 4 人 (80%)、ユニークリーチが 3 人 (60%) となります（両指標とも、1 回目放送からの累積である点に注意してください）。このように、グロスリーチは同一人物の接触を毎回カウントしますが、ユニークリーチは最初の 1 回しかカウントしない点が異なります。

2 章 マーケティングにデータサイエンスを導入する

す。また、平均フリークエンシーは、グロスリーチをユニークリーチで割ることで、 $4/3 \approx 1.33$ 回と計算できます。

3 回目以降の放送についても、同様の手順で計算します。なお、グロスリーチとユニークリーチは人数で示す場合と割合で示す場合があります。ここまでは説明のため両方を併記してきましたが、以降は基本的に割合で表記することにします*3。

ここまで、広告接触を測る指標としてグロスリーチ、ユニークリーチ、平均フリークエンシーを紹介しました。このうち、広告認知の獲得を目的とする場合、特に重視されるのはユニークリーチです。なぜなら、同じ人に何度も CM を見てもらうよりも、一度でも多くの人に CM を見てもらう方が、認知を広げる上で効果的だと考えられているためです。そのため、本書ではユニークリーチに焦点を当てて分析を進めていきます。

2.1.3 指標の測定

テレビ CM の結果を確認する指標として、認知、想起、グロスリーチ、ユニークリーチ、平均フリークエンシーを紹介しました。それでは、これらの指標を実際に測定したデータはどのように入手すればよいのでしょうか？

例えば、自社製品のテレビ CM を日本全国で放送した場合を考えてみましょう。理想的には、日本のすべての人に CM を見たかどうかや製品を知っているかを確認したいところですが、全員に調査を行うのは時間とコストがかかりすぎるため現実的ではありません。そこで、一般的には、日本全体（母集団）を代表するようにランダムに選ばれた人々（標本）に調査を行い、その結果から全体の状況を推測するという手法がとられます。

このような標本調査のデータは、マーケティングリサーチ会社に依頼することで入手できます。

まず、認知度や想起率といった指標を知るためには、株式会社マクロミル*4や株式会社クロス・マーケティング*5のようなリサーチ会社にアンケート調査を依頼するのが一般的です。ただし、アンケート調査には時間とコストがかかるため、すべての製品で頻繁に行うことは難しく、キャンペーンの前後や月次・四半期ごとなど、予算に応じて重要なタイミングを選んで実施されることが多いです。

* 3 テレビ CM においては、割合で表したグロスリーチを GRP や TRP と呼びます。テレビ CM に関する概念をすべて紹介するのは本書の内容を超えるので詳細は割愛しますが、GRP は世帯単位の CM に接触した量を表す指標、TRP は個人単位の CM に接触した量を表す指標です。

* 4 <https://www.macromill.com/>

* 5 <https://www.cross-m.co.jp/>

2.1 マーケティング活動としてのテレビ CM

次に、グロスリーチやユニークリーチといった指標については、テレビ視聴データに特化したリサーチ会社に依頼してデータを取得するのが一般的です。例えば、株式会社ビデオリサーチ^{*6}や、筆者が所属する REVISIO 株式会社などが、テレビ視聴データを専門とするリサーチ会社にあたります。本書では REVISIO のデータを用いて分析を進めるため、ここで REVISIO のデータ取得方法について紹介します。

REVISIO では、関東地方にお住まいの約 4,500 人を対象として、テレビの視聴行動データを収集・蓄積しています。調査への協力に同意いただいたご家庭のテレビに人体認識センサーを設置し、その技術を用いて「誰が」「いつ」テレビを見ているかを自動で計測し、データ化しています（図 2.3）。



■ 図 2.3: REVISIO のデータ取得技術

この視聴データと、放送された CM の情報、そして調査参加者のデモグラフィック情報（年齢、性別など）を組み合わせることで、「どのような人が、どのテレビ CM に、どれだけ接触したか」という詳細な情報をデータとして蓄積しています。

テレビ視聴データの取得は人体認識センサーによって自動的に行われるため、特定の CM に限らず、地上波で放送されたすべてのテレビ CM について、個々の放送回ごとに「誰が接触したか」という詳細なデータを取得することが可能です。ただし、このような詳細なデータの購入には相応のコストがかかるため、実際にはキャンペーン終了時点の集計データなど、より限定的な形で利用されるケースもあります。

* 6 <https://www.videor.co.jp/>

2 章 マーケティングにデータサイエンスを導入する

本書では、このようにして REVISIO が測定・提供しているグロスリーチやユニークリーチのデータを用いて、テレビ CM の効果分析を進めていきます*7。

2.1.4 テレビ CM の出稿形態

テレビ CM を放送するには、まず CM を放送するための放送枠を購入する必要があります。放送枠を購入する方法として、大きく分けて**タイム**と**スポット**の2種類があります。本章から4章まではスポットを、5章ではタイムを扱います。以下では、タイムとスポットのそれぞれについて、基本的な仕組みを解説します。

タイム

タイムとは、放送してほしいテレビ番組を具体的に指定して契約する方式です*8。

⋮	放送局A	放送局B	放送局C	放送局D	放送局E
16 時	ドラマA			ニュースC	バラエティE
			ニュースB		
17 時	バラエティB	ニュースA		バラエティD	
					ニュースD
18 時	バラエティC		アニメA	ドラマB	
⋮					

■ 図 2.4: 番組表の例

図 2.4 のような番組表を参考に、多数の番組候補の中から CM を放送したい番組の放送枠を購入します。番組の放送枠を購入すると、契約期間中（通常は半年間）、毎週その番組内で CM が放送されます（図 2.5）。

* 7 REVISIO が提供するデータについてより詳しく知りたい方は、公式ページ <https://revisio.co.jp/> をご確認ください

* 8 より詳細な解説は、放送局や広告会社の解説記事、例えば <https://www.tv-asahi.co.jp/ex/sales/column/time/> などを参照してください。

2.1 マーケティング活動としてのテレビ CM

番組Bの放送枠を購入すると、半年間、番組Bで毎週CMが放送される



■ 図 2.5: タイムの例

例えば、日本テレビの月曜 21 時台の番組『しゃべくり 007』^{*9}で CM を放送したい場合、その番組の放送枠を購入すれば、契約期間中は毎週『しゃべくり 007』の放送中に CM が流れます。

放送枠の価格は、基本的に**視聴率**^{*10} に応じて変動し、視聴率が高い番組ほど価格も高くなる傾向があります。

タイムで放送枠を購入した企業は、その番組のスポンサーとして「提供」クレジットが表示されます。テレビ番組の冒頭に「この番組は、〇〇（企業名）の提供でお送りします」といったアナウンスが流れるのを聞いたことがある方も多いでしょう。この提供クレジットを見れば、どの企業がその番組のタイム枠を購入しているかがわかります。次にテレビを見る機会があれば、ぜひ提供表示にも注目してみてください。

タイムのメリットとデメリットを以下にまとめます。

- メリット
 - 番組の視聴者層に合わせたターゲティングを行いやすい
 - 提供クレジットによりスポンサーとして認知され、企業イメージや信頼性の向上につながりやすい

* 9 執筆時点（2025 年 5 月）の情報です。

* 10 視聴率はその番組を視聴した世帯または個人の割合で、株式会社ビデオリサーチが測定した視聴率が主に利用されています。本書では割合で表示したグロスリーチが対応します。詳細は以下を参照してください。<https://www.videor.co.jp/digestplus/article/tv240926.html>

2 章 マーケティングにデータサイエンスを導入する

- 繰り返し同じ番組で CM が放送されるため、番組イメージとブランドイメージを結び付けやすい
- デメリット
 - スポットと比較して費用が高額になりやすい
 - 契約期間が長い（通常は半年間）
 - 人気の番組の放送枠は、すでに他のスポンサーで埋まっていることがある

スポット

スポットはタイムとは異なり、特定の番組を指定するのではなく、放送局^{*11}と放送時間帯のパターンを指定して契約する方式です。放送時間帯は細かく指定できず、図 2.6 に示すような、あらかじめ定められたパターンの中から、ターゲットとする視聴者が多く含まれると想定されるものを選択します。例えば、「逆 L」を選択すると、平日の夜と深夜、土日の全日で CM が放送されます。

	全日							ヨの字							コの字							逆L						
	月	火	水	木	金	土	日	月	火	水	木	金	土	日	月	火	水	木	金	土	日	月	火	水	木	金	土	日
朝																												
午前																												
昼																												
午後																												
夕方																												
夜																												
深夜																												

■ 図 2.6: スポットの代表的な出稿パターン

放送局と時間帯パターンを決めたら、次に CM を出稿する「量」を決定します。スポットの契約では、通常、「CM を 500 回放送する」といった放送回数ではなく、「延べ 500%分の視聴者に CM を届ける」というように、目標とするグロスリーチの総量に基づいて契約します。

この契約方式では、グロスリーチ 1%あたりの単価があらかじめ決められていることが一般的です^{*12}。仮にグロスリーチ 1%あたりの単価が 10 万円の場合、「総額 5,000 万円でグロスリーチ 500%分の CM を出稿する」といった契約になります。本書では、このグロスリーチ 1%あたりの単価（例：10 万円）を**パーコスト**、契約

* 11 主に、日本テレビ、テレビ朝日、TBS、テレビ東京、フジテレビの 5 局とその系列局から指定します。

* 12 実際には、出稿金額が一定水準を超える、他の放送局よりも多く予算を割り振るなどの条件で割引が適用されることがあります。議論が複雑になるので、本書では割引は考慮しないことにします。

2.1 マーケティング活動としてのテレビ CM

の総額（例：5,000 万円）を**出稿金額**、目標とするグロスリーチの総量（例：500%）を**出稿量**と呼びます。なお、放送時間帯を限定する（例えば、特定の曜日や時間帯に絞る）パターンほど、パーコストは高くなる傾向があります。したがって、一般的には「全日」→「ヨの字」→「コの字」→「逆 L」の順にパーコストが高くなります。

スポットのメリットとデメリットを以下にまとめます。

- メリット
 - タイムと比較して費用を抑えやすく、予算に応じた柔軟な出稿計画を立てやすい
 - 短期間からの出稿が可能で、特定のキャンペーン期間などに合わせた機動的な展開ができる
 - 多様な時間帯に放送されるため、より幅広い視聴者層にリーチできる可能性がある
- デメリット
 - 特定の番組を指定できないため、タイムほど精密なターゲティングは難しい
 - 提供クレジットが表示されないため、タイムほどの企業名認知効果は期待しにくい

2.1.5 スポットにおけるグロスリーチとユニークリーチの関係

本節では、スポットを例にテレビ CM の分析を進めるため、スポットにおけるグロスリーチとユニークリーチの関係を詳しく見ていきましょう。

前述したように、パーコストが固定されている場合、

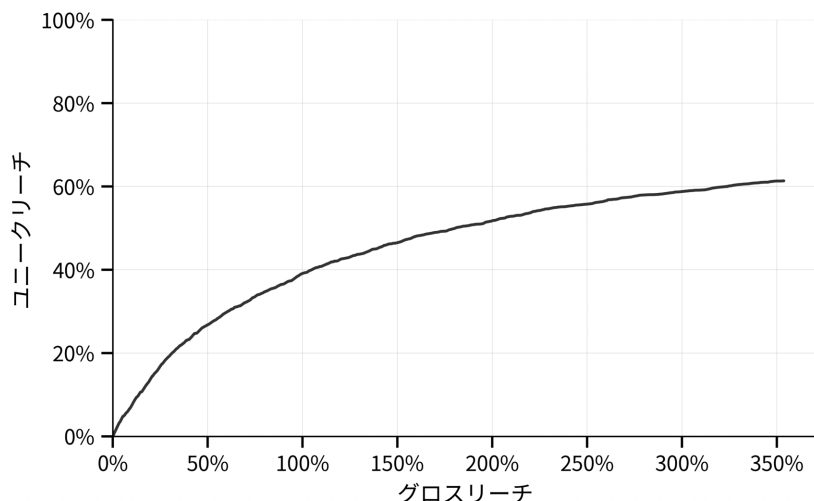
$$\text{出稿金額} = \text{パーコスト} \times \text{出稿量}$$

であるため、出稿量と出稿金額は一对一で対応します。つまり、CM の出稿金額が決まると、同時に出稿量（グロスリーチ）も決まる関係にあります。そのため、本書ではグロスリーチを操作対象（数理最適化における決定変数）、ユニークリーチをその結果（数理最適化における目的関数）として扱います。なお、広告出稿金額を決定変数とすることもできますが、本書ではユニークリーチとの関連性が明白な CM 出稿量（グロスリーチ）を決定変数として考えることにします。

2 章 マーケティングにデータサイエンスを導入する

グロスリーチを通じてユニークリーチをコントロールするには、両者の関係性を理解する必要があります。図 2.7 は、グロスリーチとユニークリーチの関係性を示しています。横軸にグロスリーチ、縦軸にユニークリーチがとられています。

一般的に、グロスリーチとユニークリーチの間には、図 2.7 に示すような非線形の関係が見られます*13。この曲線を**リーチカーブ**と呼ぶことにします。



■ 図 2.7: リーチカーブ

この図からは、以下の点が読み取れます。

- グロスリーチが 0% のときは、ユニークリーチも 0%
- グロスリーチが増加しても、ユニークリーチは 100% を超えることはない
- グロスリーチが増加するにつれて、ユニークリーチの伸びは徐々に鈍化する

このように、図 2.7 のようなグロスリーチとユニークリーチの関係性がわかっていれば、「広告予算が決まっている場合に獲得できるユニークリーチ」を予測したり、「目標とするユニークリーチを達成するために最低限必要な予算」を決定する最適化（意思決定）が可能になります。

* 13 繰り返しになりますが、横軸のグロスリーチをパーコスト倍したものが出稿金額なので、出稿金額とユニークリーチにも同様の関係がみられます。

2.1 マーケティング活動としてのテレビ CM

まず、予測について考えてみましょう。仮に、グロスリーチ 1%あたりのコストが 10 万円だとしましょう。予算が 1,000 万円の場合、グロスリーチ 100%分の CM 出稿が可能です。図 2.7 から、これにより約 40%のユニークリーチを獲得できることがわかります。

次に、最適化について考えてみましょう。調査の結果、CM 接触者の 80%が製品を認知したと仮定します（簡単のため、CM 非接触者は誰も製品を認知しないものとしします）。この場合、32%の製品認知を得るには、 $0.32/0.8 = 0.4$ 、つまり 40%のユニークリーチが必要となります。そして、図 2.7 から、40%のユニークリーチを獲得するには約 100%のグロスリーチが必要だとわかります。

このように、グロスリーチとユニークリーチの関係性が明らかであれば、それに基づいてさまざまな予測や意思決定を行うことが可能です。

2.1.6 実務における課題

前述したように、グロスリーチとユニークリーチの関係が既知であれば、予測や意思決定に活用できます。例えば、過去にテレビ CM 出稿を実施しており、そのテレビ CM に対してすべての放送回に対するグロスリーチとユニークリーチのデータが入手できれば、そこからリーチカーブを可視化し、前述のような意思決定を行うことが可能です。

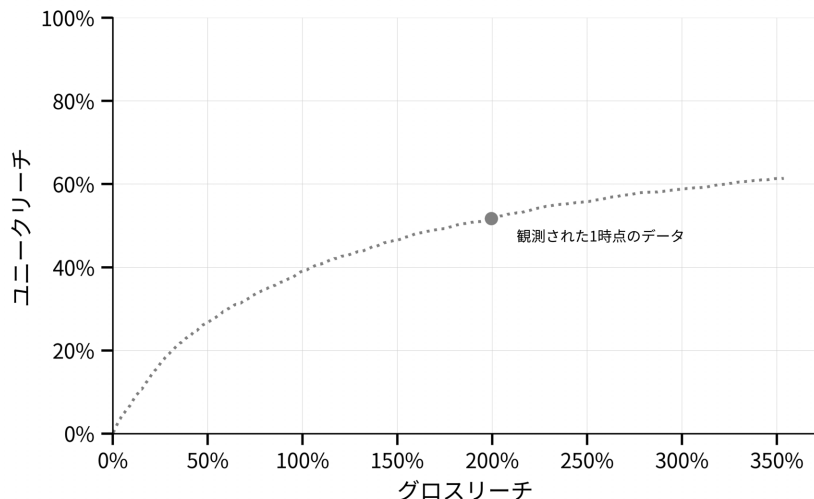
しかし、実務では、キャンペーン終了時点、つまり最終的なグロスリーチとユニークリーチのデータしか手に入らないケースも少なくありません。言い換えると、表 2.2 に示すような、1 時点のデータしか得られない状況があるということです。

▼ 表 2.2: 調査時点の結果

出稿金額	パーコスト	グロスリーチ	ユニークリーチ
2000 万円	10 万円	200%	52%

この最終時点のデータだけでは、図 2.8 のようにリーチカーブ上の 1 点がわかるに過ぎず、カーブ全体の形状を把握することは困難です。

2章 マーケティングにデータサイエンスを導入する



■ 図 2.8: 実務において手に入るデータは少数の場合がある

実務では、このようにデータが限られた中で、目標とするユニークリーチを達成するために必要なグロスリーチを推定する必要が生じます。本章では、このような状況下でリーチカーブを推定する方法について考えていきます。

本節では、テレビ CM を用いたマーケティング活動に関する基本的な知識を解説しました。次節からは、具体的なビジネス課題を取り上げ、3 ステップのフレームワークを用いてその解決策を探っていきます。

2.2

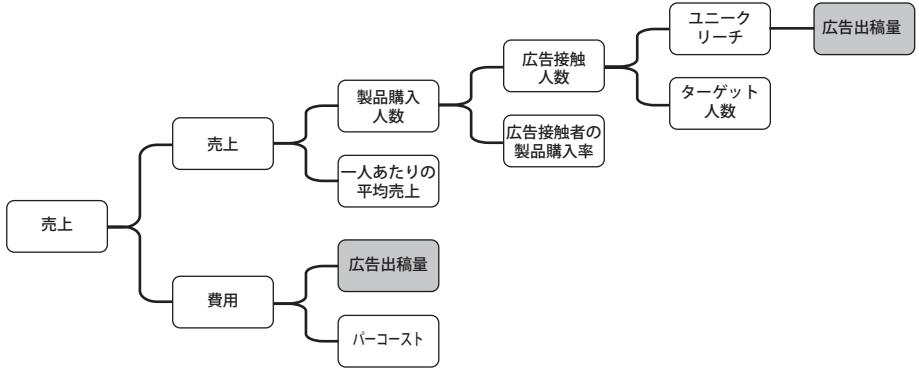
ステップ1：ビジネス課題を数理最適化問題として定式化する

ビジネス課題として「テレビ CM の出稿量を最適化したい」という要望が出てきたとします。実務では、得てしてこのようなあいまいな課題が提示されますが、このままではそもそも何を達成できればいいのかが明確ではありません。そこで、3 ステップフレームワークを用いてこのビジネス課題をデータサイエンスの問題に落とし込み、課題を明確化し、解決策を検討します。

まずはステップ1です。ビジネス課題を数理最適化問題として定式化しましょう。

最初に決定変数について考えます。テレビ CM の出稿形態としては、スポットを想定しているとしましょう。前述のように、スポットで検討可能なアクションは、

2.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する



■ 図 2.9: CM 出稿量と利潤の関係を分解

放送局、放送時間帯、そして出稿量です。今回は放送局と放送時間帯はすでに決まっていて、調整できるのは出稿量のみだったとしましょう。よって、決定変数は CM の出稿量、つまりグロスリーチとし、これを g で表します。また、グロスリーチ 1%あたりのコストが固定されているとし、これを c で表します。このとき、テレビ CM の出稿金額は $100cg$ となります（グロスリーチ 1%が $g = 0.01$ なので、100倍する必要があることに注意してください）。なお、出稿金額 $100cg$ を決定変数とすることもできますが、ここでは出稿量 g を決定変数として考えることにします。

次に、目的関数、つまり最適化したい指標を考えます。「テレビ CM の出稿量を最適化したい」という要望そのものには、最適化したい指標が言及されていないので、ステークホルダーと議論して指標を決定する必要があります。その結果、最適化したい指標が利潤に決まったとします。利潤を確率変数 P で表すと、出稿量で条件付けた利潤の期待値 $\mathbb{E}[P | g]$ が最大化したい目的関数です。

出稿量と利潤の関係を直接的にモデリングするのは難しいので、問題を分解していきましょう。試行錯誤の結果、図 2.9 のような分解を考えたとします。

まず、利潤 P は売上 S と費用 C に分解できます。

$$\begin{aligned}\mathbb{E}[P | g] &= \mathbb{E}[S - C | g] \\ &= \mathbb{E}[S | g] - \mathbb{E}[C | g]\end{aligned}$$

ここで、パースコストは c で固定されているので、 $\mathbb{E}[C | g] = cg$ です。元の式に代入して

$$\mathbb{E}[P | g] = \mathbb{E}[R | g] - 100cg$$

2 章 マーケティングにデータサイエンスを導入する

を得ます。

次に、残った部分である売上 S と CM 出稿量 g の関係について考えます。今回は、別の分析から、 $100q\%$ の人が製品を購入し、その平均売上は $\bar{s}$ 円で、一方で CM に接触していない人はまったく製品を購入しないという結果が得られたとします。

CM に接触したかどうかを確率変数 R で表すとします。 R は CM に一度でも接触したら 1、一度も接触していなければ 0 をとります。CM 出稿量 g で条件付けた R の期待値を $\mathbb{E}[R | g]$ とします。 R は 0 か 1 をとる確率変数なので、 $\mathbb{E}[R | g] = \Pr(R = 1 | g)$ です。ここで、 $\Pr(R = 1 | g)$ はグロスリーチが g の場合のユニークリーチ（CM 接触者の割合）を表していると解釈できます。

仮にターゲットの人数が n 人の場合、売上は一人あたりの平均売上 $\bar{s}$ に CM に接触して製品を購入する人数 $qn \Pr(R = 1 | g)$ を掛ければよいので、

$$\mathbb{E}[S | g] = \bar{s}qn \Pr(R = 1 | g)$$

です。

売上を元の利潤の式に戻すことで、最大化したい利潤関数

$$\mathbb{E}[P | g] = \bar{s}qn \Pr(R = 1 | g) - 100cg$$

を得ます。

目的関数が設定できたので、最後に制約条件を考えます。当たり前ですが、負の広告出稿はできないので $g \geq 0$ です。また、広告予算の上限を b で表すことにすると、予算制約は

$$100cg \leq b$$

となります。

これらをまとめると、「テレビ CM の出稿を通じて利潤を最大化したい」という課題は、以下の制約付き利潤最大化問題として定式化できます。

$$\begin{aligned} \max_g \quad & \bar{s}qn \Pr(R = 1 | g) - 100cg \\ \text{s.t.} \quad & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

課題のあいまいさがなくなり、解くべき課題を明確化することに成功しています。あとは、グロスリーチとユニークリーチの関係である $\Pr(R = 1 | g)$ 、つまり

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

リーチカーブをデータから推定し、この最適化問題を解くことで、元のビジネス課題に対する最適化アクションを知ることができます。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ステップ 1 で数理最適化問題の定式化ができました。最適化問題における未知の部分はグロスリーチとユニークリーチの関係 $\Pr(R = 1 | g)$ なので、これを推定できるようなデータを取得する必要があります。

前述のようにテレビ CM の接触データは十分に手に入らないことが多く、今回も調査時点のデータしか手に入らなかったとします。このような状況でも、得られたデータからできる限り妥当なリーチカーブを推定する方法を考えていきます。

2.3.1 データの準備と可視化

ここからは実際にデータを使って分析していきましょう。

まずは必要なライブラリを読み込みます。その他のライブラリは、必要に応じて都度読み込んでいきます。

```
In[1]: import json # ファイル読み込み

import matplotlib.pyplot as plt # 可視化
import numpy as np # データ処理
import polars as pl # データ処理
import seaborn as sns # 可視化
from matplotlib.axes import Axes # 型ヒント
from matplotlib.figure import Figure # 型ヒント
from matplotlib.ticker import PercentFormatter # 可視化

In[2]: # 乱数を固定
np.random.seed(42)

# データの表示設定
pl.Config.set_float_precision(precision=3)

# 可視化の設定
with open("../setting/visualization.json") as f:
    setting = json.load(f)
sns.set_theme(**setting)
```

2 章 マーケティングにデータサイエンスを導入する

```
sns.set_palette("gray", 4)
```

次に、分析に利用するデータを読み込みます。

分析に利用するデータは、REVISIO 株式会社が保有する、あるテレビ CM に対する接触データです。約 4,500 人に対する CM 接触データからグロスリーチとユニークリーチを集計してあります。実際には REVISIO 株式会社はすべての CM 放送時点におけるグロスリーチとユニークリーチのデータを保有していますが、今回は特定の 1 時点しかデータが手に入らないという制約のある状況を想定します^{*14}。これは、限られた情報から全体の傾向を推測するという、実際のデータ分析でよくある課題に取り組むためです。

この設定のもと、以下の 2 種類のデータを読み込みます。

- `df_reach`: すべての放送時点のデータ。本来は観測できない、構築したモデルの予測精度を確認するためのデータ。よって、モデルの学習（パラメータの推定）には利用しない
- `df_reach_observed`: 限られた調査時点のみのデータ。こちらは実際に分析で利用可能。モデルの学習は、このデータのみを用いて行う

重要な点として、モデル構築の際は `df_reach_observed` の情報しか使えないことに注意してください。`df_reach` は、あくまで後でモデルの性能を評価するための参照用データです。

```
In[3]: # データの読み込み

# 全時点のデータ
df_reach = pl.read_csv("../data/ch02/reach.csv")

# 調査時点のデータ
df_reach_observed =
    pl.read_csv("../data/ch02/reach_observed.csv")

# 観測できない全時点のデータの確認
df_reach
```

* 14 同様の問題設定は、Goerg (2014) でも検討されています。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
Out[3]: shape: (1_165, 3)
```

t	g	r
i64	f64	i64
1	0.003	0.003
2	0.007	0.007
3	0.013	0.012
4	0.015	0.015
5	0.024	0.023
...	...	...
1161	3.526	0.613
1162	3.528	0.613
1163	3.529	0.613
1164	3.532	0.613
1165	3.537	0.613

`df_reach` は三つの変数で構成されています。`t` は CM 放送の時点です。`t = 3` なら 3 回目の放送です。

`g` と `r` は、それぞれ `t` 時点目のグロスリーチとユニークリーチを表しています。数字は%ではなく小数です。この CM は累計で 1,165 回放送され、最終放送時点ではグロスリーチが約 354%、ユニークリーチが約 61%です。

`df_reach_observed` は `df_reach` から `t = 600` の時点のみを抜き出したものです。このとき、グロスリーチは 186.9%、ユニークリーチは 50.5%でした。

```
In[4]: # 観測できた1点のみのデータを確認
df_reach_observed
```

```
Out[4]: shape: (1, 3)
```

t	g	r
i64	f64	i64
600	1.859	0.505

2 章 マーケティングにデータサイエンスを導入する

なお、df_reach には調査時点である $t = 600$ 時点以外のデータも入っていますが、これは「もしこのキャンペーンでより少なく／多く CM を放送していたらこうなっていた」という検証用のデータになります。繰り返しになりますが、今回の問題設定で実際に観測できるのは df_reach_observed に入っている $t = 600$ 時点のデータのみです。

それでは、リーチカーブを可視化しましょう。実際には観測できないリーチカーブを点線で描画し、実際に観測できたデータポイントを 1 点で描画します。

```
In[5]: def plot_reach_curve(
        df_reach: pl.DataFrame,
        df_reach_observed: pl.DataFrame,
        title: str
    ) -> tuple[Figure, Axes]:
    """
    実際には観測できないリーチカーブと、
    観測できるデータポイント1点を可視化する
    """

    fig, ax = plt.subplots()

    ax.plot(
        df_reach["g"],
        df_reach["r"],
        linestyle="dotted",
        color="0.5",
        label="観測できない値",
        zorder=1,
    ) # 観測できないリーチカーブ全体
    ax.scatter(
        df_reach_observed["g"],
        df_reach_observed["r"],
        color="0.5",
        zorder=2,
        label="観測できた値",
    ) # 観測できる1点
    ax.legend()

    # 軸の目盛りを%表示にする
    ax.xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.yaxis.set_major_formatter(
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

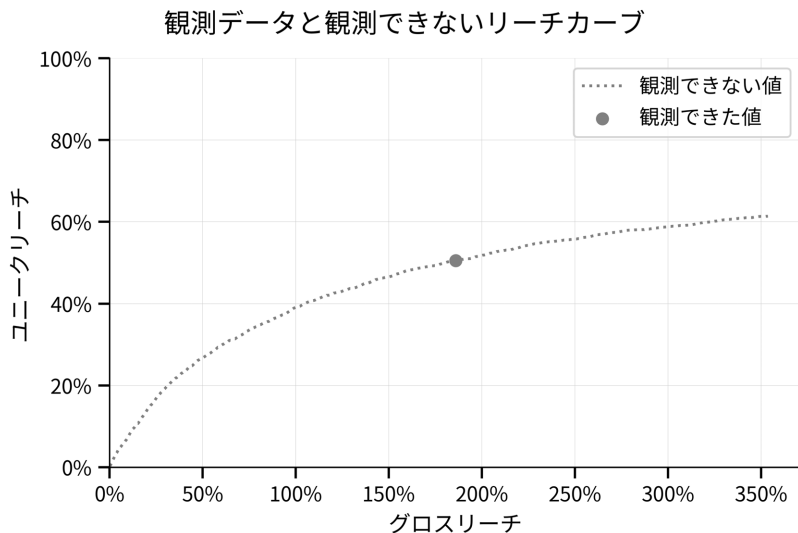
PercentFormatter(xmax=1, decimals=0)
)
ax.set_xlim(0, None)
ax.set_ylim(0, 1)
ax.set_xlabel("グロスリーチ")
ax.set_ylabel("ユニークリーチ")

fig.suptitle(title)

return fig, ax

# リーチカーブの可視化
fig, ax = plot_reach_curve(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    title="観測データと観測できないリーチカーブ",
)

```



2 章 マーケティングにデータサイエンスを導入する

2.3.2 機械学習モデルによる推定

さて、データの準備が整ったので、リーチカーブの推定を行しましょう。予測問題といえば、Random Forest、Gradient Boosting Decision Tree、Neural Net のような機械学習モデルがまず想起されますが、1 章の身長から体重を予測する例でもみたように、データが少ない状況では、複雑なモデルを利用しても予測精度が低くなるが多々あります。

実際に、複雑な機械学習モデルではリーチカーブをうまく推定できないことを確認してみましょう。モデルの学習と予測には scikit-learn を利用します。まずはデータを scikit-learn の入力形式に合わせて変形します。

```
In[6]: # scikit-learnの入力形式に合わせる

# 学習用データ。1点のみ
g_train = df_reach_observed[["g"]].to_numpy()
r_train = df_reach_observed["r"].to_numpy()

# テスト用データ。観測されていないデータも含む全時点データ
g_test = df_reach[["g"]].to_numpy()
```

機械学習モデルには Random Forest を採用することにしましょう^{*15}。scikit-learn の ensemble モジュールから RandomForestRegressor クラスを読み込み、モデルの学習と予測を行います。Random Forest のハイパーパラメータはすべてデフォルトにします（学習の並列化と乱数固定だけ行っています）。

```
In[7]: from sklearn.ensemble import RandomForestRegressor

# Random Forestによるリーチカーブの推定
rf = RandomForestRegressor(
    n_jobs=-1, random_state=42
) # 全コアで並列化。乱数シードを固定
rf.fit(g_train, r_train)
r_pred_rf = rf.predict(g_test) # リーチカーブの推定
```

リーチカーブの推定結果を r_pred_rf に格納しました。予測がうまくいっているかを確認するため、観測されない本当のリーチカーブと、Random Forest で推定されたリーチカーブを並べて可視化してみます。

* 15 本書では Random Forest などの機械学習モデルに関する説明は行いません。興味のある読者は Hastie et al. (2009) をご確認ください。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

In[8]: def plot_reach_curve_with_prediction(
        df_reach: pl.DataFrame,
        df_reach_observed: pl.DataFrame,
        r_pred: np.ndarray,
        prediction_label: str,
        title: str,
    ) -> tuple[Figure, Axes]:
    """
    観測されない本当のリーチカーブと、
    推定されたリーチカーブを可視化する
    """

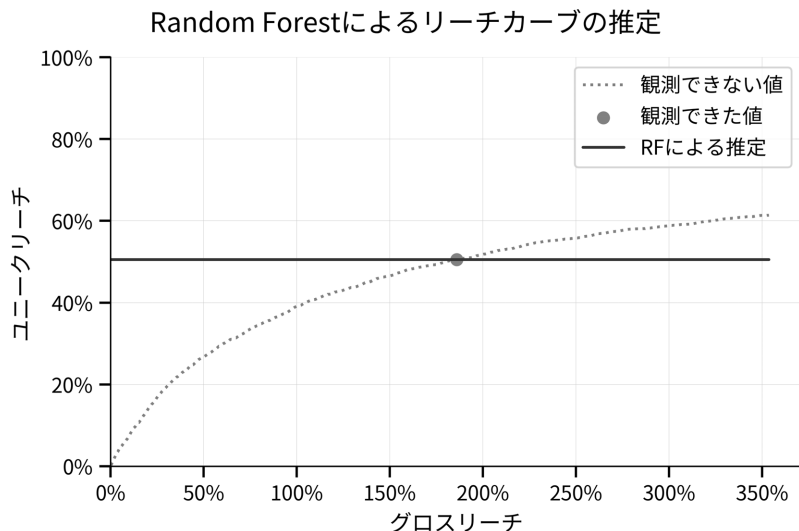
    fig, ax = plot_reach_curve(
        df_reach, df_reach_observed, title
    )
    ax.plot(
        df_reach["g"], r_pred, label=prediction_label
    ) # 推定されたリーチカーブを追加
    ax.legend()

    return fig, ax

# Random Forestによるリーチカーブの可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_rf,
    prediction_label="RFによる推定",
    title="Random Forestによるリーチカーブの推定",
)

```

2 章 マーケティングにデータサイエンスを導入する



データが1点しかないので、Random Forest ではリーチカーブを平らに推定しています。これは、グロスリーチの大小にかかわらずユニークリーチが一定であると予測していることを意味します。

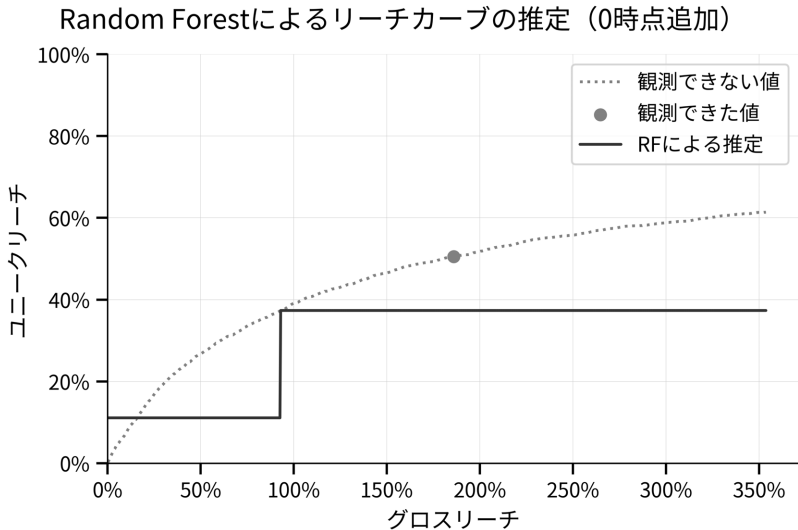
ところで、グロスリーチが0のときはユニークリーチも0になることはわかっているのですが、推定を改善するためこの情報をデータに追加することもできます。念のため、0時点を追加してリーチカーブを推定してみましょう。

```
In[9]: # 0時点の情報を追加
g_train_2 = np.concatenate(
    [np.zeros_like(g_train), g_train], axis=0
)
r_train_2 = np.concatenate(
    [np.zeros_like(r_train), r_train], axis=0
)

# Random Forestによるリーチカーブの推定
rf2 = RandomForestRegressor(
    n_jobs=-1, random_state=42
) # 全コアで並列化。乱数シードを固定
rf2.fit(g_train_2, r_train_2)
r_pred_rf2 = rf2.predict(g_test) # リーチカーブの推定
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
# Random Forestによるリーチカーブの可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_rf2,
    prediction_label="RFによる推定",
    title="Random Forestによるリーチカーブの推定（0時点追加）",
)
```



推定したリーチカーブは平らでこそなくなりましたが、真のリーチカーブをうまく近似できているとは言えません。このように、データが極端に少ない場合は、複雑な機械学習モデルに必要なだけの情報を与えることができず、うまく予測が行えません。

2.3.3 線形回帰モデルによる推定

複雑な機械学習モデルではリーチカーブをうまく推定できませんでした。そこで、代わりに単純な線形回帰モデルを利用することにします。線形回帰モデルでは、グロスリーチで条件付けたユニークリーチの条件付き期待値に対して、線形な関係を仮定します。

2 章 マーケティングにデータサイエンスを導入する

$$\mathbb{E}[R | g] = \beta g$$

ここで、 g はグロスリーチを、 R はユニークリーチを表します。グロスリーチが 0 の場合はユニークリーチも 0 なので、いわゆる切片なしのモデルになっています。回帰係数 β は回帰直線の傾きで、グロスリーチが 1 だけ (つまり 100% だけ) 大きくなったときに、ユニークリーチがどれだけ大きくなるかを表していると解釈できます。

それでは、上記の単純な線形回帰モデルをリーチカーブに当てはめましょう。線形回帰モデルの学習には `scikit-learn` の `linear_model` モジュールから `LinearRegression` クラスを読み込んで利用します。

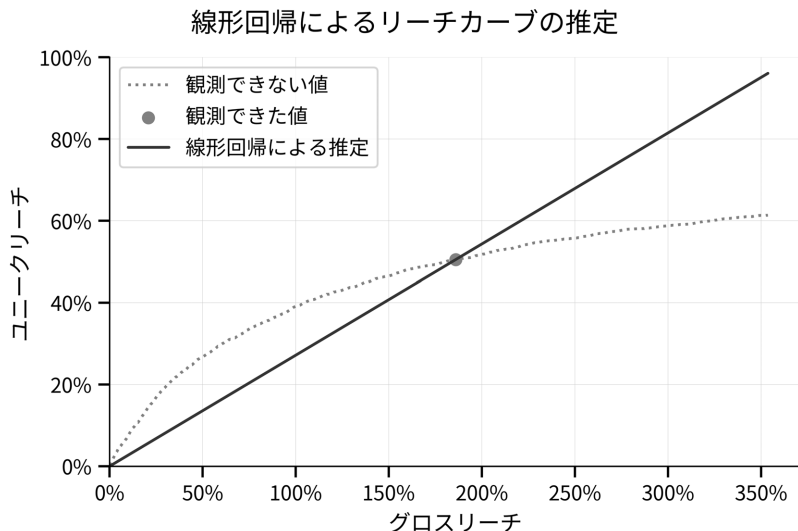
```
In[10]: from sklearn.linear_model import LinearRegression

# 切片を0に固定
lr_linear = LinearRegression(fit_intercept=False)
# 1点で学習
lr_linear.fit(g_train, r_train)
# リーチカーブの推定
r_pred_linear = lr_linear.predict(g_test)
```

リーチカーブの推定結果を `r_pred_linear` に格納しました。観測されない本当のリーチカーブと、線形モデルで推定されたリーチカーブを並べて可視化してみます。

```
In[11]: fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_linear,
    prediction_label="線形回帰による推定",
    title="線形回帰によるリーチカーブの推定",
)
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



線形回帰モデルは、真のリーチカーブをうまく推定できていません。画像の範囲内であっても近似できていたとはいえないく、さらにグロスリーチを増加させるとユニークリーチの予測が 100% を突き抜けてしまいます。ユニークリーチは「一度でも CM に接触した人の割合」なので、100% 以上の数値をとることは想定しておらず、線形回帰モデルによる推定は現実的な制約条件を満たしません。そもそも、グロスリーチとユニークリーチの関係は非線形であることを想定しているので、それを直線で近似することが望ましくありません^{*16}。

可視化の結果から、リーチカーブの近似は直線では難しいことがあらためてわかりました。そこで、次はグロスリーチを対数変換することで非線形な関係を捉えるアプローチを試します。

2.3.4 対数変換による推定

説明変数^{*17}に非線形な変換を施すことで、線形回帰モデルで曲線を引くことがで

* 16 推定するグロスリーチの区間が狭ければ、局所的に直線で近似できるケースはあります。

* 17 回帰モデルにおいて、結果を予測・説明するために使う入力側の変数を説明変数、予測・説明の対象となる出力側の変数を目的変数と呼びます。

2 章 マーケティングにデータサイエンスを導入する

きます*¹⁸。非線形な関係を捉えるためのポピュラーな変換の一つが対数変換です。対数変換では、説明変数であるグロスリーチの自然対数をとったものを学習に利用します。

$$\mathbb{E}[R \mid g] = \beta \log(g + 1)$$

ここで、グロスリーチ g が 0 の場合、ユニークリーチ R も 0 になってほしいことに注意してください。この条件を満たすために、 $\log(g)$ ではなく $\log(g + 1)$ という変換を行っています ($\log(1) = 0$ であることを利用しています)。

それでは、グロスリーチを対数変換してリーチカーブの推定を行いましょう。 $\log(g + 1)$ という対数変換には NumPy の `log1p` 関数が利用できます。

```
In[12]: # 対数変換した場合のリーチカーブの推定

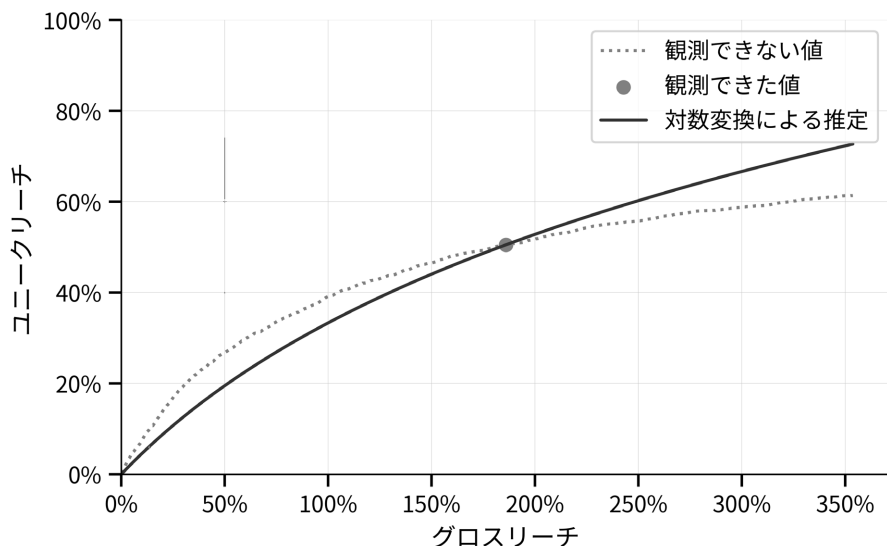
# 切片を0に固定
lr_log = LinearRegression(fit_intercept=False)
# log1p(x) = log(x + 1)
lr_log.fit(np.log1p(g_train), r_train)
# リーチカーブの推定
r_pred_log = lr_log.predict(np.log1p(g_test))

# 対数変換で推定したリーチカーブの可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_log,
    prediction_label="対数変換による推定",
    title="対数変換によるリーチカーブの推定",
)
```

*** 18** 線形回帰モデルは回帰係数 β に関して線形になっていればよく、説明変数 X に関する線形性の制限はありません。例えば、 $\mathbb{E}[Y \mid x] = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$ のような多項式や、 $\mathbb{E}[Y \mid x] = \beta_0 + \beta \log(x)$ のような対数、 $\mathbb{E}[Y \mid x] = \beta_0 + \beta \sqrt{x}$ のような平方根を使った定式化も可能です。一方で、回帰係数 β に関しては線形である必要があります。例えば、 $\mathbb{E}[Y \mid x] = \beta_0 x^\beta$ のような定式化は線形回帰モデルという枠組みでは扱えません。この場合は非線形回帰モデルという枠組みを用いる必要があります。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

対数変換によるリーチカーブの推定



対数変換を行うことで、推定されたリーチカーブが曲線になり、直線よりはうまく真のリーチカーブを予測できているように見えます。ただし予測結果の当てはまりは悪く、ユニークリーチが小さい部分では過小評価を、大きい部分では過大評価をしてしまうという、バイアスがかかっていることが見てとれます。

2.3.5 既存の予測モデルの限界

ここまでの分析で、複雑な機械学習モデルも、単純な線形回帰モデルも、真のリーチカーブをうまく推定できないことがわかりました。今回の課題設定には、利用できるデータが調査時点の一つのデータポイントのみであり、データから得られる情報が少ないという特徴があります。このようにデータから得られる情報が少ない状況では、複雑な機械学習モデルを単純に使用するのではなく、データ生成過程に対するドメイン知識に基づいた分析者の信念・仮説を積極的にモデルに取り込み、情報不足を補う必要があります。

線形回帰モデルの場合は「線形」という強い仮定をモデルに課しており、この仮定によって手元のデータから得られる情報の不足を補っていると考えられます。ただし、今回のビジネス課題では線形性の仮定はうまく現実を表すことがで

2 章 マーケティングにデータサイエンスを導入する

きませんでした。線形回帰モデルの失敗は、仮定が現実と合っていないければ間違った予測をしてしまうことを示しています。したがって、次にとるべきアプローチは、より現実在即した妥当性の高い仮説を立て、それを数理モデルとして表現することです。

以降の分析では、リーチカーブの背後にあるメカニズムに着目します。そのメカニズムを数理モデルで表現することで、データの情報量の不足を補い、妥当な予測を可能にすることを目指します。

2.3.6 CM 接触の数理モデルの構築

ここまでの議論で、少ないデータからリーチカーブを推定するためには、ドメイン知識を反映した数理モデルの力を必要とすることがわかりました。ここからは、数理モデルを具体的に構築するために、グロスリーチとユニークリーチの関係、つまりリーチカーブを数式を使って表現していきます。

ユニークリーチは少なくとも 1 回以上 CM に接触した人の割合です。ユニークリーチを理解するために、まずはリーチカーブの背後にある、より基本的な情報について考えましょう。それは、「CM に何回接触したのか」という接触回数のばらつきです。「CM に 1 回だけ接触した人は 10%」「CM に 2 回接触した人は 30%」のように、各接触回数がどのくらいの割合で出現しているか、つまり CM 接触回数の分布について考えます。

CM 接触回数はフリークエンシーとも呼ばれます。そこで、 $F = 1, 2, \dots$ を CM への接触回数を表す確率変数とします。このとき、CM 接触回数の分布は、CM に f 回接触する確率 $\Pr(F = f)$ で表現できます。

図 2.10 は CM 接触回数の分布の一例です。この例では、CM に一度も接触しない確率が 60%、1 回だけ接触する確率が 30%、2 回接触する確率が 10%となっています。

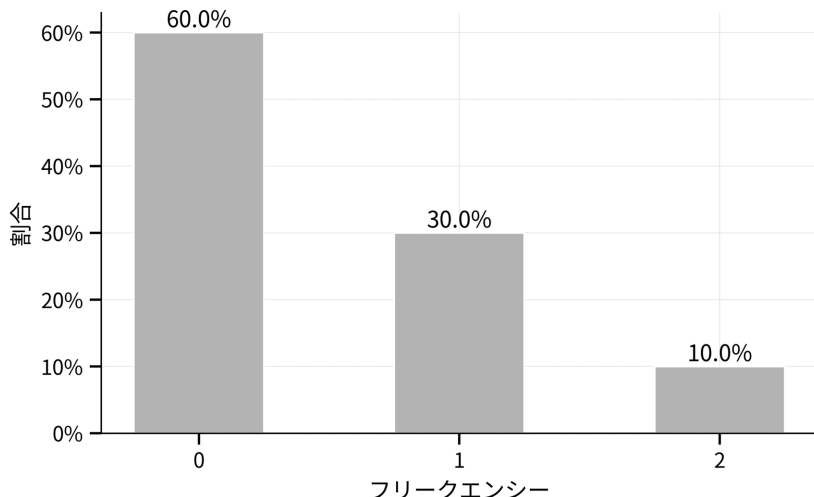
さて、いま考えたいことはリーチカーブの推定です。つまり、グロスリーチ g が与えられた場合のユニークリーチをうまく予測することです。

まずはグロスリーチについて考えましょう。図 2.10 の CM 接触回数の分布を例にとると、グロスリーチは以下で計算できます。

$$0 \times 0.6 + 1 \times 0.3 + 2 \times 0.1 = 0.5$$

計算の結果、グロスリーチは 50%でした。これは延べ 50%の人に CM を見てもらえていることを意味します。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 2.10: CM 接触回数の分布の例

上記のグロスリーチの計算式をよく見ると、CM 接触回数をその接触回数の出現確率で加重平均しています。ということは、グロスリーチは CM 接触回数 F の期待値

$$\mathbb{E}[F] = \sum_{f=0}^{\infty} f \Pr(F = f)$$

と同じ計算をしており、「グロスリーチは CM 接触回数の期待値」と解釈することができます。よって、グロスリーチが与えられた下での CM 接触回数の分布の期待値は、グロスリーチに等しくなることに注意してください。

$$\mathbb{E}[F | g] = \sum_{f=0}^{\infty} f \Pr(F = f | g) = g$$

次はユニークリーチについて考えます。ユニークリーチは CM に 1 回以上接触した人の割合です。CM に 1 回以上接触したら 1、一度も接触しなければ 0 をとる確率変数を R とします。つまり、ユニークリーチ R とフリークエンシー F には以下の関係があります。

$$\mathbb{E}[R | g] = \Pr(R = 1 | g) = \Pr(F \geq 1 | g)$$

よって、グロスリーチ g が与えられた際のユニークリーチの条件付き期待値を $r(g)$ とすると、

2 章 マーケティングにデータサイエンスを導入する

$$\begin{aligned} r(g) &= \mathbb{E}[R \mid g] \\ &= \Pr(F \geq 1 \mid g) \\ &= 1 - \Pr(F = 0 \mid g) \end{aligned}$$

であり、1 回以上 CM に接触した人の割合は、全体の 100% から CM に一度も接触しなかった人の割合を引けば計算できることがわかります。

具体的に図 2.10 を例にとって計算してみましょう。

$$r(0.5) = 1 - 0.6 = 0.4$$

であり、グロスリーチが 50% の場合のユニークリーチは 40% となります。

これらの記法を用いて、解くべきビジネス課題「テレビ CM の出稿を通じて利潤を最大化したい」をあらためて記載します。

$$\begin{aligned} \max_g \quad & \bar{s} q n r(g) - 100 c g \\ \text{s.t.} \quad & 100 c g \leq b \\ & g \geq 0 \end{aligned}$$

結局、この問題を解くためには、リーチカーブ $r(g)$ の推定が必要です。また、ここまでの議論から、リーチカーブの推定にはグロスリーチが与えられた下での 0 回接触の確率 $\Pr(F = 0 \mid g)$ の推定が必要であることがわかりました。

それでは、どのようにして $\Pr(F = 0 \mid g)$ を推定すればよいのでしょうか？ この 0 回接触の確率だけをピンポイントで推定するアプローチも考えられますが、今回は「CM 接触回数全体の分布 $\Pr(F = f \mid g)$ をよく知られた何らかの分布で近似するアプローチ」を考えることにします。

それでは、どのような分布を使えば CM 接触回数の分布をうまく近似できるのでしょうか？ CM 接触回数は頻度であり、0 以上の整数をとる離散確率分布が候補となります。CM 接触回数のような、「ある期間の中で事象が何回発生するか」を記述する際によく用いられる確率分布として、ポアソン分布が知られています*19。

* 19 ポアソン分布は、「1 時間あたりに店舗に来る顧客の数」や「1 ページあたりに含まれる誤字の数」など、比較的稀な事象が発生する回数をモデル化するのに適しています。どのような状況でどの分布を用いられるかの判断は、数理統計学やベイズ統計学の専門書の記載を参考にしてください。例えば、久保川 (2024) や Lambert (2018) が参考になります。

2.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

2.3.7 ポアソン分布によるリーチカーブの推定

ポアソン分布の導入

CM 接触回数の分布を近似できそうだとし、ポアソン分布が候補に挙がりました。CM 接触回数 F がポアソン分布に従うと仮定した場合、CM にちょうど f 回接触する確率 $\Pr(F = f | \theta)$ は、以下の確率関数で与えられます。

$$\begin{aligned}\Pr(F = f | \theta) &= \text{Poisson}(f | \theta) \\ &= \frac{1}{\Gamma(f+1)} \theta^f e^{-\theta}\end{aligned}$$

パラメータ θ はポアソン分布の形状を決定する唯一のパラメータです。 θ の値が決まれば、CM 接触回数の分布の形が一意に定まります。

パラメータ θ はポアソン分布の期待値を表します^{*20}。つまり、ポアソン分布は「平均して θ 回事象が発生する」ような状況を記述する分布と言えます。CM 接触回数の文脈で言えば、平均接触回数が θ であるような状況を想定していることになります。CM 接触回数の分布の期待値はグロスリーチ g と等しいことを利用すると、ポアソン分布のパラメータは $\theta = g$ で固定され、分布の形状が一意に定まります。

なお、ガンマ関数 $\Gamma(f)$ は階乗を連続値に一般化したもので、 f が整数の場合は階乗と等しくなります ($\Gamma(f+1) = f!$)^{*21}。

それでは、 $\theta = 1, 2, 5$ の3パターンでポアソン分布を可視化してみましょう。それぞれグロスリーチが100%、200%、500%の場合に相当します。

ポアソン分布の確率関数は自分で書き下す必要はなく、SciPyのstatsモジュールに`poisson.pmf`メソッドが用意されています。なお、本書のパラメータ名とメソッドの引数名が異なることに注意してください。引数`k`がCM接触回数 f に、引数`mu`がパラメータ θ に相当します。

```
In[13]: from scipy.stats import poisson
```

```
def plot_poisson(
    f: np.ndarray, theta_list: list[float]
) -> tuple[Figure, Axes]:
    """ポアソン分布を可視化"""
```

* 20 なお、ポアソン分布のパラメータ θ は分散でもあります。ポアソン分布は期待値と分散が等しい確率分布です。

* 21 ポアソン分布は離散分布なので、確率関数は階乗で定義することが多いですが、後述するガンマ分布と記法を揃えるために、今回はガンマ関数を利用しました。

2 章 マーケティングにデータサイエンスを導入する

```
fig, ax = plt.subplots()

markers = ["o", "D", "^", "x"]
linestyles = ["solid", "dashed", "dashdot", "dotted"]
for i, theta in enumerate(theta_list):
    y = poisson.pmf(k=f, mu=theta) # ポアソン分布の確率関数
    ax.plot(
        f,
        y,
        marker=markers[i % len(markers)],
        linestyle=linestyles[i % len(linestyles)],
        label=f"Poisson$(f \mid \theta = \{\theta\})$",
    )

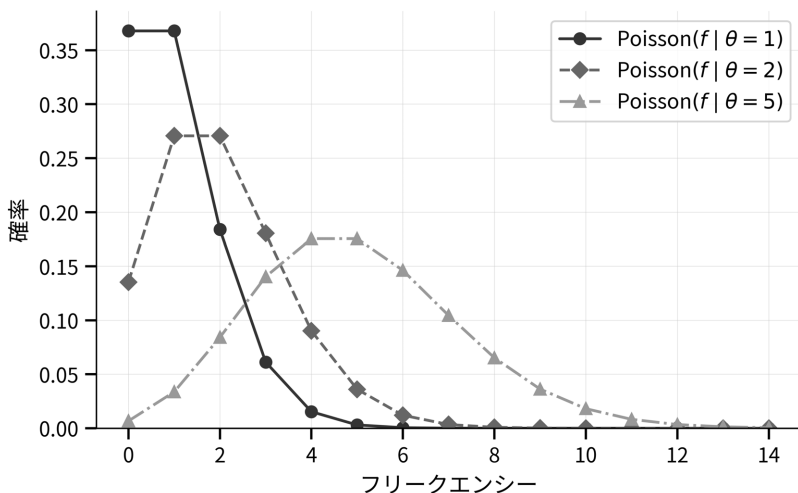
ax.legend()
ax.set_ylim(0, None)
ax.set_xlabel("フリークエンシー")
ax.set_ylabel("確率")
fig.suptitle("ポアソン分布の例")

return fig, ax

# ポアソン分布の可視化
fig, ax = plot_poisson(f=np.arange(15), theta_list=[1, 2, 5])
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ポアソン分布の例



θ が大きくなるということは、ポアソン分布の期待値が大きくなるということなので、分布が右に寄っていくのが見てとれます。また、 θ は分散でもあるので、 θ が大きくなるにつれて分布も広がっていきます。

ユニークリーチは少なくとも 1 回 CM に接触する確率なので 0 回接触の確率を 100%から引けば求まります。例えば、 $\theta = 1$ のときは 0 回接触の確率が約 37%なので、ユニークリーチは約 63%です。

ポアソン分布を仮定したリーチカーブ

具体的な θ の値に対応するポアソン分布の形状を確認することで、ポアソン分布に対する理解を深めました。次は、より一般的に、ポアソン分布を仮定した場合のリーチカーブを考えましょう。

リーチカーブはグロスリーチ g が与えられたときのユニークリーチを表します。CM 接触回数の分布の期待値はグロスリーチと等しく、いま CM 接触回数の分布をポアソン分布で近似しているので、ポアソン分布の期待値を表すパラメータ θ は g と等しくなります。よって、リーチカーブは以下で記述できます。

$$\begin{aligned} r(g) &= 1 - \Pr(F = 0 \mid \theta = g) \\ &= 1 - \frac{1}{\Gamma(0+1)} g^0 e^{-g} \end{aligned}$$

2 章 マーケティングにデータサイエンスを導入する

$$= 1 - e^{-g}$$

ここで、2 行目への式変形にはポアソン分布の確率関数を利用しています。

それでは、実際にリーチカーブを描画してみましょう。リーチカーブを作成する関数 `predict_reach_poisson` を定義し、その結果を可視化します。

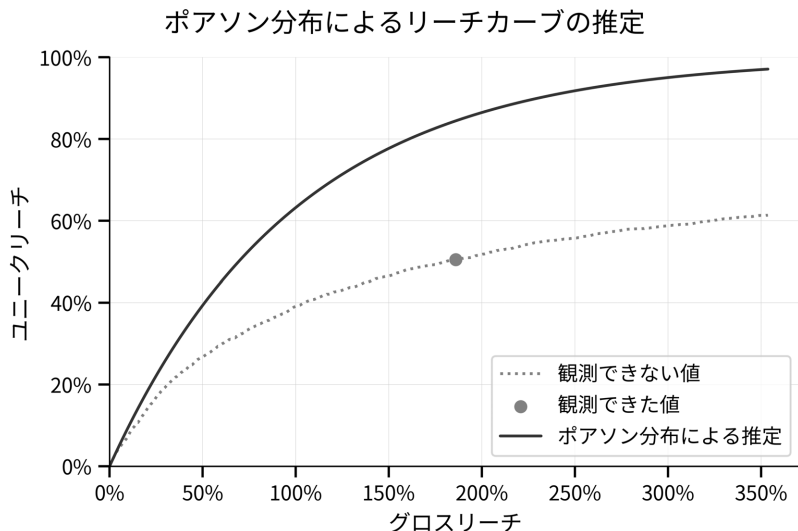
```
In[14]: def predict_reach_poisson(
        g: np.ndarray | pl.Series
    ) -> np.ndarray:
        """ポアソン分布を仮定したリーチカーブ"""

        # 100%から非接触率を引いたものがユニークリーチ
        return 1 - poisson.pmf(k=0, mu=g)

# リーチカーブを計算
r_pred_poisson = predict_reach_poisson(g=df_reach["g"])

# 可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_poisson,
    prediction_label="ポアソン分布による推定",
    title="ポアソン分布によるリーチカーブの推定",
)
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



ポアソン分布を仮定したリーチカーブは、実際のリーチカーブをうまく近似できていないことが見てとれます。線形回帰モデルを利用したリーチカーブの推定よりも誤差が大きくなっており、別の手段を考える必要があります。

ポアソン分布によるリーチカーブの問題点

ここまでの分析で、ポアソン分布を単純に利用しても、真のリーチカーブをうまく推定できないことがわかりました。この問題の原因を探索するために、リーチカーブだけでなく、その背後にある CM 接触回数の分布をポアソン分布がうまく近似できているかを確認しましょう。

そのために、CM 接触回数の分布のデータを読み込みます。

```
In[15]: # 調査時点のCM接触回数の分布を読み込む
df_frequency_observed = pl.read_csv(
    "../data/ch02/frequency_observed.csv"
)

# データの確認
df_frequency_observed
```

2 章 マーケティングにデータサイエンスを導入する

Out[15]: shape: (61, 4)

t	g	f	prop
i64	f64	i64	f64
600	1.859	0	0.495
600	1.859	1	0.180
600	1.859	2	0.098
600	1.859	3	0.064
600	1.859	4	0.039
...	...	...	...
600	1.859	56	0.000
600	1.859	57	0.000
600	1.859	58	0.000
600	1.859	59	0.000
600	1.859	60	0.000

`df_frequency_observed` は、`df_reach_observed` と同じ時点の、CM 接触回数の分布に関する情報が格納されています。

- `t` : 時点。今回は一時点のみ
- `f` : CM 接触回数
- `prop` : `f` 回接触の人の割合

0 回接触の割合が 49.5%なので、ユニークリーチが 50.5%であることもわかります（これは、`df_reach_observed` に入っている数字と一致します）。なお、この CM 接触回数の分布の情報は、あくまで検証のためのデータであり、観測できない情報であることに注意してください。繰り返しになりますが、今回の問題設定で観測できるのは、`df_reach_observed` に入っている調査時点のグロスリーチとユニークリーチのみです。

観測できないデータであることは念頭に置きながら、`df_frequency_observed` を使って CM 接触回数の分布を可視化してみましょう。可視化対象の CM 接触回

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

数を大きくとりすぎると見づらいなので、0 回接触から 15 回接触までの割合を可視化します。

```
In[16]: def plot_histogram(
    df_frequency_observed: pl.DataFrame,
    f_max: int,
    title: str
) -> tuple[Figure, Axes]:
    """CM接触回数の分布を可視化"""

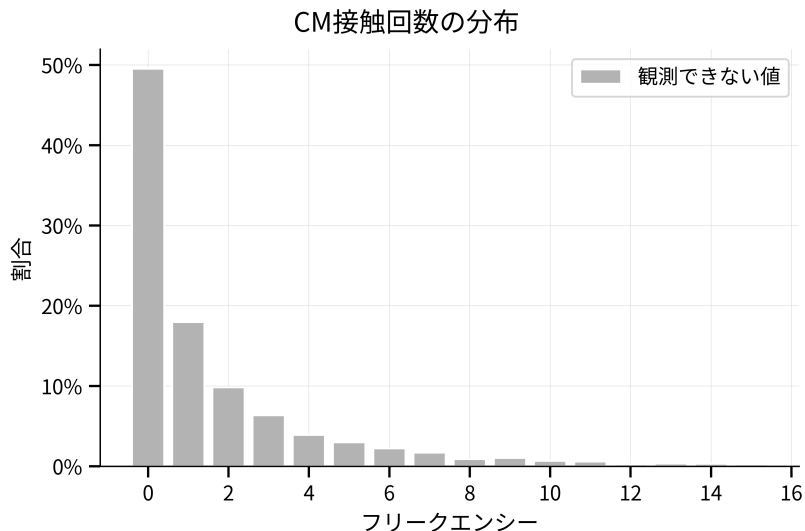
    # 横に長くなりすぎるとうまく可視化できないで、
    # 必要であればf_max以下のデータのみを使う
    df_to_use = df_frequency_observed.filter(
        pl.col.f <= f_max
    )
    fig, ax = plt.subplots()

    ax.bar(
        df_to_use["f"],
        df_to_use["prop"],
        label="観測できない値",
        color="0.7",
    )
    ax.legend()
    ax.yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set_xlabel("フリークエンシー")
    ax.set_ylabel("割合")
    fig.suptitle(title)

    return fig, ax

# フリークエンシーの分布を可視化
f_max = 15
fig, ax = plot_histogram(
    df_frequency_observed=df_frequency_observed,
    f_max=f_max,
    title="CM接触回数の分布",
)
```

2 章 マーケティングにデータサイエンスを導入する



0 回接触の割合が最も大きく、接触回数が多くなるにつれて徐々に割合が小さくなっていることが見てとれます。

次に、この CM 接触回数の分布に推定されたポアソン分布を重ね合わせてみましょう。

```
In[17]: def plot_histogram_with_distribution(
    df_frequency_observed: pl.DataFrame,
    prop_pred: np.ndarray,
    prop_label: str,
    f_max: int,
    title: str,
) -> tuple[Figure, Axes]:
    """CM接触回数の分布と、近似する分布を重ね合わせて可視化"""

    fig, ax = plot_histogram(
        df_frequency_observed, f_max, title
    )
    ax.plot(
        range(f_max + 1),
        prop_pred,
        marker="o",
        label=prop_label
    ) # 近似する分布を追加
```

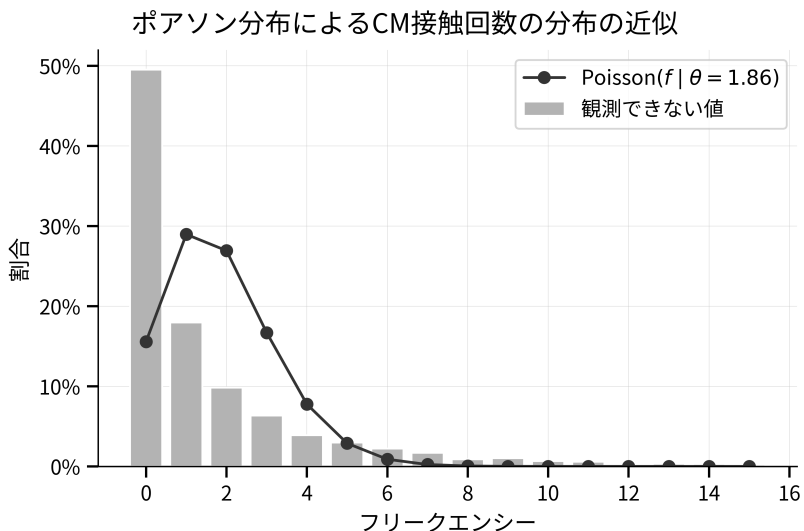
2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
ax.legend()

return fig, ax

# ポアソン分布を仮定したCM接触回数の分布
# 期待値は調査時点のグロスリーチを指定
prop_poisson = poisson.pmf(
    k=range(f_max + 1), mu=df_reach_observed["g"]
)

# CM接触回数分布とポアソン分布を重ね合わせて可視化
fig, ax = plot_histogram_with_distribution(
    df_frequency_observed=df_frequency_observed,
    prop_pred=prop_poisson,
    prop_label=f"Poisson$(f \mid \theta = \{df_reach_observed[\"g\"]').item(): .2f)$",
    f_max=f_max,
    title="ポアソン分布によるCM接触回数の分布の近似",
)
```



上図から、グロスリーチの値を期待値として利用した $\theta = 1.86$ のポアソン分布では、CM 接触回数の分布をうまく近似できていないことが見てとれます。

2 章 マーケティングにデータサイエンスを導入する

2.3.8 負の二項分布によるリーチカーブの推定

ポアソン分布の合成

なぜ CM 接触回数の分布をポアソン分布ではうまく近似できなかったのでしょうか？ その原因を探るため、CM 接触回数の分布がどのように形成されるのかについて、あらためて考えてみましょう。

CM 接触回数の分布は、ある集団に所属する一人ひとりが CM に接触した回数の分布です。CM 接触回数の分布を単純にポアソン分布で近似することは、「集団に所属する全員が共通の平均接触回数 θ をもつ」と暗に仮定していたことになります。しかし、集団にはテレビをよく見るので CM に接触しやすい個人もいれば、テレビをほとんど見ないので CM になかなか接触しない個人もいると考えたほうが自然です。

つまり、個人の CM 接触のしやすさには異質性が存在するはずです。この個人の異質性を考慮に入れると、各個人の CM 接触回数はその個人固有のパラメータをもつポアソン分布に従い、最終的に観測される集団としての CM 接触回数の分布は、パラメータ θ の値が異なるポアソン分布が合成されたものと考えることができそうです。

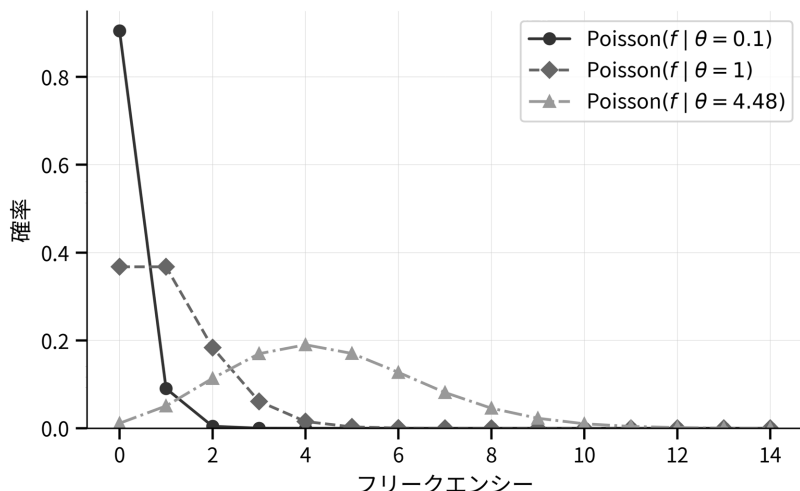
この「パラメータの異なるポアソン分布の合成」というアイデアはうまく機能するのでしょうか。理解を深めるのために、具体的な数値例を使って考えてみましょう。ある集団に所属する個人は、「テレビをほとんど見ない人」、「ある程度見る人」、「よく見る人」の三つのグループに分けられ、それぞれのグループは 3 分の 1 ずつを占めているとします。そして、彼らの CM 接触回数の期待値はそれぞれ $\theta = 0.1, 1, 4.48$ だたしとしましょう（これらの値は、観測されたグロスリーチが 1.86 だったことに合わせて、三つのグループの CM 接触回数の期待値を平均すると 1.86 になるように数字を調整したものです）。

ポアソン分布を合成する前に、いったん各パラメータごとのポアソン分布を確認しておきましょう。

```
In[18]: # パラメータの異なるポアソン分布を可視化
        theta_list = [0.1, 1, 4.48] # パラメータのリスト
        fig, ax = plot_poisson(f=np.arange(15), theta_list=theta_list)
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ポアソン分布の例



期待値 θ が小さいと左寄りの分布、 θ が大きいと右寄りの分布になります。前述のように、 θ は分散でもあるので、 θ が大きくなるほど分布の広がりも大きくなります。

ここで、集団全体の CM 接触回数の分布を考えてみましょう。三つのグループはそれぞれ集団の 3 分の 1 ずつを占めるという設定から、集団全体の CM 接触回数の分布は、以下のように同じ重みでポアソン分布を合成したものになります。

$$\frac{1}{3}\text{Poisson}(f \mid \theta = 0.1) + \frac{1}{3}\text{Poisson}(f \mid \theta = 1) + \frac{1}{3}\text{Poisson}(f \mid \theta = 4.48)$$

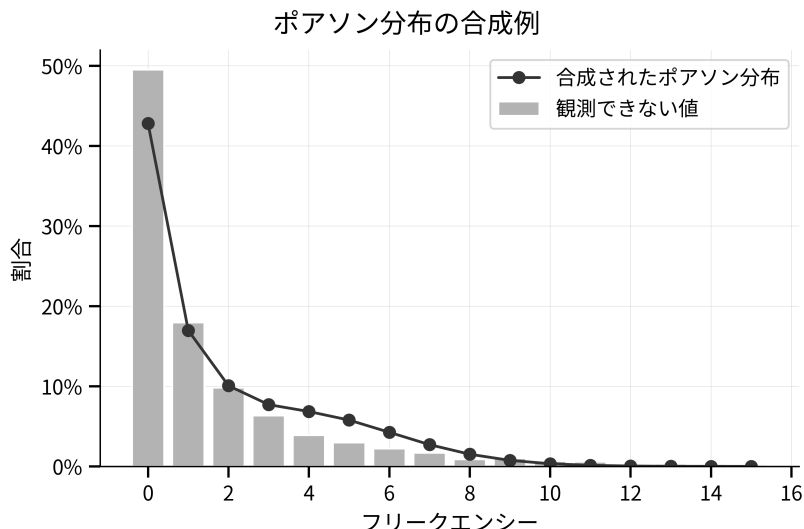
実際に合成されたポアソン分布を可視化します。

```
In[19]: # ポアソン分布を合成。接触回数ごとに確率の平均をとればよい
prop_mixed = np.zeros(f_max + 1)
for theta in theta_list:
    prop_mixed += poisson.pmf(
        k=np.arange(f_max + 1), mu=theta
    ) / len(theta_list)

# 合成された分布を可視化
fig, ax = fig, ax = plot_histogram_with_distribution(
    df_frequency_observed=df_frequency_observed,
    prop_pred=prop_mixed,
    prop_label="合成されたポアソン分布",
```

2章 マーケティングにデータサイエンスを導入する

```
f_max=f_max,
title="ポアソン分布の合成例",
)
```



まだまだ改良の余地はありますが、合成されたポアソン分布は単純なポアソン分布より CM 接触回数の分布をうまく近似できています。このポアソン分布の合成というアイデアが有力であることがわかったので、以降ではより良いポアソン分布の合成について考えていきます。

積分によるポアソン分布の合成

先ほどは異なるパラメータ θ の値をもつ 3 種類のポアソン分布を同じ重みで合成しましたが、もちろん 3 種類以上の分布を異なる重みで合成することも可能です。例えば、 m 個のポアソン分布の合成は、以下の加重平均で考えることができます。

$$\Pr(F = f) = \sum_{j=1}^m w_j \text{Poisson}(f \mid \theta_j),$$

$$\text{s.t. } \sum_{j=1}^m w_j = 1$$

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ここで、 j 番目のポアソン分布のパラメータを θ_j 、その合成の重みを w_j としています。 w_j は加重平均の重みなので、足して 1 になるという制約を加えています。

さらに、パラメータ θ は 0 以上の値をもつ連続値であることを考慮すると、積分を用いたより柔軟な合成を考えることができます。

$$\begin{aligned} \Pr(F = f) &= \int_0^\infty \text{Poisson}(f | \theta) p(\theta) d\theta, \\ \text{s.t. } \int_0^\infty p(\theta) d\theta &= 1 \end{aligned}$$

ここで、 $p(\theta)$ は合成の重みと解釈でき、加重平均による合成の w_j に相当します。

ポアソン分布の合成の重みとしてのガンマ分布

さて、積分による合成を実行するためには集団内の θ の分布をうまく表現できるような確率分布 $p(\theta)$ を見つける必要があります。0 以上の値をもつ連続値が従う確率分布としてガンマ分布が知られているので、今回はこれを使うことにしましょう*22。

ガンマ分布の確率密度関数は以下で与えられます。

$$\begin{aligned} p(\theta | \alpha, \beta) &= \text{Gamma}(\theta | \alpha, \beta) \\ &= \frac{\beta^\alpha}{\Gamma(\alpha)} \theta^{\alpha-1} e^{-\beta\theta} \end{aligned}$$

ガンマ分布のパラメータは (α, β) ですが、CM 接触回数の文脈ではグロスリーチ、つまり期待値が与えられる都合上、期待値を直接指定できるようパラメータを変換しておくとう便利です。ガンマ分布の期待値は α/β なので、 $\mu = \alpha/\beta$ と変換しておくと、ガンマ分布の確率密度関数は以下で再表現できます。

$$\begin{aligned} p(\theta | \mu, \alpha) &= \text{Gamma}\left(\theta | \alpha, \frac{\alpha}{\mu}\right) \\ &= \frac{\left(\frac{\alpha}{\mu}\right)^\alpha}{\Gamma(\alpha)} \theta^{\alpha-1} e^{-\frac{\alpha}{\mu}\theta} \end{aligned}$$

ここで、パラメータ μ は定義から分布の期待値を表します。 μ は「各個人の CM 接触回数の期待値」の期待値なので、集団としての CM 接触回数の期待値を表しています。パラメータ α は分布の密集度を表していて、 α が大きいほど分布が期待値

* 22 ガンマ分布はポアソン分布のパラメータの分布として、特にベイズ統計学でよく利用されます。「共役性」という性質から、パラメータの事後分布が事前分布と同じ形になり、数学的に扱いやすいからです。詳細は Lambert (2018) をご確認ください

2 章 マーケティングにデータサイエンスを導入する

の周りに密集します。つまり、 α は個人の CM 接触回数の期待値がどの程度似通っているかを表します。

実際にいくつかのパラメータでガンマ分布を可視化してみましょう。期待値を $\mu = 2$ に固定して、 $\alpha = 1, 2, 10$ と動かしてみます。

ポアソン分布と同様、ガンマ分布の確率密度関数も SciPy の stats モジュールに `gamma.pdf` メソッドとして用意されています。本書のパラメータ名とメソッドの引数名が異なることに注意してください。引数 `a` がパラメータ α 、引数 `scale` が μ/α に相当します。

```
In[20]: from scipy.stats import gamma

def plot_gamma(
    theta: np.ndarray,
    mu_list: list[float],
    alpha_list: list[float]
) -> tuple[Figure, Axes]:
    """ガンマ分布を可視化"""

    fig, ax = plt.subplots()

    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, (mu, alpha) in enumerate(zip(mu_list, alpha_list)):
        # ガンマ分布の確率密度関数
        y = gamma.pdf(x=theta, a=alpha, scale=mu / alpha)
        ax.plot(
            theta,
            y,
            linestyle=linestyle[i % len(linestyle)],
            label=f"Gamma$(\\theta \\mid \\mu = \\{mu\\}, \\alpha$
            = {alpha})$",
        )

    ax.legend()
    ax.set_ylim(0, None)
    ax.set_xlabel("$\\theta$")
    ax.set_ylabel("確率密度")

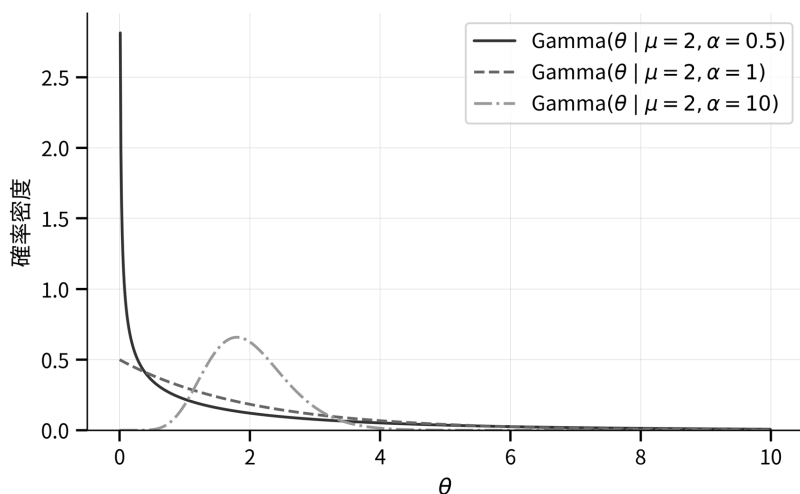
    fig.suptitle("ガンマ分布の例")

    return fig, ax
```


2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
# ガンマ分布を可視化
fig, ax = plot_gamma(
    theta=np.arange(0, 10, 0.01),
    mu_list=[2, 2, 2],
    alpha_list=[0.5, 1, 10]
)
```

ガンマ分布の例



期待値が同じでも、パラメータ α が大きくなると期待値 $\mu = 2$ の周りに分布が密集していく様子が見てとれます。

$\alpha = 0.5$ の場合は、 $\theta = 0$ 付近の割合が大きい一方で、 $\theta > 4$ の割合も一定あります。つまり、テレビをほとんど見ない人もいれば、テレビをよく見る人もいる状態を表していると解釈できます。一方で、 $\alpha = 10$ の場合、 θ は 1 から 3 程度の区間に多く分布しており、集団内のテレビ視聴が比較的均一である状態と解釈できます。

負の二項分布の導出

これまでの議論から、ガンマ分布を用いて、ポアソン分布のパラメータ θ の分布を柔軟に表現できそうなのことがわかりました。それでは、ガンマ分布を用いてポアソン分布を合成してみましょう。

2 章 マーケティングにデータサイエンスを導入する

$$\begin{aligned}
& \Pr(F = f \mid \mu, \alpha) \\
&= \int_0^\infty \text{Poisson}(f \mid \theta) \text{Gamma}\left(\theta \mid \alpha, \frac{\alpha}{\mu}\right) d\theta \\
&= \int_0^\infty \left(\frac{1}{\Gamma(f+1)} \theta^f e^{-\theta} \right) \left(\frac{\left(\frac{\alpha}{\mu}\right)^\alpha}{\Gamma(\alpha)} \theta^{\alpha-1} e^{-\frac{\alpha}{\mu}\theta} \right) d\theta \\
&= \frac{\left(\frac{\alpha}{\mu}\right)^\alpha}{\Gamma(f+1)\Gamma(\alpha)} \int_0^\infty \theta^{\alpha+f-1} e^{-(\frac{\alpha}{\mu}+1)\theta} d\theta \\
&= \frac{\left(\frac{\alpha}{\mu}\right)^\alpha}{\Gamma(f+1)\Gamma(\alpha)} \frac{\Gamma(\alpha+f)}{\left(\frac{\alpha}{\mu}+1\right)^{\alpha+f}} \underbrace{\int_0^\infty \frac{\left(\frac{\alpha}{\mu}+1\right)^{\alpha+f}}{\Gamma(\alpha+f)} \theta^{\alpha+f-1} e^{-(\frac{\alpha}{\mu}+1)\theta} d\theta}_{=1} \\
&= \frac{\Gamma(\alpha+f)}{\Gamma(f+1)\Gamma(\alpha)} \left(\frac{\frac{\alpha}{\mu}}{\frac{\alpha}{\mu}+1} \right)^\alpha \left(\frac{1}{\frac{\alpha}{\mu}+1} \right)^f \\
&= \frac{\Gamma(\alpha+f)}{\Gamma(f+1)\Gamma(\alpha)} \left(\frac{\alpha}{\mu+\alpha} \right)^\alpha \left(\frac{\mu}{\mu+\alpha} \right)^f
\end{aligned}$$

一見、式変形が複雑そうに見えますが、一つひとつ手順を追うと難しい操作は行っていないことがわかります。

- 1 行目の：前述の「積分を用いたポアソン分布の合成」に従った
- 2 行目への変形：単純にポアソン分布とガンマ分布の定義を代入した
- 3 行目への変形： θ と関係ない項を積分の外に出した
- 4 行目への変形：同じ数を掛けて割っている（5 行目への変形のためによく使うテクニック）
- 5 行目への変形：積分内はガンマ分布 $\text{Gamma}\left(\theta \mid \alpha + f, \frac{\alpha}{\mu} + 1\right)$ であり、確率密度関数を全区間で積分すると 1 となる
- 6 行目への変形：最後に、分数部分を整理した

実は、このようにガンマ分布を利用してポアソン分布を合成した確率分布には名前があり、負の二項分布（Negative Binomial Distribution）と呼ばれています。

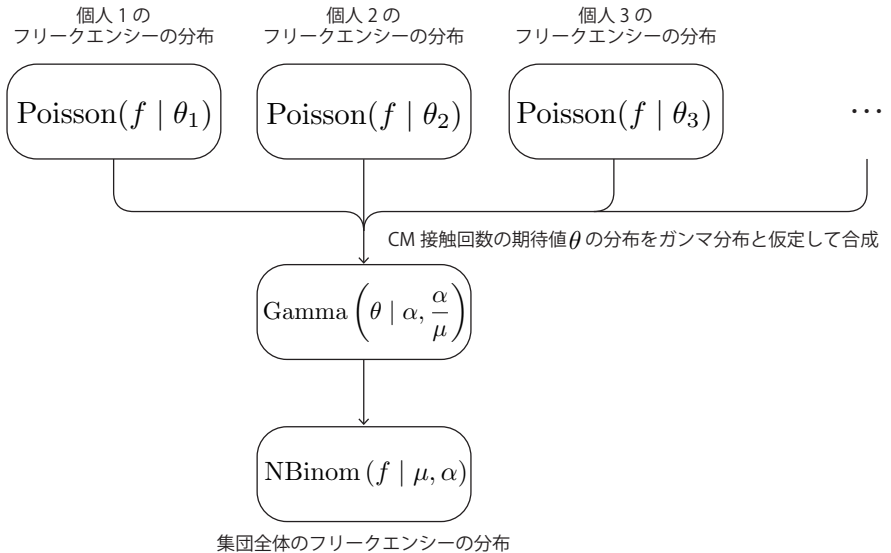
$$\begin{aligned}
& \Pr(F = f \mid \mu, \alpha) \\
&= \text{NBinom}(f \mid \mu, \alpha)
\end{aligned}$$

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

$$= \frac{\Gamma(\alpha + f)}{\Gamma(f + 1)\Gamma(\alpha)} \left(\frac{\alpha}{\mu + \alpha}\right)^\alpha \left(\frac{\mu}{\mu + \alpha}\right)^f$$

ポアソン分布と同じく、負の二項分布の期待値は μ なので、グロスリーチが与えられた場合は $\mu = g$ として扱います。ポアソン分布にはパラメータが θ の一つしかなく、 θ が期待値と分散の両方に対応していました。一方で、負の二項分布は、期待値を表すパラメータ μ だけでなく、分布の密集度を定めるパラメータ α をもちます。 α の値を調整することで、ポアソン分布よりも柔軟に CM 接触回数の分布を近似することができます。

ガンマ分布を用いたポアソン分布の合成のイメージを図 2.11 にまとめました。



■ 図 2.11: ガンマ分布を用いたポアソン分布の合成イメージ

それでは、いくつかのパラメータで負の二項分布を可視化してみましょう。例によって、負の二項分布の確率関数は SciPy の stats モジュールに `nbinom.pdf` メソッドとして用意されています。本書のパラメータ名とメソッドの引数名が異なることに注意してください。引数 `k` が CM 接触回数 f に、引数 `p` が $\alpha/(\mu + \alpha)$ に、引数 `n` が α に対応します。

2 章 マーケティングにデータサイエンスを導入する

```

In[21]: from scipy.stats import nbinom

def plot_nbinom(
    f: np.ndarray,
    mu_list: list[float],
    alpha_list: list[float]
) -> tuple[Figure, Axes]:
    """負の二項分布を可視化"""

    fig, ax = plt.subplots()

    markers = ["o", "D", "^", "x"]
    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, (mu, alpha) in enumerate(zip(mu_list, alpha_list)):
        y = nbinom.pmf(
            k=f, p=alpha / (mu + alpha), n=alpha
        ) # 負の二項分布の確率関数
        ax.plot(
            f,
            y,
            marker=markers[i % len(markers)],
            linestyle=linestyle[i % len(linestyle)],
            label=f"NBinom$(f \mid \mu = \{mu\}, \alpha = \{alpha\})$",
        )

    ax.legend()
    ax.set_ylim(0, None)
    fig.suptitle("負の二項分布の例")

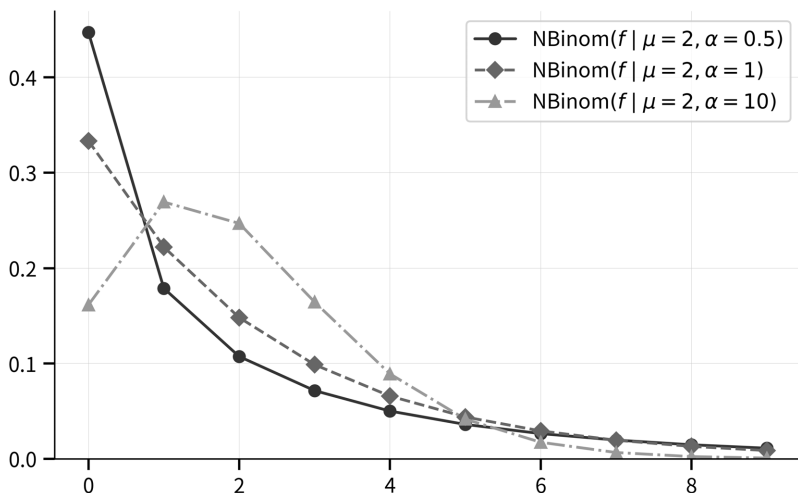
    return fig, ax

# 負の二項分布を可視化
fig, ax = plot_nbinom(
    f=np.arange(10),
    mu_list=[2, 2, 2],
    alpha_list=[0.5, 1, 10]
)

```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

負の二項分布の例



ガンマ分布の性質を引き継いで、パラメータ α の値が小さい場合は CM 接触回数が多くても少ない人もいること、 α の値が大きくなるにつれて接触回数の分布が期待値 $\mu = 2$ の周りに密集していくことが見てとれます*²³。

負の二項分布によるリーチカーブの推定

いよいよ本題に入りましょう。負の二項分布を用いてリーチカーブを推定してみましょう。負の二項分布の定義から、リーチカーブは以下で導出できます。

$$\begin{aligned}
 r(g; \alpha) &= 1 - \Pr(F = 0 \mid \mu = g, \alpha) \\
 &= 1 - \frac{\Gamma(\alpha + 0)}{\Gamma(0 + 1) \Gamma(\alpha)} \left(\frac{\alpha}{g + \alpha} \right)^\alpha \left(\frac{g}{g + \alpha} \right)^0 \\
 &= 1 - \left(\frac{\alpha}{g + \alpha} \right)^\alpha
 \end{aligned}$$

- 1 行目の変形：ユニークリーチは CM に 1 回以上接触した個人の割合なので、非接触者の割合を 100% から引くことで求める
- 2 行目への変形：負の二項分布の定義を代入

* 23 なお、 $\alpha \rightarrow \infty$ の場合、負の二項分布はポアソン分布と一致します。 θ の分布であるガンマ分布が期待値 μ の周りに限りなく密集することが理由です。

2 章 マーケティングにデータサイエンスを導入する

- 3 行目への変形： $\Gamma(1) = 0! = 1$ であることに注意しながら、数式を整理した

実際に、 $\alpha = 0.5, 1, 10$ の場合でリーチカーブを可視化してみましょう。

```
In[22]: def predict_reach_nbinom(
        g: np.ndarray | pl.Series, alpha: float
    ) -> np.ndarray:
        """負の二項分布を仮定したリーチカーブ"""

        return 1 - nbinom.pmf(k=0, p=alpha / (g + alpha), n=alpha)

def plot_reach_curve_with_predictions(
    df_reach: pl.DataFrame,
    df_reach_observed: pl.DataFrame,
    r_preds: list[np.ndarray],
    prediction_labels: list[str],
    title: str,
) -> tuple[Figure, Axes]:
    """負の二項分布を仮定したリーチカーブを複数描く"""

    fig, ax = plot_reach_curve(
        df_reach, df_reach_observed, title
    )

    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, (r_pred, prediction_label) in enumerate(
        zip(r_preds, prediction_labels)
    ):
        ax.plot(
            df_reach["g"],
            r_pred,
            linestyle=linestyle[i % len(linestyle)],
            label=prediction_label,
        )

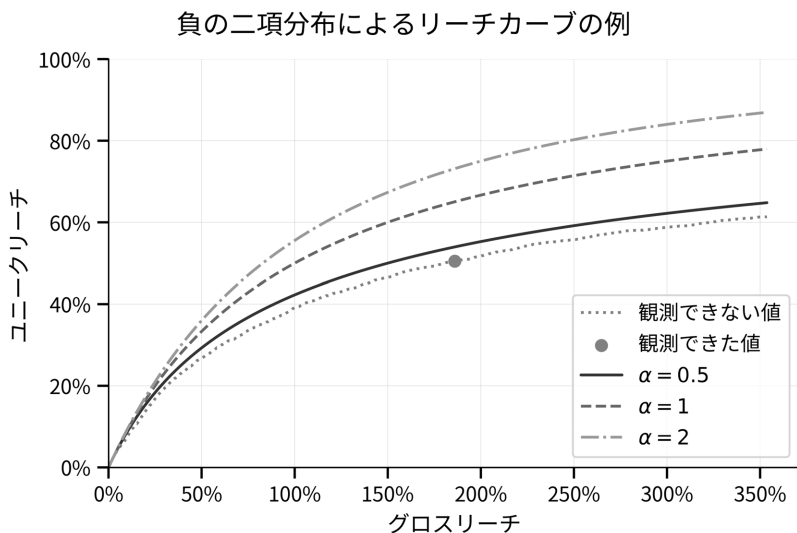
    ax.legend()

    return fig, ax

# 負の二項分布を仮定したリーチカーブの可視化
alpha_list = [0.5, 1, 2]
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
fig, ax = plot_reach_curve_with_predictions(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_preds=[
        predict_reach_nbinom(df_reach["g"], alpha)
        for alpha in alpha_list
    ],
    prediction_labels=[
        f"$\\alpha = {alpha}$" for alpha in alpha_list
    ],
    title="負の二項分布によるリーチカーブの例",
)
```



ポアソン分布を仮定すると、決まった一つのリーチカーブしか引くことができませんでしたが、負の二項分布ではパラメータ α の値に応じてさまざまなリーチカーブを引くことができます。特に、 $\alpha = 0.5$ の場合は比較的实际のリーチカーブに近い線が引けていることが見てとれます。

先ほどは α を 0.5, 1, 10 で決め打ちしましたが、最も当てはまりが良さそうな α をデータから推定してみましょう。今回の問題設定では、手に入っているデータは調査時点のグロスリーチとユニークリーチのみです。調査時点のグロスリーチを g' 、ユニークリーチを r' とします。最適なパラメータ $\hat{\alpha}$ は、グロスリーチ g' が与

2 章 マーケティングにデータサイエンスを導入する

えられた下で観測されたユニークリーチ r' との誤差を最小化すればいいので、以下で計算できます。

$$\begin{aligned}\hat{\alpha} &= \arg \min_{\alpha > 0} (r' - r(g'; \alpha))^2 \\ &= \arg \min_{\alpha > 0} \left(r' - \left(1 - \left(\frac{\alpha}{g' + \alpha} \right)^\alpha \right) \right)^2\end{aligned}$$

ここで、2 行目への変形は、導出したリーチカーブの定義を代入しています。

それでは、実際に最適なパラメータ $\hat{\alpha}$ を計算してみましょう。誤差の最小化には SciPy の `optimize` モジュールにある `minimize_scalar` 関数を使います。最小化したい関数を引数 `fun` に、探索する範囲を引数 `bounds` に与えます。戻り値の `x` に最適な $\hat{\alpha}$ が格納されています。

```
In[23]: from scipy.optimize import minimize_scalar
        from sklearn.metrics import mean_squared_error

def estimate_optimal_alpha(
    g: np.ndarray | pl.Series, r: np.ndarray | pl.Series
) -> float:
    """データに最も当てはまるalphaを求める"""

    return minimize_scalar(
        fun=lambda alpha: mean_squared_error(
            r, predict_reach_nbinom(g=g, alpha=alpha)
        ), # 最小化したい関数
        bounds=(0.01, 100), # 推定範囲。alphaは0より大きい
    ).x

# 最適なalphaの推定
alpha_hat = estimate_optimal_alpha(
    g=df_reach_observed["g"], r=df_reach_observed["r"]
)
print(f"alpha_hat = {alpha_hat:.2f}")
```

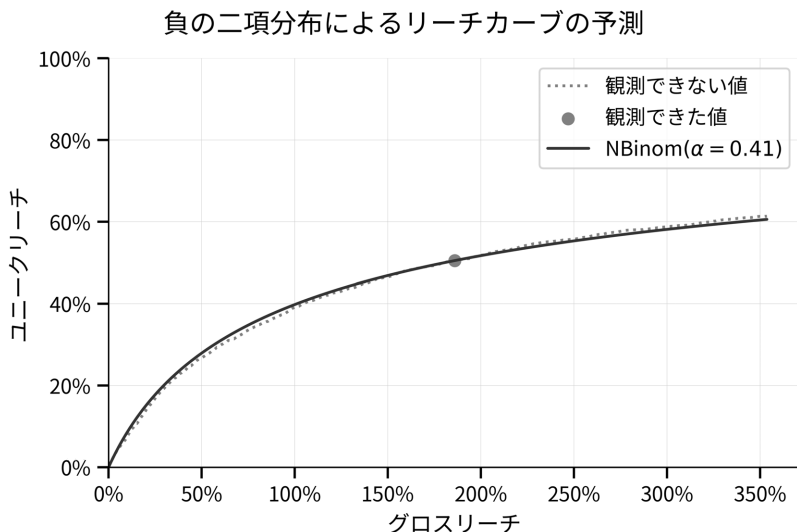
```
Out[23]: alpha_hat = 0.41
```

データに最も当てはまるのは $\hat{\alpha} = 0.41$ でした。この $\hat{\alpha}$ を使ってリーチカーブを可視化してみましょう。

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
In[24]: # 負の二項分布を用いたリーチカーブ
r_pred_nbinom = predict_reach_nbinom(
    g=df_reach["g"], alpha=alpha_hat
)

# 推定したリーチカーブを可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach,
    df_reach_observed=df_reach_observed,
    r_pred=r_pred_nbinom,
    prediction_label=f"NBinom$(\\alpha = {alpha_hat:.2f})$",
    title="負の二項分布によるリーチカーブの予測",
)
```



利用したデータは1点しかないにもかかわらず、リーチカーブ全体をうまく推定できていることがわかります。

これが数理モデルの力です。データの背後に存在するメカニズムをうまくモデリングすることができれば、たとえデータから得られる情報が不足していても信頼性の高い推定を行うことができます。

2 章 マーケティングにデータサイエンスを導入する

リーチカーブをうまく推定することができたので、元の課題だった利潤最大化問題を解くことができるようになりました。すぐにステップ3に移ってもよいのですが、その前に数理モデルを構築したことのさらなる利点も紹介しておきます。

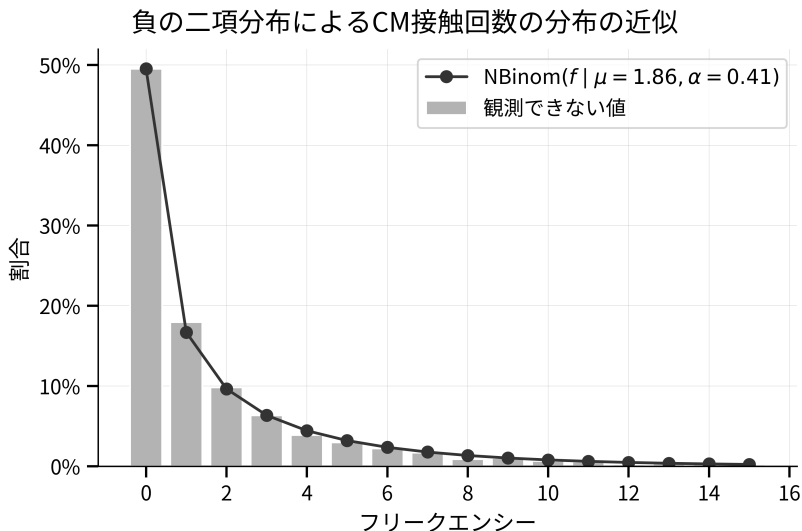
負の二項分布による CM 接触回数の分布の推定

数理モデルを構築したことのさらなる利点は、リーチカーブだけでなく CM 接触回数の分布そのものを推定できることです。これを確認するために、負の二項分布で CM 接触回数の分布を近似できているかを可視化しましょう。

```
In[25]: # 負の二項分布を用いたCM接触回数の分布
prop_nbinom = nbinom.pmf(
    k=range(f_max + 1),
    p=alpha_hat / (df_reach_observed["g"] + alpha_hat),
    n=alpha_hat
)

# CM接触回数の分布と負の二項分布を重ね合わせて可視化
fig, ax = plot_histogram_with_distribution(
    df_frequency_observed=df_frequency_observed,
    prop_pred=prop_nbinom,
    prop_label=f"NBinom$(f \\mid \\mu = \\\n    {df_reach_observed[\"g\"].item(): .2f}, \\alpha = \\\n    {alpha_hat: .2f})$",
    f_max=f_max,
    title="負の二項分布によるCM接触回数の分布の近似",
)
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



負の二項分布は CM 接触回数の分布をうまく近似できていることがわかります。グロスリーチ（期待値）とユニークリーチ（1 回以上接触の確率）の情報しか使っていないにもかかわらず、CM 接触回数の分布をうまく近似することに成功しています。これは注目すべき結果と言えるでしょう。

CM 接触回数の分布そのものを推定できたので、1 回以上の接触で定義したリーチカーブだけでなく、複数回の接触で定義したリーチカーブも推定できることを次に示します。

負の二項分布によるリーチ $f+$ の推定

これまで、CM 接触に関する指標として、グロスリーチとユニークリーチだけを考えてきました。ユニークリーチは少なくとも 1 回接触した人の割合です。ただし、CM に 1 回だけ接触したからといって、必ずしも製品・サービスを認知してくれるとは限りません。そこで、CM を 1 回だけ見てもらうのではなく、CM を 3 回以上見てもらいたいと考える場面も出てきます。このような、「 f 回以上 CM を見た人の割合」はリーチ $f+$ と呼ばれています。ユニークリーチはリーチ $f+$ の特殊ケースで、リーチ $1+$ と呼ぶこともあります。

前述のように、負の二項分布で CM 接触回数の分布全体を近似できているので、そこからリーチ $f+$ も推定することができます。

2 章 マーケティングにデータサイエンスを導入する

$$\begin{aligned}
 r_{f+}(g; \alpha) &= \Pr(F \geq f \mid \mu = g, \alpha) \\
 &= 1 - \Pr(F \leq f - 1 \mid \mu = g, \alpha) \\
 &= 1 - \sum_{f'=0}^{f-1} \frac{\Gamma(\alpha + f')}{\Gamma(f' + 1) \Gamma(\alpha)} \left(\frac{\alpha}{g + \alpha} \right)^{\alpha} \left(\frac{g}{g + \alpha} \right)^{f'}
 \end{aligned}$$

実際にリーチ $f+$ を計算してみましょう。具体的には、リーチ 1+, リーチ 3+, リーチ 10+ を計算します。負の二項分布の累積分布関数は `nbinom.cdf` メソッドで実装されています。引数の扱いは `nbinom.pmf` メソッドと同様です。

```
In[26]: def predict_reach_f_plus_nbinom(
        g: np.ndarray | pl.Series, alpha: float, f_plus: int
    ) -> np.ndarray:
        """リーチf+の計算"""

        return 1 - nbinom.cdf(
            f_plus - 1, p=alpha / (g + alpha), n=alpha
        ) # 負の二項分布の累積分布関数

def plot_reach_f_plus_curves_nbinom(
    g: np.ndarray | pl.Series,
    alpha: float,
    f_plus_list: list[int]
) -> tuple[Figure, Axes]:
    """リーチf+の可視化"""

    fig, ax = plt.subplots()

    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, f_plus in enumerate(f_plus_list):
        reach_f_plus = predict_reach_f_plus_nbinom(
            g=g, alpha=alpha, f_plus=f_plus
        )
        ax.plot(
            g,
            reach_f_plus,
            linestyle=linestyle[i % len(linestyle)],
            label=f"R{f_plus}+",
        )

    ax.legend()
```

2.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

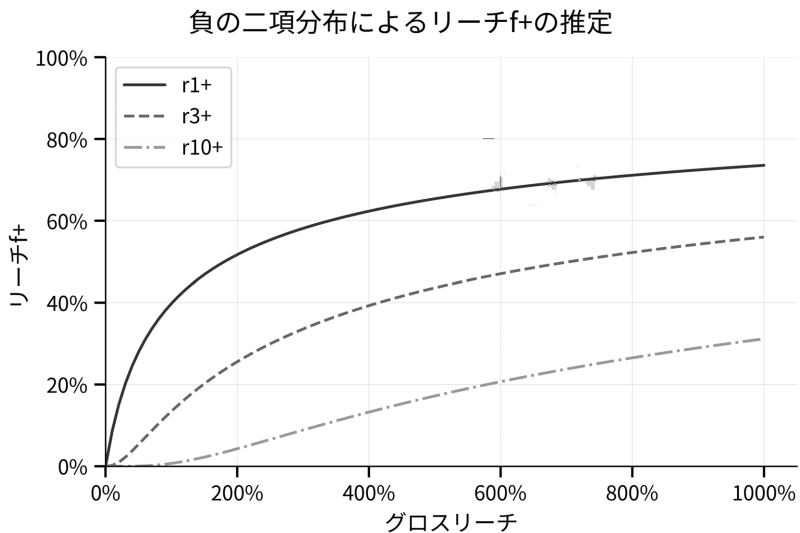
```

ax.xaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.yaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.set_xlim(0, None)
ax.set_ylim(0, 1)
ax.set_xlabel("グロスリーチ")
ax.set_ylabel("リーチf+")
fig.suptitle("負の二項分布によるリーチf+の推定")

return fig, ax

# リーチf+の可視化
fig, ax = plot_reach_f_plus_curves_nbinom(
    g=np.linspace(0, 10, 100),
    alpha=alpha_hat,
    f_plus_list=[1, 3, 10]
)

```



2 章 マーケティングにデータサイエンスを導入する

まず、リーチ 1+は最初に大きく伸びて徐々に伸び方が鈍くなっていきますが、リーチ 3+とリーチ 10+はそうではありません。次に、リーチ 3+を見ると、最初の伸びは緩やかで、そこから徐々に伸び始め、最終的にまた伸び方が鈍くなっていきます。最後に、リーチ 10+はその傾向がさらに強くなっており、グロスリーチ 1,000%の時点では、まだ伸び方は鈍くなっていないように見えます。このように、リーチ $f+$ は CM に f 回接触する個人が出てくるまでは増加しないので、最初はリーチが伸びないということが起きます。

リーチカーブを単純な予測モデルで近似するだけでは、このようにリーチ $f+$ のカーブを推定することはできませんが、データの背後の構造に対して数理モデルを構築すると、このような派生分析も可能になります。これも数理モデルを用いる利点の一つと言えるでしょう。

2.4

ステップ3：数理最適化問題を解いて最適なアクションを導出する

ステップ2では最適化問題の未知の部分をデータの力を借りて推定することができました。未知の部分がなくなった最適化問題を解くことで、最適な意思決定を導くことができます。学習した CM 出稿量とユニークリーチの関係を代入すると、最適化問題は以下のように定式化できます。

$$\begin{aligned} \max_g \quad & \bar{s}qn \left(1 - \left(\frac{\hat{\alpha}}{g + \hat{\alpha}} \right)^{\hat{\alpha}} \right) - 100cg \\ \text{s.t.} \quad & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

仮に、ターゲットセグメント n が 1,000 万人で、CM 接触者の 1%が製品を購入し ($q = 0.01$)、一人あたりの平均売上が $\bar{s} = 0.2$ (万円) だったとしましょう。また、グロスリーチ 1%あたりの CM 出稿コストが $c = 10$ (万円)、トータルの予算上限が $b = 5000$ (万円) だったとすると、解くべき問題は

$$\begin{aligned} \max_g \quad & 0.2 \times 0.01 \times 10^7 \times \left(1 - \left(\frac{0.41}{g + 0.41} \right)^{0.41} \right) - 100 \times 10 \times g \\ \text{s.t.} \quad & 100 \times 10 \times g \leq 5000 \\ & g \geq 0 \end{aligned}$$

2.4 ステップ3：数理最適化問題を解いて最適なアクションを導出する

です。

この場合の最適な出稿量を計算してみましょう。ここでは、解析的に、または数値計算で解を求めるのではなく、各グロスリーチに対して利潤を計算したものを網羅的に可視化することで最適解を求めることにします。

予算上限が5,000万円、グロスリーチ1%あたりのCM出稿コストが10万円であることから、グロスリーチが0%から500%までの範囲で利潤を計算していきます。

```
In[27]: def plot_optimal_g(
        s: float,
        q: float,
        n: int,
        c: float,
        b: float,
        alpha: float
    ) -> tuple[Figure, Axes]:
    g = np.linspace(0, b / (100 * c), 1001)
    r = predict_reach_nbinom(g=g, alpha=alpha)
    p = s * q * n * r - 100 * c * g

    max_index = np.argmax(p) # 利潤を最大化するインデックス
    # そのときのグロスリーチ、ユニークリーチ、利潤
    optimal_g = g[max_index]
    optimal_r = r[max_index]
    optimal_p = p[max_index]

    fig, ax = plt.subplots()
    ax.plot(g, p, c=".7", zorder=1)
    ax.scatter(
        optimal_g,
        optimal_p,
        label=f"最適点({optimal_g * 100: .0f}%, {optimal_p: .0f}万円)",
        zorder=2,
    )
    ax.legend()
    ax.xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set(
        xlabel="グロスリーチ（出稿量）",
        ylabel="利潤（万円）"
    )
```

2 章 マーケティングにデータサイエンスを導入する

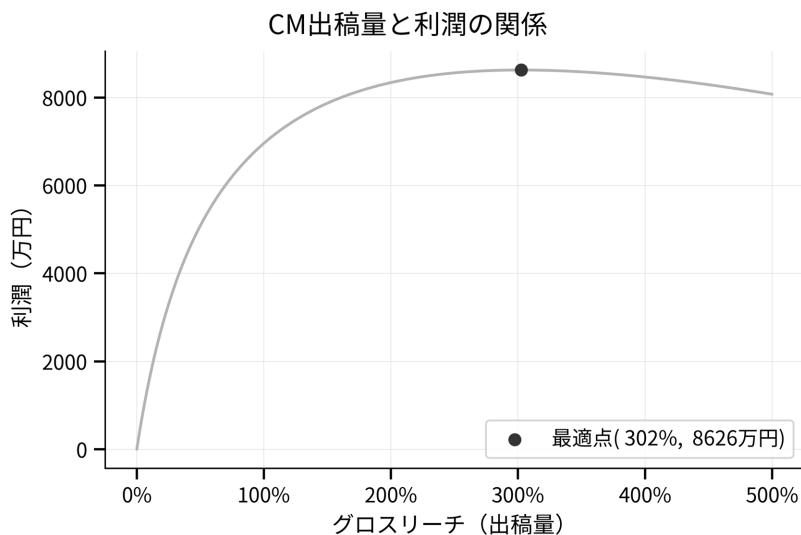
```
fig.suptitle("CM出稿量と利潤の関係")

print(f"optimal_g: {optimal_g: .2f}")
print(f"optimal_r: {optimal_r: .2f}")
print(f"optimal_p: {optimal_p: .2f}")

return fig, ax

# CM出稿量と利潤の関係を可視化
fig, ax = plot_optimal_g(
    s=0.2,
    q=0.01,
    n=1000 * 10000,
    c=10,
    b=5000,
    alpha=alpha_hat
)
```

```
Out[27]: optimal_g: 3.02
         optimal_r: 0.58
         optimal_p: 8626.44
```



結果を見ると、グロスリーチ 302%までは CM 出稿を増やすほど利潤が増加しますが、それ以上増やしても利潤は減少することが見てとれます。よって、CM をグロスリーチ 302%分だけ出稿して、58%のユニークリーチを達成し、8,626 万円の利潤を獲得することが最適なアクションであることがわかりました。こうして得られた知見をもとに、実際のマーケティング戦略を立案することができます。

もちろん、実務においては、このように数理最適化で求めた最適解をそのまま実行するのは現実的ではない場面も多々あります。戦略の決定には多角的な視点が必要です。数理最適化の結果だけを見て判断するのではなく、データ分析はあくまで判断に数理的な視点を加えるためのツールであるということを忘れないようにしましょう。

2.5 本章のまとめ

本章では、「テレビ CM の出稿量を最適化したい」というビジネス課題を題材として、3 ステップフレームワークを用いて課題解決に取り組みました。

まず、ステップ 1 では、数理最適化問題の定式化を通じて、最適化したい指標、制約条件、とりうるアクションを明確化しました。具体的には、最大化したい指標を利潤、制約条件を予算制約、とりうるアクションを CM 出稿量とすることで、ビジネス課題を制約付き利潤最大化問題

$$\begin{aligned} \max_g \quad & \bar{s} q n \Pr(R = 1 \mid g) - 100cg \\ \text{s.t.} \quad & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

として定式化し、データサイエンスで扱える問題に変換しました。

ステップ 1 で定式化した数理最適化問題には未知の部分としてグロスリーチとユニークリーチの関係 $\Pr(R = 1 \mid g)$ が含まれていました。そこで、ステップ 2 ではこの未知の部分データをから推定しました。特にデータが不足している状況を想定して、複雑な機械学習モデルや単純な線形回帰モデルではうまく予測ができないことを示し、ドメイン知識を反映させた数理モデルを用いた推定の有用性を示しました。具体的には、CM 接触回数の分布を負の二項分布で近似し、そこからリーチカーブ

2 章 マーケティングにデータサイエンスを導入する

$$r(g; \alpha) = 1 - \left(\frac{\alpha}{g + \alpha} \right)^\alpha$$

を導出することで、グロスリーチとユニークリーチの関係を推定しました。

最後に、ステップ 3 では、ステップ 2 で推定したリーチカーブを代入した数理最適化問題

$$\begin{aligned} \max_g \quad & \bar{s}qn \left(1 - \left(\frac{\hat{\alpha}}{g + \hat{\alpha}} \right)^{\hat{\alpha}} \right) - 100cg \\ \text{s.t.} \quad & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

を解くことで、利潤を最大化する最適な CM 出稿量を求めました。

このように、3 ステップフレームワークを用いることで、ビジネス課題を数理最適化問題として定式化し、データを用いて未知の部分を推定し、最適化なアクションを導くことができます。

本章では、特にステップ 2 に多くの紙面を割きました。今回の事例のように、利用可能なデータが限定的な状況では、機械学習モデルや線形回帰モデルを単純に適用しても予測がうまくいくことは少なく、結果として意思決定の質は低くなります。このような情報不足の状況でこそ、分析者がもつドメイン知識や、現象の背後にあるメカニズムに対する深い洞察を、積極的にモデルの構造に組み込むアプローチが有効です。そして、数理モデリングは、分析者の仮説を反映するための柔軟な分析手法です。リーチカーブの背後にある個人の CM 接触行動に関する仮説を、ポアソン分布やガンマ分布といった確率分布を用いた数理モデルとして具体的に記述し、そのモデルを通じてデータを解釈することで、たとえば手元のデータが乏しくとも、より現実在即し、解釈性の高い推論を行うことが可能となります。

一方で、分析者の仮説をモデルに取り込むということは、仮説が間違っていた場合、つまり想定した数理モデルが現実をうまく反映できていなかった場合は、正しい推論ができないことには注意が必要です。数理モデルに分析者の仮説を反映させるということは、もしその仮説が現実から大きく乖離していた場合、導き出される結論もまた現実とはかけ離れたものになってしまうというリスクを内包しています。したがって、数理モデルを用いた意思決定を行う際には、論理的な妥当性、ドメイン知識との整合性、(手に入る) データからのズレなど、複数の視点から仮説を検証する姿勢を忘れないようにしましょう。

今回構築した数理モデルは課題を単純化したもので、例えば、実務上は以下のような論点も考えられます。

- グロスリーチとユニークリーチの関係は、CM 出稿期間の前半と後半で一定とみなしてよいのか？ 例えば、前半ではさまざまな番組に CM を出稿し、後半では少数の番組に CM を出稿していた場合、今回の分析は成り立つのか？
- CM 出稿期間の長さは結果に影響するか？ 例えば、CM 出稿期間が短い場合と長い場合では、グロスリーチとユニークリーチの関係は変わってくるのか？ 短期間に大量出稿すると、同じ人ばかりが CM 接触してしまい、リーチが伸びないかもしれない
- 負の二項分布を用いたリーチカーブでは、CM 出稿量を無限大に近づけるとユニークリーチが 100%になると仮定しているが、そもそもテレビを見ない人も存在する。テレビ保有率を利用してユニークリーチを補正する必要があるかもしれない
- 前回のキャンペーンの結果を利用して次回キャンペーンの意思決定を行おうとしているが、そもそも前回キャンペーンのグロスリーチとユニークリーチの関係は次回も成り立つのか？ 誤差があるとすればどの程度か？

とはいえ、すべての要素を考慮した完全な数理モデルを作ることはできません。まずは重要な要素に的を絞って、小さくて扱いやすい数理モデルを構築しましょう。それをベースに、実務の条件に合わせて微調整を繰り返すのが、データサイエンスによる実践的な課題解決と筆者は考えています。

今回の課題ではグロスリーチとユニークリーチの関係性に焦点を当てましたが、マーケティングファネルの例で見たように、単に CM が「届いたか」だけでなく、それが「記憶に残ったか」という、より成果に近い KPI の分析も重要です。このような CM 認知率のモデリングは、ユニークリーチのモデリング以上の複雑さをもっています。ユニークリーチは一度でも CM に接触したかどうかだけが重要でしたが、CM 認知率は「CM への接触回数が増えるほど、その内容を認知する確率が高まる」という、接触回数に応じた効果の積み重ねを考慮に入れる必要があります。

次章では、さらに複雑な CM 認知率のモデリングにおいても、数理モデルが力を発揮することを見ていきます。

コラム：シミュレーションによる数理モデルの検証

モデル検証の重要性

2 章では、CM 接触回数の分布を負の二項分布で近似することで、リーチカーブの関数形を導出しました。今回のように数理モデルを分析者自身でカスタムして構築した場合、既存のモデルを流用するよりもミスが混入する可能性が高くなります。例えば、モデルの数式を変換していく際に計算ミスがあるかもしれませんし、理論は正しく構築できていても理論を実装に落とす際にコードにバグが含まれてしまうかもしれません。

よって、構築した数理モデルが想定通りの挙動をするか検証しておくことが重要です。そこで、本コラムではシミュレーションデータを用いて数理モデルとその実装を検証する方法を紹介します。

シミュレーションデータの設定

シミュレーションデータを用いたモデル検証のために、分析者が想定しているデータ生成過程を再現する必要があります^{*24}。今回の場合は、テレビをよく見る個人とそうではない個人がいる、つまり個人ごとに CM 接触確率が異なるということを想定しています。そこで、合計 n 人の個人に対して、個人 i の CM 接触確率を θ_i とし、 θ_i はベータ分布から独立に生成されるとします。

$$\theta_i \sim \text{Beta}(\alpha, \beta)$$

詳細は続く章で紹介しますが、ベータ分布は区間 $(0, 1)$ 上に定義される確率分布で、確率を生成する分布としてよく利用されます。なお、ベータ分布のパラメータは (α, β) の二つで、期待値は $\alpha/(\alpha + \beta)$ となります。

個人ごとの CM 接触確率が生成されたら、各個人が CM に接触するか否かをベルヌーイ分布から生成します。CM が複数回放送されたとして、個人 i が放送回 t の CM に接触したかどうかを R_{it} とします。

$$R_{it} \sim \text{Bernoulli}(\theta_i)$$

* 24 データ生成過程の具体化を分析者に強制することも、シミュレーションによる検証の利点の一つです。シミュレーションデータをうまく作れない場合、分析者がデータ生成過程をあいまいに考えていることに気づけます。

グロスリーチは延べ何回の CM 接触があったかを表す指標なので、放送回 s 時点までの累積グロスリーチ $\bar{G}_s$ は

$$\bar{G}_s = \frac{1}{n} \sum_{i=1}^n \sum_{t=1}^s R_{it}$$

で計算できます。

最後に、ユニークリーチは少なくとも一度は CM に接触した人の割合なので、放送回 s 時点までのユニークリーチ $\bar{R}_s$ は

$$\bar{R}_s = \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{t=1}^s (1 - R_{it}) \right)$$

で計算できます。CM に接触するまでは $\prod_{t=1}^s (1 - R_{it})$ はずっと 1、一度でも CM に接触するとそこからは 0 になるので、時点 s までに CM に接触したかどうかを $1 - \prod_{t=1}^s (1 - R_{it})$ で求めることができます。あとは全個人の平均をとることで s 時点までのユニークリーチを求めています。

この設定でシミュレーションデータを生成し、数理モデルを適用してリーチカーブを推定することで、数理モデルが正しく動作するかを検証します。

シミュレーションデータによる検証

それでは、シミュレーションデータによるモデルの検証を行いましょう。まずは設定に従ってシミュレーションデータを生成します。

集団の人数は $n = 5000$ 人とし、最終的な CM 放送回数は $t_{\text{total}}=100$ 回とします。また、ベータ分布のパラメータは $\alpha=1$ 、 $\beta=29$ とします。よって、平均的には 3.3%の確率で CM に接触するという設定になっています。

そして、 $t = 50$ 時点のグロスリーチとユニークリーチのみから、リーチカーブをうまく推定できるかを確認します。

```
In[28]: def generate_data(
        n: int, t_total: int, alpha: float, beta: float
    ) -> pl.DataFrame:
        """
        グロスリーチとユニークリーチのシミュレーションデータを生成する
        """

        # ベータ分布から個人ごとのCMへの接触確率を生成
        theta_i = np.random.beta(alpha, beta, n)
```

2 章 マーケティングにデータサイエンスを導入する

```
# 二項分布の試行回数を1にすることで、
# ベルヌーイ分布から各CM放送回への接触の有無を生成
r_it = np.random.binomial(1, theta_i, (t_total, n))

# グロスリーチは足すだけ
g_s = r_it.cumsum(0).mean(1)
# ユニークリーチは非接触にしてから積をとる
r_s = 1 - (1 - r_it).cumprod(0).mean(1)

# データフレームとしてまとめる
df_reach = pl.DataFrame(
    {"t": range(1, t_total + 1), "g": g_s, "r": r_s}
)

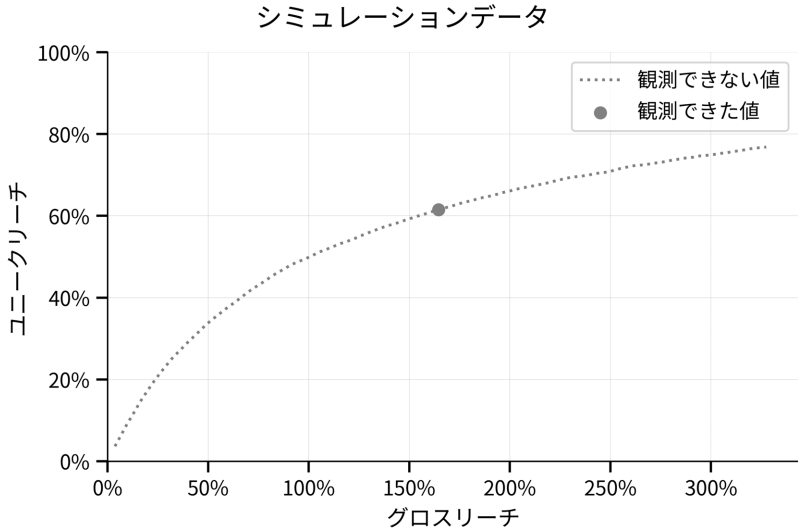
return df_reach

# シミュレーションデータを生成
df_reach_sim = generate_data(
    n = 5000, t_total = 100, alpha = 1, beta = 29
)

# 一時点のみを抽出
df_reach_sim_observed = df_reach_sim.filter(pl.col.t == 50)
```

生成されたリーチカーブを可視化しておきましょう。本編で見たリーチカーブと同様の形状が得られていることが確認できます。

```
In[29]: # 生成されたリーチカーブを可視化
fig, ax = plot_reach_curve(
    df_reach=df_reach_sim,
    df_reach_observed=df_reach_sim_observed,
    title="シミュレーションデータ",
)
```



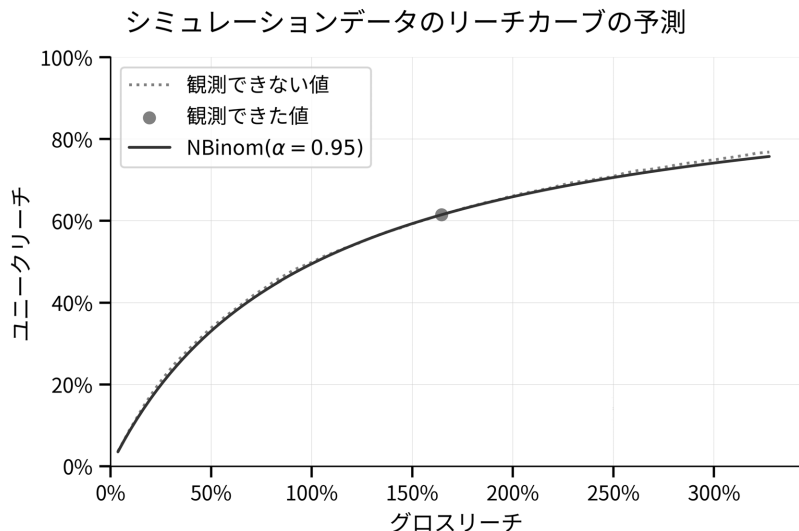
それでは、1点のみのデータを用いて負の二項分布から導出されたリーチカーブを推定し、予測結果を可視化しましょう。

```
In[30]: # 最適なalphaの推定
alpha_hat_sim = estimate_optimal_alpha(
    g=df_reach_sim_observed["g"], r=df_reach_sim_observed["r"]
)

# 負の二項分布を用いたリーチカーブ
r_pred_nbinom_sim = predict_reach_nbinom(
    g=df_reach_sim["g"], alpha=alpha_hat_sim
)

# 推定したリーチカーブを可視化
fig, ax = plot_reach_curve_with_prediction(
    df_reach=df_reach_sim,
    df_reach_observed=df_reach_sim_observed,
    r_pred=r_pred_nbinom_sim,
    prediction_label=f"NBinom$(\\alpha = \\alpha_hat_sim:.2f)$",
    title="シミュレーションデータのリーチカーブの予測",
)
```

2章 マーケティングにデータサイエンスを導入する



結果を見ると、リーチカーブを精度高く予測できていることが見てとれます。ここから、少なくともデータ生成過程が分析者の想定通りだった場合、数理モデルは正しく動作することが確認できました。

このように、シミュレーションデータを用いた検証を行うことで、分析時のミスを減らし、信頼性の高い分析結果を導くことができます。分析者自身でモデルをカスタマイズする際には、ぜひシミュレーションデータを用いた検証を行うことをおすすめします。

3章

認知形成を数理モデリングする

3章では、テレビCMの効果をより深く理解するため、製品認知率の数理モデリングに焦点を当てます。

2章では、広告が「どれだけ多くの人に届いたか」を示すユニークリーチを扱いました。これは広告効果の基本的な指標ですが、マーケティングの最終目標である購買行動につなげるためには、顧客がその製品を「知った」かどうか、つまり「認知」という、より顧客の購買行動に近い指標を分析することが重要になります。

3章では、このユニークリーチよりも複雑な認知を題材として、その背後にあるメカニズムに関するドメイン知識を具体的な数理モデルへと落とし込む方法を解説していきます。

3.1 ビジネス課題：製品認知率の改善

2 章では、「テレビ CM の出稿量最適化」という具体的なビジネス課題を取り上げ、データサイエンスの手法を用いてその解決に取り組むプロセスを解説しました。特に、グロスリーチとユニークリーチの関係性を表現する数理モデルを構築する中で、データが限られている状況においてこそ、分析者の仮説やドメイン知識をモデルに積極的に組み込むことの重要性を具体的に見てきました。

本章では、2 章で培ったこの数理モデリングの考え方をさらに一歩進め、テレビ CM の効果をより深く、そしてよりビジネスの最終成果に近い指標で捉えることを目指します。

2 章における分析の焦点は、CM の成果を「どれだけ多くの人に広告が届いたか」というユニークリーチの観点から評価することでした。ユニークリーチはマーケティング活動の基本的な成果指標の一つであることは間違いありません。しかし、マーケティングの最終的な目的が「売上を伸ばすこと」であるならば、広告が単に「届いた」だけでなく、その結果として顧客の態度や行動にどのような影響を与えたのか、という視点が不可欠です。そのために、2.1.1 節で示したように、テレビ CM のような認知型広告の重要な役割として、まだ製品を知らない潜在顧客に対して製品の存在を知らせ、興味や関心のきっかけを作ること、すなわち「製品を認知してもらうこと」があります。

そこで本章では、この「より多くの人に製品を認知してもらうこと」を新たなビジネス課題として設定します。この課題に対して、3 ステップフレームワークを用いて、具体的な解決策を導き出す方法を解説していきます。

3.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

「テレビ CM の出稿を通じて製品認知率を高めたい」というビジネス課題が与えられたとします。このビジネス課題をデータサイエンスの問題に落とし込みましょう。ビジネス課題を具体的なデータサイエンスの問題に変換するため、3 ステップフレームワークを適用します。

まずはステップ 1 です。ビジネス課題を数理最適化問題として定式化しましょう。

最初に決定変数を考えます。製品の認知率を改善するためにとれるアクションとして、大きくメディア戦略とクリエイティブ戦略の 2 種類が考えられます。

3.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

- メディア戦略：より多くのターゲットに適切な回数 CM を見てもらえるようにする
- クリエイティブ戦略：より製品のことが印象に残る CM を作成する

今回はメディア戦略に注力することにしましょう。つまり、印象に残りやすい CM の制作ではなく、より多くの人に CM を見てもらうことで製品認知を高める戦略をとります。メディア戦略としては、どの時間帯に CM を出稿するのか、どの放送局に CM を出稿するのかなどの細かい設定が考えられます。ここでは 2 章と同様に、テレビ CM の出稿形態としてはスポットを想定し、調整できる要素は CM の出稿量だけという状況を考えることにします。

そこで、CM 出稿量、つまりグロスリーチを g とします。費用は 2 章の設定と同じくグロスリーチ 1%あたりのコストが固定されているとし、 c で表します。このとき、テレビ CM の出稿金額は $100cg$ となります。グロスリーチ 1 が $g = 0.01$ なので 100 倍する必要があることに注意してください。広告出稿金額 $100cg$ を決定変数とすることもできますが、ここでは CM 出稿量 g を決定変数として考えることにします。

次に、目的関数、つまり最適化したい指標を考えます。「テレビ CM の出稿を通じて製品認知率を高めたい」という要望そのものには、いったいどの程度の製品認知率が望ましいのか、具体的な数値が含まれていません。そのため、ステークホルダーとの議論を通じて、目標の水準を決める必要があります。

ステークホルダーとの議論の結果、できるだけ少ない費用で最低 $\underline{a}$ の製品認知率を獲得することが目標になったとします。よって、最小化したい目的関数がテレビ出稿の費用で、制約条件が製品認知率を $\underline{a}$ 以上にすることになります。

制約条件を数式で表現するため、CM が認知されたかどうかを確率変数 A で表します。確率変数 A は CM が認知されていれば 1 をとり、認知されていなければ 0 をとります。CM 出稿量 g に対する製品認知の条件付き期待値を $\mathbb{E}[A | g]$ とします。なお、 A は 1 か 0 をとる確率変数なので、 $\mathbb{E}[A | g] = \Pr(A = 1 | g)$ です。 $\mathbb{E}[A | g]$ は、どのくらい CM を出稿すればどのくらいの製品認知率が獲得できるのかという、CM 出稿量と製品認知率の関係を表しています。この関係を認知カーブと呼ぶことにしましょう。

最後に、目標を達成するためなら際限なく費用をかけていいわけではないので、制約条件に予算上限 $\bar{b}$ を追加します。

以上をまとめると、ビジネス課題「テレビ CM の出稿を通じて製品認知率を高めたい」を、以下の制約付き費用最小化問題として定式化できます。

3 章 認知形成を数理モデリングする

$$\begin{aligned}
 \min_g \quad & 100cg \\
 \text{s.t.} \quad & \Pr(A = 1 \mid g) \geq \underline{a} \\
 & 100cg \leq b \\
 & g \geq 0
 \end{aligned}$$

あとは、データから条件付き認知率 $\Pr(A = 1 \mid g)$ を推定し、この最適化問題を解くことで元のビジネス課題を解決するアクションを知ることができます。

3.3

ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

3.3.1 求められるデータ

ステップ1で数理最適化問題の定式化ができました。この最適化問題を実際に解くためには、最適化問題における未知の要素、つまり条件付き認知率 $\Pr(A = 1 \mid g)$ をデータに基づいて推定する必要があります。

この条件付き認知確率 $\Pr(A = 1 \mid g)$ を精度よく推定するためには、さまざまなグロスリーチの値と、その各時点に対応する製品認知率がセットで観測されたデータが豊富にあることが理想的です。例えば、表3.1のように、CMキャンペーン期間中の異なる時点におけるグロスリーチと、そのときの製品認知率が同時に観測されているようなデータがあれば、両者の関係性を推定できそうです。

▼ 表 3.1: グロスリーチとユニークリーチの計算例

時点	グロスリーチ	製品認知率
2025 年 4 月 1 日	100%	10%
2025 年 4 月 8 日	200%	40%
2025 年 4 月 15 日	300%	60%
2025 年 4 月 22 日	400%	70%
2025 年 4 月 29 日	500%	75%
⋮	⋮	⋮

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

それでは、実際に表 3.1 のようなデータをどうやって取得すればよいでしょうか。まず、グロスリーチに関しては、2 章で利用したような CM 接触データをテレビ視聴データを専門とする調査会社に依頼して取得することが考えられます。CM 接触データを利用することで、ある時点までに累計でどの程度のグロスリーチが獲得できていたのかを把握できます。

さらに、分析のためには、グロスリーチだけでなく製品認知率のデータも必要です。時点ごとの製品認知率を知るためには、例えば、CM 放送期間に繰り返し製品認知率のアンケート調査を実施し、製品を認知していた人の割合を記録することが求められます。このとき、「ある製品を認知しているか」を質問すると、質問それ自体によって製品を認知してしまうため、同じ個人に対して複数回の認知調査を行うことは避け、毎時点で異なる個人をサンプリングして調査を行う必要があります。

このように CM 接触データとアンケート調査データを組み合わせることで、表 3.1 のようなデータを作成することができそうです。しかし、アンケート調査を高い頻度で繰り返し実施するには、相応の時間と費用がかかってしまいます。そのため、多くの企業では、投資対効果を考慮し、製品認知率の調査は CM キャンペーン終了直後の 1 回のみ、あるいは放送中に数回程度に留めることが一般的です。頻繁に詳細なデータを取得することは、コストの観点から難しいというのが実情です。

このようなビジネスの現場における現実的な制約を踏まえ、本章でも、データが限られている状況を想定した分析を行います。具体的には、あるキャンペーンに対して放送開始直後と放送終了直後の 2 時点でのみ製品認知率を調査したデータが得られた状況を扱います。この情報が限られた状況下でも、数理モデルの力を借りることで、ビジネス課題の解決に取り組む方法を解説します。

3.3.2 データの読み込みと可視化

それでは、利用するデータを確認していきましょう。まずは本章で利用するライブラリを読み込みます。その他のライブラリは、必要に応じて都度読み込んでいきます。

```
In[31]: import json # ファイル読み込み
        from typing import Self # 型ヒント

        import matplotlib.pyplot as plt # 可視化
        import numpy as np # データ処理
        import polars as pl # データ処理
        import seaborn as sns # 可視化
```

3 章 認知形成を数理モデリングする

```
from matplotlib.axes import Axes # 型ヒント
from matplotlib.figure import Figure # 型ヒント
from matplotlib.ticker import PercentFormatter # 可視化
```

```
In[32]: # 乱数を固定
np.random.seed(42)

# データの表示設定
pl.Config.set_float_precision(precision=3)

# 可視化の設定
with open("../setting/visualization.json") as f:
    setting = json.load(f)
sns.set_theme(**setting)
sns.set_palette("gray", 4)
```

次に、本章で利用するデータを読み込みます。

分析に利用するデータは、REVISIO 株式会社が保有する、あるテレビ CM に対する接触データです。約 4,500 人に対する CM 接触データからグロスリーチを集計してあります。加えて、過去に REVISIO 株式会社で実施した製品認知率の調査データを利用して、時点ごとの製品認知率のデータを人工的に作成しました*1。

今回は CM 放送開始時と放送終了時の 2 点しか調査データを得ていないという設定で分析を行います。分析の妥当性を確認するために、CM 放送中に 100 時点分の調査を行ったという設定のデータも用意しています。

実際には観測できない 100 時点のデータを `df_awareness`、実際に観測できた 2 時点のデータを `df_awareness_observed` として読み込みます。実際に推測に利用できるデータは `df_awareness_observed` のみなので、モデルの学習は `df_awareness_observed` のみで行います。

```
In[33]: # データの読み込み

# 100時点のデータ
df_awareness = pl.read_csv("../data/ch03/awareness.csv")

# 2時点のデータ
df_awareness_observed = pl.read_csv(
    "../data/ch03/awareness_observed.csv"
)
```

* 1 よって、本章で利用するデータは、純粋な実測データではなく、実測データをもとに作成した人工的なデータです。本章以外で利用するデータは、純粋な実測データです。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
# 観測できない100時点のデータの確認
df_awareness
```

Out[33]: shape: (100, 6)

t	g	r	a	n_awareness	n_total
i64	f64	f64	f64	i64	i64
1	0.032	0.032	0.487	975	2000
22	0.124	0.102	0.480	961	2000
37	0.211	0.146	0.510	1020	2000
62	0.314	0.189	0.524	1049	2000
75	0.407	0.224	0.545	1090	2000
...	...	...	...	...	...
1644	8.968	0.755	0.705	1409	2000
1662	9.075	0.756	0.719	1438	2000
1682	9.174	0.759	0.734	1468	2000
1696	9.267	0.761	0.737	1474	2000
1708	9.364	0.764	0.717	1434	2000

また、df_awareness には、過去のあるキャンペーンに対して 100 時点でグロスリーチとユニークリーチ、そして製品認知率を取得した結果が格納されています。時点ごとにその時点 t までのテレビ CM 出稿量をグロスリーチ g とし、ユニークリーチ r とともに記録しています。同時に、毎時点で 2,000 人に製品をしているかどうかのアンケート調査を行い、認知していた人数を n_awareness としてカウントし、認知していた割合を a として記録しています。

- t：何回目の CM 放送の時点か
- g：グロスリーチ
- r：ユニークリーチ
- a：製品認知率。n_awareness / n_total で計算
- n_awareness：製品を認知していた人数
- n_total：製品を認知しているかどうかを確認した人数

3 章 認知形成を数理モデリングする

次に、df_awareness_observed には、過去のあるキャンペーンに対して CM 放送開始時と放送終了時の 2 時点でグロスリーチと製品認知率を取得した結果が格納されています。

```
In[34]: # 観測できた2時点のみのデータを確認
df_awareness_observed
```

```
Out[34]: shape: (2, 6)
```

	t	g	r	a	n_awareness	n_total
	i64	f64	f64	f64	i64	i64
	1	0.032	0.032	0.487	975	2000
	1708	9.364	0.764	0.717	1434	2000

CM 放送開始時点のグロスリーチとユニークリーチはともに 3.2%で、このときの製品認知率は 48.7%です。まだほとんど CM を出稿していないので、約半数の個人は「今回のキャンペーンの CM 接触とは無関係にそもそも製品を認知していた」ということです。

さらに、可視化することでデータの傾向を確認しましょう。実際には観測できない認知カーブを点線で描画し、実際に観測できた CM 放送開始時と放送終了時のデータを点で描画します。

```
In[35]: def plot_awareness(
df_awareness: pl.DataFrame,
df_awareness_observed: pl.DataFrame,
title: str
) -> tuple[Figure, Axes]:
fig, ax = plt.subplots()

ax.plot(
df_awareness["g"],
df_awareness["a"],
linestyle="dotted",
c=".5",
label="観測できない値",
zorder=1,
)
ax.scatter(
df_awareness_observed["g"],
df_awareness_observed["a"],
```


3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

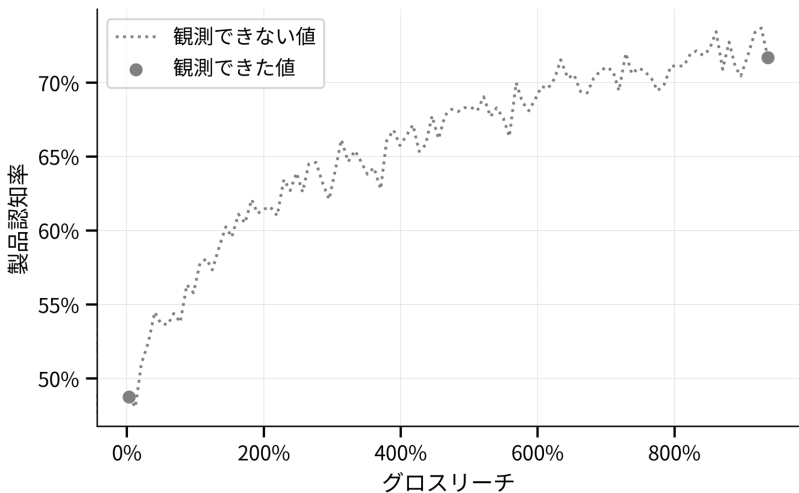
        c=".5",
        label="観測できた値",
        zorder=2,
    )
    ax.legend()
    ax.xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set(xlabel="グロスリーチ", ylabel="製品認知率")
    fig.suptitle(title)

    return fig, ax

fig, ax = plot_awareness(
    df_awareness=df_awareness,
    df_awareness_observed=df_awareness_observed,
    title="観測されない100時点のデータと観測された2時点のデータ",
)

```

観測されない100時点のデータと観測された2時点のデータ



3 章 認知形成を数理モデリングする

グロスリーチが増加するにつれて認知率は上がり、徐々に上がり方が緩やかになることが見てとれます。これは、CM 出稿量を増やしていくと、すでに製品を認知している人が CM を見る可能性が高まる一方、まだ製品を認知していない人に見てもらえる可能性は低くなるので、認知率改善の難易度が上がっていくことが影響していると考えられます。

いま、マーケティングの目標として 65%以上の製品認知を獲得したいとします。グラフのように認知率を細かく測定していれば、グロスリーチ 300%から 400%程度の CM 出稿を行えば目標を達成できそうなことがわかります。しかし、実際に観測できているのは CM 放送開始時と放送終了時の 2 時点のみです。2 時点間の製品認知率の推移は得られていないので、どの程度の CM 出稿を行えばよいのかあたりをつけることができません。

そこで、以降ではグロスリーチと製品認知率の関係 $\Pr(A = 1 | g)$ を推定することを目指します。この関係が推定できれば、目標達成に必要な CM 出稿量の水準を求めることができます。

3.3.3 機械学習モデルによる推定

データポイントが 2 点しかないので、機械学習モデルを単純に適用しても 2 章と同様にうまく予測ができないことが予想されますが、実際に確認しておきましょう。

モデルの学習と予測には scikit-learn を利用します。まずはデータを scikit-learn の入力形式に合わせて変形します。

```
In[36]: # scikit-learnの入力形式に合わせる

# 学習用データ。2点のみ
g_train = df_awareness_observed[["g"]].to_numpy()
r_train = df_awareness_observed["r"].to_numpy()
a_train = df_awareness_observed["a"].to_numpy()

# テスト用データ。観測されていないデータも含む全時点データ
g_test = df_awareness[["g"]].to_numpy()
```

機械学習モデルには Random Forest を利用します。scikit-learn の ensemble モジュールにある RandomForestRegressor クラスを利用して学習と予測を行います。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
In[37]: from sklearn.ensemble import RandomForestRegressor

# Random Forestによる認知カーブの推定
rf = RandomForestRegressor(n_jobs=-1, random_state=42)
rf.fit(g_train, a_train)
a_pred_rf = rf.predict(g_test)
```

Random Forest の予測結果を（観測できない）認知カーブと重ね合わせて可視化することで、予測の妥当性を確認します。

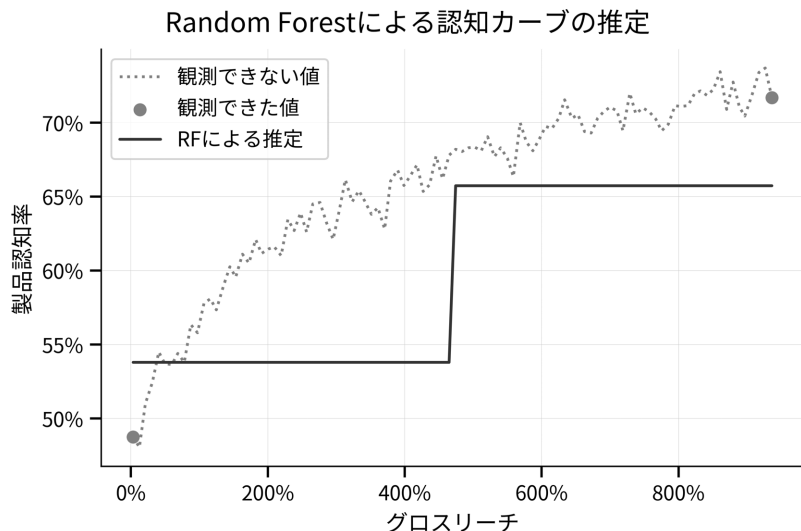
```
In[38]: def plot_awareness_with_prediction(
    df_awareness: pl.DataFrame,
    df_awareness_observed: pl.DataFrame,
    a_pred: np.ndarray,
    prediction_label: str,
    title: str,
) -> tuple[Figure, Axes]:
    """実測値に合わせてモデルによる予測値を可視化"""

    fig, ax = plot_awareness(
        df_awareness=df_awareness,
        df_awareness_observed=df_awareness_observed,
        title=title,
    )
    ax.plot(df_awareness["g"], a_pred, label=prediction_label)
    ax.legend()

    return fig, ax

# 可視化
fig, ax = plot_awareness_with_prediction(
    df_awareness=df_awareness,
    df_awareness_observed=df_awareness_observed,
    a_pred=a_pred_rf,
    prediction_label="RFによる推定",
    title="Random Forestによる認知カーブの推定",
)
```

3 章 認知形成を数理モデリングする



予想通り、Random Forest では 2 点のデータからうまく認知カーブを推定できませんでした。このように、観測されたデータポイントが少ない状況では、複雑な機械学習モデルは十分な力を発揮することができません。

3.3.4 線形回帰モデルによる推定

複雑な機械学習モデルではうまく認知カーブが推定できなかったので、次は単純な線形回帰モデルを検討しましょう。グロスリーチ g が増加するにつれて製品認知率が増加すること、その増加率は徐々に逓減していくことを表すため、グロスリーチの対数をとったモデルを考えます。

$$\Pr(A = 1 \mid g) = \alpha + \beta \log(g + 1)$$

CM 出稿が認知率に与える影響をパラメータ β で、CM 出稿がない場合の認知率をパラメータ α で表現しています。グロスリーチ g が 0 の際の認知率が α になるように、 $\log(g + 1)$ と 1 を足していることに注意してください。

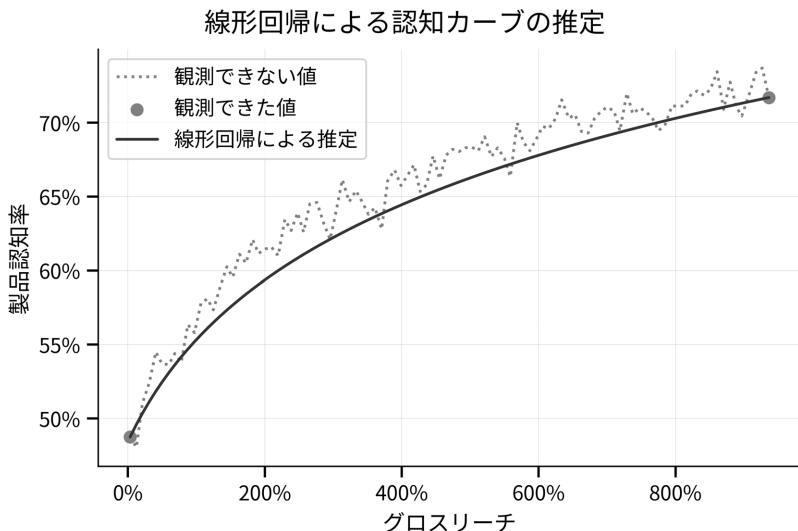
それでは、この線形回帰モデルの学習と予測を行いましょう。scikit-learn の `linear_model` から `LinearRegression` クラスを利用します。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
In[39]: from sklearn.linear_model import LinearRegression

# 対数変換した場合の認知カーブの推定
lr = LinearRegression()
lr.fit(np.log1p(g_train), a_train) #  $\log_1p(x) = \log(x + 1)$ 
a_pred_lr = lr.predict(np.log1p(g_test)) # 認知カーブの推定

# 可視化
fig, ax = plot_awareness_with_prediction(
    df_awareness=df_awareness,
    df_awareness_observed=df_awareness_observed,
    a_pred=a_pred_lr,
    prediction_label="線形回帰による推定",
    title="線形回帰による認知カーブの推定",
)
```



全体的に製品認知率を低めに推定しているとはいえ、たった2点のデータから推定を行ったことを考えると、線形回帰モデルは悪くない推定ができています。

一方で、モデルの妥当性には懸念があります。この線形回帰モデルでは、グロスリーチと認知率に関係を表現するために、グロスリーチに対数変換を施した上で線形関係を仮定しました。しかし、この対数変換という選択には、現象の背後にある

3 章 認知形成を数理モデリングする

メカニズムに基づいた明確な根拠があるわけではなく、「よくある変換手法」を適用したに過ぎません。

いま私たちは点線で表された「正解データ」を比較のために見ているので、この線形回帰モデルが「悪くない」ことを知っています。ただし、実際のビジネスの現場では、このような答え合わせはできず、手元の限られたデータと、モデルに置かれた仮定の妥当性で判断を下さなければなりません。

特に、今回のようにデータが限られている状況では、データへの当てはまり具合だけでモデルの良し悪しを判断するのは危険です。むしろ、「広告出稿が製品認知率に影響を与えるメカニズムをモデルはどの程度反映できているのか」、というロジックの現実性・妥当性が、モデルの信頼性を評価する上でより重要な判断基準となります。

この観点から見ると、単に対数変換を施しただけの線形モデルはドメイン知識に基づいた論理で支持されているわけではなく、仮定の根拠が十分とは言えません。より信頼性が高く、かつ実務での意思決定に資する推定を行うためには、広告出稿が製品認知につながるメカニズムに関するドメイン知識を、モデルの構造に反映させることが不可欠です。

そこで、これまでの議論を踏まえ、広告出稿が製品認知につながるメカニズムを反映した数理モデルを構築することで、より信頼性の高い推定を目指していきます。

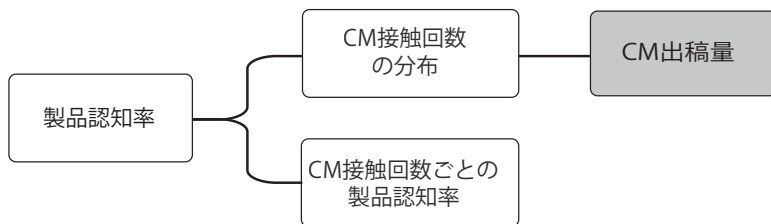
3.3.5 数理モデルによる問題の分解

それでは、数理モデルの力を借りることでグロスリーチと認知率の関係 $\Pr(A = 1 | g)$ のより良い推定を目指しましょう。そのためには、CM 出稿量と製品認知の関係性を表す数理モデルに対して、分析者の仮説を反映させる必要があります。

1 章で 3 ステップフレームワークを適用するための具体例として取り上げた「広告出稿の最適水準」に関する議論を思い出しましょう。CM 出稿量と認知の関係性を、CM 出稿量を増やすと広告を見る人数／見る回数が増え、広告を見る人数／見る回数が増えたと認知する人が増える、といった 2 段階のプロセスに分解しました(図 3.1)。

この仮説を数理モデルに落とし込み、「CM 出稿量→製品認知」といった直接的な関係を考えるのではなく、間に CM 接触回数 F を挟んだ「CM 出稿量→CM 接触回数→製品認知」という 2 段階のモデルを考えます。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 3.1: CM 出稿量と製品認知率の関係

1.4.2 節で議論したように、集団全体の製品認知率は、CM 接触回数ごとの製品認知率を、CM 接触回数の分布で重み付けて平均することで計算できます。よって、この 2 段階のモデルは以下の形で表現できます。

$$\Pr(A = 1 | g) = \sum_{f=0}^{\infty} \Pr(A = 1 | f) \Pr(F = f | g)$$

ここで、第一項の $\Pr(A = 1 | f)$ は CM 接触回数で条件付けた製品を認知する確率です。つまり、どのくらい広告に接触すればどのくらいの確率で製品を認知するのかを表しています。

次に、第二項の $\Pr(F = f | g)$ は CM 出稿量で条件付けた CM 接触回数が f 回になる確率です。つまり、広告 g だけ出稿した際に、広告に f 回接触する人がどのくらいの割合で出現するのかを表しています。

ところで、第二項の CM 出稿量 g と CM 接触回数の分布の関係性は、2 章で負の二項分布を用いてモデリングすることができています。

$$\begin{aligned} \Pr(F = f | g) &= \text{NBinom}(f | \mu = g, \alpha) \\ &= \frac{\Gamma(\alpha + f)}{\Gamma(f + 1) \Gamma(\alpha)} \left(\frac{\alpha}{g + \alpha} \right)^{\alpha} \left(\frac{g}{g + \alpha} \right)^f \end{aligned}$$

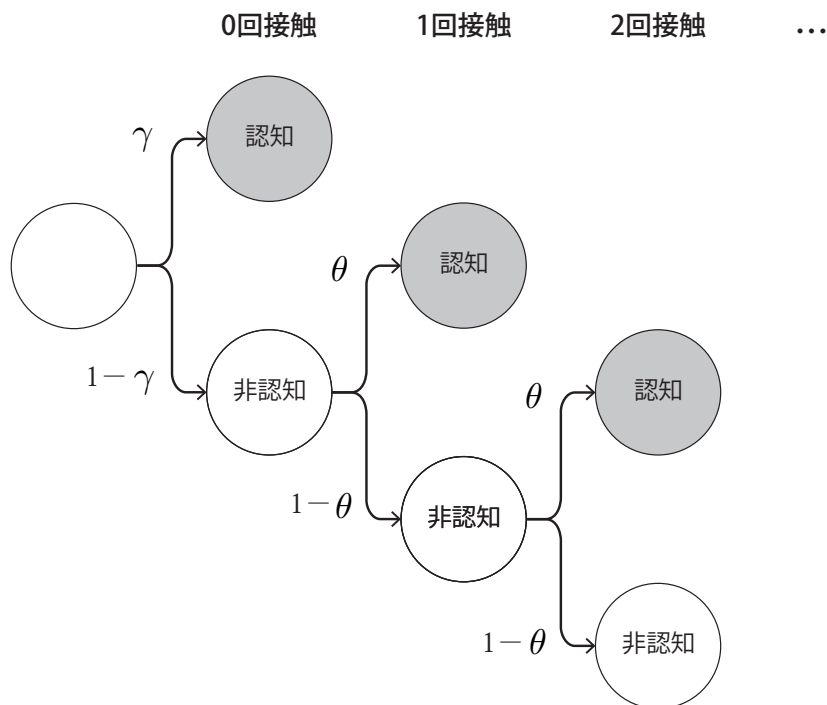
よって、以降では、まだモデリングできていない第一項の CM 接触確率と製品認知率の関係 $\Pr(A = 1 | f)$ について数理モデルを考えていきます。

3.3.6 CM 接触確率と製品認知率の関係

二つに分解された問題のうち、CM 接触と製品認知率の関係 $\Pr(A = 1 | f)$ について考えていきましょう。基本的には、1.4.2 節で議論したモデルを踏襲するので、詳細は 1.4.2 節を確認してください。

3 章 認知形成を数理モデリングする

妥当な数理モデルを構築するために、「背後のメカニズムはこうなっているはずだ」という分析者の信念をモデルに反映させていきます。具体的には、図 3.2 のような認知メカニズムを考えます。



■ 図 3.2: CM 接触と製品認知のメカニズム (再掲)

このメカニズムは、二つのパラメータ (γ, θ) をもっています。

まず、「CM 接触とは無関係に製品を認知している確率」を $\gamma \in [0, 1]$ とします。製品が元々有名な場合、CM に接触しなくても製品を認知している確率が高いと考えられるため、 γ は高い値になるでしょう。一方で、新規の製品であれば、 γ は低い値になります。CM に接触していない人 (0 回接触の人) が製品を認知している確率が γ なので、逆に製品を認知していない確率は $1-\gamma$ です。

次に、「CM に接触した際に製品を認知する確率」を $\theta \in [0, 1]$ とします。 θ は「CM 接触が製品を認知させる力」を表していると解釈できます。製品を強烈に印

3.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

象付ける CM であれば θ は大きな値をとり、逆に印象に残らないよくある CM であれば θ は小さな値をとります*。

この設定のもとで、CM に f 回接触した際の製品認知率は以下の数式で表現できます。

$$\Pr(A = 1 | f) = 1 - (1 - \gamma)(1 - \theta)^f$$

このモデルは、対数やロジスティック関数のような「よくある」関数形を使ったモデルとは異なり、認知形成に関する具体的なメカニズムが数理モデルに反映されています。メカニズムをモデルに反映させることによって、「広告なしでの認知率は100%」、「広告1回接触あたりの新規認知率は100%」といったように、パラメータが具体的な意味をもつため、モデルの解釈性や納得性が格段に高まっています。

それでは、構築した数理モデルの振る舞いを確認しておきましょう。仮に、 $\gamma = 0.5$ 、 $\theta = 0.3$ とした場合の CM 接触回数と製品認知率の関係を可視化します。

```
In[40]: def plot_awareness_frequency_relation(
        f: np.ndarray, a_pred: np.ndarray
    ) -> tuple[Figure, Axes]:
        fig, ax = plt.subplots()

        ax.plot(f, a_pred, linestyle="dotted", zorder=1)
        ax.scatter(f, a_pred, zorder=2)

        ax.yaxis.set_major_formatter(
            PercentFormatter(xmax=1, decimals=0)
        )
        ax.set(xlabel="CM接触回数", ylabel="認知率")
        fig.suptitle("CM接触回数と認知率の関係")

        return fig, ax

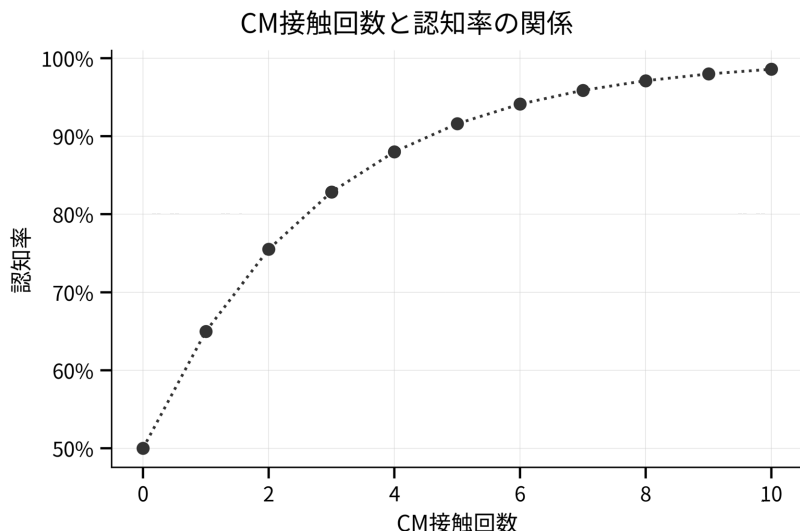
# パラメータの設定
f = np.arange(0, 11) # CM接触回数
gamma = 0.5 # 製品をそもそも知っている割合
theta = 0.3 # 製品認知率
a_pred = 1 - (1 - gamma) * (1 - theta) ** f # 認知率の推定

# 可視化
```

* 2 よって、このモデリングにおいては、クリエイティブ戦略をとることは、 θ の値を大きくする戦略をとることを意味します。

3 章 認知形成を数理モデリングする

```
fig, ax = plot_awareness_frequency_relation(f=f,
a_pred=a_pred)
```



CM 接触による認知率の増加が徐々に緩やかになっていくことが確認できます。

さて、ここまでの議論で、分析者の信念を反映させた CM 接触と製品認知率の関係に関する数理モデルを構築できました。この数理モデルはあくまで分析者の仮説を反映させたものなので、現実を完全に反映させたものではないことに注意してください。例えば、長く CM や製品に接触せずにいると、製品のことを忘れる「忘却」が起きるかもしれません。また、CM に 1 回接触した際に製品を認知する確率を θ としましたが、この「CM 接触によって製品を認知する確率」は一定ではなく個人によって異なるかもしれません。

数理モデルを利用する際には、このようなモデルの限界を認識しておくようにしましょう。分析時に置いた仮定が明示的になることが数理モデルを利用することの利点の一つです。分析結果がどのような仮定のもとで導出されたのかを把握しておくことで、より堅実な意思決定が可能となります。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

3.3.7 個人差を考慮したモデルの拡張

モデルの仮定と限界を認識することは重要ですが、モデルの限界を克服する努力も重要です。例えば、前述のモデルには「CM 接触によって製品を認知する確率は全個人で一定」という仮定を置いていました。この仮定を緩和して、「CM 接触によって製品を認知する確率は一定ではなく個人によって異なってもよい」というモデルを考えてみます。

2 章で CM 接触回数の個人差を考慮するためにポアソン分布を負の二項分布に拡張したことを思い出してください。個人ごとにパラメータの異なる複数のポアソン分布の混合分布を考えることで、個人差のある集団の分布を表現することに成功しました。具体的には、ポアソン分布のパラメータは 0 以上の連続値をとるので、合成の重み付けにガンマ分布を利用して合成しました。

2 章のアプローチにならって、CM 接触による製品を認知する確率に個人差があることを混合分布で表現する方法を考えます。CM 接触によって製品を認知する確率は名前の通り確率なので、0 から 1 の範囲に収まります。確率のような 0 から 1 の値をとる変数に対してよく使われる分布として、ベータ分布が知られています^{*3}

ベータ分布の確率密度関数は以下で定義されます。

$$\begin{aligned} f(\theta \mid \alpha, \beta) &= \text{Beta}(\theta \mid \alpha, \beta) \\ &= \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)} \end{aligned}$$

ここで、 $B(\alpha, \beta)$ はベータ関数と呼ばれていて、ガンマ関数とは以下の関係をもっています。

$$B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)}$$

ガンマ分布と同様、ベータ分布もパラメータ α と β の意味が直感的ではないので、以下のようにパラメータを変換しておくとか解釈が容易になります。

$$\begin{aligned} \mu &= \frac{\alpha}{\alpha + \beta}, \\ \nu &= \alpha + \beta \end{aligned}$$

*** 3** 2 章で解説したポアソン分布に対するガンマ分布と同様に、ベータ分布も二項分布に対する「共役性」の性質から利用されることが多いです。今回は二項分布とは直接関係しませんが、5 章のコラムでは二項分布を用いた例を紹介します。

3 章 認知形成を数理モデリングする

この変換により、 μ は期待値、 ν は分布の密集度を表すようになります。パラメータ μ は、CM 接触によって製品を認知する確率の分布の期待値を表すので、 μ が大きいということは、CM 接触の製品を認知させる力が大きいという解釈になります。また、パラメータ ν は CM 接触によって製品を認知する確率の分布の密集度を表すので、 ν が大きい場合ということは、CM 接触によって製品を認知する確率の個人差が小さいという解釈になります。

$\alpha = \mu\nu$ かつ $\beta = (1 - \mu)\nu$ であることに注意すると、ベータ分布の確率密度関数は以下で再定義できます。

$$\begin{aligned} f(\theta \mid \mu, \nu) &= \text{Beta}(\theta \mid \mu\nu, (1 - \mu)\nu) \\ &= \frac{\theta^{\mu\nu-1}(1 - \theta)^{(1-\mu)\nu-1}}{B(\mu\nu, (1 - \mu)\nu)} \end{aligned}$$

実際に、いくつかのパラメータに対してベータ分布をプロットしてみましょう。ベータ分布の確率密度関数は SciPy の stats モジュールに `beta.pdf` メソッドとして用意されています。引数 `a` がパラメータ $\alpha = \mu\nu$ を、引数 `b` が $\beta = (1 - \mu)\nu$ を意味することに注意してください。

```
In[41]: import scipy.stats

def plot_beta(
    theta: np.ndarray,
    mu_list: list[float],
    nu_list: list[float]
) -> tuple[Figure, Axes]:
    fig, ax = plt.subplots()

    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, (mu, nu) in enumerate(zip(mu_list, nu_list)):
        y = scipy.stats.beta.pdf(
            x=theta, a=mu * nu, b=(1 - mu) * nu
        )
        ax.plot(
            theta,
            y,
            linestyle=linestyle[i % len(linestyle)],
            label=f"Beta$(\\theta \\mid \\mu = \\{mu\\}, \\nu = \\{nu\\})$",
        )
```

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

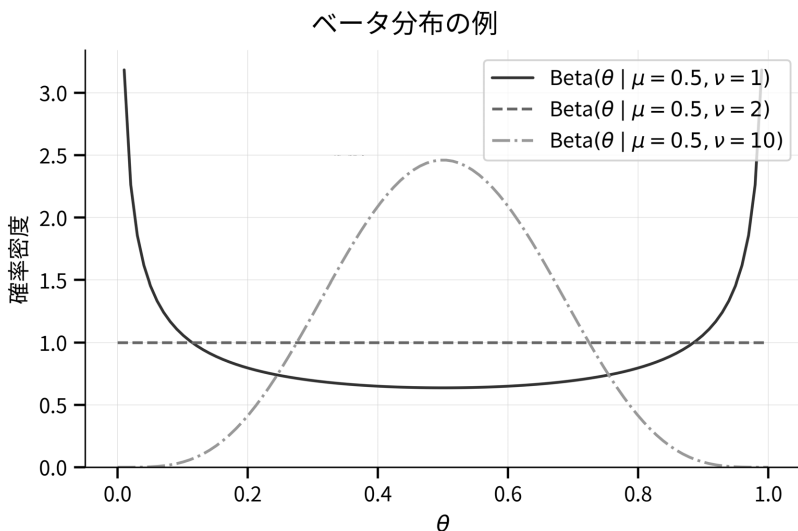
```

ax.legend()
ax.set_ylim(0, None)
ax.set_xlabel("$\\theta$")
ax.set_ylabel("確率密度")
fig.suptitle("ベータ分布の例")

return fig, ax

fig, ax = plot_beta(
    theta=np.linspace(0, 1, 100),
    mu_list=[0.5, 0.5, 0.5],
    nu_list=[1, 2, 10]
)

```



上図からも見てとれるように、 $\mu = 0.5$ かつ $\nu = 2$ の場合は、ベータ分布の確率密度関数は区間 $[0, 1]$ の一様分布と一致します。また、 ν を小さくすると分布が広がってお椀のような形になること、 ν を大きくすると分布が密集していくことがわかります。 ν を際限なく大きくしていくと、分布は 0.5 の周りに密集します。これは、実質的には CM 接触によって製品を認知する確率がどの個人も 50%であることを意味します。よって、CM 接触による製品認知確率がどの個人も一定だと仮定

3 章 認知形成を数理モデリングする

したモデルは、CM 接触による製品認知確率に個人の異質性を許したモデルにおいて、 $\nu \rightarrow \infty$ のケースと解釈することもできます。この意味で、個人差を許した新しいモデルは、より仮定の弱い柔軟なモデルになっていると解釈できます。

さて、ベータ分布で認知確率のさまざまな分布を表現できることがわかったので、これを利用して数理モデルを拡張しましょう。CM に 1 回接触した際に製品を認知する確率を表すパラメータ θ がベータ分布に従うと仮定して、2 章と同様に積分することで、CM 接触によって製品を認知する確率の個人差をモデルに盛り込みます。

$$\begin{aligned}
 & \Pr(A = 1 \mid f) \\
 &= \int_0^1 \Pr(A = 1 \mid f) \text{Beta}(\theta \mid \mu\nu, (1-\mu)\nu) d\theta \\
 &= \int_0^1 \left(1 - (1-\gamma)(1-\theta)^f\right) \left(\frac{\theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)}\right) d\theta \\
 &= \int_0^1 \frac{\theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)} d\theta \\
 &\quad - (1-\gamma) \frac{1}{B(\mu\nu, (1-\mu)\nu)} \int_0^1 \theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu+f-1} d\theta \\
 &= \underbrace{\int_0^1 \frac{\theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)} d\theta}_{=1} \\
 &\quad - (1-\gamma) \frac{B(\mu\nu, (1-\mu)\nu + f)}{B(\mu\nu, (1-\mu)\nu)} \underbrace{\int_0^1 \frac{\theta^{\mu\nu-1}(1-\theta)^{(1-\mu)\nu+f-1}}{B(\mu\nu, (1-\mu)\nu + f)} d\theta}_{=1} \\
 &= 1 - (1-\gamma) \frac{B(\mu\nu, (1-\mu)\nu + f)}{B(\mu\nu, (1-\mu)\nu)}
 \end{aligned}$$

以下で式変形の手順を解説します。

- 1 行目の変形：積分による合成を書き下した
- 2 行目への変形：単純に定義で置き換えた
- 3 行目への変形： θ と関係のない項目を積分の外に出して整理
- 4 行目への変形：第二項に同じ数 $B(\mu\nu, (1-\mu)\nu + f)$ を掛けて割った（5 行目への変形のためによく使うテクニック）
- 5 行目への変形：第一項の積分内は $\text{Beta}(\mu\nu, (1-\mu)\nu)$ 、第二項の積分内は $\text{Beta}(\mu\nu, (1-\mu)\nu + f)$ であり、確率密度関数を全区間で積分すると 1 となる

3.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

性質を利用

ベータ分布による合成を行うことで、CM 接触によって製品を認知する確率の違いを考慮した数理モデルを考えることができました。モデルを確認すると、元の「CM 接触によって製品を認知する確率は全個人で一定」と仮定したモデルでは $(1 - \theta)^f$ となっていた部分が、個人差を考慮したモデルでは $B(\mu\nu, (1-\mu)\nu+f)/B(\mu\nu, (1-\mu)\nu)$ に変わっています。パラメータが θ の一つだけから (μ, ν) の二つに増えたので、その分だけモデルの表現力が増えています。

それでは、実際に元のモデルよりも CM 接触回数と製品認知率の多様な関係を表現できることを確認しましょう。 $\theta = \mu = 0.3$ として個人ごとの認知確率の期待値が元のモデルと同じになるように設定した上で、 $\nu = 1, 2, 10$ としてベータ分布の広がり进行调整します。製品をそもそも知っている割合は先ほどと同様に $\gamma = 0.5$ としておき、CM 接触回数と製品認知率の関係を可視化します。

```
In[42]: from scipy.special import beta
```

```
def plot_awareness_frequency_relations(
    f: np.ndarray,
    gamma: float,
    theta: float,
    mu_list: list[float],
    nu_list: list[float],
) -> tuple[Figure, Axes]:
    fig, ax = plt.subplots()

    # 個人の異質性を考慮しない場合
    a = 1 - (1 - gamma) * (1 - theta) ** f
    ax.plot(
        f,
        a,
        label=f"異質性無視:  $\gamma = \{gamma\}, \theta = \{theta\}$ ",
        marker="o",
        linestyle="solid",
        zorder=1,
    )

    # 個人の異質性を考慮した場合
    markers = ["D", "^", "x"]
```

3 章 認知形成を数理モデリングする

```

linestyles = ["dashed", "dashdot", "dotted"]
for i, (mu, nu) in enumerate(zip(mu_list, nu_list)):
    a = 1 - (1 - gamma) * beta(
        mu * nu, (1 - mu) * nu + f
    ) / beta(
        mu * nu, (1 - mu) * nu
    )
    ax.plot(
        f,
        a,
        marker=markers[i % len(markers)],
        linestyle=linestyles[i % len(linestyles)],
        label=f"異質性考慮： $\gamma = \{\text{gamma}\}, \mu = \{\text{mu}\}, \nu = \{\text{nu}\}$ ",
        zorder=2,
    )

ax.legend()

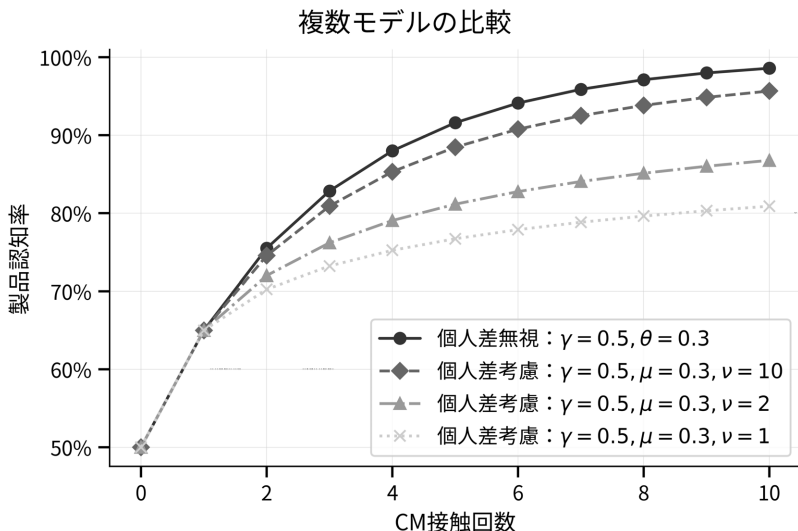
ax.yaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.set(xlabel="CM接触回数", ylabel="認知率")
fig.suptitle("複数モデルの比較")

return fig, ax

fig, ax = plot_awareness_frequency_relations(
    f=f,
    gamma=0.5,
    theta=0.3,
    mu_list=[0.3, 0.3, 0.3],
    nu_list=[10, 2, 1]
)

```


3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



グラフを確認すると、接触回数が0回の場合と1回の場合は両モデルの結果が一致していますが、2回以上の接触回数においてはパラメータ ν によって異なる結果が得られていることが見てとれます。 ν が大きいほどベータ分布は期待値 μ の周りに密集するので個人差が小さくなり、個人の異質性を考慮しないモデルの結果に近づいていきます。特に、 $\nu \rightarrow \infty$ のときは個人の異質性を考慮しないモデルと一致します。つまり、個人の異質性を考慮したモデルは、個人の異質性を考慮しないモデルを含有していると捉えることができ、より複雑な現象を表現できることがわかります。

このように、個人の異質性を考慮したモデルでは、パラメータ ν によってモデルの表現力が増していることが確認できました。次は、数理モデルのパラメータ(γ, μ, ν)をデータから推定する方法を考えます。

3.3.8 パラメータの推定方法

ここまでの議論で、「CM 出稿量→CM 接触回数→製品認知」という2段階のモデルに関して、CM 出稿量と CM 接触回数の関係 $\Pr(F = f | g)$ と、CM 接触回数と製品認知の関係 $\Pr(A = 1 | f)$ をともに数理モデリングできました。

表記の簡略化のために $a(g) = \Pr(A = 1 | g)$ とすると、2段階のモデルは

3 章 認知形成を数理モデリングする

$$\begin{aligned}
 a(g) &= \Pr(A = 1 \mid g) \\
 &= \sum_{f=0}^{\infty} \Pr(A = 1 \mid f) \Pr(F = f \mid g) \\
 &= \sum_{f=0}^{\infty} \left(1 - (1 - \gamma) \frac{B(\mu\nu, (1 - \mu)\nu + f)}{B(\mu\nu, (1 - \mu)\nu)} \right) \\
 &\quad \times \left(\frac{\Gamma(\alpha + f)}{\Gamma(f + 1) \Gamma(\alpha)} \left(\frac{\alpha}{g + \alpha} \right)^{\alpha} \left(\frac{g}{g + \alpha} \right)^f \right)
 \end{aligned}$$

とまとめられます。

この数理モデルにおいて、未知のパラメータはそもそも製品を知っている割合 γ 、CM に 1 回接触した際に製品を認知する確率の期待値 μ と密集度 ν 、そして CM 接触回数の分布の密集度 α の四つです。これらの未知のパラメータ $(\gamma, \mu, \nu, \alpha)$ をデータから推定する方法を考えましょう。2 段階にモデルを分解しているので、推定方法もそれぞれ個別に考えます。

$\Pr(F = f \mid g)$ の推定

まずは、CM 出稿量と CM 接触回数の関係 $\Pr(F = f \mid g)$ を推定する方法を考えましょう。これは 2 章で扱った CM 接触回数の分布を推定する問題と同じ方法で推定できます。

いま、CM 放送開始時点と終了時点の 2 時点のグロスリーチとユニークリーチを観測できています。時点 t のグロスリーチを g_t 、ユニークリーチを r_t とします。最適なパラメータ $\hat{\alpha}$ は、グロスリーチ g_t が与えられたもとで、観測されたユニークリーチ r_t と負の二項分布を仮定したリーチカーブの二乗誤差を最小化することで推定します。

$$\hat{\alpha} = \arg \min_{\alpha > 0} \sum_{t=1}^2 \left(r_t - \left(1 - \left(\frac{\alpha}{g_t + \alpha} \right)^{\alpha} \right) \right)^2$$

推定した $\hat{\alpha}$ を用いて、フリークエンシーの分布は負の二項分布の確率関数から

$$\widehat{\Pr}(F = f \mid g) = \frac{\Gamma(\hat{\alpha} + f)}{\Gamma(f + 1) \Gamma(\hat{\alpha})} \left(\frac{\hat{\alpha}}{g + \hat{\alpha}} \right)^{\hat{\alpha}} \left(\frac{g}{g + \hat{\alpha}} \right)^f$$

で求めます。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

$\Pr(A = 1 | f)$ の推定

次は、CM 接触回数と製品認知の関係 $\Pr(A = 1 | f)$ を推定する方法を考えます。

いま観測できているデータは、過去のあるキャンペーンに対する、CM 放送開始時点と CM 放送終了時点の 2 時点のグロスリーチ、ユニークリーチ、そして製品認知率です。特に、製品認知率に関するデータとして、2,000 人のモニターのうち何人が製品を認知しているかという人数を観測できています。そこで、製品認知人数に対するデータ生成過程を考え、その数理モデルを構築していきましょう。このような「 n 人の対象者のうち K 人が反応（ここでは製品認知）した」というデータのモデリングには二項分布を用いることが一般的です**。

$$\begin{aligned}\Pr(K = k | g) &= \text{Binomial}(k | n, a(g)) \\ &= \binom{n}{k} a(g)^k (1 - a(g))^{n-k}\end{aligned}$$

ここで、 $a(g)$ はグロスリーチが g のときの製品認知率、 n は調査対象者の人数、 k が製品を認知していた人数を表します。

二項分布から観測データが生成されたと仮定した上で、分布のパラメータを最尤法で推定します。いま、CM 放送開始時点と終了時点の 2 時点のデータが得られていて、これらのデータが同一の二項分布から独立に生成されたと仮定すると、尤度は各時点での二項分布の確率の積として、

$$\prod_{t=1}^2 \binom{n}{k_t} a(g_t)^{k_t} (1 - a(g_t))^{n-k_t}$$

で表されます。ここで、 g_t は時点 t のグロスリーチ、 k_t は時点 t の製品認知者数です。また、 $\binom{n}{k_t}$ は二項係数であり、 n 個から k_t 個を選んだときの組み合わせの数を表しています。

尤度よりも対数尤度のほうが扱いやすいので、尤度の対数をとることで対数尤度

$$\sum_{t=1}^2 \log \binom{n}{k_t} + k_t \log(a(g_t)) + (n - k_t) \log(1 - a(g_t))$$

を得ます。

最尤法では上記の対数尤度を最大化するようなパラメータを推定します。このとき、パラメータと関係のない $\log \binom{n}{k_t}$ は対数尤度の最大化に影響しないので省略します。また、製品認知は人数ではなく割合のほうが扱いやすいので、対数尤度を n

* 4 二項分布によるモデリングには、独立性の仮定が必要です。つまり、各モニターが製品を認知するかどうかは、他のモニターの認知状態とは独立に決まると仮定する必要があります。

3 章 認知形成を数理モデリングする

で割っておきます。この操作も最大化に影響は与えません。

$$\begin{aligned} \sum_{t=1}^2 \frac{k_t}{n} \log(a(g_t)) + \left(1 - \frac{k_t}{n}\right) \log(1 - a(g_t)) \\ = \sum_{t=1}^2 a_t \log(a(g_t)) + (1 - a_t) \log(1 - a(g_t)) \end{aligned}$$

ここで、 $a_t = k_t/n$ は時点 t の観測された製品認知率を表します。

ここまでの議論をまとめると、以下の対数尤度最大化問題を解くことでパラメータ (γ, μ, ν) を推定できます。

$$\begin{aligned} (\hat{\gamma}, \hat{\mu}, \hat{\nu}) = \arg \max_{(\gamma, \mu, \nu)} \sum_{t=1}^2 a_t \log(a(g_t)) + (1 - a_t) \log(1 - a(g_t)) \\ \text{s.t. } \gamma \in [0, 1] \\ \mu \in [0, 1] \\ \nu > 0 \end{aligned}$$

$$\text{where } \hat{a}(g) = \sum_{f=0}^{\infty} \Pr(A = 1 \mid f) \widehat{\Pr}(F = f \mid g)$$

$$\Pr(A = 1 \mid f) = 1 - (1 - \gamma) \frac{B(\mu\nu, (1 - \mu)\nu + f)}{B(\mu\nu, (1 - \mu)\nu)}$$

$$\widehat{\Pr}(F = f \mid g) = \frac{\Gamma(\hat{\alpha} + f)}{\Gamma(f + 1) \Gamma(\hat{\alpha})} \left(\frac{\hat{\alpha}}{g + \hat{\alpha}} \right)^{\hat{\alpha}} \left(\frac{g}{g + \hat{\alpha}} \right)^f$$

なお、CM 出稿量に対するフリークエンシーの分布 $\Pr(F = f \mid g)$ は、前述のようにグロスリーチとユニークリーチの関係から推定しておいた $\hat{\alpha}$ を用いて計算します。ここであらためて α を推定しません。推定するのは (γ, μ, ν) の三つのパラメータです。

もちろん、ここでパラメータ α もまとめて推定することはできます。それでも、 α の推定にはグロスリーチとより関連性の強いユニークリーチを用いて推定すべきです。グロスリーチと関係性が弱い製品認知率から推定すると、予測精度が低下すると考えられるからです。最終的にはグロスリーチと製品認知率の関係性をモデリングしたい場合でも、今回のユニークリーチのような有力な中間指標がある場合は、中間指標を用いてパラメータを推定することが有効です。

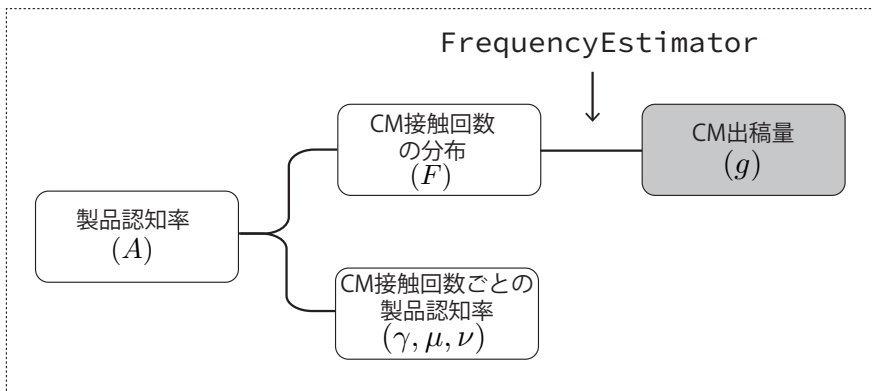
3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

3.3.9 数理モデルの実装

モデルの推定方法が決まったので、実装に落とし込みましょう。モデルは scikit-learn で用意されている関数を利用するため、scikit-learn に準拠した方法で実装します*5。scikit-learn の記法に合わせておくと、普段からこの記法に慣れ親しんでいるデータサイエンティストがモデルを利用しやすくなるというメリットもあります。

scikit-learn 準拠の回帰モデルを構築する際は、scikit-learn の base モジュールにある BaseEstimator クラスと RegressorMixin クラスを継承します*6。CM 出稿量→CM 接触回数→製品認知」という 2 段階のモデルがあるので、それぞれのモデルを BaseEstimator クラスと RegressorMixin クラスを継承して実装します (図 3.3)。

AwarenessCurveEstimator



■ 図 3.3: 実装するクラス

1 段階目のモデルの実装

まずは 1 段階目のモデルである CM 出稿量に対するフリークエンシーの分布を予測するモデルを FrequencyEstimator クラスとして実装しましょう。

* 5 次章以降で scikit-learn の cross_val_score 関数を利用する際には、scikit-learn 準拠のモデルである必要があります。

* 6 scikit-learn 準拠の予測モデルの構築について、詳細は公式のガイドラインをご確認ください。
<https://scikit-learn.org/stable/developers/develop.html#coding-guidelines>

3 章 認知形成を数理モデリングする

```

In[43]: from scipy.optimize import minimize_scalar # 最適化
        from scipy.stats import nbinom # 負の二項分布
        # 回帰モデルの基底クラス
        from sklearn.base import BaseEstimator, RegressorMixin
        from sklearn.metrics import mean_squared_error # 二乗誤差

class FrequencyEstimator(BaseEstimator, RegressorMixin):
    """
    グロスリーチとユニークリーチから
    フリークエンシーの分布を推定するモデル
    """

    def __init__(self) -> None:
        pass

    @staticmethod
    def _predict(g: np.ndarray, alpha: float) -> np.ndarray:
        """負の二項分布を仮定したリーチカーブの予測

        Args:
            g (np.ndarray): グロスリーチ
            alpha (float): 負の二項分布のパラメータ

        Returns:
            np.ndarray: リーチの予測値
        """

        return 1 - nbinom.pmf(
            k=0, p=alpha / (g + alpha), n=alpha
        )

    def fit(self, X: np.ndarray, y: np.ndarray) -> Self:
        """グロスリーチとユニークリーチの関係を学習

        Args:
            X (np.ndarray): 説明変数。グロスリーチを想定
            y (np.ndarray): 目低変数。ユニークリーチを想定

        Returns:
            Self: 学習済みモデル
        """

```

3.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

```

# 二乗誤差を最小化するalphaを推定して保存
self.alpha_: float = minimize_scalar(
    fun=lambda alpha: mean_squared_error(
        y, self._predict(g=X, alpha=alpha)
    ), # 最小化したい関数
    bounds=(0.01, 100), # 推定範囲。alphaは0より大きい
).x

return self

def predict(self, X: np.ndarray) -> np.ndarray:
    """学習済みのパラメータを使ってユニークリーチを予測

    グロスリーチとユニークリーチの関係のモデルなので、
    predictメソッドはユニークリーチを返すようにしている。
    実際は、predict_frequencyメソッドを使って、
    フリークエンシーを予測することが多い

    Args:
        X (np.ndarray): 説明変数。グロスリーチを想定

    Returns:
        np.ndarray: ユニークリーチの予測値
    """

    return self._predict(g=X, alpha=self.alpha_)

def predict_frequency(
    self, X: np.ndarray | float, f_max: int
) -> np.ndarray:
    """学習済みのパラメータを使ってフリークエンシーを予測

    Args:
        X (np.ndarray | float): 説明変数。グロスリーチを想定
        f_max (int): 予測するフリークエンシーの最大値

    Returns:
        np.ndarray:
            フリークエンシーの分布の予測値
            f_max + 1列のデータが返ることに注意
    """

    f = np.arange(f_max + 1)
    return nbinom.pmf(

```

3 章 認知形成を数理モデリングする

```

        k=f,
        p=self.alpha_ / (X + self.alpha_),
        n=self.alpha_
    )

```

FrequencyEstimator クラスの実装を順に説明していきます。

まず、_predict メソッドは負の二項分布を仮定したリーチカーブを計算するメソッドです。

次に、fit メソッドは説明変数 X と目的変数 y を受け取って、負の二項分布のパラメータ α を推定するメソッドです*7。ここで、 X は CM 出稿量つまりグロスリーチを、 y はユニークリーチを受け取ることを想定しています。

fit メソッドの内部では _predict メソッドが呼ばれています。SciPy の optimize モジュールにある minimize_scalar 関数を利用して、実際に観測されたユニークリーチと _predict メソッドで予測したユニークリーチの二乗誤差を最小化することでパラメータ α を推定しています。

predict メソッドは学習したパラメータ α を用いて、CM 出稿量に対してユニークリーチを予測するメソッドです。このクラスの役割はユニークリーチではなく CM 接触回数の分布を推定することなので、predict メソッドの多用は想定していません。

与えられたグロスリーチに対する CM 接触回数の予測は predict_frequency メソッドで行います。分布を予測するので、このメソッドの返り値は 1 列ではなく $f_{\max} + 1$ 列のデータが返ってくることに注意してください。

それでは、実際にモデルの学習を行いましょう。

```

In[44]: # モデルの学習
fe = FrequencyEstimator().fit(g_train, r_train)

```

predict_frequency メソッドの返り値の shape を確認しておきましょう。

```

In[45]: # フリークエンシー分布の予測値のshapeを確認
f_max = 15
fe.predict_frequency(g_train, f_max=15).shape

```

```

Out[45]: (2, 16)

```

* 7 学習したパラメータは α のようにアンダースコアを付けています。scikit-learn において、学習したパラメータを保持する属性にはアンダースコアを付ける慣習があるため、それに合わせる形にしています。

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

与えたデータが 2 時点のデータで、f_max が 15 だったので、shape は (2, 16) になっています。

予測したフリークエンシーの分布も可視化しておきます。まずはグロスリーチが 100% の場合の予測値を確認します。

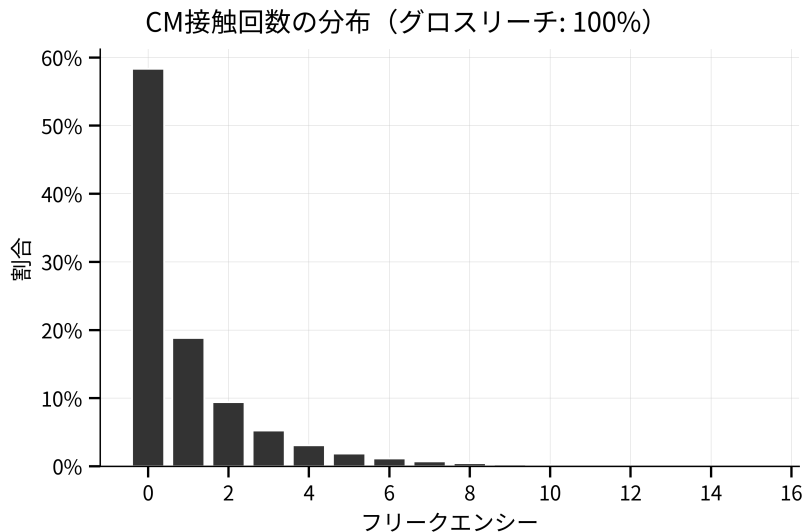
```
In[46]: def plot_frequency_distribution(
        pr_f_given_g_pred: np.ndarray, f_max: int, title: str
    ) -> tuple[Figure, Axes]:
        """フリークエンシーの分布を可視化"""

        fig, ax = plt.subplots()
        ax.bar(np.arange(f_max + 1), pr_f_given_g_pred)
        ax.yaxis.set_major_formatter(
            PercentFormatter(xmax=1, decimals=0)
        )
        ax.set(xlabel="フリークエンシー", ylabel="割合")
        fig.suptitle(title)

        return fig, ax

# グロスリーチが100%の場合を可視化
g = 1
fig, ax = plot_frequency_distribution(
    pr_f_given_g_pred=fe.predict_frequency(X=g, f_max=f_max),
    f_max=f_max,
    title=f"CM接触回数の分布 (グロスリーチ: {g:.0\\%})",
)
```

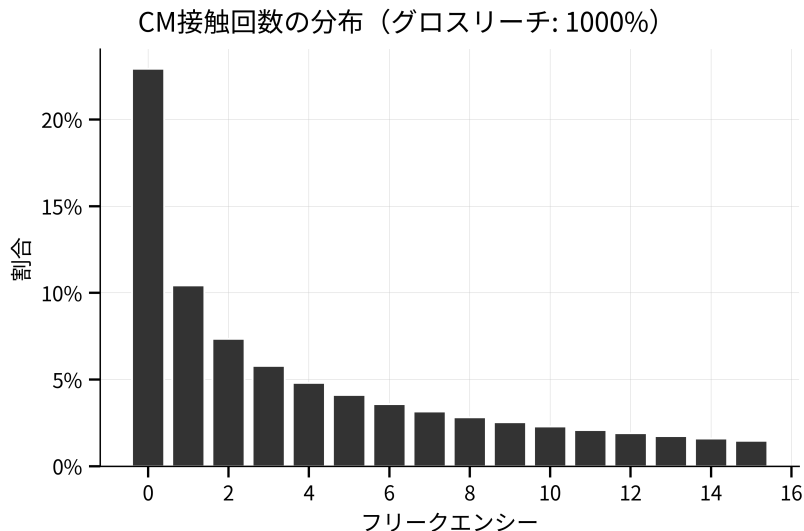
3 章 認知形成を数理モデリングする



グロスリーチが 1000% の場合の予測値も出力して比較してみましょう。

```
In[47]: # グロスリーチが1000%の場合を可視化
g = 10
fig, ax = plot_frequency_distribution(
    pr_f_given_g_pred=fe.predict_frequency(X=g, f_max=f_max),
    f_max=f_max,
    title=f"CM接触回数の分布 (グロスリーチ: {g:.0\%})",
)
```

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



グロスリーチが 100%のときは 8 回接触までが最大でしたが、グロスリーチが 1,000%になると分布が大きく右に動き、15 回以上の接触も発生すると予測しています（実際は 16 回以上の接触も発生していますが、見切れています）。

また、グロスリーチが 1,000%の状態でも、正規分布のようなベル型の形状にはならず、右に大きく伸びた分布を予測していることが見てとれます。

以上で、1 段階目の CM 出稿量とフリークエンシーの分布のモデルを `FrequencyEstimator` クラスとして実装することができました。次は、2 段階目のモデルの実装に移りましょう。

2 段階目のモデルの実装

次は、CM 接触回数と製品認知率のモデルを `AwarenessEstimator` クラスとして実装しましょう。このクラスは、先ほど実装した学習済みの `FrequencyEstimator` クラスを受け取って、CM 接触回数と製品認知率の関係を新たに学習することで認知カーブを推定します。

```
In[48]: from scipy.optimize import minimize
```

```
class AwarenessCurveEstimator(BaseEstimator, RegressorMixin):
    """認知カーブを推定するモデル"""
```

3 章 認知形成を数理モデリングする

```

def __init__(self, frequency_estimator:
    FrequencyEstimator, f_max: int) -> None:
    """
    Args:
        frequency_estimator (FrequencyEstimator):
            学習済みのフリークエンシー分布の推定モデル
        f_max (int):
            モデルの学習と予測に利用する、
            フリークエンシーの最大値。
            f_max + 1 回接触の発生確率がほぼ0になるくらい、
            大きな値を設定する必要がある。
            大きくしすぎると計算時間がかかるので、
            必要十分な大きさにする。
            200程度が目安
    """

    self.frequency_estimator = frequency_estimator
    self.f_max = f_max

    @staticmethod
    def _calc_pr_a_given_f(
        f_max: int, gamma: float, mu: float, nu: float
    ) -> np.ndarray:
        """CM接触回数に対する製品認知率を予測

        Args:
            f_max (int): 何回接触までを出力するか
            gamma (float): 製品をそもそも知っている割合
            mu (float): CMに1回接触したときの製品認知率の期待値
            nu (float): CMに1回接触したときの製品認知率の密集度

        Returns:
            np.ndarray: 製品認知率
        """

        f = np.arange(f_max + 1)
        return 1 - (1 - gamma) * beta(
            mu * nu, (1 - mu) * nu + f
        ) / beta(
            mu * nu, (1 - mu) * nu
        )

    def _predict(

```

3.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

```

        self,
        X: np.ndarray,
        gamma: float,
        mu: float,
        nu: float
    ) -> np.ndarray:
        """グロスリーチに対する製品認知率を計算

    Args:
        X (np.ndarray): 説明変数。グロスリーチを想定
        gamma (float): 製品をそもそも知っている割合
        mu (float): CMに1回接触したときの製品認知率の期待値
        nu (float): CMに1回接触したときの製品認知率の密集度

    Returns:
        np.ndarray: 製品認知率
        """

    # グロスリーチに対するCM接触回数の分布を予測
    pr_f_given_g = (
        self.frequency_estimator.predict_frequency(
            X, self.f_max
        )
    )

    # CM接触回数ごとの製品認知率を予測
    pr_a_given_f = self._calc_pr_a_given_f(
        self.f_max, gamma, mu, nu
    )

    # 二つを合わせてグロスリーチに対する製品認知率を計算
    return pr_a_given_f @ pr_f_given_g.T

def _negative_log_likelihood(
    self,
    params: np.ndarray,
    X: np.ndarray,
    y: np.ndarray,
) -> np.float_:
    """最小化したい負の対数尤度

    Args:
        params (np.ndarray): 推定するパラメータ
            gamma, mu, nuの順

```

3 章 認知形成を数理モデリングする

```

        X (np.ndarray): 説明変数。グロスリーチを想定
        y (np.ndarray): 目的変数。製品認知率を想定

Returns:
    np.float_: 負の対数尤度
    """

    # グロスリーチから製品認知率を予測
    gamma, mu, nu = params
    pr_a_given_g = self._predict(X, gamma, mu, nu)

    # minimize関数は最小化なので、
    # 対数尤度ではなく負の対数尤度を返す
    return -np.mean(
        y * np.log(pr_a_given_g)
        + (1 - y) * np.log(1 - pr_a_given_g)
    )

def fit(self, X: np.ndarray, y: np.ndarray) -> Self:
    """モデルの学習

    Args:
        X (np.ndarray): 説明変数。グロスリーチを想定
        y (np.ndarray): 目的変数。認知率を想定

    Returns:
        Self: 学習済みモデル
    """

    result = minimize(
        fun=lambda params: self._negative_log_likelihood(
            params=params, X=X, y=y
        ),
        x0=np.array([0.5, 0.5, 1]), # 初期値
        bounds=[
            (0.01, 0.99), (0.01, 0.99), (0.01, 10)
        ], # 推定範囲
        tol=1e-10, # 収束判定
    )

    # 学習したパラメータを保存
    self.gamma_, self.mu_, self.nu_ = result.x

    return self

```

3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
def predict(self, X: np.ndarray) -> np.ndarray:
    """
    学習済みのパラメータを使って、
    グロスリーチに対する製品認知率を予測

    Args:
        X (np.ndarray): 説明変数。グロスリーチを想定

    Returns:
        np.ndarray: 製品認知率の予測値
    """

    return self._predict(
        X, self.gamma_, self.mu_, self.nu_
    )

def predict_frequency_awareness_relation(
    self, f_max: int
) -> np.ndarray:
    """
    学習済みのパラメータを使って、
    CM接触回数に対する製品認知率を予測

    Args:
        f_max (int): 何回接触までを出力するか

    Returns:
        np.ndarray: 製品認知率の予測値
    """

    return self._calc_pr_a_given_f(
        f_max, self.gamma_, self.mu_, self.nu_
    )
```

AwarenessEstimator クラスの実装を順に説明していきます。

まず、__init__メソッドでは、学習済みの FrequencyEstimator クラスを受け取って初期化を行います。同時に、モデルの学習と予測に利用するフリークエンシーの最大値 f_max を設定します。モデルを適切に学習するためには、f_max + 1 回接触の発生確率がほぼ 0 になるくらい大きな値を設定する必要があります。今回の問題設定では 200 程度で十分です。

3 章 認知形成を数理モデリングする

次に、`_pr_a_given_f` メソッドは、CM 接触回数ごとの製品認知率を予測するメソッドです。 $\Pr(A = 1 | f)$ に対する数理モデルをそのまま実装しています。ここに $\Pr(F = f | g)$ の予測結果も合わせて、グロスリーチから製品認知率を予測するのが `_predict` メソッドです。

パラメータの推定には最尤法を用います。パラメータが複数あるので、ここまで使ってきた `minimize_scaler` 関数ではなく、SciPy の `optimize` モジュールにある `minimize` 関数を利用します。`minimize` 関数は最大化ではなく最小化を行う関数なので、対数尤度をマイナスにした負の対数尤度を用意する必要があり、これを `neg_log_likelihood` メソッドで実装しています。

`fit` メソッドではパラメータの推定を行います。`minimize` 関数では、最小化した目的関数を引数 `fun` に与えます。パラメータの推定範囲は、`minimize_scaler` 関数と同じく引数 `bounds` で指定できます。逆に、`minimize_scaler` 関数にはなかったものとして、引数 `x0` で初期値を設定する必要があります。最適化したい関数が複雑な場合は初期値によって結果が異なる場合があるので注意してください。学習したパラメータは `gamma_`、`mu_`、`nu_` として保持されます。

最後に、`predict` メソッドと `predict_frequency_awareness_relation` メソッドは、学習したパラメータを用いて予測値を出力します。`predict` メソッドはグロスリーチに対する製品認知率を、`predict_frequency_awareness_relation` メソッドは CM 接触回数に対する製品認知率を返します。

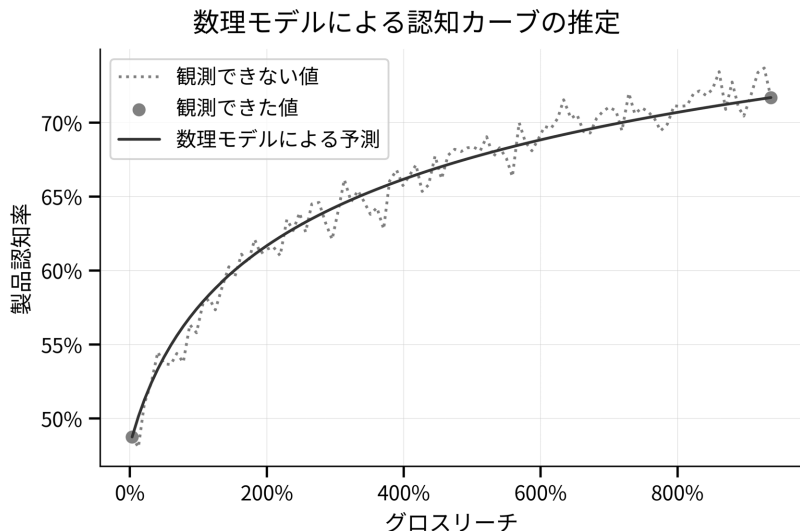
以上でモデルが実装できたので、認知カーブを確認してみましょう。

```
In[49]: # モデルの学習
ace = AwarenessCurveEstimator(
    frequency_estimator=fe, f_max=200
)
ace.fit(X=g_train, y=a_train)

# 認知カーブの推定
a_pred_ace = ace.predict(X=g_test)

# 認知カーブの可視化
fig, ax = plot_awareness_with_prediction(
    df_awareness=df_awareness,
    df_awareness_observed=df_awareness_observed,
    a_pred=a_pred_ace,
    prediction_label="数理モデルによる予測",
    title="数理モデルによる認知カーブの推定",
)
```


3.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



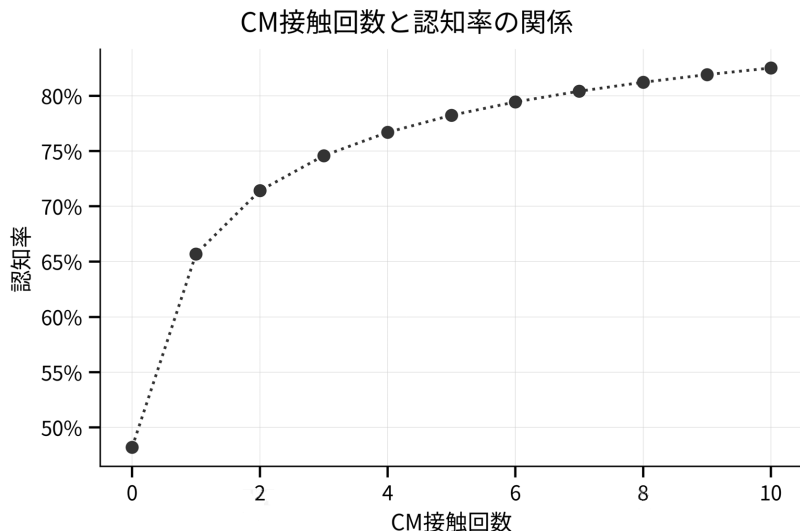
実測値に対して、数理モデルによる予測値がよく一致していることがわかります。対数変換を加えた線形回帰モデルによる予測結果も悪くはありませんでしたが、さらに精度の高い予測ができていることが確認できます。

さらに、構築した数理モデルは「CM 出稿量に対する CM 接触回数の分布」と「CM 接触回数に対する製品認知率」の 2 段階に関係性を分解しているので、それぞれの関係性を個別に可視化することもできます。CM 出稿量に対する CM 接触回数の分布は先ほど `FrequencyEstimator` で確認したので、ここでは CM 接触回数と製品認知率の関係を可視化しましょう。

```
In[50]: f_max = 10
        f = np.arange(f_max + 1)
        pr_a_given_f_pred = ace.predict_frequency_awareness_relation(
            f_max=f_max
        )

        # 可視化
        fig, ax = plot_awareness_frequency_relation(
            f, a_pred=pr_a_given_f_pred
        )
```

3 章 認知形成を数理モデリングする



CM に接触していない場合の認知率、つまり製品をそもそも知っている割合が50%弱であること、CM 接触回数が増えるにつれて認知率は上昇するがいずれ飽和するという予測結果が読み取れます。

さらに学習されたパラメータを確認しましょう。

```
In[51]: print(f"gamma: {ace.gamma_: .2f}")
        print(f"mu: {ace.mu_: .2f}")
        print(f"nu: {ace.nu_: .2f}")
```

```
Out[51]: gamma:  0.48
         mu:   0.34
         nu:   0.98
```

特に、パラメータ μ を見ることで、CM に一度接触した際に製品を認知する確率は平均的には34%と予測していることがわかります。

このように、分析者自身の仮説を反映させた数理モデルを構築することで、モデルの中身を把握し、なぜモデルがこのような予測結果を出すのかを理解することができます。モデルの振る舞いを知り、モデルの限界を理解することで、より安全で説得的な意思決定が可能となります。

3.4 ステップ 3：数理最適化問題を解いて最適なアクションを導出する

3.4

ステップ 3：数理最適化問題を解いて最適なアクションを導出する

ステップ 1 では「テレビ CM の出稿を通じて製品認知率を高めたい」というビジネス課題を数理最適化問題に落とし込みました。ステップ 2 では、最適化問題の未知の部分である、CM 出稿量と CM 認知率の関係 $a(g)$ に対して数理モデルを構築し、データの力を借りて未知のパラメータを推定しました。そして、このステップ 3 では、未知の部分が無くなった最適化問題を解くことで、最適なアクションを導出します。

学習した CM 出稿量と製品認知率の関係を $\hat{a}(g)$ とすると、目標認知率を達成するために必要最低限の費用を求めるための、制約付き費用最小化問題は以下のように定式化できます。

$$\min_g \quad 100cg$$

$$\text{s.t.} \quad \hat{a}(g) \geq \underline{a}$$

$$100cg \leq b$$

$$g \geq 0$$

$$\text{where} \quad \hat{a}(g) = \sum_{f=0}^{\infty} \widehat{\Pr}(A = 1 \mid f) \widehat{\Pr}(F = f \mid g)$$

$$\widehat{\Pr}(A = 1 \mid f) = 1 - (1 - \hat{\gamma}) \frac{B(\hat{\mu}\hat{\nu}, (1 - \hat{\mu})\hat{\nu} + f)}{B(\hat{\mu}\hat{\nu}, (1 - \hat{\mu})\hat{\nu})}$$

$$\widehat{\Pr}(F = f \mid g) = \frac{\Gamma(\hat{\alpha} + f)}{\Gamma(f + 1)\Gamma(\hat{\alpha})} \left(\frac{\hat{\alpha}}{g + \hat{\alpha}} \right)^{\hat{\alpha}} \left(\frac{g}{g + \hat{\alpha}} \right)^f$$

仮に、製品認知率の目標値 $\underline{a}$ が 65% で、予算上限 b は 1 億円、グロスリーチ 1% あたりの費用 c は 10 万円だったとします。この場合の最適解を計算しましょう。すでに可視化したグラフから大まかな解を見つけることもできますが、今回は数値計算による解法を実装します。

最適化問題を解くためには、SciPy の `optimize` モジュールにある `minimize` 関数を利用します。`minimize` 関数は引数 `constraints` で制約条件を指定できます。不等式制約の場合は `{'type': 'ineq', 'fun': 制約条件の関数}` として指定します。このとき、制約条件は「制約条件の関数が 0 以上の値を返す」ように指定します。

3 章 認知形成を数理モデリングする

```
In[52]: c = 10 # グロスリーチ1%あたりのコスト（万円単位）
b = 10000 # 予算上限（万円単位）
a_lb = 0.65 # 製品認知率の目標値

# 制約条件のもとで費用を最小化
result = minimize(
    fun=lambda g: 100 * c * g,
    x0=1,
    bounds=[(0, None)],
    constraints=[
        # 予算制約
        {"type": "ineq", "fun": lambda g: b - 100 * c * g},
        # 目標を達成する制約
        {"type": "ineq", "fun": lambda g:
            ace.predict(np.array([g])) - a_lb},
    ],
)

print(
    f"目標を達成するには{result.x[0]:.0%}のグロスリーチが必要です"
)
print(f"その場合の費用は{result.fun:.0f}万円です")
```

Out[52]: 目標を達成するには334%のグロスリーチが必要です
その場合の費用は3344万円です

最適化問題を解いた結果、334%のグロスリーチが必要であること、そのためには 3,340 万円のテレビ出稿費用が必要であることがわかりました。

なお、この結果は製品認知率の「期待値」が 65%を超える水準を導出しており、実際の製品認知率をもっと低くなる可能性もあります。よって、もっと保守的な見積もりを行う場合は、製品認知率の不確実性を考慮して 65%ではなく 70%を達成する CM 出稿量を求めたほうがいいかもしれません*8。

このように、この結果を実務で使う上ではさまざまな議論が必要ですが、数理的な側面から得られた知見の一つとしてマーケティング戦略に役立てることができるでしょう。

* 8 不確実性を考慮した意思決定を行う有力な選択肢としてベイズ推定があります。ベイズ推定の理論と実践に踏み込むことは本書の範疇を超えるため、ベイズ統計学の専門書を参考にしてください。特に Robert (2001) は意思決定についての解説が詳しいです。その他にも、McElreath (2020)、Lambert (2018)、Hoff (2010) はベイズ統計学の入門書として優れています。

3.5 本章のまとめ

本章では、「テレビ CM の出稿を通じて製品認知率を高めたい」というビジネス課題を題材とし、3 ステップフレームワークを用いて課題解決に取り組みました。

まず、ステップ 1 では、数理最適化問題の定式化を通じて、最適化したい指標、制約条件、とりうるアクションを明確にしました。具体的には、とりうるアクションを CM 出稿量とし、目標とする製品認知率の水準をできるだけ小さいコストで達成するという課題を、制約付き費用最小化問題

$$\begin{aligned} \min_g \quad & 100cg \\ \text{s.t.} \quad & \Pr(A = 1 \mid g) \geq \underline{a} \\ & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

として定式化することで、データサイエンスで扱える問題に変換しました。

ステップ 1 で定式化した数理最適化問題には、未知の部分としてグロスリーチと製品認知率の関係 $\Pr(A = 1 \mid g)$ が含まれていました。そこで、ステップ 2 ではこの未知の関係性に対して数理モデルを構築し、データを用いてパラメータを推定しました。特に、データが不足している状況を想定して、数理モデルを用いた推定の妥当性・有用性を示しました。CM 接触回数の分布は 2 章と同じく負の二項分布で近似し、さらに CM 接触回数と製品認知率の関係性を

$$\Pr(A = 1 \mid f) = 1 - (1 - \gamma) \frac{B(\mu\nu, (1 - \mu)\nu + f)}{B(\mu\nu, (1 - \mu)\nu)}$$

とモデリングすることで、2 時点のデータから CM 接触回数と製品認知率の関係を推定しました。

最後に、ステップ 3 では、ステップ 2 で推定した認知カーブを代入した数理最適化問題

$$\begin{aligned} \min_g \quad & 100cg \\ \text{s.t.} \quad & \hat{a}(g) \geq \underline{a} \\ & 100cg \leq b \\ & g \geq 0 \end{aligned}$$

3 章 認知形成を数理モデリングする

$$\begin{aligned} \text{where } \hat{a}(g) &= \sum_{f=0}^{\infty} \widehat{\Pr}(A=1 \mid f) \widehat{\Pr}(F=f \mid g) \\ \widehat{\Pr}(A=1 \mid f) &= 1 - (1 - \hat{\gamma}) \frac{B(\hat{\mu}\hat{\nu}, (1 - \hat{\mu})\hat{\nu} + f)}{B(\hat{\mu}\hat{\nu}, (1 - \hat{\mu})\hat{\nu})} \\ \widehat{\Pr}(F=f \mid g) &= \frac{\Gamma(\hat{\alpha} + f)}{\Gamma(f + 1)\Gamma(\hat{\alpha})} \left(\frac{\hat{\alpha}}{g + \hat{\alpha}}\right)^{\hat{\alpha}} \left(\frac{g}{g + \hat{\alpha}}\right)^f \end{aligned}$$

を解くことで、目標達成に必要な CM 出稿量を求めました。

このように、3 ステップフレームワークを用いることで、ビジネス課題を数理最適化問題として定式化し、データを用いて未知の部分を推定し、最適化なアクションを導くことができます。

2 章と同じく、今回構築した数理モデルもビジネス課題や取り巻く状況を単純化したもので、実務上は以下のような論点も考えられます。

- CM は認知することもあれば、逆に忘れることもある。CM 接触から時間がたてば忘却することをモデルに含める必要があるかもしれない
- CM 出稿期間の長さは結果に影響するか？例えば、短期間で大量に CM を出稿すると、SNS や日常会話で話題になり、CM 接触以外の経路を通じて認知率が向上するかもしれない
- 最適化問題に製品認知率の不確実性を取り込む必要はないか？「90%の確率で目標が達成される」ような CM 出稿量の水準はどの程度か？
- 過去のキャンペーンの結果を利用して、未来のキャンペーンの意思決定を行うとしているが、そもそも過去キャンペーンのグロスリーチと製品認知率の関係は未来でも成り立つか？誤差があるとすればどの程度か？
- 今回の分析で得られた結果を因果関係と解釈しても問題ないか？「製品を認知している人が広告に接触しやすい」のようなバイアスはないか？バイアスがあるとすればどのように軽減すればよいか？

数理モデルは一度構築して終わりではなく、分析者が置かれた状況に応じて、アップデートしていく必要があります。上記のような論点を洗い出し、重要な要素をモデルに組み込んでいくことで、より現実的で、よりビジネスに意思決定に利用しやすいモデルを作っていくことができます。

さて、2 章と 3 章では広告出稿先が一つしかない場合を考えましたが、マーケティングの実務において広告媒体は必ずしも一つとは限りません。次章では、複数

の広告媒体がある状況を題材として、広告出稿の予算配分を最適化する問題に取り組みます。

4章

連続最適化で広告予算を配分 する

これまでの章では、単一の広告出稿媒体を前提として、その効果を数理モデルで捉える方法論を探索してきました。しかし、実際のマーケティングでは、テレビ CM、デジタル広告など、複数の広告出稿媒体を効果的に組み合わせることが求められます。

媒体が複数になると、媒体ごとのコストパフォーマンス、異なる媒体間でのユニークリーチの重複といった、新たな複雑性が生じます。そこで 4 章では、この実務的な課題に対応するため、数理モデルの対象を複数媒体へと拡張し、限りある広告予算を各媒体へどのように配分すべきかという問題に取り組みます。

4.1 ビジネス課題：広告予算配分の最適化

前章までは、主に単一の広告出稿媒体を対象として、その出稿量と成果の関係性を数理モデルで分析し、最適な CM 出稿量を導出しました。

2 章では、グロスリーチとユニークリーチの関係をモデル化し、テレビ CM 出稿を通じた利潤最大化という課題に取り組みました。続く 3 章では、ユニークリーチからさらに一步踏み込み、グロスリーチと製品認知率という、より複雑な関係性に対して数理モデルを構築しました。これらの分析を通じて、特にデータが限られた状況下で、ドメイン知識をモデルに組み込むことの重要性を確認しました。

これまでの分析では「広告出稿媒体が一つである」という前提を置いていましたが、実際のマーケティング活動においては、広告出稿の選択肢は一つとは限らず、むしろ複数の媒体を組み合わせてキャンペーンを展開するのが一般的です。例えば、テレビ CM だけでなく YouTube などのデジタル広告を併用する、あるいはテレビ CM においても複数の放送局に CM を出稿するという状況は日常的に発生します。

このような複数の広告出稿を考慮するためには、より複雑な分析が求められます。第一に、コスト効率やターゲットへの到達しやすさが媒体ごとに異なります。第二に、より重要な点として、異なる媒体に同じ人が接触する可能性、すなわち複数媒体でのユニークリーチの重複の可能性があります。これらの要素を正確に評価しなければ、キャンペーン全体の成果を正しく評価することができません。



■ 図 4.1: 複数媒体の最適化

4.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

複数の広告出稿媒体が選択肢として存在するならば、「限られた総広告予算を、どの媒体にどれだけ配分すれば、キャンペーン全体の成果を最大化できるのか？」という問題意識が生まれます。

そこで本章では、この「広告予算配分の最適化」を新たなビジネス課題として取り上げます（図 4.1）。そして、このビジネス課題に対して、これまでと同様に 3 ステップフレームワークを適用し、最適なアクションを導出する方法を解説します。

4.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

それでは、ビジネス課題「広告予算配分の最適化」を数理最適化問題として定式化していきましょう。

まずは最適化したい指標を考えます。「広告予算配分を最適化したい」という要望そのものには、最適化したい指標が言及されていません。ステークホルダーと議論して、具体的な指標を考える必要があります。

議論の結果、最大化したい指標は「複数媒体合算のユニークリーチ」であるという結論に至ったとします*1。ここで、「複数媒体合算のユニークリーチ」とは、媒体を問わず CM に少なくとも一度は接触した割合を指します。

いま、出稿媒体が $j = 1, 2$ の二つがあるとし、媒体 j で放送された CM に一度でも接触したことがあれば 1、そうでなければ 0 をとる確率変数を R_j とします。さらに、いずれかの媒体で CM に接触したことを表す確率変数を $R_{1 \cup 2}$ とします。 $\cup$ は和集合を表す記号で「媒体 1 または媒体 2 の CM に接触した」というニュアンスを表現しています。 $R_{1 \cup 2}$ はいずれかの媒体で CM に接触したら 1、そうでなければ 0 をとる確率変数なので、 R_j を用いて以下のように表すことができます。

$$\begin{aligned} R_{1 \cup 2} &= 1 - (1 - R_1)(1 - R_2) \\ &= R_1 + R_2 - R_1 R_2 \end{aligned}$$

なお、「複数媒体合算のユニークリーチ」という表記が煩雑なので、以降では単に「合算のユニークリーチ」と表記します。

次に決定変数を考えます。今回もテレビ CM の出稿形態としてスポットを想定します。ただし、前回までとは違い、CM を出稿する放送局は一つではなく二つだ

* 1 実際の指標選定では、キャンペーンの目的、測定の実現可能性、などを総合して指標を選択します。最終成果である売上や、3 章で取り扱った製品認知率などの指標も優良な候補です。今回は、「できるだけ多くの人々に情報を届ける」という広告の基本的な成果を最大化することを重視して、測定が容易で、媒体横断的な評価にも馴染みやすいユニークリーチが選ばれた、という状況を考えます。

4 章 連続最適化で広告予算を配分する

とします。つまり、決定変数は、どちらの広告出稿媒体（放送局）にどのくらいの CM を出稿するか、つまり広告出稿媒体ごとのグロスリーチだとしましょう*2。グロスリーチをそれぞれ (g_1, g_2) と表記します。最大化したい指標は合算のユニークリーチだったので、目的関数はグロスリーチで条件付けた合算のユニークリーチ $\mathbb{E}[R_{1 \cup 2} \mid g_1, g_2]$ です。

最後に制約条件を考えます。費用は 2 章の設定と同じくグロスリーチ 1%あたりのコストが固定されているとし、それぞれ (c_1, c_2) だったとしましょう。よって、媒体 1 に配分する費用は $100c_1g_1$ 、媒体 2 に配分する費用は $100c_2g_2$ です。この合計が、広告予算の上限 b を超えないという制約条件を課します。

以上をまとめると、ビジネス課題「広告予算配分の最適化」を以下の制約付きユニークリーチ最大化問題として定式化できます。

$$\begin{aligned} \max_{g_1, g_2} \quad & \mathbb{E}[R_{1 \cup 2} \mid g_1, g_2] \\ \text{s.t.} \quad & 100c_1g_1 + 100c_2g_2 \leq b \\ & g_1, g_2 \geq 0 \end{aligned}$$

この数理最適化問題における未知の部分は合算のユニークリーチ $\mathbb{E}[R_{1 \cup 2} \mid g_1, g_2]$ なので、この関係性をデータを用いて推定することで、最適な広告予算配分を求めることができます。

4.3

ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

4.3.1 必要なデータ

ステップ 1 で数理最適化問題の定式化ができました。最大化問題における未知の部分は合算のユニークリーチ $\mathbb{E}[R_{1 \cup 2} \mid g_1, g_2]$ なので、この条件付き期待値を推定できるようなデータを取得する必要があります。

まずは必要なデータについて考えましょう。各媒体のグロスリーチと複数媒体合算のユニークリーチの関係性を知りたいので、各媒体のグロスリーチと、合算のユニークリーチは最低限必要です。例えば、表 4.1 のように各媒体のグロスリーチと

* 2 この例では、テレビ CM を題材とし媒体は放送局を想定していますが、媒体として YouTube などを想定しても以降の議論は適用可能です。

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

合算のユニークリーチが同時に観測されているようなデータがあれば、この条件付き期待値を推定できそうです。

▼ 表 4.1: グロスリーチとユニークリーチが観測されているデータ

キャンペーン	グロスリーチ (媒体 1)	グロスリーチ (媒体 2)	ユニークリーチ (複数媒体合算)
1	100%	300%	70%
2	200%	200%	50%
3	300%	100%	60%
⋮	⋮	⋮	⋮

この要件で社内のデータを収集したところ、過去の 20 キャンペーンに対して、各媒体のグロスリーチと合算のユニークリーチが観測されているデータが手に入ったとしましょう。そのデータをもとに、以降の分析を進めていきます。

4.3.2 データの読み込みと可視化

それでは、本章で利用するデータを確認していきましょう。
まずは本書を通じて利用するライブラリを読み込みます。その他必要なライブラリは利用する際に適宜読み込んでいきます。

```
In[53]: import json # ファイル読み込み
        from typing import Self # 型ヒント

        import matplotlib.pyplot as plt # 可視化
        import numpy as np # データ処理
        import polars as pl # データ処理
        import seaborn as sns # 可視化
        from matplotlib.axes import Axes # 型ヒント
        from matplotlib.figure import Figure # 型ヒント
        from matplotlib.ticker import PercentFormatter # 可視化

In[54]: # 乱数を固定
        np.random.seed(42)

        # データの表示設定
        pl.Config.set_float_precision(precision=3)
```

4 章 連続最適化で広告予算を配分する

```
# 可視化の設定
with open("../setting/visualization.json") as f:
    setting = json.load(f)
sns.set_theme(**setting)
sns.set_palette("gray", 4)
```

今回利用するデータセットを読み込みます。

```
In[55]: # データの読み込み
df_total_reach = pl.read_csv(
    "../data/ch04/df_total_reach.csv"
)

# データの確認
df_total_reach
```

Out[55]: shape: (20, 5)

	i	g	r	g_1	g_2
	i64	f64	f64	f64	f64
	1	1.895	0.497	0.934	0.961
	2	2.106	0.524	1.540	0.566
	3	3.633	0.551	0.725	2.908
	4	2.100	0.505	1.520	0.580
	5	1.179	0.430	0.640	0.540
	...	...	...	...	...
	16	1.347	0.426	0.859	0.488
	17	3.316	0.604	2.343	0.973
	18	1.441	0.429	0.755	0.685
	19	2.059	0.502	1.445	0.614
	20	1.538	0.461	0.805	0.733

データは REVISIO 株式会社が保有するグロスリーチとユニークリーチのデータです。一つひとつのキャンペーンに対して、約 4,500 人の CM 接触データからグロスリーチとユニークリーチを計算しています。二つの広告出稿媒体は匿名化した地上波の放送局です。

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

df_total_reach には過去の全 20 回のキャンペーン ($i = 1, \dots, 20$) に関して、各媒体のグロスリーチ (g_1, g_2) と、複数媒体合算のユニークリーチ r が格納されています。また、単純に各媒体のグロスリーチを足すと複数媒体合算のグロスリーチになるので、それを g としてもっています。

2 章の 1 点、3 章の 2 点ほどは極端なケースではないですが、今回のデータは 20 点しかなく、小規模データしか保有していないケースを考えています*3。以降では、このような小規模のデータから合算のユニークリーチをうまく推定する方法を考えていきます。

まずはデータの可視化からはじめましょう。

```
In[56]: def plot_total_reach(
        g: np.ndarray | pl.Series,
        r: np.ndarray | pl.Series,
        title: str,
    ) -> tuple[Figure, Axes]:
        """合算のグロスリーチとユニークリーチの関係を可視化"""

        fig, ax = plt.subplots()
        ax.scatter(g, r)
        ax.xaxis.set_major_formatter(
            PercentFormatter(xmax=1, decimals=0)
        )
        ax.yaxis.set_major_formatter(
            PercentFormatter(xmax=1, decimals=0)
        )
        ax.set(
            xlabel="グロスリーチ (合算)",
            ylabel="ユニークリーチ (合算)"
        )
        fig.suptitle(title)

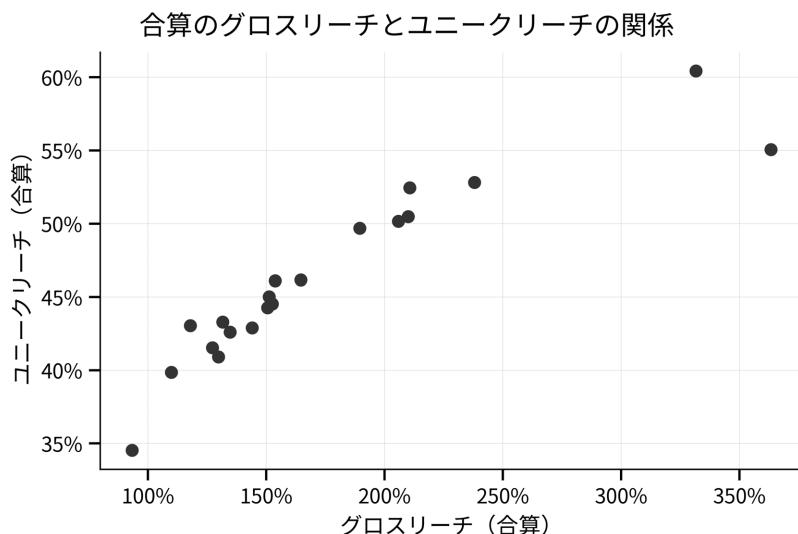
        return fig, ax

# 合算のグロスリーチに占める媒体1のグロスリーチの割合
df_total_reach = df_total_reach.with_columns(
    g_share_1=pl.col.g_1 / pl.col.g
)
```

* 3 規模の大きい企業やブランドが適切にデータを保管していればさらに多くのキャンペーンデータを保有している可能性もありますが、現実的にはそのような状況はあまり多くありません。

4 章 連続最適化で広告予算を配分する

```
# 可視化
fig, ax = plot_total_reach(
    g=df_total_reach["g"],
    r=df_total_reach["r"],
    title="合算のグロスリーチとユニークリーチの関係",
)
```



横軸に合算のグロスリーチが、縦軸に合算のユニークリーチが表示されています。媒体ごとのグロスリーチとユニークリーチの関係と同様、合算のグロスリーチが増加するにつれて、合算のユニークリーチも増加すること、その増加率は徐々に減少していることが読み取れます。

さらに、合算のグロスリーチが同程度でも、合算のユニークリーチはキャンペーンごとにばらつきがあることも見てとれます。ここから、同じ CM 出稿量でも、媒体ごとの出稿量を調整することで合算のユニークリーチを高めることができる可能性があるとわかります。

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

4.3.3 機械学習モデルによる予測

データが確認できたので、各媒体のグロスリーチから合算のユニークリーチを予測するモデルを作成しましょう。まずはデータを scikit-learn の入力形式に変換します。

```
In[57]: # scikit-learnの入力形式に合わせて変換
G = df_total_reach[["g_1", "g_2"]].to_numpy()
r = df_total_reach["r"].to_numpy()
```

それでは予測モデルを構築します。まずは機械学習モデルである Random Forest を用いた予測モデルを構築します。ハイパーパラメータはデフォルトにしておきます。

```
In[58]: from sklearn.ensemble import RandomForestRegressor

# モデルの準備
rf = RandomForestRegressor(n_jobs=-1, random_state=42)
```

Random Forest のような柔軟性の高い機械学習モデルは学習データに容易にオーバーフィットしてしまいます。よって、予測精度を評価するためには、学習用のデータセットと精度評価用のデータセットを分けて予測精度を評価する必要があります。そこで、今回は **Leave-One-Out Cross Validation (LOOCV)** を用いて、予測精度を評価することにします*4。

図 4.2 は、データポイントが 20 点の場合の LOOCV の処理を表しています。LOOCV はデータを 1 点だけ評価用のデータとして取り除き、残った 19 点を学習用データとして利用します。学習用データと評価用データを入れ替えながら、合計で 20 回の学習と評価を行います。

Cross Validation を用いて予測精度を評価するために、scikit-learn の model_selection モジュールから cross_val_predict 関数を用います。この関数は Cross Validation による予測値を返します。

cross_val_predict は、estimator 引数に予測モデル、X 引数に説明変数、y 引数に目的変数を指定します。また、引数 cv でデータの分割数を指定できます。今

* 4 もちろん、LOOCV ではなく、単に学習用データと評価用データを分割する方法も考えられます。今回はデータポイントが 20 点と少ないので、学習にできるだけ多くのデータ点を使いたいこと、学習と予測に時間がかからないことから、LOOCV を選択しました。予測精度の評価についての詳細は、門脇他 (2019) をご確認ください。

4 章 連続最適化で広告予算を配分する

精度評価用にデータを1点だけ取り除き、残りの19点で学習を行う



■ 図 4.2: Leave-One-Out Cross Validation による予測精度の評価

回はデータポイントの数、つまり 20 点で指定しているので、20 点中 19 点のデータで学習し、学習に使わなかった残りの 1 点のデータを予測する、LOOCV と同じ処理が実行されます。

```
In[59]: from sklearn.model_selection import cross_val_predict

# Leave-One-Out Cross Validationによる予測結果
cv = r.shape[0]
r_pred_rf = cross_val_predict(
    estimator=rf, X=G, y=r, cv=cv, n_jobs=-1
)
```

それでは、この予測結果を実測値と比較してみましょう。

```
In[60]: from sklearn.metrics import r2_score # 決定係数

def plot_compare_actual_prediction(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    prediction_label: str,
    title: str,
```

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

) -> tuple[Figure, Axes]:
    """予測値と実測値を比較"""

    # 決定係数
    r2 = r2_score(y_true=y_true, y_pred=y_pred)

    y = np.concatenate([y_true, y_pred])
    x = np.linspace(y.min(), y.max(), 100) # 対角線

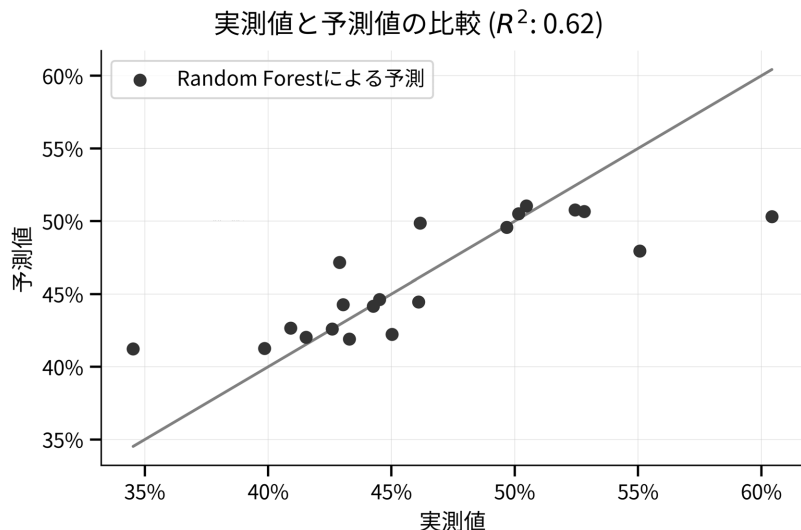
    fig, ax = plt.subplots()
    ax.scatter(
        y_true,
        y_pred,
        label=prediction_label,
        zorder=2
    )
    ax.plot(x, x, c=".5", zorder=1)
    ax.legend()
    ax.xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set(xlabel="実測値", ylabel="予測値")
    fig.suptitle(f"{title} ( $R^2$ : {r2:.2f})")

    return fig, ax

# 可視化
fig, ax = plot_compare_actual_prediction(
    y_true=r,
    y_pred=r_pred_rf,
    prediction_label="Random Forestによる予測",
    title="実測値と予測値の比較",
)

```

4 章 連続最適化で広告予算を配分する



横軸に実測値、縦軸に予測値がとられています。散布図の点は一つひとつの実測値と予測値の対応関係を表します。右上がりの対角線は実測値と予測値が完全に一致している場合を表しているので、対角線上にデータが集まっているほど予測精度が高いことを意味します。散布図を見ると、中央付近は相対的に対角線上にデータが集まっていますが、左端では過大な、右端では過小な予測をしていることも見てとれます。

散布図だけでは予測精度がどの程度かわかりにくいので、定量的な評価指標を用いて予測精度を評価します。定量的な予測精度の評価には決定係数 R^2 を用いました。

$$R^2 = 1 - \sum_{i=1}^n \frac{(y_i - \hat{y}_i)^2}{(y_i - \bar{y})^2}$$

ここで、 n はデータポイントの数 (つまり $n = 20$) を、 y_i はキャンペーン i の目的変数の実測値、 $\hat{y}_i$ はキャンペーン i の目的変数の予測値、 $\bar{y}$ は目的変数の実測値の平均を表しています。

決定係数は、モデルがどれだけ実際のデータのばらつきを説明できるかを示す指標です。具体的には、予測値と実測値のずれ (残差) の二乗和と、実測値が目的変数の平均からどれだけ離れているかの二乗和を比較し、モデルが単に平均値を使った

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

予測よりもどれだけ改善しているかを数値で表します。値は 1 に近いほど予測精度が高く、モデルが平均値よりも悪い予測をする場合は 0 を下回ることもあります。

決定係数は scikit-learn の `metrics` モジュールから `r2_score` 関数を用いて計算しました。計算された Random Forest の決定係数は 0.62 でした。Random Forest の予測値との乖離は、目的変数の平均値との乖離と比較して 6 割程度に抑えられています。

他に選択肢がなければこのモデルを用いるしかありませんが、まだその判断をする段階ではありません。さらなる予測精度を求めて、他のモデルも試してみることにしましょう。

4.3.4 線形回帰モデルによる予測

次に、線形回帰モデルによる予測を行います。グロスリーチがユニークリーチに与える影響が徐々に小さくなっていくことを示すため、説明変数の対数をとったモデルを考えます。

$$\mathbb{E}[R_{1 \cup 2} \mid g_1, g_2] = \beta_1 \log(g_1 + 1) + \beta_2 \log(g_2 + 1)$$

媒体 j の CM 出稿量が合算のユニークリーチに与える影響をパラメータ β_j で表しています。全媒体のグロスリーチが 0 の際は合算のユニークリーチも 0 になるように、 $\log(g_j + 1)$ と 1 を足していることに注意してください。

それでは、この線形回帰モデルの学習と予測を行いましょう。scikit-learn の `linear_model` から `LinearRegression` クラスを利用します。

```
In[61]: from sklearn.linear_model import LinearRegression

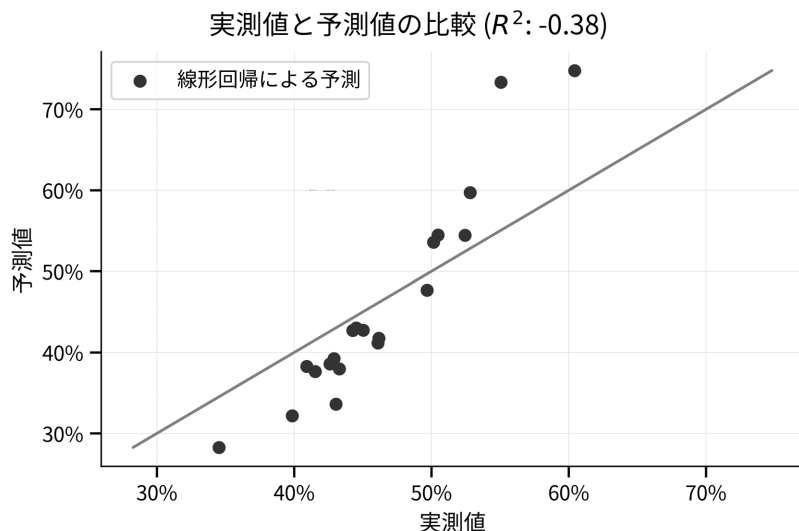
# 切片なしの線形回帰モデルの準備
lr = LinearRegression(fit_intercept=False)

# Leave-One-Out Cross Validationによる予測結果
r_pred_lr = cross_val_predict(
    lr, np.log1p(G), r, cv=cv, n_jobs=-1
)

# 可視化
fig, ax = plot_compare_actual_prediction(
    y_true=r,
    y_pred=r_pred_lr,
    prediction_label="線形回帰による予測",
```

4 章 連続最適化で広告予算を配分する

```
title="実測値と予測値の比較",
)
```



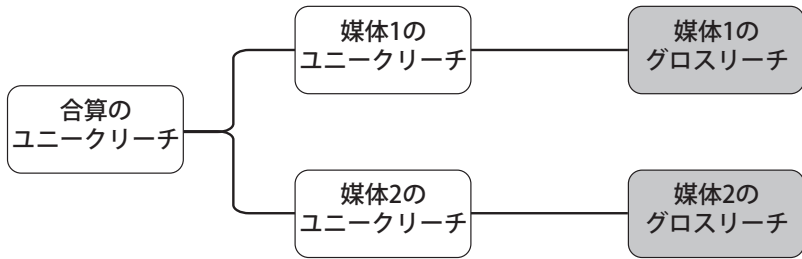
散布図を確認すると、実測値が高い場合に予測値も高くなるという右上がりの関係こそ見られますが、対角線からは大きく離れた点が多く見られます。実際、線形回帰モデルの決定係数は負の値をとっており、単に目的変数の平均で予測するよりも予測精度が悪いことがわかります。

4.3.5 数理モデルの構築

合算のユニークリーチ $E[R_{1 \cup 2} \mid g_1, g_2]$ を線形回帰モデルや機械学習モデルで推定しましたが、Cross Validation で予測精度を評価したところ、これらのアプローチでは満足する予測精度は得られませんでした。

そこで、以降ではデータ生成過程に対するドメイン知識を駆使した数理モデルを構築するアプローチをとります。各媒体のグロスリーチと合算のユニークリーチの関係性を直接モデリングするのではなく、問題を分解しましょう。これまでの議論を踏まえると、各媒体のグロスリーチから各媒体のユニークリーチが決まり、そこから合算のユニークリーチが決まるというメカニズムを想定することができそうです (図 4.3)。

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 4.3: 各媒体のグロスリーチと合算のユニークリーチの関係

この方針で問題を分解していきましょう。まず、条件付き期待値を変形すること

$$\begin{aligned}
 \mathbb{E}[R_{1 \cup 2} \mid g_1, g_2] &= \mathbb{E}[R_1 + R_2 - R_1 R_2 \mid g_1, g_2] \\
 &= \mathbb{E}[R_1 \mid g_1, g_2] + \mathbb{E}[R_2 \mid g_1, g_2] - \mathbb{E}[R_1 R_2 \mid g_1, g_2] \\
 &= \mathbb{E}[R_1 \mid g_1] + \mathbb{E}[R_2 \mid g_2] - \mathbb{E}[R_1 R_2 \mid g_1, g_2]
 \end{aligned}$$

を得ます。

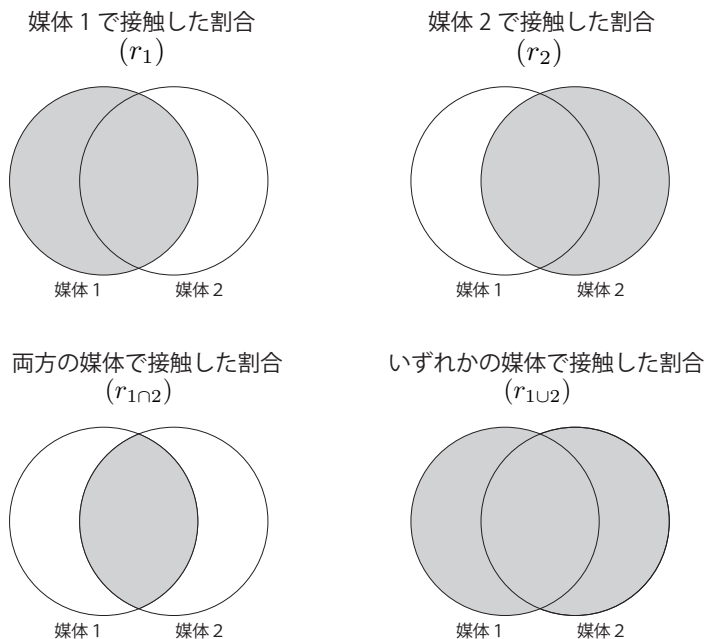
- 1 行目の変形：単に定義を代入した
- 2 行目への変形：期待値の線形性を利用して展開
- 3 行目への変形：媒体 1 の CM に接触するかどうか媒体 2 の出稿量 g_2 は関係がないことを仮定した（媒体 1 と媒体 2 を逆にした場合も同様に仮定）。通常、「ある放送局で CM 出稿量を増やすと、別の放送局での CM 接触が増加する」のような動きは考えづらいので、これは現実的な仮定だと考えられる

さらに、条件付き期待値の記載を簡単にするために、

$$\begin{aligned}
 r_j(g_j) &= \mathbb{E}[R_j \mid g_j] \\
 r_{1 \cap 2}(g_1, g_2) &= \mathbb{E}[R_1 R_2 \mid g_1, g_2] \\
 r_{1 \cup 2}(g_1, g_2) &= \mathbb{E}[R_{1 \cup 2} \mid g_1, g_2]
 \end{aligned}$$

と表記することにします。ここで、 $\cap$ は積集合を意味していて、 $r_{1 \cap 2}(g_1, g_2)$ は「媒体 1 の CM に接触して、かつ媒体 2 の CM にも接触した」というニュアンスを表現しています。これらの関係をベン図として図 4.4 にまとめました。

4 章 連続最適化で広告予算を配分する



■ 図 4.4: ユニークリーチのベン図

まず、図 4.4 の左上は媒体 1 が単体で獲得したユニークリーチを、右上は番組 2 が単体で獲得したユニークリーチを表しています。次に、二つの媒体に対するユニークリーチの重複を左下に表しています。最後に、右下が二つの媒体合算のユニークリーチを表しています。

この記法で元の式を置き換えると

$$r_{1\cup 2}(g_1, g_2) = r_1(g_1) + r_2(g_2) - r_{1\cap 2}(g_1, g_2)$$

と書けます。第 1 項と第 2 項は（グロスリーチで条件付けたときの）各媒体で獲得できるユニークリーチを、第 3 項は二つの媒体のリーチの重複を表しています。ここから、合算のユニークリーチを求めるには、まず各媒体のユニークリーチを足して、そこから重複部分を引けばよいことがわかりました。

ここで、媒体 j 単体のグロスリーチとユニークリーチの関係 $r_j(g_j)$ は、2 章と同様に負の二項分布から導出した関数形

$$r_j(g_j) = 1 - \left(\frac{\alpha_j}{g_j + \alpha_j} \right)^{\alpha_j}$$

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

を利用すればよいでしょう*5。一方で、重複部分に対するモデリングはまだ検討しておらず、数理モデルを構築する必要があります。

4.3.6 重複リーチのモデリング

独立性を仮定したモデリング

それでは、重複リーチのモデリングを検討しましょう。まずは二つの媒体の重複リーチを取り上げます。媒体 1 と媒体 2 の重複 $r_{1 \cap 2}(g_1, g_2)$ について考えてみましょう。 $R_1 R_2$ は R_1 と R_2 の両方が 1 のときだけ 1 をとり、そうでなければ 0 をとることに注意すると、期待値の定義から

$$\begin{aligned} r_{1 \cap 2}(g_1, g_2) &= \mathbb{E}[R_1 R_2 \mid g_1, g_2] \\ &= 0 \cdot 0 \cdot \Pr(R_1 = 0, R_2 = 0 \mid g_1, g_2) \\ &\quad + 1 \cdot 0 \cdot \Pr(R_1 = 1, R_2 = 0 \mid g_1, g_2) \\ &\quad + 0 \cdot 1 \cdot \Pr(R_1 = 0, R_2 = 1 \mid g_1, g_2) \\ &\quad + 1 \cdot 1 \cdot \Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2) \\ &= \Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2) \end{aligned}$$

と変形できます。つまり、媒体 1 と媒体 2 の両方の CM に接触する確率 $\Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2)$ を求めれば、2 媒体の重複リーチが求められます。

それでは、媒体 1 と媒体 2 の両方の CM に接触する確率 $\Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2)$ をどのようにして求めればよいでしょうか。一番単純な方法は、 R_1 と R_2 が (g_1, g_2) で条件付けた際に) 独立であると仮定してしまうことです*6。条件付き独立性を仮定すれば、同時確率を確率の積に分解できるので、

$$\begin{aligned} r_{1 \cap 2}(g_1, g_2) &= \Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2) \\ &= \Pr(R_1 = 1 \mid g_1, g_2) \Pr(R_2 = 1 \mid g_1, g_2) \\ &= \Pr(R_1 = 1 \mid g_1) \Pr(R_2 = 1 \mid g_2) \\ &= r_1(g_1) r_2(g_2) \end{aligned}$$

* 5 2 章ではポアソン分布から導出したリーチカーブと区別するために $r(g; \alpha)$ とパラメータ α を明示していました。本章では負の二項分布から導出したリーチカーブしか利用せず、区別の必要がないので、記法の簡略化のため α_j を省略して $r_j(g_j)$ と表記しています。

* 6 これは、「媒体 1 の CM に接触しやすい人は、媒体 2 の CM にも接触しやすい」とような状況がないことを仮定しています。現実的には、「テレビ好きはどちらの媒体の視聴時間も長いので、どちらの媒体でも CM 接触確率が高い」とような状況になっている可能性があります。よって、この仮定は現実的ではないので、後ほど仮定を緩めたモデルを検討します。

4 章 連続最適化で広告予算を配分する

と変形できます。これは、独立性を仮定すれば、重複リーチは単純に各媒体のユニークリーチの積で計算できるということを言っています。仮に、媒体 1 のユニークリーチが 40% で、媒体 2 のユニークリーチが 60% なら、重複リーチは $0.4 \times 0.6 = 0.24$ で 24% となります。

これを元の式に代入すると以下の関係を得ます。

$$r_{1 \cup 2}(g_1, g_2) = r_1(g_1) + r_2(g_2) - r_1(g_1)r_2(g_2)$$

ここから、独立性の仮定が成り立っているならば、媒体ごとのリーチカーブ $r_j(g_j)$ を推定すれば、そこから合算のユニークリーチ $r_{1 \cup 2}(g_1, g_2)$ も推定できることがわかります。

今回のモデリングでは、媒体ごとのグロスリーチから直接合算のユニークリーチを予測するのではなく、図 4.3 で見たように

- 媒体ごとのグロスリーチから媒体ごとのユニークリーチを予測
- 媒体ごとのユニークリーチから合算のユニークリーチを予測

という 2 段階のステップに分けて予測を行います。これは、3 章で行った「CM 出稿量と CM 認知率の関係」を、「CM 出稿量と CM 接触回数」と「CM 接触回数と CM 認知率の関係」に分解したのと同じアプローチです。数理モデルを考えることで、大きくて複雑な問題を小さくて簡単な問題に分解し、それぞれの問題を個別に扱うことができるようになります。

2 段階目のモデルは各媒体のユニークリーチが入力になっていることを明示するために、以降では

$$r_{1 \cup 2}(r_1(g_1), r_2(g_2)) = r_1(g_1) + r_2(g_2) - r_1(g_1)r_2(g_2)$$

と記載することにしましょう。

1 段階目のモデルは、各媒体のグロスリーチから各媒体のユニークリーチを予測するモデルです。これは、前述のように 2 章で実施した負の二項分布から導出したリーチカーブで推定できます。なので、以降では、媒体ごとのユニークリーチから合算のユニークリーチを予測する 2 段階目のモデルに焦点を当てましょう。

データによる検証

モデルを 2 段階に分けたので、各媒体のグロスリーチと合算のユニークリーチだけでなく、各媒体のユニークリーチが中間変数として要求されます。よって、各媒

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

体のユニークリーチのデータを入手する必要があります。幸いなことに、過去の 20 キャンペーンに対する各媒体のユニークリーチのデータが追加で手に入ったとしましょう。

それでは、追加で手に入ったデータを読み込みます。

```
In[62]: # 各媒体のユニークリーチが追加されたデータの読み込み
df_total_reach_with_single_reach = pl.read_csv(
    "../data/ch04/df_total_reach_with_single_reach.csv"
)

# データの確認
df_total_reach_with_single_reach
```

Out[62]: shape: (20, 7)

	i	g	r	g_1	g_2	r_1	r_2
	i64	f64	f64	f64	f64	f64	f64
1	1.895	0.497	0.934	0.961	0.368	0.313	
2	2.106	0.524	1.540	0.566	0.451	0.253	
3	3.633	0.551	0.725	2.908	0.296	0.475	
4	2.100	0.505	1.520	0.580	0.430	0.240	
5	1.179	0.430	0.640	0.540	0.306	0.266	
...	...	...	...	...	...	...	
16	1.347	0.426	0.859	0.488	0.337	0.226	
17	3.316	0.604	2.343	0.973	0.526	0.339	
18	1.441	0.429	0.755	0.685	0.293	0.274	
19	2.059	0.502	1.445	0.614	0.417	0.256	
20	1.538	0.461	0.805	0.733	0.333	0.276	

`df_total_reach_with_single_reach` には、既存のデータに加えて、各媒体のユニークリーチ (r_1 , r_2) が追加されています。

このデータを用いて、そもそも独立性の仮定が成り立っているかを確認してみましょう。もし独立性の仮定が成り立っているなら、キャンペーン i の合算のユニークリーチ $r_{i,1 \cup 2}$ は、各媒体のユニークリーチ $r_{i,j}$ を用いて以下のように表せるはずです。

4 章 連続最適化で広告予算を配分する

$$r_{i,1\cup 2} = r_{i,1} + r_{i,2} - r_{i,1}r_{i,2}$$

実際には、各媒体の CM への接触は独立ではない可能性があります。今回は媒体として地上波の異なる放送局を考えていますが、

- ・ パターン 1 (CM 接触に正の相関がある場合)
 - 「テレビをよく見る」、「テレビはあまり見ない」という、どの放送局の視聴にも共通する属性がある
 - よって、ある放送局の視聴時間が長い人は他の放送局の視聴時間も長く、逆に、ある放送局の視聴時間が短い人は他の放送局の視聴時間も短い
 - 結果として、ある放送局の CM に接触している人は、他の放送局の CM にも接触していることが多くなる。つまり、重複リーチが大きくなる
- ・ パターン 2 (CM 接触に負の相関がある場合)
 - テレビ視聴は可処分時間の奪い合いである
 - よって、ある放送局の視聴時間が長い人は他の放送局の視聴時間が短く、逆に、ある放送局の視聴時間が短い人は他の放送局の視聴時間が長い
 - 結果として、ある放送局の CM に接触している人は、他の放送局の CM には接触しないことが多くなる。つまり、重複リーチは小さくなる

という 2 通りのパターンが考えられます (図 4.5)。

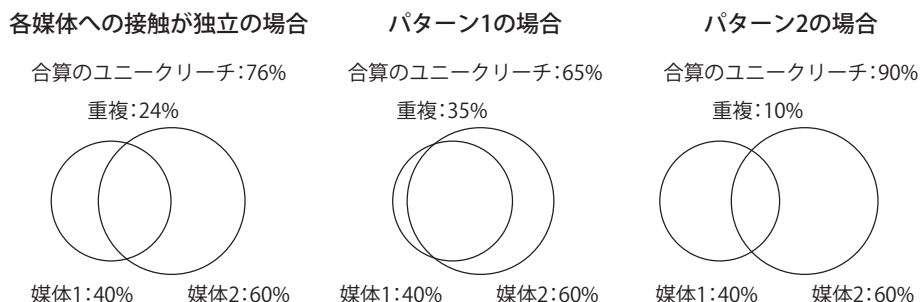
パターン1: テレビをよく見る人と見ない人がいる			パターン2: テレビを視聴時間は可処分時間の奪い合い		
	放送局1の 視聴時間	放送局2の 視聴時間		放送局1の 視聴時間	放送局2の 視聴時間
	6 時間 / 週	8 時間 / 週		1 時間 / 週	7 時間 / 週
	2 時間 / 週	1 時間 / 週		6 時間 / 週	2 時間 / 週

■ 図 4.5: テレビの視聴パターンに対する仮説

4.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

現実的には両者が混在しているパターンが多いでしょう。図 4.6 に、各媒体の CM への接触が独立の場合と、そうではない場合（パターン1、パターン2）の数値例をまとめました。例えば、媒体1のユニークリーチが40%、媒体2のユニークリーチが60%だったとします。各媒体のCMへの接触が独立の場合、重複リーチは 0.4×0.6 で24%となります。このとき、合算のユニークリーチは $0.4 + 0.6 - 0.24$ で76%です。

もしパターン1が多い場合は媒体間でリーチの重複が多くなるので（例：35%）、合算のユニークリーチは独立の場合より小さくなります（例：65%）。逆に、パターン2が多い場合は媒体間でリーチの重複が少なくなるので（例：10%）、合算のユニークリーチは独立の場合より大きくなります（例：90%）。



■ 図 4.6: パターンごとの重複リーチの数値例

それでは、独立性の仮定ははたして現実的な仮定なのか、データで確認してみましょう。

```
In[63]: # 後でscikit-learnで使うのでNumPyに変換
R = df_total_reach_with_single_reach[
    ["r_1", "r_2"]
].to_numpy()

# 実際の重複リーチ。r_1 + r_2 - r_1or2 で重複部分 r_1and2 になる
r_mult = R.sum(axis=1) - r

# 独立性を仮定した重複リーチの予測値
r_mult_pred = R.prod(axis=1)

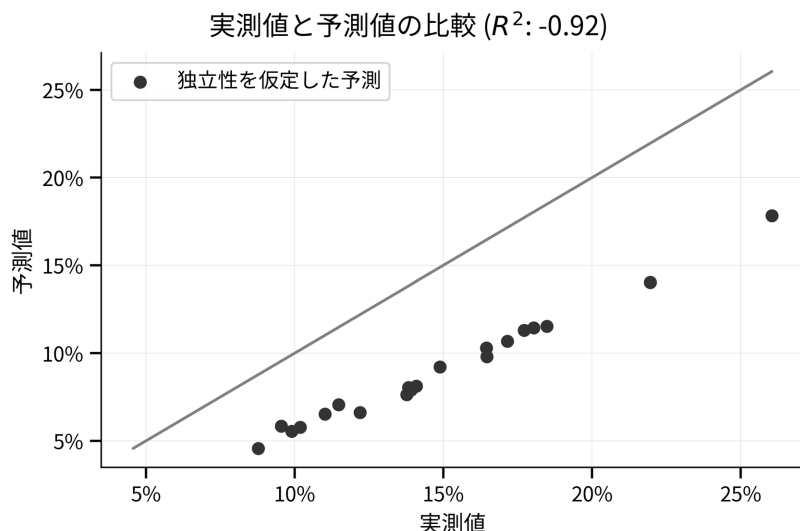
# 可視化で独立かどうかを確認
fig, ax = plot_compare_actual_prediction(
```

4 章 連続最適化で広告予算を配分する

```

y_true=r_mult,
y_pred=r_mult_pred,
prediction_label="独立性を仮定した予測",
title="実測値と予測値の比較",
)

```



散布図の点が対角線上にプロットされていれば、独立性を仮定した重複リーチの予測値と重複リーチの実測値が等しいので、独立性の仮定が成り立っていると解釈できます。実際に散布図を確認すると、点は対角線の下エリアにまとまっており、独立性を仮定した重複リーチの予測値は、重複リーチの実測値よりも小さい傾向が見てとれます。よって、独立性を仮定したモデリングは重複リーチを過小に予測してしまうことがわかります。独立性を仮定したモデリングではデータをうまく説明できず、この仮定はあまり現実的ではないことがわかりました。

また、独立性を仮定した予測値が実測値より小さいということは、媒体間の CM 接触にはパターン 2 の「可処分時間の奪い合い」の影響よりも、パターン 1 の「テレビをよく見る人と、あまり見ない人がいる」の影響が強かったと考えられます。

散布図をよく確認すると、対角線から大きく離れてはいますが、予測値と実測値には概ね線形の関係があることも見てとれます。独立性を仮定した予測値と線形の関係があるということは、独立性を仮定したモデリングはまったく的外れというわ

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

けではなく、部分的には有用である可能性を示唆しています。そこで、独立性を仮定したモデルに調整を加えるというアプローチでより現実に応じたモデルを構築することを目指します。

4.3.7 重複リーチの補正

補正係数による重複リーチの調整

ここまでの分析から、各媒体での CM 接触を独立と仮定したモデルでは実際のデータをうまく説明できないことがわかりました。一方で、予測値と実測値には概ね線形の関係が見られ、独立性を仮定したモデルを調整すればデータをうまく予測できそうなこともわかりました。それでは、各媒体での CM 接触を独立と仮定したモデル

$$r_{1 \cup 2}(g_1, g_2) = r_1(g_1) + r_2(g_2) - r_1(g_1)r_2(g_2)$$

をどのように調整すればよいかを考えましょう。

あらためて、媒体 1 と媒体 2 の重複リーチ $r_{1 \cap 2}(g_1, g_2)$ について考えます。 $r_{1 \cap 2}(g_1, g_2)$ を変形していくと

$$\begin{aligned} r_{1 \cap 2}(g_1, g_2) &= \Pr(R_1 = 1, R_2 = 1 \mid g_1, g_2) \\ &= \Pr(R_1 = 1 \mid R_2 = 1, g_1, g_2) \Pr(R_2 = 1 \mid g_1, g_2) \\ &= \Pr(R_1 = 1 \mid R_2 = 1, g_1) \Pr(R_2 = 1 \mid g_2) \\ &= \Pr(R_1 = 1 \mid R_2 = 1, g_1) r_2(g_2) \end{aligned}$$

を得ます。

- 1 行目の変形：単に定義による置き換え
- 2 行目への変形：同時確率と条件付き確率の関係*7を利用
- 3 行目への変形：媒体 2 の出稿量は媒体 1 の CM への接触には影響しないと仮定したことを利用（媒体 1 と媒体 2 を入れ替えても同じ）*8
- 4 行目への変形：第二項を定義により置き換え

* 7 確率変数 (X_1, X_2) に関して、 $\Pr(X_1 = x_1, X_2 = x_2) = \Pr(X_1 = x_1 \mid X_2 = x_2) \Pr(X_2 = x_2)$ という関係が成り立ちます。

* 8 条件付けた変数（ここでは g_2 ）が条件付けられた変数（ここでは R_1 ）に影響を与えない場合、条件付けた変数を条件付けから除外できます。つまり $\Pr(R_1 = 1 \mid g_1, g_2) = \Pr(R_1 = 1 \mid g_1)$ が成り立ちます。

4 章 連続最適化で広告予算を配分する

媒体間の CM 接触に独立性を仮定している場合は

$$\begin{aligned}\Pr(R_1 = 1 \mid R_2 = 1, g_1) &= \Pr(R_1 = 1 \mid g_1) \\ &= r_1(g_1)\end{aligned}$$

であり、これを元の式に戻すと

$$r_{1 \cap 2}(g_1, g_2) = r_1(g_1)r_2(g_2)$$

となります。

ただし、この独立性を仮定する方法ではデータをうまく説明できないことはもうわかっています。そこで、何らかの調整を加える方法を考えましょう。

一番単純な調整として、補正係数を掛けるという方法です。具体的には、補正係数 η を掛けて

$$\begin{aligned}\Pr(R_1 = 1 \mid R_2 = 1, g_1) &= \eta \Pr(R_1 = 1 \mid g_1) \\ &= \eta r_1(g_1)\end{aligned}$$

という調整を行います。つまり、媒体 2 の CM に接触したという条件が与えられたら、媒体 1 の CM に接触する確率が η 倍になるとモデリングしています。これを元の式に戻すと

$$r_{1 \cap 2}(g_1, g_2) = \eta r_1(g_1)r_2(g_2)$$

を得ます。

このモデリングは妥当でしょうか。まず、補正係数 η は 0 以上の値をとり、その値によって媒体 1 と媒体 2 の CM 接触の関係性を表現することができます。

- η が 0 以上 1 未満の場合、媒体 1 と媒体 2 の CM 接触には負の相関がある
- η が 1 の場合、媒体 1 と媒体 2 の CM 接触は独立
- η が 1 より大きい場合、媒体 1 と媒体 2 の CM 接触には正の相関がある

独立性を仮定したモデルを $\eta = 1$ のケースとして含有しているので、このモデルは独立性を仮定したモデルを一般化した、より表現力の高いモデルと解釈することができます。

一方で、この方法では、各媒体のリーチと補正係数の組み合わせによっては、現実的な制約を満たさないケースが出てきます。例えば、 $r_1 = 0.6$ 、 $r_2 = 0.5$ 、 $\eta = 3$ というケースを考えると、

4.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

$$\begin{aligned} r_{1\cap 2} &= 3 \times 0.6 \times 0.5 \\ &= 0.9 \end{aligned}$$

となり、重複リーチが媒体ごとのリーチ（0.6 または 0.5）よりも大きくなってしまいます。

このように、構築した数理モデルがどのような振る舞いをして、どこに限界があるのかを知ることは、分析から得られた知見を実務で活用する上で欠かせないステップです。モデルの考察を欠かすと、モデル想定外の振る舞いをしていることに気づかず、意思決定を誤ってしまう可能性があります。

もちろん、これは理論上発生しうるということだけで、現実的な補正係数とユニークリーチの水準でモデルを適用する範囲ではまったく問題が起きない可能性もあります。また、今回の目的は合算のユニークリーチを予測することであり、個別の重複リーチを精度高く予測することは必須の要件ではありません。すべての状況に適合した完璧なモデルを構築するのは難しいので、目的を達成するのに十分なモデルを構築することを目指しましょう。

このようなモデルの限界を認識しながら、今回はこの補正係数によるモデリングを採用することにします。具体的には、重複リーチに対する補正係数を導入したモデルは以下になります。

$$r_{1\cup 2}(g_1, g_2) = r_1(g_1) + r_2(g_2) - \eta r_1(g_1)r_2(g_2)$$

以降では、この補正係数による数理モデルがうまくデータに当てはまるのか、予測精度を確認していきましょう。

補正係数の推定

各媒体のユニークリーチから合算のユニークリーチを予測するモデルを作成するために、データに当てはまるような補正係数 η を推定する必要があります。

補正係数の推定は最小二乗法で行います。つまり、キャンペーン i の合算のユニークリーチの実測値 $r_{i,1\cup 2}$ と、モデルから予測される合算のユニークリーチ $\hat{r}_{1\cup 2}(r_{i,1}, r_{i,2})$ の二乗誤差を最小化するような補正係数 $\hat{\eta}$ を求めます。

具体的には、以下の最小化問題の解が $\hat{\eta}$ です。

$$\begin{aligned} \hat{\eta} &= \arg \min_{\eta \geq 0} \frac{1}{n} \sum_{i=1}^n (r_{i,1\cup 2} - r_{1\cup 2}(r_{i,1}, r_{i,2}))^2 \\ \text{where } r_{1\cup 2}(r_{i,1}, r_{i,2}) &= r_{i,1} + r_{i,2} - \eta r_{i,1} r_{i,2} \end{aligned}$$

4 章 連続最適化で広告予算を配分する

ここで、このモデルは各媒体のユニークリーチから合算のユニークリーチを予測するモデルなので、説明変数は各媒体のユニークリーチ (r_1, r_2) であり、各媒体のグロスリーチ (g_1, g_2) ではないことに注意してください。

予測モデルの実装

補正係数の推定方法が決まったので、予測モデルを `TotalReachEstimator` クラスとして実装しましょう。

Random Forest を用いた実装と同じく、予測精度の評価には `scikit-learn` の `cross_val_predict` 関数を利用したいので、`scikit-learn` と回帰モデルの実装を合わせます。そのために、`scikit-learn` の `base` モジュールにある `BaseEstimator` クラスと `RegressorMixin` クラスを継承します。

なお、今回の問題設定では広告出稿できる媒体は二つだけですが、より汎用的に m 個の媒体に対応できるように実装しました。結果として、多少コードが複雑になっています。

```
In[64]: import math # 組み合わせの数の計算にcombを使いたいので

from scipy.optimize import minimize # 最適化
# 回帰モデルの基底クラス
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.metrics import mean_squared_error # 二乗誤差
# 掛け算の項目を作成
from sklearn.preprocessing import PolynomialFeatures

class TotalReachEstimator(RegressorMixin, BaseEstimator):
    """
    各媒体のユニークリーチから合算のユニークリーチを予測するモデル
    """

    def __init__(self):
        pass

    def _transform(self, X: np.ndarray) -> np.ndarray:
        """重複リーチを表す説明変数を追加し、符号も追加

        Args:
            X (np.ndarray):
                説明変数。各媒体のユニークリーチを想定
```

4.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

```

Returns:
    np.ndarray: 重複リーチの説明変数を追加した説明変数
    """
    # 重複リーチの説明変数を追加
    X_pf = PolynomialFeatures(
        degree=self.m_, # 重複リーチの最大重複数
        include_bias=False, # 切片を含めない
        interaction_only=True, # 掛け算した項目のみに限定
    ).fit_transform(X)

    # 奇数項なら1、偶数項なら-1を掛ける
    signs: list[int] = []
    for k in range(1, self.m_ + 1):
        # m個からk個抜き出す組み合わせの数
        n_params = math.comb(self.m_, k)
        sign = 1 if k % 2 != 0 else -1
        signs.extend([sign] * n_params) # 追加

    return X_pf * signs

def _predict(
    self, X: np.ndarray, eta: np.ndarray
) -> np.ndarray:
    """重複部分を補正して合算のユニークリーチを予測

    Args:
        X (np.ndarray):
            説明変数。重複リーチを含む
        eta (np.ndarray):
            重複リーチの補正係数

    Returns:
        np.ndarray: 合算のユニークリーチの予測値
        """

    # 最初のm個は重複リーチではないので、1を掛ける
    eta_add_ones = np.concatenate([np.ones(self.m_), eta])

    return eta_add_ones @ X.T

def fit(self, X: np.ndarray, y: np.ndarray) -> Self:
    """
    各媒体のユニークリーチと合算のユニークリーチの関係を学習

```

4 章 連続最適化で広告予算を配分する

```

Args:
    X (np.ndarray):
        説明変数。各媒体のユニークリーチを想定
    y (np.ndarray):
        目的変数。合算のユニークリーチを想定

Returns:
    Self: 学習済モデル
"""

# 媒体数を保存
self.m_ = X.shape[1]

# 重複リーチの説明変数を追加
X_trans = self._transform(X=X)

# 最適な補正係数etaを推定
self.eta_ = minimize(
    fun=lambda eta: mean_squared_error(
        y, self._predict(X=X_trans, eta=eta)
    ),
    x0=np.ones(2**self.m_ - self.m_ - 1), # 初期値
    bounds=[(0, 10)]
    * (2**self.m_ - self.m_ - 1), # 制約
    tol=1e-10,
).x

return self

def predict(self, X: np.ndarray) -> np.ndarray:
    """合算のユニークリーチを予測

    Args:
        X (np.ndarray):
            説明変数。各媒体のユニークリーチを想定

    Returns:
        np.ndarray: 合算のユニークリーチの予測値
    """

    # 重複リーチの説明変数を追加
    X_trans = self._transform(X=X)

    return self._predict(X=X_trans, eta=self.eta_)

```

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

それでは、実装したメソッドを一つひとつ確認していきましょう。

まず、`_predict` メソッドでは、補正係数を使って合算のユニークリーチを予測します。ここで、`PolynomialFeatures` クラスを使って、重複リーチを表す掛け算の説明変数を作っています。具体的に、今回の問題設定のように $m = 2$ の場合は、 r_1 と r_2 に加えて $r_1 r_2$ が説明変数として追加され、合計 3 個の説明変数が生成されます。仮に $m = 3$ の場合は、 (r_1, r_2, r_3) に加えて、

$$r_1 r_2, r_1 r_3, r_2 r_3, r_1 r_2 r_3$$

が追加され、合計 7 個の説明変数が生成されます。

より一般的には、 $\sum_{k=1}^m \binom{m}{k} = 2^m - 1$ 個の説明変数が生成されます。補正係数は重複リーチを表す説明変数の数だけ必要なので、媒体ごとのユニークリーチを表す最初の m 個を除いた $2^m - m - 1$ 個です。よって、媒体数が多すぎると、推定すべき補正係数の数が爆発的に増えてしまい、推定が安定しない可能性があります。このような状況下では別のアプローチをとる必要があり、5 章であらためて考察します。

なお、作成された説明変数はそのまま使うのではなく、奇数個の媒体の重複リーチは足して、偶数個の媒体の重複リーチは引くという符号の調整を行っています。実際、 $m = 3$ の場合を例にすると、

$$\begin{aligned} r_{1 \cup 2} &= r_1 + r_2 + r_3 \\ &\quad - \eta_{12} r_1 r_2 - \eta_{13} r_1 r_3 - \eta_{23} r_2 r_3 \\ &\quad + \eta_{123} r_1 r_2 r_3 \end{aligned}$$

なので、奇数個の媒体の重複リーチは足して、偶数個の媒体の重複リーチは引かなければならないことがわかります。そこで、符号を `signs` として保存しておき、これを掛けることで説明変数の符号を調整しています。なお、 k 媒体の重複リーチの項目は m 媒体から k 媒体を抜き出す組み合わせの数だけあります。そこで、組み合わせの数を `math` モジュールにある `comb` 関数を使って計算しています。

次に、`_predict` メソッドでは、重複リーチ項が追加された説明変数を使って合算のユニークリーチを予測します。このメソッドでは、パラメータ `eta` の最初の m 個に 1 を追加したものを `eta_add_ones` として保存し、これを利用して合算リーチの予測値を計算しています。なぜこのような操作をしているのでしょうか。一般的な状況を考えるとイメージをつかみづらいので、 $m = 3$ の場合を例にしましょう。

4 章 連続最適化で広告予算を配分する

合算のユニークリーチの計算は以下のように行列を使って表すことができます。

$$r_{1\cup 2} = r_1 + r_2 + r_3 - \eta_{12}r_1r_2 - \eta_{13}r_1r_3 - \eta_{23}r_2r_3 + \eta_{123}r_1r_2r_3$$

$$= \begin{pmatrix} 1 & 1 & 1 & \eta_{12} & \eta_{13} & \eta_{23} & \eta_{123} \end{pmatrix} \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ -r_1r_2 \\ -r_1r_3 \\ -r_2r_3 \\ r_1r_2r_3 \end{pmatrix}$$

ここから、 $m = 3$ の場合は、パラメータ η の最初の三つに 1 を追加したものをを使って、行列の内積を計算することで合算のユニークリーチを計算することができます。_predict メソッドでは、これを一般化したものが実装されています。

さらに、fit メソッドでは、データから補正係数 η を推定し、 $\eta_{\text{eta_}}$ として保存しています。補正係数 η を推定するために、minimize 関数の中で、最小化したい目的関数、 η の初期値、 η の上限下限を設定しています。まず、パラメータの推定には最小二乗法を用いるので、最小化したい目的関数は二乗誤差になります。

次に、 η の初期値は 1 で設定しています。媒体ごとの CM 接触が独立の場合を想定した 1 が最も自然な初期値と考えました。

補正係数は 0 以上の値をとるので、 η の下限は 0、上限は 10 としています。理論上はより大きな値をとる可能性もありますが、10 を上限とすることで、推定が安定する可能性を高めています。

最後の predict メソッドは単純です。推定された補正係数 $\eta_{\text{eta_}}$ を使って、合算のユニークリーチを予測しています。

データを用いた検証

実装が完了したので、TotalReachEstimator クラスを使って、各媒体のユニークリーチと合算のユニークリーチの関係性を学習し、補正係数を確認してみましょう。

```
In[65]: # 全データで学習して補正係数を推定
tre = TotalReachEstimator().fit(R, r)

# 補正係数を確認
tre.eta_.round(2)[0]
```

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

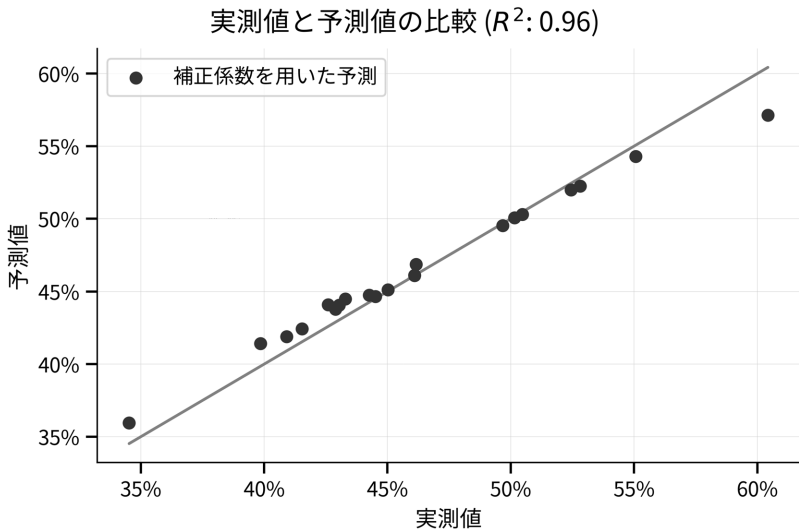
Out[65]: 1.61

補正係数は 1.61 であり、媒体 1 と媒体 2 の CM 接触における正の相関をうまく学習できていることがわかります。

次は、この補正係数を用いた合算のユニークリーチの予測精度を確認するため、Cross Validation を行います。scikit-learn と仕様を合わせたので、`cross_val_predict` 関数が使えます。

```
In[66]: # Leave-One-Out Cross Validationによる予測結果
tre = TotalReachEstimator()
r_pred_tre = cross_val_predict(tre, R, r, cv=cv)

# 予測精度の可視化
fig, ax = plot_compare_actual_prediction(
    y_true=r,
    y_pred=r_pred_tre,
    prediction_label="補正係数を用いた予測",
    title="実測値と予測値の比較",
)
```



4 章 連続最適化で広告予算を配分する

予測結果を見ると、実測値と予測値が概ね一致しています。予測精度を表す決定係数も 0.96 という高い水準にあり、補正係数によるモデリングはがデータに適合していると言えます。

なお、この予測は各媒体のユニークリーチから合算のユニークリーチを予測したものです。一方で、線形回帰モデルや機械学習モデルの予測精度は、各媒体のグロスリーチから合算のユニークリーチを予測したものです。なので、単純に比較することはできません。この 2 段階目のモデルだけでなく、1 段階目である各媒体のグロスリーチから各媒体のユニークリーチを予測するモデルも実装し、二つのモデルを統合することで初めて各媒体のグロスリーチから合算のユニークリーチを予測するモデルが完成します。予測精度の比較は、この統合モデルを使って行う必要があります。

後回しになってしまいましたが、統合モデルを構築するために 1 段階目のモデルを実装していきましょう。

4.3.8 各媒体のユニークリーチの予測

ユニークリーチの数理モデル

それでは、各媒体のリーチカーブを推定するモデルを実装します。2 章の分析を通じて、CM 接触回数の分布は負の二項分布でうまく近似できることがわかったので、今回も負の二項分布を利用しましょう。

負の二項分布を仮定すると、媒体 j のユニークリーチは以下の形で推定できます。

$$r_j(g_j) = 1 - \left(\frac{\alpha_j}{g_j + \alpha_j} \right)^{\alpha_j}$$

モデルの推定方法

モデルのパラメータは α_j で、媒体 j ごとにグロスリーチとユニークリーチの実測値から、パラメータ α_j を個別に推定します。具体的には、実測値と予測値の二乗誤差を最小化するようなパラメータ $\hat{\alpha}_j$ を求めます。

$$\begin{aligned} \hat{\alpha}_j &= \arg \min_{\alpha_j > 0} \frac{1}{n} \sum_{i=1}^n (r_{i,j} - r_j(g_{i,j}))^2 \\ &= \arg \min_{\alpha_j > 0} \frac{1}{n} \sum_{i=1}^n \left(r_{i,j} - \left(1 - \left(\frac{\alpha_j}{g_{i,j} + \alpha_j} \right)^{\alpha_j} \right) \right)^2 \end{aligned}$$

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

予測モデルの実装

それでは、各媒体のリーチカーブを推定するコードを実装しましょう。予測精度の評価には scikit-learn の `cross_val_predict` 関数を利用したいので、scikit-learn と回帰モデルの実装を合わせます。そのために、scikit-learn の base モジュールにある `BaseEstimator` クラスと `RegressorMixin` クラスを継承した `UniqueReachEstimator` クラスを実装していきます。

なお、このクラスは 3 章で実装した `FrequencyEstimator` クラスとほとんど同一です。

```
In[67]: from scipy.optimize import minimize_scalar # 最適化
        from scipy.stats import nbinom # 負の二項分布

class UniqueReachEstimator(RegressorMixin, BaseEstimator):
    """グロスリーチからユニークリーチを予測するモデル"""

    def __init__(self) -> None:
        pass

    @staticmethod
    def _predict(g: np.ndarray, alpha: float) -> np.ndarray:
        """負の二項分布を仮定したリーチカーブの予測

        Args:
            g (np.ndarray): グロスリーチ
            alpha (float): 負の二項分布のパラメータ

        Returns:
            np.ndarray: リーチの予測値
        """

        return 1 - nbinom.pmf(
            k=0, p=alpha / (g + alpha), n=alpha
        )

    def fit(self, X: np.ndarray, y: np.ndarray) -> Self:
        """グロスリーチとユニークリーチの関係を学習

        Args:
            X (np.ndarray): 説明変数。グロスリーチを想定
            y (np.ndarray): 目低変数。ユニークリーチを想定
```

4 章 連続最適化で広告予算を配分する

```

Returns:
    Self: 学習済みモデル
    """

    # 二乗誤差を最小化するalphaを推定して保存
    self.alpha_: float = minimize_scalar(
        fun=lambda alpha: mean_squared_error(
            y, self._predict(g=X, alpha=alpha)
        ), # 最小化したい関数
        bounds=(0.01, 100), # 推定範囲。alphaは0より大きい
    ).x

    return self

def predict(self, X: np.ndarray) -> np.ndarray:
    """学習済みのパラメータを使ってユニークリーチを予測

    Args:
        X (np.ndarray): 説明変数。グロスリーチを想定

    Returns:
        np.ndarray: ユニークリーチの予測値
    """

    return self._predict(g=X, alpha=self.alpha_)

```

FrequencyEstimator クラスの記載と重複しますが、UniqueReachEstimator クラスの各メソッドを一つひとつ見ていきましょう。

まず、_predict メソッドは負の二項分布を仮定したリーチカーブを計算するメソッドです。

次に、fit メソッドは説明変数 X と目的変数 y を受け取って、負の二項分布のパラメータ α_* を推定するメソッドです。ここで、 X は媒体 j のグロスリーチを、 y は媒体 j のユニークリーチを受け取ることを想定しています。

fit メソッドの内部では _predict メソッドが呼ばれています。SciPy の optimize モジュールにある minimize_scalar 関数を利用して、実際に観測されたユニークリーチと _predict メソッドで予測したユニークリーチの二乗誤差を最小化することでパラメータ α_* を推定しています。

最後に、predict メソッドは学習したパラメータ α_* を用いて、CM 出稿量に対してユニークリーチを予測するメソッドです。FrequencyEstimator クラスと

4.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

は異なり、UniqueReachEstimator クラスはこの predict メソッドを使ってユニークリーチを予測することが主な目的です。

以上で UniqueReachEstimator クラスの実装は完了です。

データを用いた検証

UniqueReachEstimator クラスが実装できたので、Cross Validation で予測精度を確認してみましょう。scikit-learn と仕様を合わせたので、cross_val_predict 関数が使えます。

UniqueReachEstimator クラスは一つの媒体に対するグロスリーチとユニークリーチの関係を推定するモデルです。なので、三つの媒体に対してそれぞれモデルを構築する必要があります。

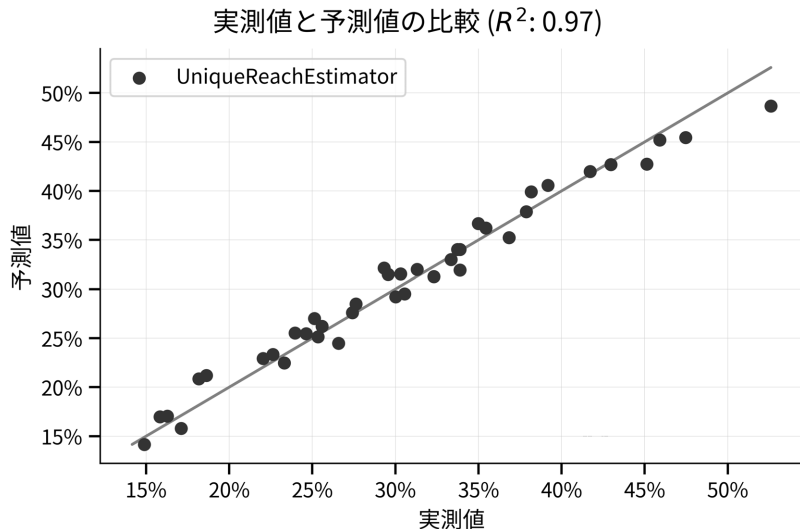
```
In[68]: # 媒体数
m = R.shape[1]

# 予測値を格納する変数。shapeが(20,2)になっていることに注意
R_pred_ure = np.zeros_like(R)
for j in range(m):
    # Leave-One-Out Cross Validationによる予測結果
    ure = UniqueReachEstimator()
    R_pred_ure[:, j] = cross_val_predict(
        ure, G[:, j], R[:, j], cv=cv, n_jobs=-1
    )
```

それでは、予測結果を可視化します。

```
In[69]: # 媒体が二つあるので、
# 実測値Rも予測値R_pred_ureもshapeが(20,2)のndarrayになっている
# flattenでshapeを(40,)に変換し、
# 複数媒体をまとめて予測精度を計算する
fig, ax = plot_compare_actual_prediction(
    y_true=R.flatten(),
    y_pred=R_pred_ure.flatten(),
    prediction_label="UniqueReachEstimator",
    title="実測値と予測値の比較",
)
```

4 章 連続最適化で広告予算を配分する



決定係数は 0.97 であり、実際のユニークリーチを精度よく予測できていることがわかります。各媒体のリーチカーブも可視化しておきましょう。

```
In[70]: def plot_unique_reach_with_prediction(
    G: np.ndarray,
    R: np.ndarray,
    estimators: list[UniqueReachEstimator],
) -> tuple[Figure, Axes]:
    """各媒体のユニークリーチの実測値とリーチカーブを可視化"""

    fig, ax = plt.subplots()

    # 予測結果
    g = np.linspace(0, G.max(), num=100)

    #
    媒体ごとに実際のユニークリーチと推定されたリーチカーブをプロット
    m = G.shape[1]
    markers = ["o", "D", "^", "x"]
    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for j in range(m):
        ax.scatter(
            G[:, j],
            R[:, j],
```

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

        marker=markers[j % m],
        label=f"媒体{j+1}実測値",
        zorder=1,
    )
    ax.plot(
        g,
        estimators[j].predict(g),
        linestyle=linestyles[j % m],
        label=f"媒体{j+1}予測値",
        zorder=2,
    )

    ax.legend()
    ax.xaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set_xlim(0, None)
    ax.set_ylim(0, None)
    ax.set(xlabel="グロスリーチ", ylabel="ユニークリーチ")
    fig.suptitle(
        "各媒体のユニークリーチの実測値とリーチカーブの予測"
    )

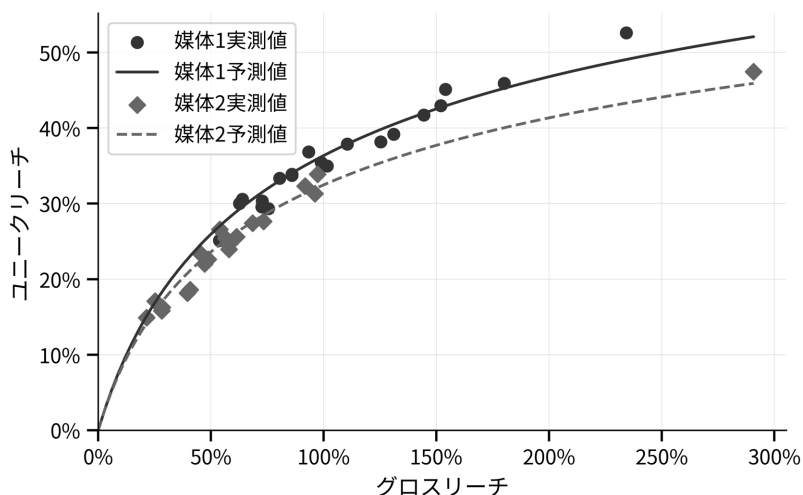
    return fig, ax

# リーチカーブの可視化
# 全データで学習したモデルを使ってリーチカーブを予測
ur_estimators = [
    UniqueReachEstimator().fit(G[:, j], R[:, j])
    for j in range(m)
]
fig, ax = plot_unique_reach_with_prediction(
    G=G, R=R, estimators=ur_estimators
)

```

4 章 連続最適化で広告予算を配分する

各媒体のユニークリーチの実測値とリーチカーブの予測



各媒体のグロスリーチとユニークリーチの関係をうまく予測できることが見てとれます。また、同じグロスリーチでも、媒体 1 のユニークリーチの方が媒体 2 のユニークリーチよりも大きいこともわかります。

よって、もしグロスリーチ 1%あたりのコストが同じなら、媒体 1 に予算を多く配分した方がユニークリーチを増加できる可能性が示唆されています。

4.3.9 予測モデルの統合

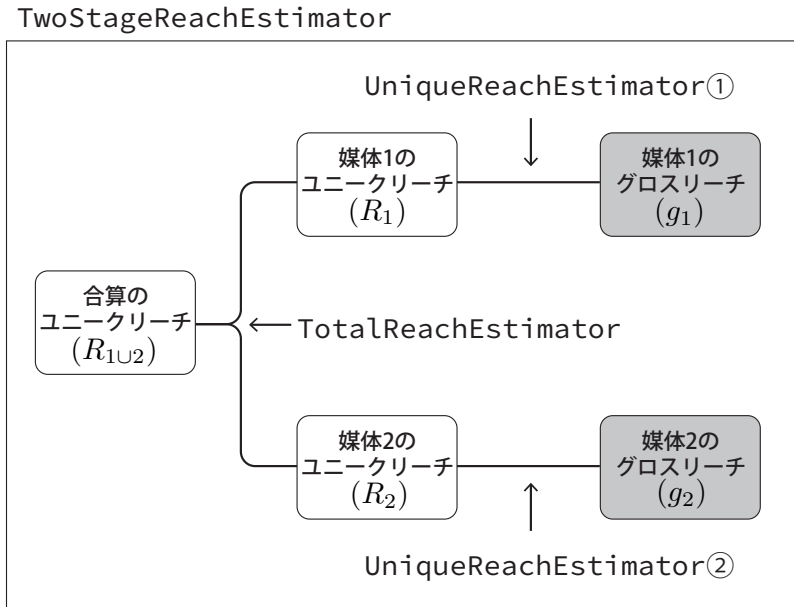
統合モデルのアルゴリズム

1 段階目の「各媒体のグロスリーチから各媒体のユニークリーチを予測するモデル」である `UniqueReachEstimator` クラスと、2 段階目の「各媒体のユニークリーチから合算のユニークリーチを予測するモデル」である `TotalReachEstimator` クラスが実装できました。最後に、二つのモデルをまとめた `TwoStageReachEstimator` クラスを実装することで、各媒体のグロスリーチから合算のユニークリーチを予測できるようにしましょう。

クラスの役割を把握するために、学習と予測のアルゴリズムを整理しておきましょう。各クラスの関係性は図 4.7 にまとめておきます。

まず、学習フェーズでは、二つのクラスを個別に学習します。`UniqueReachEstimator` クラスでは、媒体 $j = 1, 2$ に対して、グロスリーチとユニークリーチのデータ

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 4.7: 各クラスの関係性

$\{(g_{i,j}, r_{i,j})\}_{i=1}^n$ から、リーチカーブ

$$r_j(g_j) = 1 - \left(\frac{\alpha_j}{g_j + \alpha_j} \right)^{\alpha_j}$$

のパラメータ α_j を推定します。

並行して、TotalReachEstimator では、媒体ごとのユニークリーチと合算のユニークリーチのデータ $\{(r_{i,1}, r_{i,2}, r_{i,1\cup 2})\}_{i=1}^n$ から、

$$r_{1\cup 2}(r_1, r_2) = r_1 + r_2 - \eta r_1 r_2$$

のパラメータ η を推定します。

次に、予測フェーズでは、推定されたパラメータを用いて 2 段階の予測を行います。媒体ごとのグロスリーチ (g_1, g_2) が与えられたら、1 段階目の UniqueReachEstimator クラスは、

$$\hat{r}_j(g_j) = 1 - \left(\frac{\hat{\alpha}_j}{g_j + \hat{\alpha}_j} \right)^{\hat{\alpha}_j}$$

から媒体ごとのユニークリーチ $(\hat{r}_1, \hat{r}_2)$ を予測します。

4 章 連続最適化で広告予算を配分する

この予測値を 2 段階目の TotalReachEstimator クラスの入力として、学習された関係

$$\hat{r}_{1 \cup 2}(\hat{r}_1, \hat{r}_2) = \hat{r}_1 + \hat{r}_2 - \hat{\eta} \hat{r}_1 \hat{r}_2$$

から合算のユニークリーチ $\hat{r}_{1 \cup 2}$ を予測します。

統合モデルの実装

クラスの関係性が確認できたので、TwoStageReachEstimator クラスを実装しましょう。基本的には、UniqueReachEstimator クラスと TotalReachEstimator クラスを組み合わせるだけなので、実装は簡単です。ただし、TwoStageReachEstimator クラスは、説明変数の各媒体のグロスリーチ G、目的変数である合算のユニークリーチ r に加えて、中間変数として各媒体のユニークリーチ R を受け取る必要があります。よって、通常の scikit-learn のモデルとは少し異なる形で実装する必要があります。BaseEstimator クラスと RegressorMixin クラスを継承していません。

```
In[71]: class TwoStageReachEstimator:
    """
    各媒体のグロスリーチから合算のユニークリーチを予測するモデル
    """

    def __init__(self):
        pass

    def fit(
        self, G: np.ndarray, R: np.ndarray, r: np.ndarray
    ) -> Self:
        """二つのモデルを個別に学習

        Args:
            G (np.ndarray): 各媒体のグロスリーチ
            R (np.ndarray): 各媒体のユニークリーチ
            r (np.ndarray): 合算のユニークリーチ

        Returns:
            Self: 学習済みモデル
        """

        # 媒体数を保存
        self.m_ = G.shape[1]
        # 媒体ごとにグロスリーチとユニークリーチの関係を学習
```


4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```

self.ur_estimators_: list[UniqueReachEstimator] = [
    UniqueReachEstimator().fit(G[:, j], R[:, j])
    for j in range(self.m_)
]

# 媒体ごとの、ユニークリーチと、
# 合算のユニークリーチの関係を学習
self.tr_estimator_ = TotalReachEstimator().fit(R, r)

return self

def predict(self, G: np.ndarray) -> np.ndarray:
    """
    各媒体のグロスリーチから、
    合算のユニークリーチを2段階で予測

    Args:
        G (np.ndarray): 各媒体のグロスリーチ

    Returns:
        np.ndarray: 合算のユニークリーチの予測値
    """

    # 各媒体のグロスリーチから各媒体のユニークリーチを予測
    R_pred = np.column_stack(
        [
            self.ur_estimators[j].predict(G[:, j])
            for j in range(self.m_)
        ]
    )

    # 予測された各媒体のユニークリーチから、
    # 合算のユニークリーチを予測
    r_pred = self.tr_estimator_.predict(R_pred)

    return r_pred

def predict_individual_reach(
    self, G: np.ndarray
) -> np.ndarray:
    """
    各媒体のグロスリーチから、
    合算前の各媒体のユニークリーチと重複リーチを予測

```

4 章 連続最適化で広告予算を配分する

```

Args:
    G (np.ndarray): 各媒体のグロスリーチ

Returns:
    np.ndarray:
        各媒体のユニークリーチと重複リーチの予測値
    """

# 各媒体のグロスリーチから各媒体のユニークリーチを予測
R_pred = np.column_stack(
    [
        self.ur_estimators_[j].predict(G[:, j])
        for j in range(self.m_)
    ]
)

# 予測された各媒体のユニークリーチから重複リーチを予測
R_pred_trans = PolynomialFeatures(
    degree=self.m_,
    include_bias=False, # 切片を含めない
    interaction_only=True, # 掛け算した項目のみに限定
).fit_transform(R_pred)

# 補正係数で補正して出力
eta_add_ones = np.concatenate(
    [np.ones(self.m_), self.tr_estimator_.eta_]
)

return R_pred_trans * eta_add_ones

```

まず、fit メソッドでは、UniqueReachEstimator クラスと TotalReachEstimator クラスの fit メソッドを呼び出し、各モデルの学習を行っています。

次に、predict メソッドでは、UniqueReachEstimator クラスと TotalReachEstimator クラスの predict メソッドを呼び出し、各モデルの予測を行っています。予測の際には各媒体のグロスリーチだけが与えられ、各媒体のユニークリーチはわかりません。そこで、UniqueReachEstimator クラスで各媒体のユニークリーチを予測し R_pred に格納します。この R_pred を TotalReachEstimator クラスの入力として、合算のユニークリーチを予測しています。

最後に、合算前の各媒体のユニークリーチや重複リーチを確認するために、predict_individual_reach メソッドを実装しています。

4.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

以上で TwoStageReachEstimator クラスの実装は完了です。このモデルの予測精度をデータで検証していきましょう。

データを用いた予測精度の検証

それでは、Cross Validation で TwoStageReachEstimator クラスの予測精度を確認してみましょう。前述のように、TwoStageReachEstimator クラスは入力として (G, R, r) の三つが必要なので、scikit-learn の実装に合わせていません。よって、cross_val_predict 関数が使えません。

そこで、cross_val_predict 関数を使わずに、scikit-learn の model_selection モジュールに実装されている LeaveOneOut クラスを使って Cross Validation を行います。LeaveOneOut クラスは、データのうち一つを評価用データに、残りを学習用データに分割する Leave-One-Out Cross Validation が実装されたクラスで、ここまでの Cross Validation と同様に予測精度を評価できます。

それでは、Cross Validation を行いましょう。split メソッドを使うことで、学習用データと評価用データに分割されたインデックスを取得し、そのインデックスを用いてデータを分割しています。

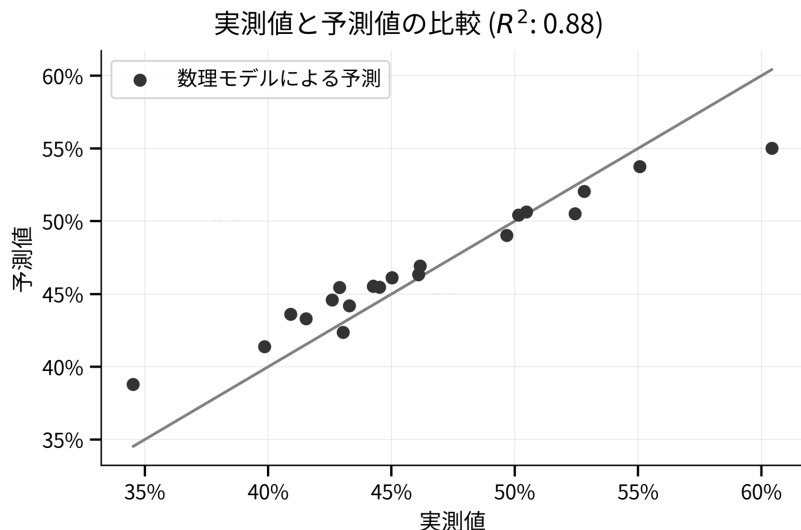
```
In[72]: from sklearn.model_selection import LeaveOneOut

# Leave-One-Out Cross Validationによる予測
# 予測結果の保存先
r_pred_tsre = np.zeros_like(r)
for i_train, i_val in LeaveOneOut().split(G):
    tsre = TwoStageReachEstimator()
    # 学習用データで学習
    tsre.fit(G[i_train], R[i_train], r[i_train])
    # 評価用データで予測
    r_pred_tsre[i_val] = tsre.predict(G[i_val])
```

Cross Validation による予測結果を可視化します。

```
In[73]: # 予測精度の可視化
fig, ax = plot_compare_actual_prediction(
    y_true=r,
    y_pred=r_pred_tsre,
    prediction_label="数理モデルによる予測",
    title="実測値と予測値の比較",
)
```

4 章 連続最適化で広告予算を配分する



2 段階の数理モデルによる予測の決定係数は 0.88 であり、Random Forest の予測精度である 0.62 と比較して大幅な改善が見られます。

この結果から、負の二項分布と補正係数を用いた 2 段階の数理モデルが、各媒体のグロスリーチから合算のユニークリーチを精度よく予測できることがわかりました。十分に予測精度の高いモデルが構築できたので、ステップ 3 に移り、このモデルを用いて最適な広告予算配分を決定しましょう。

4.4

ステップ 3：数理最適化問題を解いて最適なアクションを導出する

4.4.1 最適解の算出

ステップ 2 では最適化問題の未知の部分である、各媒体のグロスリーチと合算のユニークリーチの関係 $r_{1\cup 2}(r_1(g_1), r_2(g_2))$ をデータの力を借りて推定することができました。未知の部分がなくなった最適化問題を解くことで、最適な意思決定を導くことができます。学習した CM 出稿量と CM 認知率の関係を $\hat{r}_{1\cup 2}(\hat{r}_1(g_1), \hat{r}_2(g_2))$ とすると、最適化問題は以下のように定式化できます。

$$\max_{g_1, g_2} \hat{r}_{1\cup 2}(\hat{r}_1(g_1), \hat{r}_2(g_2))$$

4.4 ステップ3：数理最適化問題を解いて最適なアクションを導出する

$$\text{s.t. } 100c_1g_1 + 100c_2g_2 \leq b$$

$$g_1, g_2 \geq 0$$

$$\text{where } \hat{r}_{1 \cup 2}(\hat{r}_1(g_1), \hat{r}_2(g_2)) = \hat{r}_1(g_1) + \hat{r}_2(g_2) - \hat{\eta} \hat{r}_1(g_1) \hat{r}_2(g_2)$$

$$\hat{r}_j(g_j) = 1 - \left(\frac{\hat{\alpha}_j}{g_j + \hat{\alpha}_j} \right)^{\hat{\alpha}_j} \text{ for } j = 1, 2$$

この最適化問題を解くことで、最適な広告予算配分を決定することができます。仮に、予算上限 b が 3,000（万円）で、各媒体のグロスリーチ 1%あたりのコスト (c_1, c_2) がともに 10（万円）だったとします。この場合の最適な広告予算配分を求めてみましょう。

最適化問題を解くために、SciPy の optimize モジュールにある minimize 関数を利用します。

```
In[74]: def optimize(
    estimator: TwoStageReachEstimator,
    b: float,
    c: np.ndarray,
) -> tuple[float, np.ndarray, np.ndarray]:
    """合算のユニークリーチを最大化する予算配分を計算"""

    # 媒体数
    m = c.shape[0]

    result = minimize(
        fun=lambda g: -estimator.predict(
            g.reshape(1, -1)
        ), # 最小化なので-1を掛ける
        x0=np.zeros(m), # 初期値。なんでもいい
        bounds=[(0, None)] * m, # グロスリーチは0以上
        constraints=[
            {
                "type": "ineq",
                "fun": lambda g: b - 100 * c @ g
            }
        ], # 予算制約
        tol=1e-10,
    )

    g_opt = result.x # 最適なグロスリーチの配分
    s_opt = 100 * c * g_opt / b # 最適な予算シェア
    r_opt = -result.fun # 最大化された合算のユニークリーチ
```

4 章 連続最適化で広告予算を配分する

```
print(f"最適なグロスリーチ: {g_opt.round(3)}")
print(f"最適な予算配分 (金額) : {(b * s_opt).round(0)}")
print(f"最適な予算配分 (割合) {s_opt.round(3)}")
print(f"獲得できる合算のユニークリーチ: {r_opt:.3f}")

return r_opt, g_opt, s_opt

# 予算上限 (万円単位)
b = 3000
# グロスリーチ1%あたりのコスト (万円単位)
c = np.array([10, 10])

# 全データで学習
tsre = TwoStageReachEstimator().fit(G, R, r)

# 最適配分を計算
r_opt, g_opt, s_opt = optimize(estimator=tsre, b=b, c=c)
```

```
Out[74]: 最適なグロスリーチ: [2.13  0.87]
          最適な予算配分 (金額) : [2130.  870.]
          最適な予算配分 (割合)  [0.71  0.29]
          獲得できる合算のユニークリーチ: 0.548
```

最適化の結果を確認すると、予算の約 7 割を媒体 1 に配分することでユニークリーチを最大化できることがわかります。この結果は、各媒体のグロスリーチとユニークリーチの関係で見た、同じグロスリーチなら媒体 1 のユニークリーチの方が媒体 2 のユニークリーチよりも大きいという結果と整合的です。

4.4.2 最適化のインパクト

最適な予算配分はわかりましたが、最適でない配分にした場合と比べて、最適配分はどの程度ユニークリーチを増加させることができるのでしょうか。最適配分でもあまりユニークリーチが増加しない場合は、最適配分を採用するメリットが薄れます。そこで、最適でない配分も含めた、すべての予算配分に対してユニークリーチを計算し、最適配分との差を確認してみましょう。

4.4 ステップ 3：数理最適化問題を解いて最適なアクションを導出する

```

In[75]: def plot_optimal_share(
    estimator: TwoStageReachEstimator,
    b: int | float,
    c: np.ndarray,
) -> tuple[Figure, Axes]:
    """最適な予算配分を計算"""

    # 予算に占める媒体1のコストの割合
    s = np.linspace(0, 1, num=1001)
    # 予算配分から計算した各媒体のグロサリーチ
    g = b * np.array([s, 1 - s]) / (100 * c[:, None])

    # 予測モデルでユニークリーチを予測
    r_pred = estimator.predict(g.T)

    max_index = np.argmax(r_pred) #
    ユニークリーチを最大化するインデックス
    optimal_s, optimal_r = (
        s[max_index],
        r_pred[max_index],
    ) # そのときの予算配分とユニークリーチ

    # ユニークリーチを最小化するインデックス
    min_index = np.argmin(r_pred)
    worst_s, worst_r = (
        s[min_index],
        r_pred[min_index],
    ) # そのときの予算配分とユニークリーチ

    fig, ax = plt.subplots()
    ax.plot(s, r_pred, c=".7", zorder=1)
    ax.scatter(
        optimal_s,
        optimal_r,
        marker="o",
        label=f"最適({optimal_s * 100: .1f}%, {optimal_r *
100: .1f}%)",
        zorder=2,
    )
    ax.scatter(
        worst_s,
        worst_r,
        marker="x",

```

4 章 連続最適化で広告予算を配分する

```

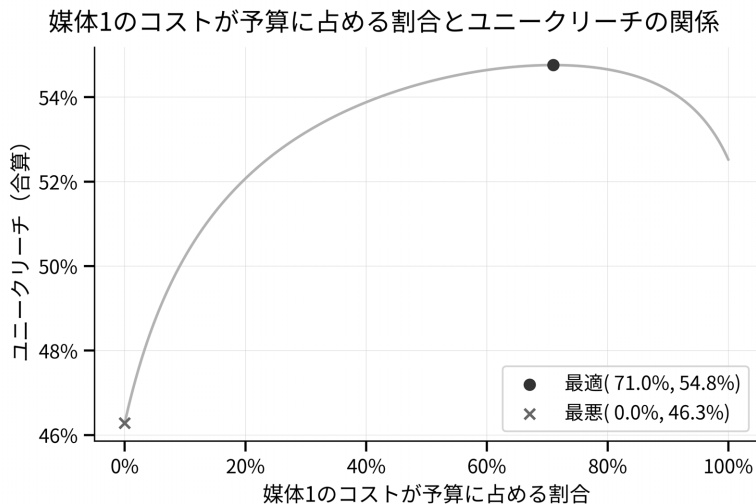
label=f"最悪({worst_s * 100:.1f}%, {worst_r *
100:.1f}%)",
zorder=3,
)
ax.legend()
ax.xaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.yaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.set(
    xlabel="媒体1のコストが予算に占める割合",
    ylabel="ユニークリーチ (合算)"
)
fig.suptitle(
    "媒体1のコストが予算に占める割合とユニークリーチの関係"
)

return fig, ax

```

各配分におけるユニークリーチの予測値を可視化

```
fig, ax = plot_optimal_share(estimator=tsre, b=b, c=c)
```



4.4 ステップ3：数理最適化問題を解いて最適なアクションを導出する

横軸には媒体1の広告予算の割合を、縦軸には合算のユニークリーチをとっています。媒体1に70.7%の広告予算を配分することで、合算のユニークリーチを最大化できることがわかります。この場合の合算のユニークリーチは54.8%です。

一方で、媒体2にすべての予算を配分すると、同じ予算3,000万円の中でユニークリーチを最小化することができます。この場合の合算のユニークリーチは46.3%です。

また、最適配分と最悪配分のユニークリーチの差は8.5%ポイントであることが読み取れます。よって、予算が3,000万円で、グロスリーチ1%あたりのコストが10万円の場合は、最適配分は最悪配分よりもユニークリーチを8.5%ポイント増加できると言えます。

ベンチマークと比較することで最適化のインパクトを定量的に評価でき、最適化の結果を意思決定に反映させる際にステークホルダーとの議論が円滑になります。

以上で、複数媒体でのユニークリーチを最大化する予算配分が計算でき、元々のビジネス課題に一定の回答を与えることができました。

予測結果の整合性の確認

最後に、予測結果の整合性を確認しましょう。今回の補正係数を用いたモデルでは、重複リーチが各媒体のリーチより大きくなってしまう可能性があります。少なくとも最適配分を探索した範囲で重複リーチが各媒体のリーチより大きくなっていないか確認しておきましょう。

TwoStageReachEstimator クラスの predict_individual_reach メソッドを使って、各媒体のユニークリーチや重複リーチを出力し、重複リーチと各媒体のリーチの関係を確認します。

```
In[76]: def validate_prediction(
        estimator: TwoStageReachEstimator,
        b: int | float,
        c: np.ndarray,
    ) -> np.bool_:
    """最適化で用いた設定に対して、
    重複リーチが各媒体のユニークリーチを超えないか確認
    """

    # 予算に占める媒体1のコストの割合
    s = np.linspace(0, 1, num=101)
    # 予算配分から計算した各媒体のグロスリーチ
    g = b * np.array([s, 1 - s]) / (100 * c.reshape(-1, 1))
```

4 章 連続最適化で広告予算を配分する

```

# 予測モデルでユニークリーチを予測
# 媒体ごとのユニークリーチと、重複部分のユニークリーチを予測
R_pred_individual = estimator.predict_individual_reach(
    g.T
)

# 各媒体のユニークリーチから重複リーチを引く
diff = R_pred_individual - R_pred_individual[:, [2]]

# 0未満になっていると重複リーチの方が大きい
# すべての予算配分で、0以上になっていることを確かめる
return np.all(diff >= 0)

```

```
validate_prediction(estimator=tsre, b=b, c=c)
```

Out[76]: True

重複リーチが各媒体のリーチよりも大きくなっていることはありませんでした。よって、今回の予算配分の最適化で探索した範囲に関しては、モデルの妥当性が保たれていると言えます。

4.5 本章のまとめ

本章では、「広告予算配分の最適化」というビジネス課題を題材に、3ステップフレームワークを用いて課題解決に取り組みました。

まず、ステップ1では、数理最適化問題の定式化を通じて、最適化した指標、制約条件、とりうるアクションを明確にしました。具体的には、最大化したい指標を合算のユニークリーチ、制約条件を予算制約、とりうるアクションをCM出稿量とすることで、ビジネス課題を制約付きユニークリーチ最大化問題

$$\begin{aligned}
 & \max_{g_1, g_2} \quad \mathbb{E}[R_{1 \cup 2} \mid g_1, g_2] \\
 & \text{s.t.} \quad 100c_1g_1 + 100c_2g_2 \leq b \\
 & \quad \quad g_1, g_2 \geq 0
 \end{aligned}$$

として定式化し、データサイエンスで扱える問題に変換しました。

ステップ1で定式化した数理最適化問題には、未知の部分として各媒体のグロスリーチと合算のユニークリーチの関係性 $\mathbb{E}[R_{1\cup 2} \mid g_1, g_2]$ が含まれていました。そこで、ステップ2ではこの未知の関係に対して数理モデルを構築し、データからパラメータを推定する方法を検討しました。具体的には、各媒体のグロスリーチと合算のユニークリーチの関係性 $\mathbb{E}[R_{1\cup 2} \mid g_1, g_2]$ を直接推定することが難しいので、「各媒体のグロスリーチと各媒体のユニークリーチの関係性」と「各媒体のユニークリーチと合算のユニークリーチの関係性」を個別に数理モデリングすることで、単純に機械学習モデルを用いるよりも解釈性が高く、予測精度も高いモデルを構築しました。

具体的には、1段階目の「各媒体のグロスリーチと各媒体のユニークリーチの関係性」を負の二項分布を仮定したリーチカーブ

$$r_j(g_j) = 1 - \left(\frac{\alpha_j}{g_j + \alpha_j} \right)^{\alpha_j}$$

としてモデリングしました。さらに、2段階目の「各媒体のユニークリーチから合算のユニークリーチの関係性」を

$$r_{1\cup 2}(r_1(g_1), r_2(g_2)) = r_1(g_1) + r_2(g_2) - \eta r_1(g_1)r_2(g_2)$$

としてリーチの重複を η で補正するモデルを構築しました。最後に、二つのモデルを統合することで、各媒体のグロスリーチから合算のユニークリーチを予測するモデルを構築しました。

最後に、ステップ3では、ステップ2で推定した関係性を代入した数理最適化問題

$$\begin{aligned} & \max_{g_1, g_2} \quad \hat{r}_{1\cup 2}(\hat{r}_1(g_1), \hat{r}_2(g_2)) \\ & \text{s.t.} \quad 100c_1g_1 + 100c_2g_2 \leq b \\ & \quad \quad g_1, g_2 \geq 0 \\ & \text{where} \quad \hat{r}_{1\cup 2}(\hat{r}_1(g_1), \hat{r}_2(g_2)) = \hat{r}_1(g_1) + \hat{r}_2(g_2) - \hat{\eta}\hat{r}_1(g_1)\hat{r}_2(g_2) \\ & \quad \quad \hat{r}_j(g_j) = 1 - \left(\frac{\hat{\alpha}_j}{g_j + \hat{\alpha}_j} \right)^{\hat{\alpha}_j} \quad \text{for } j = 1, 2 \end{aligned}$$

を解くことで、与えられた制約のもとで合算のユニークリーチを最大化する広告予算配分を求めました。このように、3ステップフレームワークを用いることで、ビジネス課題を数理最適化問題として定式化し、データを用いて未知の部分の推定し、最適なアクションを導くことができます。

4 章 連続最適化で広告予算を配分する

ただし、本章の途中でもふれたように、今回の数理モデルにもいくつかの論点が考えられます。

- 補正係数 η による補正は単純すぎないか？ 理論上、 η が大きいときには重複リーチが各媒体のリーチよりも大きくなってしまふことがある。もっと現実的な制約を満たすモデリングはないか？
- 今回は媒体が二つの状況を考えているので問題はなかったが、媒体数が増えると説明変数と補正係数の数が爆発的に増えてしまう。その場合、どうやって計算量を削減するか？
- 今回の問題設定ではユニークリーチのみに焦点を当てていたが、フリークエンシーの重複を知りたい場合はどうすればよいか？
- 今回の問題設定では中間変数である各媒体のユニークリーチが計測できていた。もし中間変数が計測できない場合はどう対応すればいいか？

このような論点が実務で問題になる場合は、状況に応じて数理モデルを調整していく必要があるでしょう*。

本章では広告出稿媒体が二つのケースを考えましたが、実務ではさらに多くの広告出稿媒体が存在する可能性があります。例えば、テレビ CM の出稿形態として CM を放送する番組を指定する「タイム」と呼ばれる広告出稿方法があります（詳細は 2.1.4 節を参照）。この場合、選択可能な番組の候補は数十から百に及ぶ場合もあり、その候補の中から適切な番組を選択するのは容易ではありません。5 章では、このような広告出稿媒体が多数存在する場合の最適化問題を取り扱い、実務での活用方法を解説します。

* 9 例えば、Danaher (1991) と Danaher and Smith (2011) は最初の三つの問題に取り組んでいます。

5章

離散最適化で広告出稿番組を 選択する

4章では、複数媒体のリーチ重複を「補正係数」を用いて調整し、予算配分を最適化する手法を構築しました。しかし、このアプローチでは媒体数が増えると推定すべき係数の数が爆発的に増加し、限られたデータではモデルの信頼性を保つのが難しくなる、という課題を抱えています。

5章ではこの限界を克服するため、多くの広告媒体を扱うケースを想定し、補正係数に頼らない新たな方法で数理モデリングを行い、予算配分を最適化する方法を解説します。

5.1 ビジネス課題：番組選択の最適化

4 章までは、テレビ CM の出稿方法として、特定の番組を指定せずに CM を出稿する「スポット」を例として、出稿量や複数媒体への予算配分といったビジネス課題に取り組んできました。

しかし、2 章でふれたように、テレビ CM にはもう一つ主要な出稿方式があります。それが、特定のテレビ番組を指定し、その番組のスポンサーとして CM を放送する「タイム」と呼ばれる方式です。タイムの大きな特徴は、単に CM が放送されるだけでなく、「この番組は〇〇（企業名や製品名）の提供でお送りします」といった提供アナウンスが伴う点にあります。提供アナウンスを通じてスポンサーとして認知されることで、企業イメージや信頼性が向上し、視聴者に対して製品やサービスをより深く、効果的に印象付けることが期待されます。

こうした効果を最大限に得るためには、どの番組に CM を出稿するかが重要な選択となりますが、CM 出稿を検討する番組の候補は数十、場合によっては百以上にものぼることがあります。したがって、限られた広告予算の中で、これらの多数の候補の中からどの番組を選び、どのように組み合わせればキャンペーン全体の効果を最大化できるのか、という新たな課題が生じます。

そこで本章では、「番組選択の最適化」を新たなビジネス課題として取り上げます。この課題に対してこれまで繰り返し活用してきた 3 ステップフレームワークを適用し、最適な番組候補を導き出す方法を解説していきます。

5.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

それでは、ビジネス課題「番組選択の最適化」を数理最適化問題として定式化していきましょう。まずは最適化したい指標を考えます。「広告出稿する番組の選択を最適化したい」という要望そのものには、最適化したい指標が言及されていません。ステークホルダーと議論して、具体的な指標を考える必要があります。

議論の結果、最大化したい指標は「複数番組合算のユニークリーチ」であるという結論に至ったとしましょう。ここで、「複数番組合算のユニークリーチ」とは、番組を問わず少なくとも一度は CM に接触した人の割合を指します。例えば、表 5.1 のような接触状況であれば、5 人中 4 人が少なくとも一つの番組で CM に接触しているので、複数番組合算のユニークリーチは 80%となります。

5.2 ステップ 1：ビジネス課題を数理最適化問題として定式化する

▼ 表 5.1: 複数の番組の CM に対する接触状況の例

	番組 1 で接触	番組 2 で接触	番組 3 で接触	いずれかの番組で接触
A さん	1	1	1	1
B さん	1	0	0	1
C さん	1	1	0	1
D さん	0	1	0	1
E さん	0	0	0	0

いま、広告を出稿可能な番組の候補が m 個あるとし、各番組を $j = 1, \dots, m$ で表します。そして、番組 j で放送された CM に一度でも接触したことがあれば 1、そうでなければ 0 をとる確率変数を R_j とします。さらに、CM を出稿した番組のうち、いずれかの番組で CM に接触したことを表す確率変数を R とします。 R はいずれかの番組で CM に接触したら 1、そうでなければ 0 をとります。なお、「複数番組合算のユニークリーチ」という表記が煩雑なので、以降では単に「合算のユニークリーチ」と表記します。

次に決定変数を考えます。今回動かせる変数はどの番組に CM を出稿するかです。表記を簡潔にするため、どの番組を選んだかをベクトル $\mathbf{d} = (d_1, \dots, d_j, \dots, d_m)$ で表現することにします。 $\mathbf{d}$ の j 番目の要素 d_j は、番組 j を選択したら 1 を、そうでなければ 0 をとります。例えば、番組候補が $m = 4$ 個あって、番組 1 と番組 3 に CM を放送するなら $\mathbf{d} = (1, 0, 1, 0)$ となります。以降では、選択した番組の組み合わせを番組群と呼ぶことにします。

最大化したい指標は合算のユニークリーチだったので、目的関数は選択した番組群 $\mathbf{d}$ で条件付けた合算のユニークリーチ $\mathbb{E}[R \mid \mathbf{d}]$ です。

最後に制約条件を考えます。トータルの予算を b とし、番組 j に CM を放送するために必要なコストを c_j とします。このとき、番組群 $\mathbf{d}$ に対するコストを予算内に収めるという制約は

$$\sum_{j=1}^m c_j d_j \leq b$$

と表現できます。

以上をまとめると、ビジネス課題「番組選択の最適化」を以下の制約付きユニークリーチ最大化問題として定式化できます。

$$\max_{\mathbf{d}} \quad \mathbb{E}[R \mid \mathbf{d}]$$

5 章 離散最適化で広告出稿番組を選択する

$$\begin{aligned} \text{s.t. } & \sum_{j=1}^m c_j d_j \leq b \\ & d_j \in \{0, 1\} \quad \forall j = 1, \dots, m \end{aligned}$$

「番組選択の最適化」というあいまいだったビジネス課題を、「番組候補の中から予算に収まるようにいくつかの番組に CM を流すことで、ユニークリーチをできるだけ大きくしたい」という形で明確化し、数式で表現できました。番組 j に CM を放送するために必要なコスト c_j とトータルの予算 b はわかっているので、上記の問題で未知の部分は合算のユニークリーチ $\mathbb{E}[R \mid \mathbf{d}]$ だけです。この関係性をデータを用いて推定することで、最適な番組群を求めることができます。

5.3

ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

5.3.1 必要なデータ

ステップ 1 で数理最適化問題の定式化ができました。最大化問題の未知の部分は、選択した番組と合算のユニークリーチの関係 $\mathbb{E}[R \mid \mathbf{d}]$ なので、次にやるべきことは、この条件付き期待値をデータから推定することです。

それでは、「どの番組群を選べばどのくらいの合算のユニークリーチになるのか」という関係性を知るためにはどのようなデータが必要でしょうか。ストレートに考えると、「どの番組に CM を流したのか」と、結果として「どのくらいの合算のユニークリーチが実現したのか」というデータがあればよさそうです。

▼ 表 5.2: 企業ごとの CM 出稿番組と合算ユニークリーチ

企業	出稿番組	合算ユニークリーチ
1	[1, 2]	30%
2	[1, 2, 3]	50%
3	[3, 4]	40%
⋮	⋮	⋮

表 5.2 は企業がどの番組に CM を出稿し、結果としてどのくらいの合算のユニークリーチを獲得したかを示すデータです。CM を放送した番組の組み合わせが十

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

分にあれば、上記のデータから条件付き期待値を推定できると考えられます。しかし、すべての番組の組み合わせを網羅したデータを入手することは現実的には困難です。仮に番組の候補が 10 番組の場合でも、各番組 CM を放送したかどうかの組み合わせは $2^{10} = 1024$ 通り存在します。すべての組み合わせを網羅できるようにうまく CM を放送した企業を見つけてきて、合算のユニークリーチを測定することは現実的ではないでしょう。

よって、以降では一部の番組の組み合わせに対してのみデータが得られている、という実務的な状況を取り扱い、条件付き期待値を推定する方法を考えます。

5.3.2 データの読み込みと可視化

それでは、本章で利用するデータを確認していきましょう。

まずは本書を通じて利用するライブラリを読み込みます。その他必要なライブラリは利用する際に適宜読み込んでいきます。

```
In[77]: import json # ファイル読み込み

import matplotlib.pyplot as plt # 可視化
import numpy as np # データ処理
import polars as pl # データ処理
import seaborn as sns # 可視化
from matplotlib.axes import Axes # 型ヒント
from matplotlib.figure import Figure # 型ヒント
from matplotlib.ticker import PercentFormatter # 可視化
```

```
In[78]: # 乱数を固定
np.random.seed(42)

# データの表示設定
pl.Config.set_float_precision(precision=3)
pl.Config.set_tbl_cols(14)
np.set_printoptions(precision=3)

# 可視化の設定
with open("../setting/visualization.json") as f:
    setting = json.load(f)
sns.set_theme(**setting)
sns.set_palette("gray", 4)
```

5 章 離散最適化で広告出稿番組を選択する

今回の分析で利用するデータセットを読み込みます。今回のデータは、データフレームの要素として `list` をもつため、`csv` 形式ではなく `parquet` 形式^{*1}で保存しています。`parquet` 形式のデータの読み込みには `Polars` の `read_parquet` 関数を用います。

```
In[79]: # データの読み込み
df_company_reach = pl.read_parquet(
    "../data/ch05/company_reach.parquet"
)

# データの確認
df_company_reach
```

Out[79]: shape: (420, 118)

is_train	k	r	sum_d	j_list	d_001	d_002	...	d_113
i32	u32	f64	u32	list[u32]	u8	u8	...	u8
1	1	0.072	1	[1]	1	0	...	0
1	2	0.164	2	[1, 61]	1	0	...	0
1	3	0.211	5	[1, 30, ...107]	1	0	...	0
1	4	0.316	11	[1, 15, ...110]	1	0	...	0
1	5	0.266	6	[1, 23, ...107]	1	0	...	0
...	...	...	...	...	...	...	...	...
0	416	0.018	1	[110]	0	0	...	0
0	417	0.093	2	[110, 113]	0	0	...	1
0	418	0.035	1	[111]	0	0	...	0
0	419	0.087	1	[112]	0	0	...	0
0	420	0.068	1	[113]	0	0	...	1

データは REVISIO 株式会社が保有する企業ごとの出稿番組とユニークリーチのデータです。個々の企業に対して、約 4,500 人の CM 接触データからユニークリーチを計算しています。

`df_company_reach` の各列には以下の情報が格納されています。

* 1 `parquet` 形式は、高い圧縮効率、高速な読み書き、多様なデータ型の対応といった、さまざまな利点をもつデータ保存形式です。詳細は <https://parquet.apache.org/> を確認してください。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

- `is_train`：学習用データなら 1、評価用データなら 0 をとるフラグ
- `k`：企業を表すインデックス
- `r`：番組合算のユニークリーチ
- `sum_d`：CM を出稿した番組の総数
- `j_list`：CM を出稿した番組のリスト。番組の総数は 113 番組
- `d_001 ~ d_113`：番組 `j` に出稿したかどうかを表すフラグ。予測モデルに説明変数として用いるために、出稿していれば 1、そうでなければ 0 をとる変数に `j_list` を変換したもの

`df_company_reach` には、企業ごとに、CM を出稿した番組と獲得できたユニークリーチが記録されています。例えば、企業 2 は番組 1 と番組 61 に CM を出稿し、合算のユニークリーチは 16.4%となっています。一方で、企業 417 は番組 110 と番組 113 に CM を出稿し、合算のユニークリーチは 9.3%となっています。よって、同じ二つの番組に CM を流すのであれば、企業 1 は企業 416 より効率的な番組群を選択できていると言えます（もちろん、番組に CM を出稿するコストが同じとは限らないので、本当に効率的かどうかは別途分析が必要です）。

なお、`df_company_reach` には、学習用データと評価用データが含まれています。学習用データは数理モデルのパラメータの学習に利用し、評価用データは学習したモデルの性能を評価するために利用します。

	期間1	期間2
企業1	学習に利用	利用しない
企業2		
企業3		
企業4		
企業5		
企業6	利用しない	評価に利用
企業7		
企業8		
企業9		
企業10		

■ 図 5.1: 学習用データと評価用データの分割

5 章 離散最適化で広告出稿番組を選択する

構築した予測モデルの性能を評価するために、図 5.1 のようにデータを 4 分割し、そのうち二つを学習用データと評価用データとして利用しています。期間 1 でモデルの学習を行い、期間 2 でモデルの性能を評価します。このとき、同じ企業のデータを学習用データと評価用データの両方に用いると、モデルの性能が過大評価されてしまうため、期間 1 の学習に用いた企業とは別の企業に対して期間 2 で予測を行うようにしています。

それでは、学習用データと評価用データを分割しておきましょう。なお、予測精度の評価では、モデルが重複リーチをうまく考慮できているかを確認したいので、複数の番組に CM を出稿している企業に限定して予測精度を評価することにします。

```
In[80]: # 学習用データと評価用データに分割
# 評価用データは、複数番組に出稿がある企業に限定
df_company_reach_train = df_company_reach.filter(
    pl.col.is_train == 1
)
df_company_reach_valid = df_company_reach.filter(
    pl.col.is_train == 0, pl.col.sum_d > 1
)

company_count_train = df_company_reach_train.shape[0]
company_count_valid = df_company_reach_valid.shape[0]

print(f"学習用データに含まれる企業数：{company_count_train}")
print(f"評価用データに含まれる企業数：{company_count_valid}")
```

```
Out[80]: 学習用データに含まれる企業数：213
         評価用データに含まれる企業数：92
```

学習用データとして 213 点、評価用データとして 92 点のデータが確保できています。よって、4 章までの状況と比較すると、データ量自体は大幅に増えていると言えます。一方で、番組の組み合わせは $2^{113} \approx 10^{34}$ 通り存在するため、すべての組み合わせを網羅するには圧倒的にデータが足りていないとも言えます。

それでは、分割した学習用データを可視化してみましょう。横軸に番組数、縦軸に合算のユニークリーチをとり、学習用データの散布図をプロットします。この散布図を見ることで、番組数と合算のユニークリーチの関係性を把握することができます。

5.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

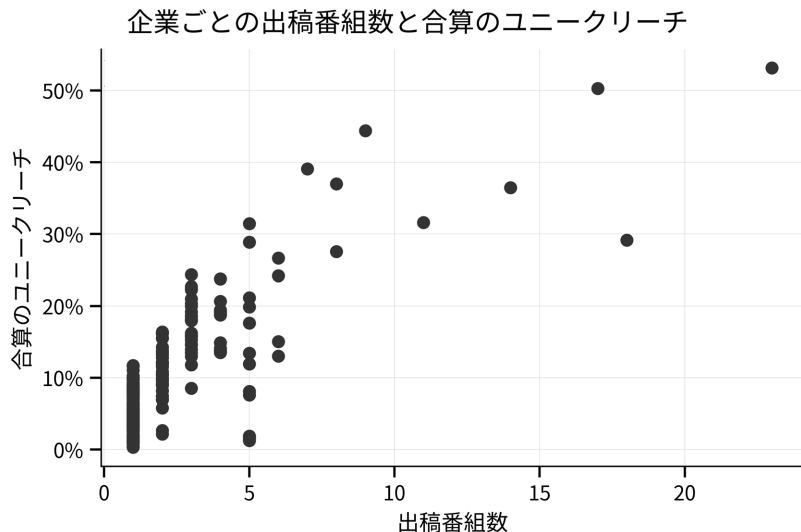
```
In[81]: def plot_scatter(
        x: np.ndarray | pl.Series,
        y: np.ndarray | pl.Series,
        xlabel: str,
        ylabel: str,
        title: str,
    ) -> tuple[Figure, Axes]:
        """散布図を描く"""

        fig, ax = plt.subplots()
        ax.scatter(x, y)
        ax.set(xlabel=xlabel, ylabel=ylabel)
        ax.yaxis.set_major_formatter(
            PercentFormatter(xmax=1, decimals=0)
        )
        fig.suptitle(title)

        return fig, ax

# 散布図を可視化
fig, ax = plot_scatter(
    x=df_company_reach_train["sum_d"],
    y=df_company_reach_train["r"],
    xlabel="出稿番組数",
    ylabel="合算のユニークリーチ",
    title="企業ごとの出稿番組数と合算のユニークリーチ",
)
```

5 章 離散最適化で広告出稿番組を選択する



基本的には、より多くの番組に CM を出稿することで合算のユニークリーチが増加する傾向があります。ただし、同じ番組数でも、合算のユニークリーチにはばらつきがあります。ここから、効率的に番組群を選択することで、ユニークリーチを増加させうる可能性を読み取れます。

5.3.3 線形回帰モデルによる予測

それでは、選択した番組から合算のユニークリーチを予測するモデルを構築していきます。 $\mathbb{E}[R \mid \mathbf{d}]$ を推定する最も単純なアプローチとして、番組 j を選択していれば 1、そうでなければ 0 をとるダミー変数 d_j を説明変数とした線形回帰モデル

$$\mathbb{E}[R \mid \mathbf{d}] = \sum_{j=1}^m \beta_j d_j$$

が考えられます。

ただし、この線形回帰モデルは重複リーチを一切考慮できていないので、番組数が増えると合算のユニークリーチが過大に推定される可能性があります。モデルに工夫の余地は残されていますが、ひとまずはベースラインとしてこの線形回帰モデルを構築してみましょう。

5.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

モデルの学習と予測を行うため、まずはデータを scikit-learn の入力形式に変換します。

```
In[82]: # 学習用データ
# ダミー変数の部分を説明変数として抽出
X_train = df_company_reach_train.select(
    pl.selectors.starts_with("d_")
).to_numpy()
# 目的変数は合算のユニークリーチ
y_train = df_company_reach_train["r"].to_numpy()

# 評価用データ
X_valid = df_company_reach_valid.select(
    pl.selectors.starts_with("d_")
).to_numpy()
y_valid = df_company_reach_valid["r"].to_numpy()
```

次に、モデルの学習と予測を行います。線形回帰モデルには、scikit-learn の `linear_model` モジュールから `LinearRegression` クラスを利用します。

```
In[83]: from sklearn.linear_model import LinearRegression

# モデルの学習。切片を含めないため、fit_intercept=False
lr = LinearRegression(
    fit_intercept=False).fit(X_train, y_train
)

# 予測
y_pred_lr = lr.predict(X_valid)
```

線形回帰モデルによる学習と予測ができたので、実測値と比較して予測精度を確かめましょう。予測精度の指標としては決定係数を用います。

```
In[84]: from sklearn.metrics import r2_score # 決定係数

def plot_compare_actual_prediction(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    prediction_label: str,
    title: str,
) -> tuple[Figure, Axes]:
```

5 章 離散最適化で広告出稿番組を選択する

```

"""予測値と実測値を比較"""

# 決定係数
r2 = r2_score(y_true=y_true, y_pred=y_pred)

y = np.concatenate([y_true, y_pred])
x = np.linspace(y.min(), y.max(), 100) # 対角線

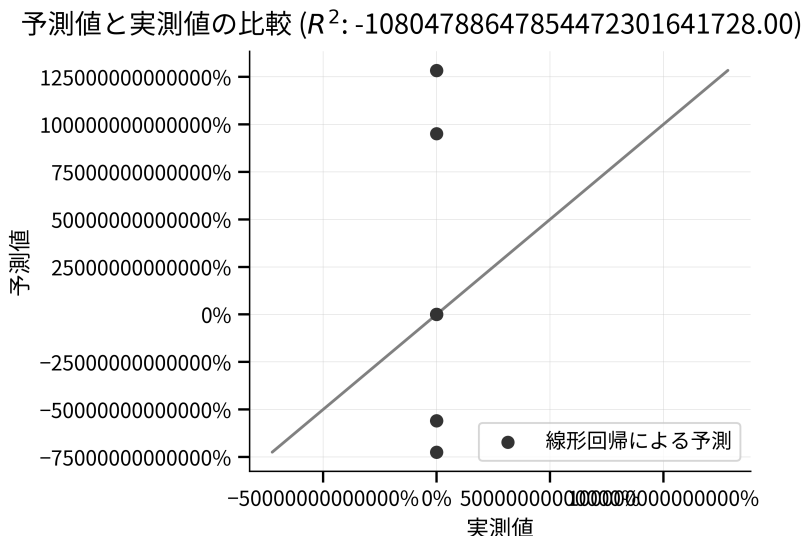
fig, ax = plt.subplots()
ax.scatter(
    y_true, y_pred, label=prediction_label, zorder=2
)
ax.plot(x, x, c=".5", zorder=1)
ax.legend()
ax.xaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.yaxis.set_major_formatter(
    PercentFormatter(xmax=1, decimals=0)
)
ax.set(xlabel="実測値", ylabel="予測値")
fig.suptitle(f"{title} ( $R^2$ : {r2:.2f})")

return fig, ax

# 線形回帰モデルの予測値と実測値を可視化
fig, ax = plot_compare_actual_prediction(
    y_true=y_valid,
    y_pred=y_pred_lr,
    prediction_label="線形回帰による予測",
    title="予測値と実測値の比較",
)

```


5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



結果を見ると、決定係数は大きな負の値をとっており、モデルが予測を大きく外していることがわかります（極端に桁数の大きい予測値をとってしまっており、目盛りの数字も潰れてしまっています）。

これは「完全な多重共線性」の影響です。完全な多重共線性とは、説明変数間に線形従属関係がある場合に発生する問題で、この問題が発生すると回帰係数が一意に定まらなくなります*2。例えば、「番組 1 に CM を出稿している企業は常に番組 2 にも CM を出稿している」といった場合*3に完全な多重共線性が起きます。この場合、番組 1 と番組 2 が合算のユニークリーチに与える影響を区別することができません。

完全な多重共線性が発生しているかどうかは、説明変数の行列のランクから確認できます。説明変数の数と行列のランクが一致しているならば、完全な多重共線性は発生していません。逆に、説明変数の数よりも行列のランクが小さい場合、説明変数に線形従属関係があるので、完全な多重共線性が発生しているとわかります。

* 2 線形従属はないが、説明変数間に強い相関がある場合は（完全ではない）多重共線性と呼ばれます。（完全ではない）多重共線性の場合は回帰係数が一意に定まらないという問題は起きませんが、推定の信頼性が低下する可能性があります。この問題の詳細は Hansen (2022) をご確認ください。

* 3 「同じターゲットセグメントをもつ企業が CM を出稿する番組は似通う傾向がある」などの理由からこのような状況が起きます。

5 章 離散最適化で広告出稿番組を選択する

```
In[85]: print(f"説明変数の列数: {X_train.shape[1]}")
        print(f"行列のランク: {np.linalg.matrix_rank(X_train)}")
```

```
Out[85]: 説明変数の列数: 113
        行列のランク: 105
```

ランクが列数よりも小さく、完全な多重共線性が発生しています。よって、単純に線形回帰モデルを適用するだけでは、合算のユニークリーチをうまく予測できないことがわかりました。

5.3.4 リッジ回帰による予測

完全な多重共線性が発生している場合、このままでは線形回帰モデルを使うことができないので、完全な多重共線性に対処する方法を検討する必要があります。対処法の一つに、線形従属である変数を説明変数から削除するという方法があります。ただ、この処理は削除した説明変数に対応する番組への CM 出稿を考慮しないことを意味します。これは今回の問題設定に対してはふさわしくない処理なので、別の方法を考えます。

完全な多重共線性への他の対処法として正則化が挙げられます。正則化を用いた回帰モデルの代表的な手法にリッジ回帰⁴があり、完全な多重共線性で回帰係数の推定が不安定になっている状態を説明変数の数を減らすことなく回避できます。

リッジ回帰を利用する場合、正則化の強さを制御するハイパーパラメータ α を指定する必要があります。 α が大きいほど正則化の効果が強くなり、推定が安定します。ただし、正則化を強くすると、回帰係数が 0 方向に引っ張られるという性質があり、推定の安定性と回帰係数の確からしさのバランスをとる必要があります。本来は Cross Validation を用いて最適な α を選択するべきですが、ここでは

* 4 今回の文脈では、過学習の抑制ではなく、完全な多重共線性によるランク落ちに対処するために正則化を利用します。ただし、特に機械学習の文脈では、正則化はモデルの複雑さに制約を加えることで過学習を抑制する用途で利用することが多いです。学習の際に回帰係数の大きさに応じたペナルティを課すことで、モデルの複雑さを抑制し、安定性を高めます。特に、リッジ回帰は、説明変数の係数の二乗和にペナルティを加え、説明変数の係数が大きくなりすぎることを防ぎます。

$$\min_{\beta_1, \dots, \beta_m} \underbrace{\sum_{k=1}^l \left(r_k - \sum_{j=1}^m \beta_j d_{k,j} \right)^2}_{\text{線形回帰モデルと同じ二乗誤差}} + \underbrace{\alpha \sum_{j=1}^m \beta_j^2}_{\text{リッジ回帰で付け加える正則化項}}$$

ここで、 α は正則化の強さを制御するハイパーパラメータで、 α が大きいほど正則化の効果が強くなり、回帰係数が 0 に近づきます。リッジ回帰の詳細に興味のある読者は Hansen (2022) や van Wieringen (2015) をご確認ください。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

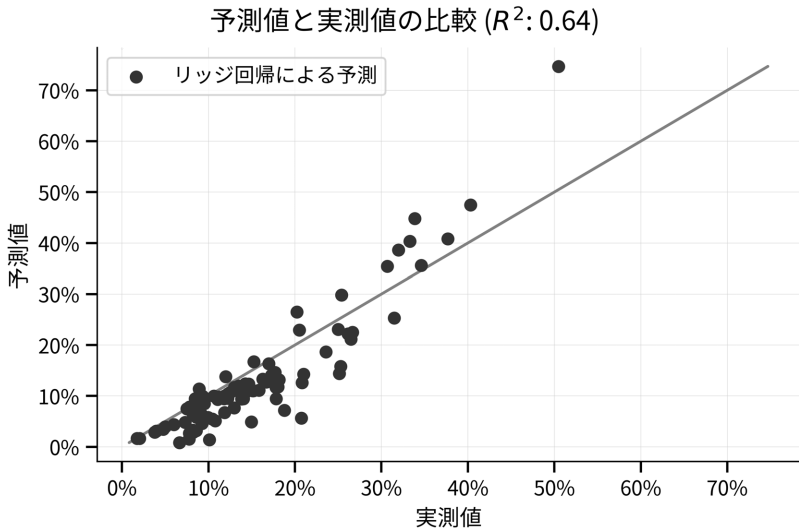
$\alpha = 1$ と決め打ちしてリッジ回帰を行きましょう。リッジ回帰には scikit-learn の `linear_model` モジュールから `Ridge` クラスを利用します。

```
In[86]: from sklearn.linear_model import Ridge

# モデルの学習。切片を含めないため、fit_intercept=False
# alphaはデフォルトが1だが、明示的に指定
ridge = Ridge(
    fit_intercept=False, alpha=1, random_state=42
).fit(X_train, y_train)

# 予測
y_pred_ridge = ridge.predict(X_valid)

# リッジ回帰の予測値と実測値を可視化
fig, ax = plot_compare_actual_prediction(
    y_true=y_valid,
    y_pred=y_pred_ridge,
    prediction_label="リッジ回帰による予測",
    title="予測値と実測値の比較",
)
```



5 章 離散最適化で広告出稿番組を選択する

リッジ回帰によって極端な予測が抑えられ、予測精度が改善したことが見てとれます。とはいえ、決定係数は 0.64 とまだまだ改善の余地があります。

散布図を見ると、合算のユニークリーチが小さい左下の部分では、対角線よりも下に点がプロットされており、実測値よりも予測値が小さくなっています。これは、リッジ回帰は正則化により回帰係数が 0 に近づいている影響と考えられます。

一方で、合算のユニークリーチが大きい右上の部分では、対角線よりも上に点がプロットされており、実測値よりも予測値が大きくなっています。合算のユニークリーチが大きいということは、選択されている番組数が多いことを意味します。リッジ回帰は線形回帰モデルと同様に重複リーチを考慮できていないので、番組数が増えると合算のユニークリーチが過大に推定されることが影響していると考えられます。

5.3.5 機械学習モデルによる予測

線形回帰モデルやリッジ回帰モデルよりも高い予測精度を求めて、機械学習モデルを検討してみましょう。予測モデルには Random Forest を利用します。scikit-learn の ensemble モジュールから RandomForestRegressor クラスを読み込みます。

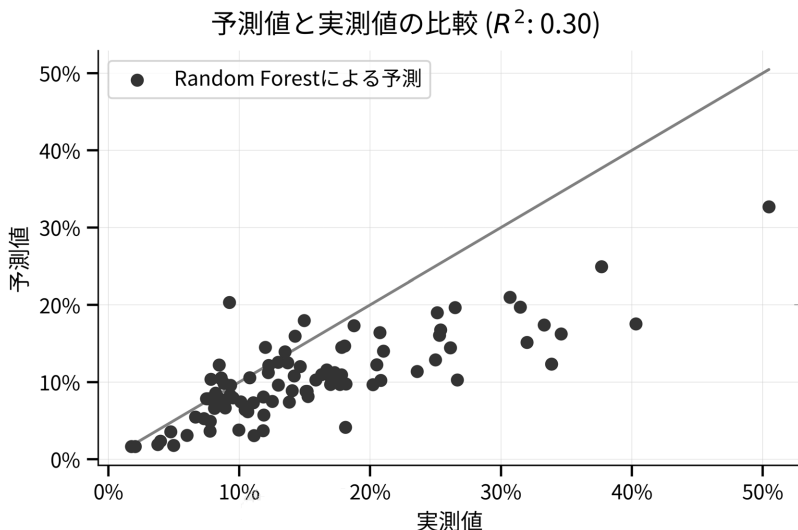
```
In[87]: from sklearn.ensemble import RandomForestRegressor

# モデルの学習と予測
rf = RandomForestRegressor(
    n_jobs=-1, random_state=42
).fit(X_train, y_train)
y_pred_rf = rf.predict(X_valid)
```

Random Forest による学習と予測ができたので、実測値と比較して予測精度を確かめましょう。予測精度の指標としては決定係数を用います。

```
In[88]: # 可視化
fig, ax = plot_compare_actual_prediction(
    y_true=y_valid,
    y_pred=y_pred_rf,
    prediction_label="Random Forestによる予測",
    title="予測値と実測値の比較",
)
```

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



Random Forest による予測精度は、決定係数が 0.3 程度となっており、あまり予測精度が高いとは言えません。また、散布図の点は対角線より下に打たれており、実測値より合算のユニークリーチを小さく予測する傾向があることが見てとれます。特に、番組数が増加しているであろう右上の部分でも予測値が大きくなりえないことから、番組数の増加に対する合算のユニークリーチの増加をうまく捉えられていないように見えます。この問題を解決するために、番組数を直接的に説明変数として取り込んでみましょう。

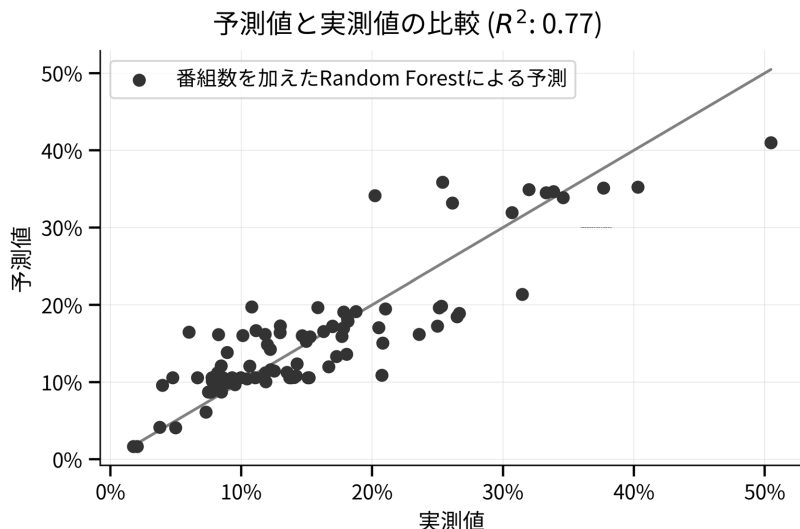
```
In[89]: # 番組数を説明変数として追加
sum_d_train = df_company_reach_train["sum_d"].to_numpy()
sum_d_valid = df_company_reach_valid["sum_d"].to_numpy()
X_train2 = np.column_stack([X_train, sum_d_train])
X_valid2 = np.column_stack([X_valid, sum_d_valid])

# モデルの学習と予測
rf2 = RandomForestRegressor(
    n_jobs=-1, random_state=42
).fit(X_train2, y_train)
y_pred_rf2 = rf2.predict(X_valid2)

# Random Forestによる予測値と実測値を可視化
fig, ax = plot_compare_actual_prediction()
```

5 章 離散最適化で広告出稿番組を選択する

```
y_true=y_valid,
y_pred=y_pred_rf2,
prediction_label="番組数を加えたRandom Forestによる予測",
title="予測値と実測値の比較",
)
```



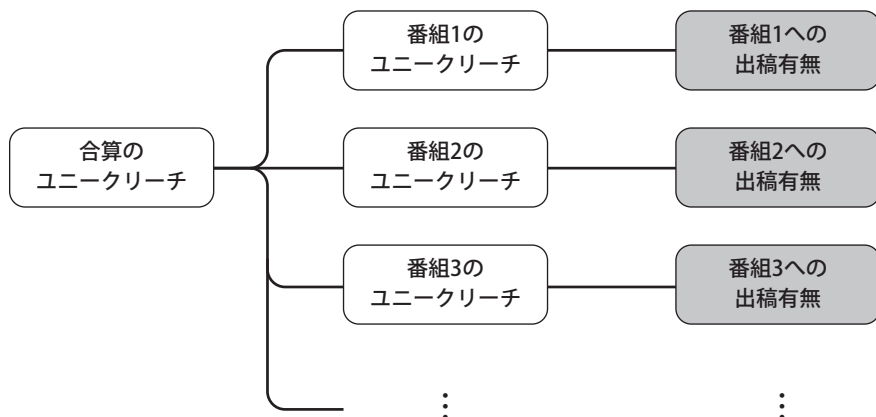
決定係数は 0.77 となり、予測精度が改善できたことが見てとれます。このように、機械学習モデルは特徴量エンジニアリングを行うことで予測精度を高めることができます。本書は特徴量エンジニアリングについて詳しく扱うわけではないので、この方向性での予測精度の改善はここまでにしておき、数理モデルの検討に移りましょう。

5.3.6 数理モデルの構築

独立性の仮定と補正係数の限界

ここまで、選択した番組群に対するユニークリーチ $\mathbb{E}[R \mid \mathbf{d}]$ を直接予測するモデルを構築してきました。他の方法として、4 章で学んだように、各番組のユニークリーチを組みわせて全体を予測するアプローチも考えられます (図 2.9)。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する



■ 図 5.2: 合算のユニークリーチの分解例

具体的に、選択した番組群に対する合算のユニークリーチを各番組のユニークリーチで表現してみましょう。

$$\begin{aligned}
 \mathbb{E}[R \mid \mathbf{d}] &= \Pr(R = 1 \mid \mathbf{d}) \\
 &= \Pr\left(\bigvee_{j=1}^m (R_j = 1) \mid \mathbf{d}\right) \\
 &= 1 - \Pr\left(\bigwedge_{j=1}^m (R_j = 0) \mid \mathbf{d}\right)
 \end{aligned}$$

- 1 行目の変形：ベルヌーイ分布に従う確率変数の期待値は、確率変数が 1 をとる確率と等しいことを利用した
- 2 行目への変形：合算のユニークリーチを「いずれかの番組の CM に接触したことがある割合」と定義したことを利用した。ここで、 $\bigvee$ は論理和を表す
- 3 行目への変形：「いずれかの番組の CM に接触したことがある確率」は「いずれの番組の CM にも接触したことがない確率」を 100% から引くことで求められることを利用した。ここで、 $\bigwedge$ は論理積を表す

これ以上の式変形にはなんらかの仮定が必要です。最も単純な仮定として、各番組の CM への接触が独立であることを仮定してみましょう。

5章 離散最適化で広告出稿番組を選択する

$$\begin{aligned}
\mathbb{E}[R \mid \mathbf{d}] &= 1 - \Pr \left(\bigwedge_{j=1}^m (R_j = 0) \mid \mathbf{d} \right) \\
&= 1 - \prod_{j=1}^m \Pr(R_j = 0 \mid d_j) \\
&= 1 - \prod_{j=1}^m \Pr(R_j = 0)^{d_j} \\
&= 1 - \prod_{j=1}^m (1 - \Pr(R_j = 1))^{d_j}
\end{aligned}$$

- 1行目の変形：前述の式変形の最終形を再掲した
- 2行目への変形：各番組のCMへの接触が独立であると仮定したことを利用した
- 3行目への変形： d_j 乗にすることで、番組 j へのCMを出稿の有無 d_j で条件付けた番組 j のCMへの非接触確率は、番組にCMを出稿した $d_j = 1$ のときは $\Pr(R_j = 0)$ 、番組にCMを出稿しない $d_j = 0$ のときは絶対にCMに接触しないので100%であることを表現した
- 4行目への変形：非接触確率 $\Pr(R_j = 0)$ を接触確率 $\Pr(R_j = 1)$ を用いた形に変換した

このように、各番組のCMへの接触が独立ならば、合算のユニークリーチは各番組のユニークリーチを組み合わせて予測できることがわかります。ただ、4章で見たように、番組のCMへの接触が独立であるという仮定は現実的ではないことが多く、番組のCMへの接触には以下のような正の相関があることが予想されます。

- 各個人は「テレビ好き」または「テレビ無関心」というどの地上波番組視聴にも共通する属性をもっている
- よって、ある番組を高頻度で視聴している「テレビ好き」は他の番組も高頻度で視聴している可能性が高く、ある番組をほとんど視聴しない「テレビ無関心」は他の番組の視聴頻度も低い
- 結果として、ある番組のCMに接触した人は他の番組のCMにも接触していることが多くなる。逆に、ある番組のCMに接触していない人は他の番組のCM

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

にも接触していないことが多くなる

各番組の CM への接触が独立ではないことを考慮するために、4 章では補正係数を用いたアプローチを検討しました。しかし、番組の候補が m 個あると $2^m - m - 1$ 個の補正係数が必要になり、今回のように番組の候補が 113 個ある場合は $2^{113} - 113 - 1 \approx 10^{34}$ 個もの補正係数を推定する必要があり、現実的ではありません。

そこで、本章では、番組の候補が多い場合でも適用可能な数理モデルを検討します。

条件付き独立性の仮定

番組の CM への接触が独立であるという仮定は現実的ではないので、この仮定を緩和する方法を検討します。そもそも、なぜ独立性の仮定が現実的ではないかというと、各個人は「テレビ好き」や「テレビ無関心」のような属性をもっており、個人の属性によって CM 接触の傾向が異なるためです。逆にいうと、「テレビ好き」や「テレビ無関心」という属性を固定すれば、番組の CM への接触が独立だと仮定することは現実的だと考えられます*5。

「テレビ好き」や「テレビ無関心」のような個人のテレビ視聴傾向を表す属性を X とします。このとき、Law of Iterated Expectations (LIE)*6を用いて、合算の

* 5 この考え方は、Clegg and Peng (2022)、Chen et al. (2014)、Shen et al. (2019) などでも利用されています。

* 6 確率変数 (X, Y) に対して、

$$\mathbb{E}_X [X] = \mathbb{E}_Y [\mathbb{E}_{X|Y} [X | Y]]$$

が成り立ちます。ここで、どの確率変数に対してどの分布で期待値を計算しているかの混乱を生まないよう、期待値をとる確率変数とその分布を明示しています。具体的には、 (X, Y) が離散確率変数の場合は、

$$\begin{aligned} \mathbb{E}_X [X] &= \sum_{x \in \mathcal{X}} x \Pr(X = x) \\ \mathbb{E}_{X|Y} [X | Y] &= \sum_{x \in \mathcal{X}} x \Pr(X = x | Y) \end{aligned}$$

を意味しています。実際、数式変形から

$$\begin{aligned} \mathbb{E}_Y [\mathbb{E}_{X|Y} [X | Y]] &= \sum_{y \in \mathcal{Y}} \left(\sum_{x \in \mathcal{X}} x \Pr(X = x | y) \right) \Pr(Y = y) \\ &= \sum_{x \in \mathcal{X}} x \left(\sum_{y \in \mathcal{Y}} \Pr(X = x | y) \right) \Pr(Y = y) \end{aligned}$$

5 章 離散最適化で広告出稿番組を選択する

ユニークリーチは

$$\mathbb{E}[R \mid \mathbf{d}] = \mathbb{E}[\mathbb{E}[R \mid X, \mathbf{d}] \mid \mathbf{d}]$$

と表現できます。LIE の性質から、まずは属性 X で条件付けた期待値 $\mathbb{E}[R \mid X, \mathbf{d}]$ を求め、その後に属性 X の分布で期待値をとることで、属性 X で条件付けない期待値 $\mathbb{E}[R \mid \mathbf{d}]$ を求められます。

ここで、個人の属性 X の分布は選択した番組群 $\mathbf{d}$ とは独立と考えられるので、外側の期待値から $\mathbf{d}$ の条件付けを外すことができ、

$$\begin{aligned}\mathbb{E}[R \mid \mathbf{d}] &= \mathbb{E}[\mathbb{E}[R \mid X, \mathbf{d}] \mid \mathbf{d}] \\ &= \mathbb{E}[\mathbb{E}[R \mid X, \mathbf{d}]]\end{aligned}$$

を得ます*7。

期待値が入れ子になっていて複雑に見えるかもしれませんが、意味するところは単純です。期待値の内側は属性ごとのユニークリーチになっており、それを属性 X の分布で重みを付けて期待値をとれば、属性で条件付けない期待値を求められるということです。

特に、属性 X が離散値の場合は

$$\mathbb{E}[R \mid \mathbf{d}] = \sum_{x \in \mathcal{X}} \Pr(X = x) \mathbb{E}[R \mid X = x, \mathbf{d}]$$

であり、それぞれの属性が占める割合で重みを付けて、属性ごとの合算のユニークリーチを平均することを意味します。ここで、 $\mathcal{X}$ は属性 X のとりうる値の集合です。

$$\begin{aligned}&= \sum_{x \in \mathcal{X}} x \sum_{y \in \mathcal{Y}} \Pr(X = x, Y = y) \\ &= \sum_{x \in \mathcal{X}} x \Pr(X = x) \\ &= \mathbb{E}_X[X]\end{aligned}$$

が成り立つことがわかります（連続確率変数の場合も同様の手順で確認できます）。つまり、条件付き期待値に対して、条件付けた確率変数で期待値をとると、条件付けない期待値になります。この性質は、繰り返し期待値をとるので、Law of Iterated Expectations（繰り返し期待値の法則）と呼ばれています。詳細は久保川 (2017) などの数理統計学の専門書をご確認ください。

*** 7** 期待値をとる分布を明示すると、

$$\begin{aligned}\mathbb{E}[R \mid \mathbf{d}] &= \mathbb{E}_{X \mid \mathbf{d}}[\mathbb{E}_{R \mid X, \mathbf{d}}[R \mid X, \mathbf{d}] \mid \mathbf{d}] \\ &= \mathbb{E}_X[\mathbb{E}_{R \mid X, \mathbf{d}}[R \mid X, \mathbf{d}] \mid \mathbf{d}]\end{aligned}$$

です。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

▼ 表 5.3: 条件付き独立性を仮定した計算の数値例

属性	属性が集団に占める割合 $\Pr(X = x)$	合算のユニークリーチ $\mathbb{E}[R \mid X = x, \mathbf{d}]$
テレビ好き	0.5	0.92
テレビ無関心	0.5	0.44
集団全体	1	0.68

計算のイメージをつかむために具体例を見てみましょう。表 5.3 は、属性 X が占める割合と、属性 X で条件付けた場合の合算のユニークリーチの数値例です。この例では、テレビ好きが集団の 50% を占め、その中では 92% の人が CM に接触しています。同様に、テレビ無関心が 50% で、その中では 44% の人が CM に接触しています。ここから、最終行の集団全体のユニークリーチは以下のように計算します。

$$\begin{aligned}
 \mathbb{E}[R \mid \mathbf{d}] &= \Pr(X = \text{テレビ好き}) \mathbb{E}[R \mid X = \text{テレビ好き}, \mathbf{d}] \\
 &\quad + \Pr(X = \text{テレビ無関心}) \mathbb{E}[R \mid X = \text{テレビ無関心}, \mathbf{d}] \\
 &= 0.5 \times 0.92 + 0.5 \times 0.44 \\
 &= 0.68
 \end{aligned}$$

計算の結果、全体のユニークリーチは 68% とわかりました。具体例を見ると当たり前の計算に感じますね。このように、属性 X で条件付けた場合のユニークリーチを求めて、それを属性 X の分布で重みを付けて期待値をとることで、属性 X で条件付けないユニークリーチを求めることができます。

計算方法のイメージが湧いたので、次は LIE を用いて期待値を変形していきましょう。追加的に属性 X で条件付けていること以外は先ほどの式変形と同じです。

$$\begin{aligned}
 \mathbb{E}[R \mid \mathbf{d}] &= \mathbb{E}[\mathbb{E}[R \mid X, \mathbf{d}]] \\
 &= \mathbb{E}[\Pr(R = 1 \mid X, \mathbf{d})] \\
 &= \mathbb{E}\left[\Pr\left(\bigvee_{j=1}^m R_j = 1 \mid X, \mathbf{d}\right)\right] \\
 &= \mathbb{E}\left[1 - \Pr\left(\bigwedge_{j=1}^m R_j = 0 \mid X, \mathbf{d}\right)\right]
 \end{aligned}$$

5 章 離散最適化で広告出稿番組を選択する

ここで、属性 X で条件付けた場合の番組の CM への接触は独立と仮定します。先ほどは特に条件付けない独立性を仮定しましたが、今回は属性 X で条件付けた上で独立性を仮定していることに注意してください。この条件付き独立性の仮定は、「テレビ好き」や「テレビ無関心」という属性を固定すれば番組の CM への接触が独立だと言っているの、元々の純粋な独立性の仮定よりも弱い仮定になっています。この仮定のもとでは、

$$\begin{aligned}\mathbb{E}[R \mid \mathbf{d}] &= \mathbb{E} \left[1 - \Pr \left(\bigwedge_{j=1}^m R_j = 0 \mid X, \mathbf{d} \right) \right] \\ &= \mathbb{E} \left[1 - \prod_{j=1}^m \Pr(R_j = 0 \mid X, d_j) \right] \\ &= \mathbb{E} \left[1 - \prod_{j=1}^m \Pr(R_j = 0 \mid X)^{d_j} \right] \\ &= \mathbb{E} \left[1 - \prod_{j=1}^m (1 - \Pr(R_j = 1 \mid X))^{d_j} \right]\end{aligned}$$

と期待値を変形することができます。特に、属性 X が離散値の場合は

$$\mathbb{E}[R \mid \mathbf{d}] = \sum_{x \in \mathcal{X}} \Pr(X = x) \left(1 - \prod_{j=1}^m (1 - \Pr(R_j = 1 \mid X = x))^{d_j} \right)$$

となります。ここから、属性 X ごとに番組 j への接触確率を求めて、それを属性 X が占める割合で重みを付けて加重平均することで、属性で条件付けないユニークリーチを求めることができるとわかりました。

最後に、条件付きの独立性の仮定が、条件付けない独立性の仮定よりも弱い仮定であることを数値例で確認しておきましょう。そのために、属性ごとの番組接触が独立だとしても、集団全体では番組接触が独立とは限らないことを確認します。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

▼ 表 5.4: 条件付き独立性と条件付けない独立性の比較

属性	属性が 全体に 占める 割合	番組 1 の ユニーク リーチ	番組 2 の ユニーク リーチ	重複 リーチ	合算の ユニーク リーチ
テレビ好き	0.5	0.8	0.6	0.48	0.92
テレビ無関心	0.5	0.3	0.2	0.06	0.44
集団全体	1	0.55	0.4	0.27	0.68
条件付けない 独立性を 仮定した推定	1	0.55	0.4	0.22	0.73

表 5.4 は、表 5.3 と同じ数値例をベースに、番組ごとのユニークリーチと重複リーチを追記したものです。CM を出稿できる番組の候補は番組 1 と番組 2 の 2 番組で、その両方に CM を出稿した状況 ($\mathbf{d} = (1, 1)$) を考えています。

まず、表 5.4 の数値例において、テレビ好きでは $0.8 \times 0.6 = 0.48$ 、テレビ無関心では $0.3 \times 0.2 = 0.06$ であり、属性ごとで見ると番組ごとのユニークリーチの積が実際のユニークリーチと一致していること、つまり番組接触の独立性

$$\begin{aligned}\mathbb{E}[R_{1 \cap 2} \mid X = x, \mathbf{d} = (1, 1)] \\ = \mathbb{E}[R_1 \mid X = x, d_1 = 1] \times \mathbb{E}[R_2 \mid X = x, d_2 = 1]\end{aligned}$$

が成り立っていることが見てとれます。

一方で、条件付けない集団全体での独立性の仮定は成り立っていません。番組 1 の集団全体のユニークリーチは 55%、番組 2 の集団全体のユニークリーチは 40%です。もしも集団全体での番組接触が独立なら、集団全体の重複リーチは $0.55 \times 0.40 = 0.22$ であり、集団全体の合算のユニークリーチは $0.55 + 0.4 - 0.22 = 73\%$ となるはずですが、実際の集団全体の合算のユニークリーチは先ほど計算したように 68%であり、ここから集団全体での番組接触は独立ではないことがわかります。

以上のように、属性ごとの番組接触が独立だとしても、集団全体では番組接触が独立とは限りません。ここから、属性ごとの番組接触の独立性は、集団全体での番組接触の独立性より弱い仮定であるとわかりました。よって、条件付きの独立性の仮定は、より現実に即した仮定であると言えるでしょう。

5 章 離散最適化で広告出稿番組を選択する

個人ごとの CM 接触確率への分解

「テレビ好き」や「テレビ無関心」のような個人のテレビ視聴傾向を表す属性 X をモデルに組み込むことで、現実的には成り立っていない独立性の仮定を、より現実的な条件付き独立性の仮定に緩和することができました。一方で、適切な属性 X を定義し、観測するというのも容易ではありません。そこで、属性 X を観測するのではなく、個人ごとの番組の CM への接触確率を求めて、それを平均することで全体のユニークリーチを推定するという方法をとみましょう。これは、属性 X を属性の最小粒度である個人だと考えているということです。

いま、集団の人数を n 人とする、 $\Pr(X = i) = 1/n$ なので、合算のユニークリーチは

$$\mathbb{E}[R | \mathbf{d}] = \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \Pr(R_j = 1 | X = i))^{d_j} \right)$$

となります。

$X = i$ という表記が煩雑なので、個人 $i = 1, \dots, n$ が番組 j に接触したかどうかを $R_{i,j}$ とすると、合算のユニークリーチは

$$\mathbb{E}[R | \mathbf{d}] = \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \Pr(R_{i,j} = 1))^{d_j} \right)$$

と記載できます。

さらに記法を簡略化するため、個人 i が番組 j に接触する確率 $\Pr(R_{i,j} = 1)$ を $\pi_{i,j}$ とすると、

$$\mathbb{E}[R | \mathbf{d}] = \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \pi_{i,j})^{d_j} \right)$$

です。ここから、個人 i が番組 j の CM に接触する確率 $\pi_{i,j}$ さえ求めれば、合算のユニークリーチを計算できることがわかります。

表 5.5 に記法をまとめておきます。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

▼ 表 5.5: 各指標とその計算式

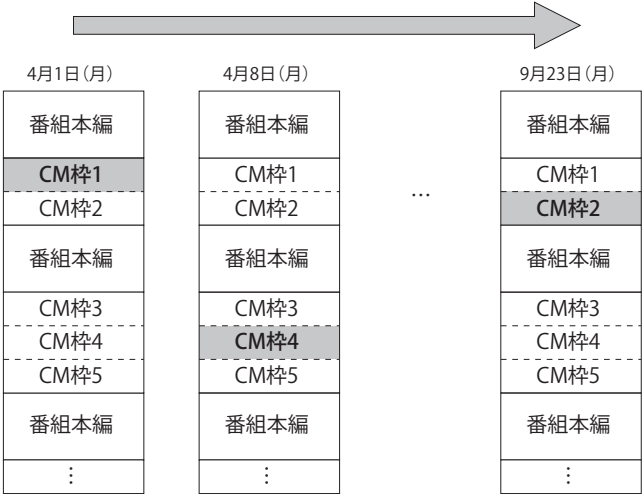
指標	計算式
個人 i が番組 j の CM に 接触する確率	$\pi_{i,j}$
番組群 d を選択した際に 個人 i がいずれかの番組の CM に 接触する確率	$1 - \prod_{j=1}^m (1 - \pi_{i,j})^{d_j}$
番組群 d を選択した際の 合算のユニークリーチ	$\frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \pi_{i,j})^{d_j} \right)$

5.3.7 数理モデルの推定

必要なデータ

ここまでの議論から、合算のユニークリーチは個人ごとに番組 j の CM に接触する確率 $\pi_{i,j}$ を求めれば計算できることがわかりました。それでは、どのようなデータがあれば $\pi_{i,j}$ を推定できるのでしょうか。

番組の放送枠を購入すると、番組内のいずれかのCM枠で毎週CMが放送される



■ 図 5.3: 番組と CM 出稿枠

5 章 離散最適化で広告出稿番組を選択する

いま知りたいのは、番組 j に CM を出稿した際に、個人 i がその CM に接触する確率 $\pi_{i,j}$ です。図 5.3 にあるように、番組には複数の CM 出稿枠があります。番組に CM を出稿すると決めると、毎週 CM 枠の中から一つが選ばれ、実際に CM が放送されます。よって、個人 i が個々の CM に接触するかどうかは確率的に決まっていると解釈すると、番組 j に出稿された CM のうち個人 i が接触した CM 割合を CM 接触確率とみなせます。

▼ 表 5.6: 必要なデータ

個人 i	番組 j	企業 k	CM 接触 $r_{i,j,k}$
1	1	1	1
1	1	2	0
1	1	3	0
1	2	1	1
1	2	4	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$

よって、表 5.6 のような番組に放送された各企業の CM に対する接触データがあれば、個人 i の番組 j の CM に接触する確率 $\pi_{i,j}$ を推定できます。ここで、 k は企業を表し、 $r_{i,j,k}$ は個人 i が番組 j に放送された企業 k の CM に接触したかどうかを表します。具体的に、表 5.6 の場合、個人 1 の番組 1 の CM 接触確率は $\hat{\pi}_{1,1} = 1/3$ 、個人 1 の番組 2 の CM 接触確率は $\hat{\pi}_{1,2} = 1/2$ です。

具体的な計算方法はわかったので、より一般的な CM 接触確率 $\pi_{i,j}$ の推定方法をまとめます。番組 j に CM を放送した企業の集合を $\mathcal{K}_j$ とします。このとき、個人 i の番組 j の CM 接触確率 $\pi_{i,j}$ は以下のように推定できます。

$$\hat{\pi}_{i,j} = \frac{1}{|\mathcal{K}_j|} \sum_{k \in \mathcal{K}_j} r_{i,j,k}$$

ここで、 $|\mathcal{K}_j|$ は企業の集合 $\mathcal{K}_j$ の要素数、つまり番組 j で放送された CM の総数です。

補正係数によるアプローチでは $2^m - m - 1$ 個の補正係数の推定が必要でしたが、こちらの個人分解のアプローチの場合は $n \times m$ 個の CM 接触確率を推定するだけで済みます。具体的に、 $m = 113$ 番組の場合、補正係数のアプローチでは約 10^{34} 個の補正係数を推定する必要がありますが、個人分解のアプローチでは $n = 10000$ 人でも約 10^6 個の CM 接触確率を推定するだけで済みます。

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

ただし、個人ごとの各番組への CM 接触確率を推定するためには、個人ごとの CM 接触という新たなデータが必要になります。そこで、調査会社に依頼を行った結果、個人ごとの CM 接触データが手に入ったとしましょう。これを使って CM 接触確率を推定します。

データの確認

それでは、新たに手に入った個人ごとの CM 接触データを用いて分析を進めます。まずはデータを確認しましょう。

```
In[90]: # 個人ごとのCM接触データを読み込む
df_individual_reach = pl.read_parquet(
    "../data/ch05/individual_reach.parquet"
)

# データの確認
df_individual_reach
```

```
Out[90]: shape: (4_567_196, 5)
```

is_train	j	k	i	r	
	i32	u32	u32	i64	i64
	1	1	1	1	0
	1	1	1	2	0
	1	1	1	3	0
	1	1	1	4	0
	1	1	1	5	0
...	...	...	...	...	...
0	113	420	4545	0	
0	113	420	4546	1	
0	113	420	4547	0	
0	113	420	4548	0	
0	113	420	4549	0	

5 章 離散最適化で広告出稿番組を選択する

`df_individual_reach` は、`df_company_reach` の元になる個人ごとの CM 接触情報が格納されています。各番組ごとに複数企業の CM が放送されており、それぞれの CM に各個人が接触したかどうかを表すデータです。

各列には以下の情報が入っています。

- `is_train`: 学習用データなら 1、評価用データなら 0 をとるフラグ
- `j`: 番組を表すインデックス
- `k`: 企業を表すインデックス
- `i`: 個人を表すインデックス
- `r`: CM に接触したかどうか。接触した場合は 1、そうでない場合は 0

このデータから、個人 i が番組 j の CM に接触した割合を計算できます。後続で NumPy を利用した行列計算を行うため、CM 接触割合を n 行 m 列の行列形式に変形します。

```
In[91]: # 学習用データを抽出
df_individual_reach_train = df_individual_reach.filter(
    pl.col.is_train == 1
)

# 個人ごとの各番組のCM接触確率を計算して横持ちで保存
df_reach_prob_train = (
    # 個人ごとの各番組のCM接触確率を計算
    df_individual_reach_train.group_by(
        "i", "j", maintain_order=True
    )
    .agg(pi=pl.col.r.sum() / pl.col.k.n_unique())
    # 横持ちに変換
    .pivot(on="j", index="i", values="pi")
)

# NumPyのndarrayに変換
PI_train = (
    df_reach_prob_train
    # 個人のインデックスが列として入っているので削除
    .drop("i")
    # n行m列の行列に変換
    .to_numpy()
)
```

5.3 ステップ 2：数理モデルを構築し、未知のパラメータをデータから推定する

```
# CM接触確率の確認
# データフレーム形式のほうがわかりやすいので、こちらを出力
df_reach_prob_train
```

Out[91]: shape: (4_549, 114)

i	1	2	3	...	111	112	113
i64	f64	f64	f64	...	f64	f64	f64
1	0.000	0.667	0.000	...	0.000	0.000	0.600
2	0.000	1.000	0.000	...	0.500	0.000	0.200
3	0.000	0.000	0.000	...	0.250	0.167	0.400
4	0.000	0.333	0.000	...	0.000	0.333	0.000
5	0.000	0.000	0.000	...	0.000	1.000	0.200
...	...	...	...	...	...	...	...
4545	0.000	0.000	0.000	...	0.000	0.000	0.000
4546	0.333	0.000	0.000	...	0.000	0.000	0.000
4547	0.000	0.000	0.000	...	0.250	0.333	0.000
4548	0.500	0.000	0.000	...	0.000	0.000	0.000
4549	0.167	0.000	0.000	...	0.000	0.000	0.000

集計結果から、例えば個人 1 の番組 113 の CM への接触確率は 60%である一方で、個人 2 の番組 113 の CM への接触確率は 20%しかないことがわかります。このように、新たに手に入ったデータでは、個人によって各番組の CM 接触確率が異なることを捉えられています。

以降では、このデータを使って選択した番組群 d に対する合算のユニークリーチを推定するモデルを実装します。

5.3.8 数理モデルの実装

それでは、構築した数理モデルを ProgramUniqueReachEstimator クラスとして実装していきましょう。

5 章 離散最適化で広告出稿番組を選択する

```
In[92]: class ProgramUniqueReachEstimator:
    """選択した番組群に対するユニークリーチを予測するモデル"""

    def __init__(self, PI: np.ndarray) -> None:
        """個人ごとの各番組のCMへの接触確率を保持

        Args:
            PI (np.ndarray): 個人ごとの各番組のCMへの接触確率
        """

        # 個人ごとの各番組への接触確率を計算して保存
        self.PI_ = PI
        self.m_ = PI.shape[1]

    def predict(self, D: np.ndarray) -> np.ndarray:
        """選択した番組群に対するユニークリーチを予測

        Args:
            D (np.ndarray): ダミー変数形式の選択した番組群

        Returns:
            np.ndarray: 合算のユニークリーチの予測値
        """

        # (1 - PI)のD乗を計算したいので、shapeをあわせる
        # (n, m)を(n, m, 1)に
        PI_ = self.PI_[ :, :, np.newaxis]
        # (1, m)を(1, m, 1)に
        D_ = D.T[np.newaxis, :, :]

        # これで、(1 - PI_) ** D_のshapeが(n, m, 1)になる
        r_pred = 1 - ((1 - PI_) ** D_).prod(1).mean(0)

        return r_pred
```

ProgramUniqueReachEstimator は、番組のユニークリーチを予測するためのクラスです。各メソッドの実装を見ていきましょう。

すでに個人ごとの各番組への CM 接触確率 $\hat{\pi}_{i,j}$ を計算できているので、__init__ メソッドで個人ごとの各番組への CM 接触確率 PI を受け取って保持しています。PI があれば合算のユニークリーチを予測できるので、fit メソッドは実装していません。

次に、predict メソッドでは、企業 $k = 1, \dots, l$ に対して

5.3 ステップ2：数理モデルを構築し、未知のパラメータをデータから推定する

$$\begin{aligned}\hat{\mathbb{E}}[R \mid \mathbf{d}_k] &= \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_{k,j}} \right) \\ &= 1 - \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_{k,j}}\end{aligned}$$

を計算しています。

5.3.9 データを用いた予測精度の検証

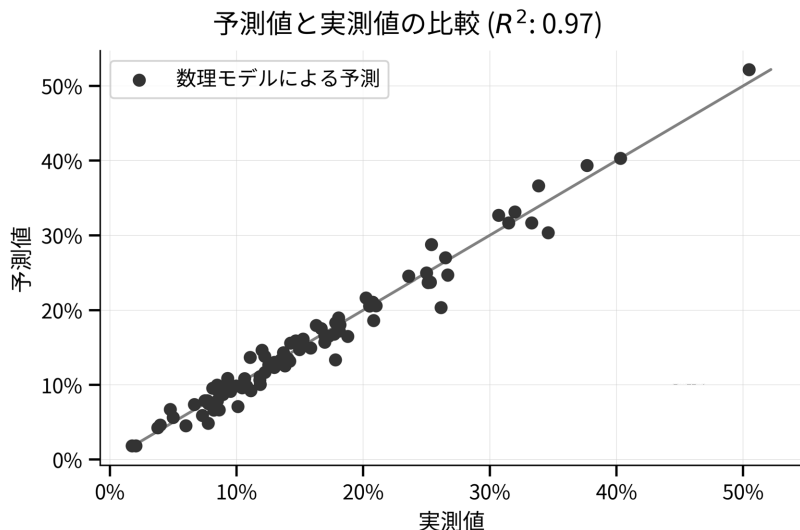
数理モデルが実装できたので、ProgramUniqueReachEstimator クラスを用いて合算のユニークリーチを予測し、予測精度を検証してみましょう。

```
In[93]: # モデルの学習
pure = ProgramUniqueReachEstimator(PI_train)

# 検証用データに対して予測
y_pred_pure = pure.predict(X_valid)

# 可視化
fig, ax = plot_compare_actual_prediction(
    y_true=y_valid,
    y_pred=y_pred_pure,
    prediction_label="数理モデルによる予測",
    title="予測値と実測値の比較",
)
```

5 章 離散最適化で広告出稿番組を選択する



決定係数は 0.97 であり、単純に機械学習モデルを適用した場合よりも高い精度で合算のユニークリーチを予測できていることがわかります。

今回は、数理モデルの力に加えて、個人ごとの CM 接触状況という情報量の多いデータが追加で手に入ったことで、予測精度が大きく改善しました。今回の例で個人ごとの CM 接触状況が必要だと特定できたように、数理モデルを構築する過程で必要な情報が明確化されることが、数理モデルを考えることのメリットの一つだと言えるでしょう。

5.4

ステップ3：数理最適化問題を解いて最適なアクションを導出する

5.4.1 離散最適化問題

ステップ 2 では未知の部分である、選択した番組群 d に対する合算のユニークリーチ $\mathbb{E}[R \mid d]$ に対する数理モデルを構築し、データから関係性を推定することができました。未知の部分が無くなった最適化問題を解くことで、最適な意思決定を導くことができます。

$$\max_d 1 - \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j}$$

5.4 ステップ3：数理最適化問題を解いて最適なアクションを導出する

$$\begin{aligned} \text{s.t. } & \sum_{j=1}^m c_j d_j \leq b \\ & d_j \in \{0, 1\} \quad \forall j = 1, \dots, m \end{aligned}$$

この最適化問題は、決定変数 d_j が $\{0, 1\}$ の離散値をとる離散最適化問題であり、4 章まで扱ってきた連続最適化問題のように簡単に解を求めることができません。予算制約を満たす番組の組み合わせを全探索することで厳密解を求めることもできますが、計算量が膨大になってしまいます。そこで、貪欲法を使って近似解^{*}を求めることにします。

5.4.2 貪欲法による近似解の探索

貪欲法は目の前の一手一手で最適な選択を繰り返すという手法です。もちろん、目の前の一手で最適な選択は、その先を考慮した最適な選択と同じであるとは限りませんが、実装が容易なこともあり、近似解を求める手法として多くの場面で利用されています。

今回の問題に対する貪欲法の実装は、「合算ユニークリーチを増加させるコストパフォーマンスが最も高い番組を選択する」ことを予算上限まで繰り返すというものにしましょう。貪欲法のアルゴリズム以下に記載します。

1. まだ選択していないすべての番組 j に対して、番組 j に CM を出稿したときと、CM を出稿しなかったときの、合算のユニークリーチの差分を計算する
2. 差分を番組 j に CM を出稿するコストで割って、コストパフォーマンスを求める
3. コストパフォーマンスが最も高い番組を追加する
4. 1 に戻る。予算内では追加的に出稿できる番組がなくなったら終了

それでは、実際に貪欲法を実装してみましょう。

```
In[94]: def greedy_search(
        estimator: ProgramUniqueReachEstimator,
        b: float,
        c: np.ndarray,
```

^{*} 8 数理最適化の文脈で、最適解を求めることが難しい場合に用いる「実用上問題のない程度に最適解に近い解」のことを近似解と呼びます。

5 章 離散最適化で広告出稿番組を選択する

```

) -> tuple[float, list[int]]:
    """
    予算内で最大のユニークリーチを達成する番組群を貪欲法で探索する
    """

    # 番組候補の数
    m = c.shape[0]
    # 選択した番組群。最初は何も選択していない
    # これを更新していく
    D = np.zeros((1, m))
    # 選択した番組のリスト
    j_list_opt: list[int] = []

    # 予算内を超えない限り番組を追加していく
    while True:
        # 番組追加前の合算ユニークリーチ
        r_before = estimator.predict(D)[0]

        # 番組を追加した場合の合算のユニークリーチの増加幅を計算
        delta = np.zeros(m) # 差分
        for j in range(m):
            # すでに選択されている番組はスキップ
            if D[0, j] == 1:
                continue

            # 番組j+1を追加
            # インデックスが0始まりなので、
            # 正確にはjではなくj+1が追加される
            D_ = D.copy()
            D_[0, j] = 1

            # 予算内に収まらない場合はスキップ
            if c @ D_.flatten() > b:
                continue

            # 番組j+1を追加した場合の
            # 合算のユニークリーチの増加幅を計算
            r_after = estimator.predict(D=D_)[0]

            delta[j] = (r_after - r_before) / c[j]

        # コストパフォーマンスが最も高い番組を追加
        j_to_add = int(np.argmax(delta))
        D[0, j_to_add] = 1

```


5.4 ステップ 3：数理最適化問題を解いて最適なアクションを導出する

```
# 予算内に収まらない場合は番組の追加をとりやめて終了
if c @ D.flatten() > b:
    D[0, j_to_add] = 0
    break

# インデックスは0始まりなので、1を足して追加
j_list_opt.append(j_to_add + 1)

# 最終的な合算のユニークリーチと、それを達成する番組群を返す
return estimator.predict(D)[0], j_list_opt
```

貪欲法を実行する関数 `greedy_search` が実装できました。

それでは、具体例に対して貪欲法で最適な番組群を探索してみましょう。話を簡単にするため、予算 b を 5,000（万円）とし、各番組に CM を流すコスト c_j を一律に 500（万円）とします^{*9}。この設定のもとでは、

$$\sum_{j=1}^m 500d_j \leq 5000$$

$$\iff \sum_{j=1}^m d_j \leq 10$$

なので、113 番組の中からユニークリーチを最大化する 10 番組を選択する問題になります。

```
In[95]: # 番組候補の数
m = X_valid.shape[1]

# 最適な番組群を選択
r_opt, j_list_opt = greedy_search(
    estimator=pure,
    b=5000,
    c=np.ones(m) * 500,
)

# 結果の確認
r_opt, j_list_opt
```

* 9 実際の価格は視聴率の高さに依存しますが、半年間継続的に CM が流れるので、そのぶん価格は高くなります。

5章 離散最適化で広告出稿番組を選択する

```
Out[95]: (0.49144801673265814, [5, 74, 32, 112, 50, 26, 76, 61, 33,
44])
```

結果を見ると、合算のユニークリーチを最大化する番組群は追加順に [5, 74, 32, 112, 50, 26, 76, 61, 33, 44] で、この番組群を選択すると合算のユニークリーチを約 49.1%獲得できることがわかりました。以上で、解きたかった課題である合算のユニークリーチを最大化する問題を解くことができました。こうして求めた番組群に CM を出稿することで、効率的に合算のユニークリーチを獲得することができます。

貪欲法の検証

最後に、上記の方法で求めた番組群が、合算のユニークリーチを効率的に獲得できているのかを確認しておきましょう。貪欲法はあくまで近似解を求める手法なので、最適な番組群を選択できているとは限りません。それでも、各企業が実際に獲得している合算のユニークリーチと同じか、それ以上の水準を達成していることは期待したいです。

そこで、番組数ごとに、合算のユニークリーチを最大化する番組群を求め、その合算のユニークリーチを計算します。同じ番組数だけ出稿した企業の合算のユニークリーチと比較することで、効率的な番組群を選択できているかを確認することになります。

```
In[96]: # 企業ごとの出稿番組の最大数
# 検証は評価用データで行う
max_sum_d = df_company_reach_valid.select(
    pl.col.sum_d.max()
).item()

# 貪欲法で最大の番組数に対する最適な番組群を探索
_, j_list_opt = greedy_search(
    estimator=pure,
    b=max_sum_d,
    c=np.ones(m),
)

# 番組数ごとに最適な合算のユニークリーチを計算
# j_list_optはコストパフォーマンスが高い順に
# 番組を追加しているので、
# j_list_optの先頭から番組を追加していくと、
```

5.4 ステップ3：数理最適化問題を解いて最適なアクションを導出する

```
# 番組数ごとに貪欲法で最適解を探索した場合と
# 同じ結果を得ることができる
r_opts = np.zeros(max_sum_d)
for i, sum_d in enumerate(range(1, max_sum_d + 1)):
    # 番組数ごとに最適な番組群を選択
    # インデックスに変換しないといけなので1を引いている
    D = np.zeros((1, m))
    D[0, np.array(j_list_opt[:sum_d]) - 1] = 1

    # 番組数ごとの最適な合算のユニークリーチ
    r_opts[i] = pure.predict(D)[0]
```

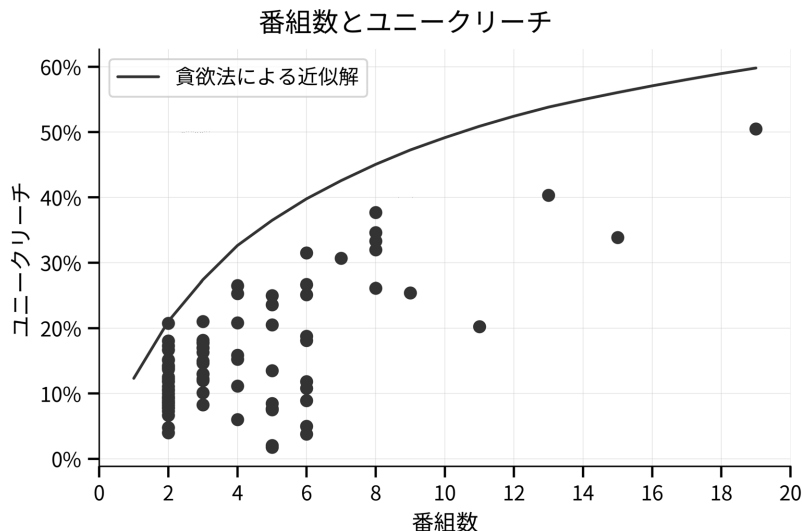
最適化の結果と実際の企業のユニークリーチを可視化して比較します。

```
In[97]: def plot_scatter_with_optimal_reach(df: pl.DataFrame, r_opts:
        np.ndarray):
        fig, ax = plot_scatter(
            x=df.get_column("sum_d"),
            y=df.get_column("r"),
            xlabel="番組数",
            ylabel="ユニークリーチ",
            title="番組数とユニークリーチ",
        )

        # 貪欲法による近似解をプロット
        ax.plot(
            range(1, r_opts.shape[0] + 1),
            r_opts,
            label="貪欲法による近似解"
        )
        ax.legend()
        ax.set_xticks(np.arange(0, r_opts.shape[0] + 2, 2))

        # 評価用データで実測値と貪欲法による近似解を可視化
        plot_scatter_with_optimal_reach(
            df=df_company_reach_valid, r_opts=r_opts
        )
```

5 章 離散最適化で広告出稿番組を選択する



可視化結果を見ると、番組数が同じであれば、常に貪欲法による近似解で獲得できる合算のユニークリーチが実測値を上回っています。ここから、貪欲法が番組群を効率的に選択できていることが確認できました。

5.5 本章のまとめ

本章では、「番組選択の最適化」というビジネス課題を題材に、3ステップフレームワークを用いて課題解決に取り組みました。

まず、ステップ1では、数理最適化問題の定式化を通じて、最適化した指標、制約条件、とりうるアクションを明確にしました。具体的には、最大化したい指標を合算のユニークリーチ、制約条件を予算制約、とりうるアクションをCMを出稿する番組の選択とすることで、ビジネス課題を制約付きユニークリーチ最大化問題

$$\begin{aligned}
 & \max_{\mathbf{d}} \quad \mathbb{E}[R \mid \mathbf{d}] \\
 & \text{s.t.} \quad \sum_{j=1}^m c_j d_j \leq b \\
 & \quad d_j \in \{0, 1\} \quad \forall j = 1, \dots, m
 \end{aligned}$$

として定式化し、データサイエンスで扱える問題に変換しました。

ステップ1で定式化した数理最適化問題には、未知の部分として選択した番組群と合算のユニークリーチの関係 $\mathbb{E}[R \mid \mathbf{d}]$ が含まれていました。そこで、ステップ2ではこの未知の部分に対して数理モデルを構築し、数理モデルのパラメータをデータから推定する方法を検討しました。具体的には、個人ごとの CM 接触確率を用いた数理モデル

$$\begin{aligned}\widehat{\mathbb{E}}[R \mid \mathbf{d}] &= \frac{1}{n} \sum_{i=1}^n \left(1 - \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j} \right) \\ &= 1 - \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j}\end{aligned}$$

を構築することで、合算のユニークリーチを予測可能にしました。

最後に、ステップ3では、ステップ2で推定した関係性を代入した数理最適化問題

$$\begin{aligned}\max_{\mathbf{d}} \quad & 1 - \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j} \\ \text{s.t.} \quad & \sum_{j=1}^m c_j d_j \leq b \\ & d_j \in \{0, 1\} \quad \forall j = 1, \dots, m\end{aligned}$$

をもとに、与えられた制約のもとで合算のユニークリーチを最大化する番組群を探索しました。この問題は離散最適化問題なので、貪欲法を利用することで近似的な最適解を求めました。

このように、3ステップフレームワークを用いることで、ビジネス課題を数理最適化問題として定式化し、データを用いて未知の部分の推定し、最適なアクションを導くことができます。

もちろん、今回のモデリングにもいくつかの論点が考えられます。

- 今回は個人ごとの CM 接触データが手に入ったが、集計されたデータしか手に入らない可能性もある。その場合はどう対応するか
- 番組に対する過去の CM 接触データから将来を予測しているが、新しく始まった番組、放送時間帯や出演者が変更された番組をどう扱うか

5 章 離散最適化で広告出稿番組を選択する

- ・貪欲法は最適解を出すことを保証しない。より良い解を効率的に求めるための方法はないか

このような論点が実務で問題になる場合は、状況に応じて数理モデルを調整していく必要があるでしょう。

以上で、本編としての分析は終了です。以降で、本編では踏み込まなかった論点である「フリークエンシー」と「番組の貢献度」について検討します。

コラム：フリークエンシー分布の推定

本編の分析で、合算のユニークリーチを最大化する番組群を選択することができました。本編では選択した番組群に対する合算のユニークリーチだけに焦点を絞っていましたが、選択した番組群に対する CM 接触回数の分布、つまりフリークエンシーに興味がある場合もあります。本コラムでは、フリークエンシーの分布を数理モデルで推定する方法を考えます。

データの確認

フリークエンシーに対する数理モデルを考える前に、まずは実際のフリークエンシーの分布を確認してみましょう。ある程度多くの番組に CM を出稿している企業が望ましいので、9 番組に CM を出稿している企業を例にとることにします。

```
In[98]: # パネルの人数
n = df_individual_reach_train.select(
    pl.col.i.n_unique()
).item()

# 9番組にCMを出稿している企業のデータを抽出
# キリのよい10番組にCMを出稿している企業がなかったので
k_to_use = (
    df_company_reach_train
    .filter(pl.col.sum_d == 9)
    .get_column("k")
    .max()
)

# 個人ごとの接触データからフリークエンシーの分布を計算
```

```
df_frequency = (
    # 企業0の出稿したCMに限定
    df_individual_reach_train.filter(pl.col("k") == k_to_use)
    .group_by("i")
    .agg(f=pl.col("r").sum()) # 個人ごとにフリークエンシーを計算
    .group_by("f")
    .agg(prop=pl.col.i.n_unique() / n) # 割合を計算
    .sort("f")
)

# データの確認
print(f"企業{k_to_use}のフリークエンシー分布")
df_frequency
```

Out[98]: 企業48のフリークエンシー分布
shape: (9, 2)

f	prop
i64	f64
0	0.556
1	0.220
2	0.111
3	0.063
4	0.029
5	0.014
6	0.005
7	0.000
8	0.000

データを見ると、フリークエンシーが0回の人 が 55.6% いること、(逆にユニークリーチは 44.4% であること) がわかります。また、ほぼ 0% なので丸められて 0% になっていますが、最大で 8 回 CM に接触している個人もいることがわかります。この企業が CM を出稿している番組数は 9 番組なので、9 回中 8 回の CM に接触している個人もいるということです。

フリークエンシーの分布を計算できたので、わかりやすく可視化しておきます。

5章 離散最適化で広告出稿番組を選択する

```

In[99]: def plot_histogram(
    df_frequency: pl.DataFrame, f_max: int, title: str
) -> tuple[Figure, Axes]:
    """フリークエンシーの分布を可視化"""

    # 横に長くなりすぎるとうまく可視化できないので、
    # 必要であればf_max以下のデータのみを使う
    df_to_use = df_frequency.filter(pl.col.f <= f_max)

    fig, ax = plt.subplots()

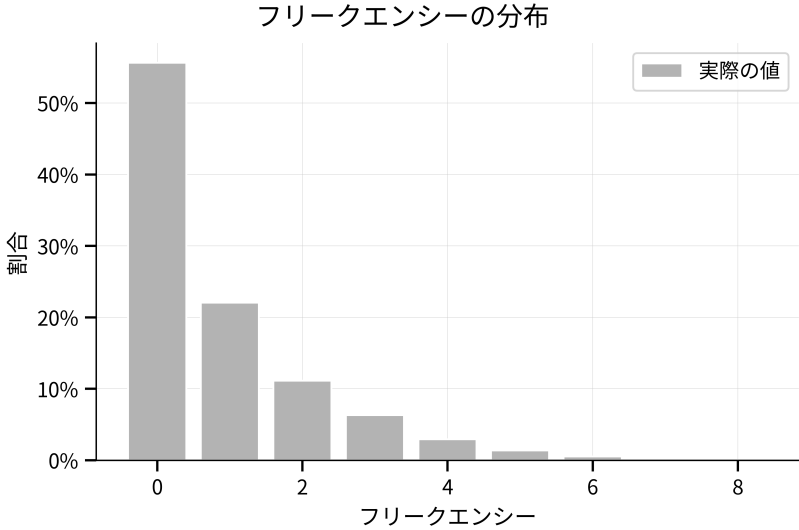
    ax.bar(
        df_to_use["f"],
        df_to_use["prop"],
        label="実際の値",
        color="0.7"
    )
    ax.legend()
    ax.yaxis.set_major_formatter(
        PercentFormatter(xmax=1, decimals=0)
    )
    ax.set_xlabel("フリークエンシー")
    ax.set_ylabel("割合")
    fig.suptitle(title)

    return fig, ax

# 選択した番組数
sum_d_to_use = (
    df_company_reach_train
    .filter(pl.col.k == k_to_use)
    .get_column("sum_d")
    .item()
)

# 可視化
fig, ax = plot_histogram(
    df_frequency=df_frequency,
    f_max=sum_d_to_use,
    title="フリークエンシーの分布"
)

```

二項分布によるモデリング

二項分布の導入

フリークエンシーの分布の実測データが確認できたので、次はこれを何らかの分布で近似できるように数理モデルを考えていきましょう。いきなり複雑な状況を考えるのは難しいので、まずは単純な状況を考えます。すべての番組 $j = 1, \dots, m$ に対して、すべての個人が同じ確率 π で接触すると仮定しましょう。このとき、 $\sum_{j=1}^m d_j$ 個の番組に CM を出稿したとすると、フリークエンシーの分布は「成功確率 π の試行を $\sum_{j=1}^m d_j$ 回実施した場合の成功回数の分布」になると解釈できます。つまり、以下の二項分布に従うと考えられます。

$$\begin{aligned} \Pr(F = f \mid \Sigma d) &= \text{Binom}(f \mid \Sigma d, \pi) \\ &= \binom{\Sigma d}{f} \pi^f (1 - \pi)^{\Sigma d - f} \end{aligned}$$

ここで、 $\sum_{j=1}^m d_j$ という表記が煩雑なので、簡略化して Σd と表記しました。今後この表記を使うことにします。

二項分布がどのような形状になるのか、いくつかのパラメータに関して可視化を行いましょう。二項分布の確率関数は SciPy の stats モジュールに `binom.pdf` メ

5 章 離散最適化で広告出稿番組を選択する

ソッドとして用意されています。引数 n がパラメータ $\sum d$ を、引数 p がパラメータ π を意味することに注意してください。

```
In[100]: from scipy.stats import binom

def plot_binom(
    f: np.ndarray, sum_d: int, pi_list: list[float]
) -> tuple[Figure, Axes]:
    """二項分布を可視化"""

    fig, ax = plt.subplots()

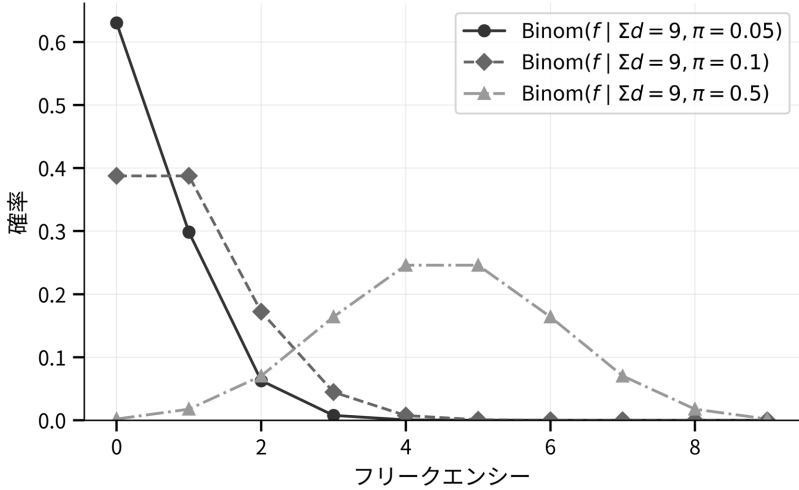
    markers = ["o", "D", "^", "x"]
    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, pi in enumerate(pi_list):
        y = binom.pmf(k=f, n=sum_d, p=pi)
        ax.plot(
            f,
            y,
            marker=markers[i % len(markers)],
            linestyle=linestyle[i % len(linestyle)],
            label=f"Binom$(f \mid \Sigma d = \{sum\_d\}, \pi$
= {pi})$",
        )

    ax.legend()
    ax.set_ylim(0, None)
    ax.set_xlabel("フリークエンシー")
    ax.set_ylabel("確率")
    fig.suptitle("二項分布の例")

    return fig, ax

# 二項分布を可視化
fig, ax = plot_binom(
    f=np.arange(10),
    sum_d=sum_d_to_use,
    pi_list=[0.05, 0.1, 0.5]
)
```

二項分布の例



企業0の選択した番組の総数は9だったので、 $\Sigma d = 9$ になっています。接触確率 π の値に応じて、二項分布の形状が変わっていることが見てとれます。

二項分布の推定

さて、それでは二項分布をデータに当てはめてみましょう。二項分布のパラメータ Σd は選択した番組の総数である9です。残りの π はデータから推定する必要があります。CM 接触確率が全個人全番組で一定であることを仮定していたので、企業48の選択した番組に対する全個人のCM 接触確率の平均で推定することが妥当と考えられます。

$$\hat{\pi} = \frac{1}{n \times \Sigma d} \sum_{i=1}^n \sum_{j=1}^m d_j \hat{\pi}_{i,j}$$

実際にデータから $\hat{\pi}$ を推定してみましょう。

```
In[101]: class BinomialFrequencyEstimator:
          """二項分布を仮定したフリークエンシーの分布を推定するモデル"""

          def __init__(self, PI: np.ndarray) -> None:
              """個人ごとのCM接触確率を保持

              Args:
```

5 章 離散最適化で広告出稿番組を選択する

```

    PI (np.ndarray): 個人iが番組jのCMに接触する確率
    """

    self.PI_ = PI

    def fit_predict(self, D: np.ndarray) -> np.ndarray:
        """フリークエンシーの分布を予測

        Args:
            D (np.ndarray): 選択した番組群。1行m列を想定

        Returns:
            np.ndarray: フリークエンシーの分布の予測値
            """
        # 選択した番組数
        sum_d = D.sum()

        # CM接触確率の全体平均。学習結果は`pi_`に格納
        self.pi_ = np.mean((self.PI_ * D).sum(1) / sum_d)

        # 二項分布を用いてフリークエンシーの分布を予測
        return binom.pmf(
            k=np.arange(sum_d + 1), n=sum_d, p=self.pi_
        )

```

学習と予測をまとめて行うメソッドを `fit_predict` メソッドとして定義しました。平均的な CM 接触確率を求めて `pi_` に格納し、二項分布を用いてフリークエンシーの分布を予測します。

実際に、企業 48 のフリークエンシーの分布をうまく近似できるかを確認してみましょう。

```

In[102]: def plot_histogram_with_distribution(
            df_frequency: pl.DataFrame,
            prop_pred: np.ndarray,
            prop_label: str,
            f_max: int,
            title: str,
        ) -> tuple[Figure, Axes]:

        """フリークエンシーの分布と、近似する分布を重ね合わせて可視化"""

        fig, ax = plot_histogram(

```

```

        df_frequency=df_frequency,
        f_max=f_max,
        title=title
    )
    ax.plot(
        range(f_max + 1),
        prop_pred,
        marker="o",
        label=prop_label
    ) # 近似する分布を追加
    ax.legend()

    return fig, ax

# 企業48に選択した番組群
D_to_use = (
    df_company_reach_train.filter(pl.col.k == k_to_use)
    .select(pl.selectors.starts_with("d_"))
    .to_numpy()
)

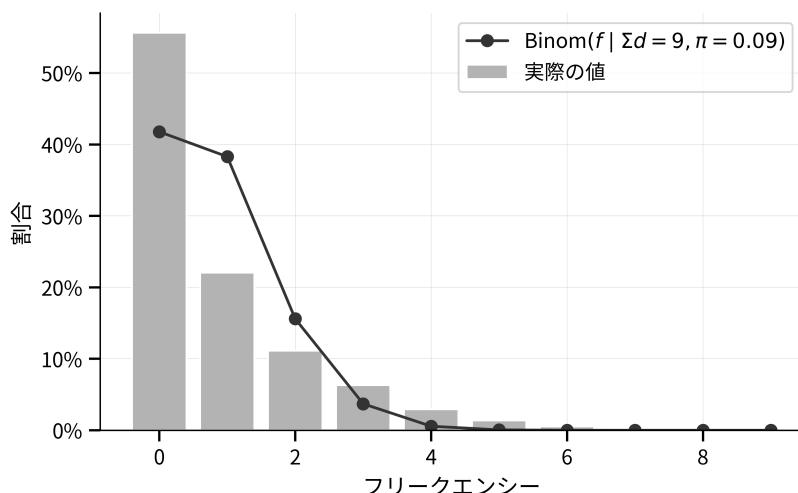
# フリークエンシーの分布の推定
bfe = BinomialFrequencyEstimator(PI=PI_train)
prop_binomial = bfe.fit_predict(D_to_use)

# フリークエンシーの分布と二項分布を重ね合あわせて可視化
fig, ax = plot_histogram_with_distribution(
    df_frequency=df_frequency,
    prop_pred=prop_binomial,
    prop_label=f"Binom$(f \\mid \\Sigma d = {sum_d_to_use}, \\pi = {bfe.pi_: .2f})$",
    f_max=sum_d_to_use,
    title="二項分布によるフリークエンシーの分布の推定",
)

```

5章 離散最適化で広告出稿番組を選択する

二項分布によるフリークエンシーの分布の推定



実際のフリークエンシーの分布と二項分布による予測値を見比べると、二項分布ではフリークエンシーの分布をうまく近似できていないことが見てとれます。よって、フリークエンシーの分布を近似するためには、他のモデリングを検討する必要があります。

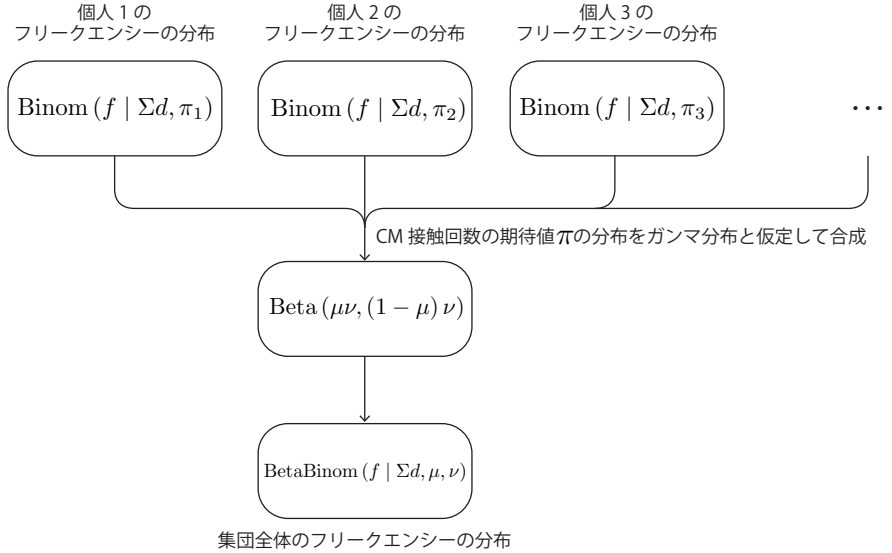
ベータ二項分布によるモデリング

ベータ分布の導入

二項分布によるモデリングではフリークエンシーの分布をうまく近似できませんでした。なぜうまく近似できなかったのでしょうか。二項分布による近似では「接触確率が一定」という仮定を置きましたが、これが現実的な仮定ではないことが原因です。実際には、個人ごとに各番組に対する接触確率は異なると考えられます。この事実をモデルに取り込む必要があります。

そこで、2章でポアソン分布を負の二項分布に拡張したように、二項分布を拡張することを目指します。2章では、ポアソン分布の期待値がガンマ分布に従うという仮定を置いてモデルを拡張しましたが、今回のパラメータ π はCM接触確率なので、別の分布を考える必要があります。確率のような区間 $[0, 1]$ の値をとる変数に対してよく使われる分布としてベータ分布があります*10。

* 10 ベータ分布の詳細な説明は 3.3.7 節をご確認ください。



■ 図 5.4: ベータ分布を用いた二項分布の合成イメージ

ベータ分布の確率密度関数は、期待値を $\mu = \alpha/(\alpha+\beta)$ 、分布の密集度を $\nu = \alpha+\beta$ と変換した

$$f(\pi \mid \mu, \nu) = \text{Beta}(\pi \mid \mu\nu, (1-\mu)\nu) \\ = \frac{\pi^{\mu\nu-1}(1-\pi)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)}$$

で定義します。

二項分布の合成

それでは、ベータ分布でさまざまな接触確率の分布を表現することで二項分布を拡張しましょう。2章でポアソン分布をガンマ分布を用いて合成したことを思い出しましょう。今回は、二項分布のパラメータ π がベータ分布に従うと仮定して、積分による分布の合成を行います（図 5.4）。

$$\Pr(F = f \mid \Sigma d, \mu, \nu) \\ = \int_0^1 \text{Binom}(f \mid \Sigma d, \pi) \text{Beta}(\pi \mid \mu\nu, (1-\mu)\nu) d\pi \\ = \int_0^1 \left(\binom{\Sigma d}{f} \pi^f (1-\pi)^{\Sigma d-f} \right) \left(\frac{\pi^{\mu\nu-1}(1-\pi)^{(1-\mu)\nu-1}}{B(\mu\nu, (1-\mu)\nu)} \right) d\pi$$

5 章 離散最適化で広告出稿番組を選択する

$$\begin{aligned}
&= \binom{\Sigma d}{f} \frac{B(\mu\nu + f, (1-\mu)\nu + \Sigma d - f)}{B(\mu\nu, (1-\mu)\nu)} \underbrace{\int_0^1 \frac{\pi^{\mu\nu+f-1} (1-\pi)^{(1-\mu)\nu + \Sigma d - f - 1}}{B(\mu\nu + f, (1-\mu)\nu + \Sigma d - f)} d\pi}_{=1} \\
&= \binom{\Sigma d}{f} \frac{B(\mu\nu + f, (1-\mu)\nu + \Sigma d - f)}{B(\mu\nu, (1-\mu)\nu)}
\end{aligned}$$

- 1 行目の変形：二項分布をベータ分布で重みを付けて合成
- 2 行目への変形：二項分布の確率密度関数とベータ分布の確率密度関数の定義をそのまま代入
- 3 行目への変形：まとめられる要素をまとめ、同じ数 $B(\mu\nu + f, (1-\mu)\nu + \Sigma d - f)$ を掛けて割ることで、第 2 項をベータ分布の確率密度関数に変形
- 4 行目の式では、確率密度関数を積分すると 1 になる性質を利用して第 2 項を消去

こうしてベータ分布をパラメータ π の重みとして合成された二項分布は、ベータ二項分布と呼ばれています。

$$\begin{aligned}
\Pr(F = f \mid \Sigma d, \mu, \nu) &= \text{BetaBinom}(f \mid \Sigma d, \mu, \nu) \\
&= \binom{\Sigma d}{f} \frac{B(\mu\nu + f, (1-\mu)\nu + \Sigma d - f)}{B(\mu\nu, (1-\mu)\nu)}
\end{aligned}$$

ベータ二項分布の期待値は $\Sigma d\mu$ で、これは二項分布の期待値が「試行回数 $\Sigma d \times$ 成功確率 μ 」であることに対応しています。ベータ分布の期待値が μ なので、ベータ二項分布の期待値は、ベータ分布の期待値を試行回数の Σd 倍したのになります。

それでは、ベータ二項分布がどのような形状をとるのか可視化で確認してみましょう。ベータ二項分布の確率関数も SciPy の stats モジュールに `betabinom.pdf` メソッドとして用意されています。引数 n がパラメータ Σd に、引数 a がパラメータ $\mu\nu$ に、引数 b が $(1-\mu)\nu$ に対応します。

```
In[103]: from scipy.stats import betabinom
```

```
def plot_betabinom(
    f: np.ndarray,
    sum_d: int,
    mu_list: list[float],
```



```

nu_list: list[float]
) -> tuple[Figure, Axes]:
    """ベータ二項分布を可視化"""

    fig, ax = plt.subplots()
    markers = ["o", "D", "^", "x"]
    linestyle = ["solid", "dashed", "dashdot", "dotted"]
    for i, (mu, nu) in enumerate(zip(mu_list, nu_list)):
        y = betabinom.pmf(
            k=f, n=sum_d, a=mu * nu, b=(1 - mu) * nu
        )
        ax.plot(
            f,
            y,
            marker=markers[i % len(markers)],
            linestyle=linestyle[i % len(linestyle)],
            label=f"BetaBinom$(f \mid \Sigma d = \{sum\_d\}, \\mu = \{mu\}, \\nu = \{nu\})$",
        )

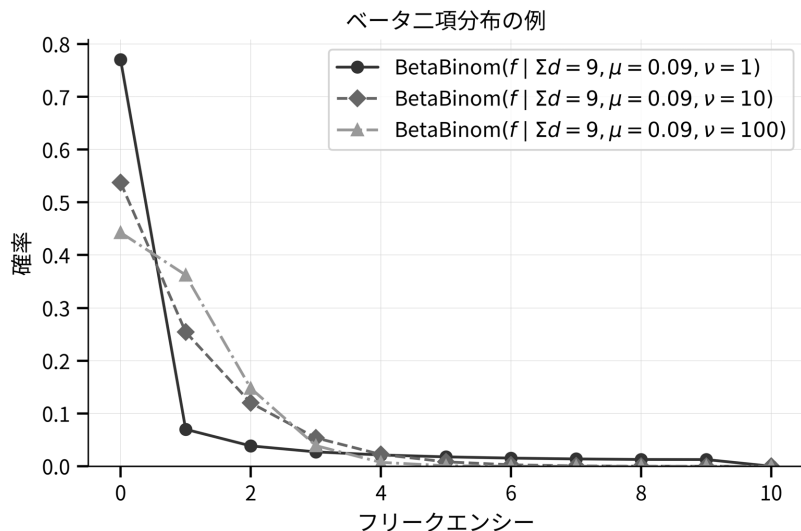
    ax.legend()
    ax.set_ylim(0, None)
    ax.set_xlabel("フリークエンシー")
    ax.set_ylabel("確率")
    ax.set_title("ベータ二項分布の例")

    return fig, ax

# ベータ二項分布の例を可視化
fig, ax = plot_betabinom(
    f=np.arange(11),
    sum_d=sum_d_to_use,
    mu_list=[bfe.pi_.round(2)] * 3,
    nu_list=[1, 10, 100],
)

```

5章 離散最適化で広告出稿番組を選択する



フリークエンシーの期待値が同じであっても、パラメータ ν に応じて分布の形状が異なることがわかります。よって、ベータ二項分布は二項分布よりも柔軟なモデリングが可能です。なお、 ν を十分に大きくすると、ベータ二項分布は二項分布に収束します。その意味で、ベータ二項分布は二項分布を一般化した分布と解釈することもできます。

ベータ二項分布の推定

さて、ここまでの議論で、ベータ二項分布を用いることで、二項分布よりも柔軟なモデリングが可能となることがわかりました。次は、ベータ二項分布のパラメータを推定する方法を考えましょう。

まず、試行回数 Σd は選択した番組の総数なのでただちにわかります。次に、 μ はベータ分布の期待値なので、二項分布の場合と同じく選択した番組群に対して全個人の接触確率を平均すればよさそうです。

$$\hat{\mu} = \frac{1}{n \times \Sigma d} \sum_{i=1}^n \sum_{j=1}^m d_j \hat{\pi}_{i,j}$$

残ったパラメータは ν です。 ν の推定には本編で推定したユニークリーチを使いましょう。まず、0 回接触の割合は、ベータ二項分布の確率関数を使って以下のよう計算できます。

$$\begin{aligned}\Pr(F=0 \mid \Sigma d, \mu, \nu) &= \binom{\Sigma d}{0} \frac{B(\mu\nu + 0, (1-\mu)\nu + \Sigma d - 0)}{B(\mu\nu, (1-\mu)\nu)} \\ &= \frac{B(\mu\nu, (1-\mu)\nu + \Sigma d)}{B(\mu\nu, (1-\mu)\nu)}\end{aligned}$$

そして、0 回接触の割合はユニークリーチを 100% から引くことで推定できます。

$$1 - \widehat{\mathbb{E}}[R \mid \mathbf{d}] = 1 - \left(1 - \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j} \right) = \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j}$$

さらに、試行回数 K と期待値 $\hat{\mu}$ はすでにわかっているので、両辺の差分をできるだけ小さくするパラメータ $\hat{\nu}$ を推定することで、データに最も当てはまるベータ二項分布を推定できます。

$$\hat{\nu} = \arg \min_{\nu} \left(\frac{1}{n} \sum_{i=1}^n \prod_{j=1}^m (1 - \hat{\pi}_{i,j})^{d_j} - \frac{B(\hat{\mu}\nu, (1-\hat{\mu})\nu + \Sigma d)}{B(\hat{\mu}\nu, (1-\hat{\mu})\nu)} \right)^2$$

ベータ二項分布を用いた数理モデルの実装

ベータ二項分布の推定方法が決まったので、数理モデルの実装を行しましょう。ProgramUniqueReachEstimator クラスの predict メソッドを利用したいので、ProgramUniqueReachEstimator クラスを継承した BetaBinomialFrequencyEstimator クラスを作成します。

```
In[104]: from scipy.optimize import minimize_scalar
```

```
class BetaBinomialFrequencyEstimator(
    ProgramUniqueReachEstimator
):
    """ベータ二項分布でフリークエンシーの分布を推定するモデル"""

    def fit_predict(self, D: np.ndarray) -> np.ndarray:
        """フリークエンシーの分布を予測

        Args:
            D (np.ndarray): 選択した番組群。1行m列を想定

        Returns:
            np.ndarray: フリークエンシーの分布の予測値
        """
```

5 章 離散最適化で広告出稿番組を選択する

```

sum_d = D.sum()
r_ = self.predict(D)[0]
self.mu_ = np.mean((self.PI_ * D).sum(1) / sum_d)

# ベータ二項分布のパラメータnuを推定
self.nu_ = minimize_scalar(
    fun=lambda nu: (
        (1 - r_)
        - betabinom.pmf(
            k=0,
            n=sum_d,
            a=self.mu_ * nu,
            b=(1 - self.mu_) * nu
        )
    )
    ** 2,
    bounds=(0, 50),
    method="bounded",
).x

return betabinom.pmf(
    k=np.arange(sum_d + 1),
    n=sum_d,
    a=self.mu_ * self.nu_,
    b=(1 - self.mu_) * self.nu_,
)

```

データによる検証

数理モデルが実装できたので、実際にフリークエンシーの分布をうまく推定できるか確認してみましょう。

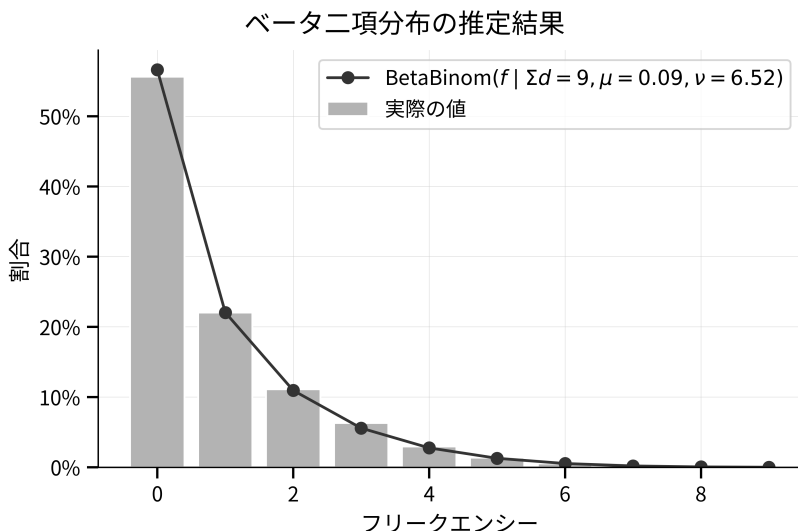
```

In[105]: # ベータ二項分布によるフリークエンシー分布の推定
bbfe = BetaBinomialFrequencyEstimator(PI=PI_train)
prop_betabinomial = bbfe.fit_predict(D=D_to_use)

# CM接触回数分布とベータ二項分布による推定結果を重ね合わせて可視化
fig, ax = plot_histogram_with_distribution(
    df_frequency=df_frequency,
    prop_pred=prop_betabinomial,
    prop_label=f"BetaBinom$(f \\mid \\Sigma d = \{sum\_d\_to\_use\}, \\\mu = \{bbfe.mu\_ : .2f\}, \\\nu = \{bbfe.nu\_ : .2f\})$",

```

```
f_max=sum_d_to_use,
title="ベータ二項分布の推定結果",
)
```

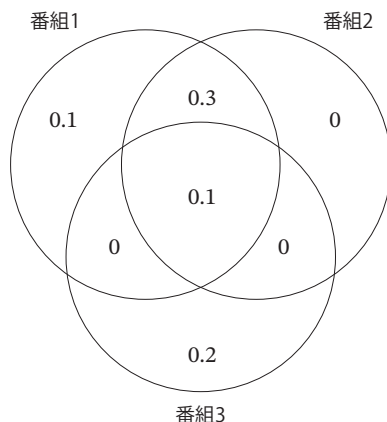


二項分布ではうまく近似できなかったフリークエンシーの分布を、ベータ二項分布はうまく近似できていることが見てとれます。以上で、選択した番組群に対して、合算のユニークリーチだけでなくフリークエンシーも推定できるようになりました。実務の状況に応じてふさわしい数理モデルを構築することで、より実践的な意思決定に役立つ分析を行えるようになるでしょう。

コラム：合算のユニークリーチを各番組の貢献度に分解する

本編では、選択した番組群に対するユニークリーチを数理モデルを用いて推定し、最大化問題を解くことで効率的な番組群を選択することができました。このとき、選択した番組の中には合算のユニークリーチを大きく増加させている番組もあれば、合算のユニークリーチには大きくは影響しない番組もあるはずです。それぞれの番組が合算のユニークリーチにどの程度「貢献」しているのかを知ることがで

5 章 離散最適化で広告出稿番組を選択する



■ 図 5.5: ユニークリーチの数値例

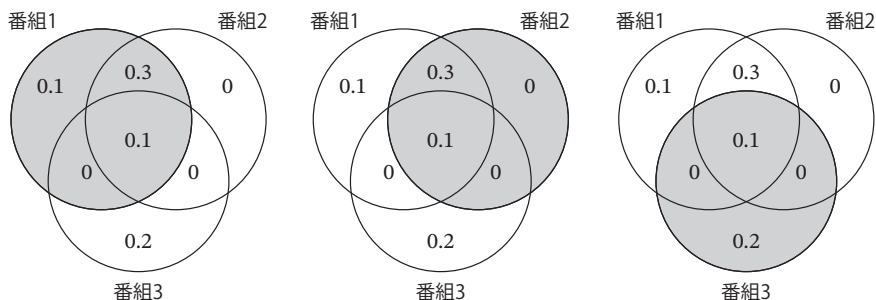
できれば、過去を振り返って番組の評価を行えるだけでなく、より貢献している番組は重要とみなして今後も継続して CM を出稿し、そうでない番組は出稿を見合わせるという将来の意思決定にも役立ちます。

そこで、以降では、そもそも何をもって番組の「貢献度」とするのかを定義し、さらに合算のユニークリーチに各番組がどの程度貢献しているのかを把握する方法を考えていきます。

貢献度の定義

合算のユニークリーチに対する各番組の「貢献度」を考えていくために、まずは具体的な数字例からはじめましょう。図 5.5 は CM 接触状況を表すベン図で、数字は各領域の割合を表しています。いま、ある企業は番組 1、番組 2、番組 3 の三つの番組に CM を出稿しています。番組 1 では $0.1 + 0.3 + 0 + 0.1 = 50\%$ の、番組 2 では $0 + 0.3 + 0 + 0.1 = 40\%$ の、番組 3 では $0.2 + 0 + 0 + 0.1 = 30\%$ のユニークリーチを獲得できています。また、番組 1、番組 2、番組 3 の合算のユニークリーチは $0.1 + 0 + 0.2 + 0.3 + 0 + 0 + 0.1 = 70\%$ です。

番組 1 と番組 3 は、その番組でしか獲得できないユニークリーチをそれぞれ 10% と 20% もっていますが、番組 2 の重複部分を見ると、番組 2 の接触者は番組 1 の



■ 図 5.6: 各番組が単体で獲得できるユニークリーチ

接触者に完全に含有されていて、番組 2 でしか獲得できないユニークリーチはありません。

このような状況で、合算のユニークリーチ 70% に対して、番組 1、番組 2、番組 3 はそれぞれどのくらい貢献していると考えればよいのでしょうか。絶対的な正解はありませんが、貢献度を計算する方法をいくつか考えられそうです。

番組単体のユニークリーチを貢献度とする

まず最初に考えられる方法は、番組が単体で獲得したユニークリーチをそのまま貢献度とする方法です。以降では、「番組が単体で獲得したユニークリーチ」を「番組単体のユニークリーチ」と呼ぶことにします。

番組単体のユニークリーチを図 5.6 に示します。グレーに塗られた領域が番組 j 単体のユニークリーチです。

番組 j の貢献度を ϕ_j とすると、この定義では各番組の貢献度は

$$\phi_1 = 0.1 + 0.3 + 0 + 0.1 = 50\%$$

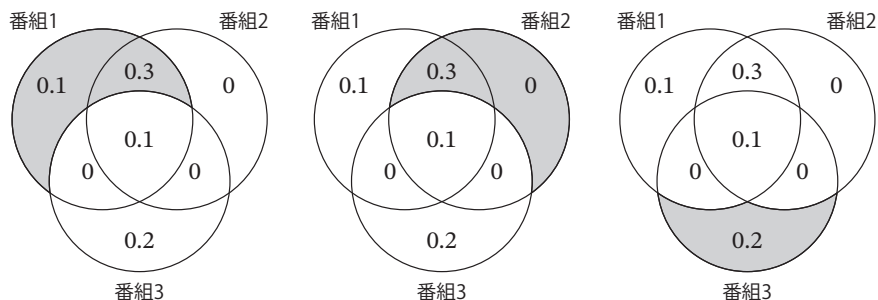
$$\phi_2 = 0 + 0.3 + 0 + 0.1 = 40\%$$

$$\phi_3 = 0.2 + 0 + 0 + 0.1 = 30\%$$

で計算します。この定義は番組が単体で獲得したユニークリーチがそのまま貢献度になるため、直感的に理解しやすいという利点があります。

一方で、番組単体のユニークリーチが大きくても、重複部分が多い場合は評価を下げるべきとも考えられます。例えば、この計算方法では番組 3 よりも番組 2 に

5 章 離散最適化で広告出稿番組を選択する



■ 図 5.7: 各番組でしか獲得できないユニークリーチ

大きな貢献度が割り当てられます。しかし、番組 2 の接触者は番組 1 の接触者に完全に含有されているため、番組 1 に CM を出稿しているという条件のもとでは、むしろ番組 3 のほうが合算のユニークリーチに貢献しているとも考えられます。

また、番組単体のユニークリーチをそのまま貢献度とする方法は、貢献度を足し合わせたときに合算のユニークリーチを超えてしまいます ($0.5 + 0.4 + 0.3 = 1.2 > 0.7$)。よって、合算のユニークリーチ 70% を貢献度に分解できているわけではありません。

番組固有のユニークリーチを貢献度とする

次に考えられる方法は、その番組だけで獲得できるユニークリーチ（番組固有のユニークリーチと呼ぶことにします）を貢献度とする方法です。

番組固有のユニークリーチを図 5.7 に示します。グレーに塗られた領域が番組 j 固有のユニークリーチです。

数値例では、番組 1 の貢献度は 10%、番組 2 の貢献度は 0%、番組 3 の貢献度は 20% となります。「CM 出稿を止める番組を一つだけ選ぶならどの番組がいいか」という実務上よく現れる問いに対して、この定義であれば「貢献度の最も低いものを止める」と答えられるので、実践的で理解しやすい定義と言えます。

一方で、単体では多くのユニークリーチを獲得できる番組 1 や番組 2 より番組 3 を優先するのは、それはそれで極端な選択にも思えます。実際、番組を「一つだけ」選ぶのであれば番組 3 が最も外しづらいですが、もし「二つ」の番組を外すなら番組 2 と番組 3 を外して番組 1 を残すことが正解です。

また、番組単体のユニークリーチを貢献度とする方法と同様に、番組固有のユニークリーチを足しても合算のユニークリーチと一致せず、貢献度を分解できていないわけではないという問題も引き続き発生しています ($0.1 + 0 + 0.2 = 0.3 < 0.7$)。

二つの貢献度を組み合わせる

番組単体のユニークリーチを貢献度とする方法も、番組固有のユニークリーチを貢献度とする方法も、それぞれ一長一短があることがわかりました。そこで、これらの方法を組み合わせることで、よりバランスのとれた貢献度を計算する方法を考えてみましょう。

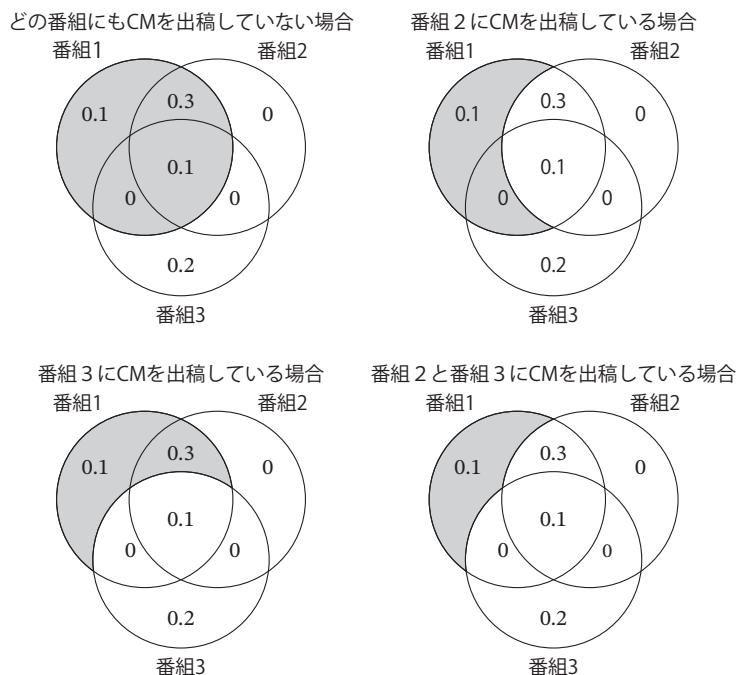
まず、番組 1 の貢献度に注目します。番組単体のユニークリーチを貢献度とする方法をあらためて考えると、これは「『他の番組に CM を出稿していない前提で』番組 1 に CM を出稿するとユニークリーチがどの程度増加するのか」を測定していると解釈できます。一方で、番組固有のユニークリーチは、「『番組 2 と番組 3 に CM を出稿しているという前提で』番組 1 に CM を出稿するとユニークリーチがどの程度増加するのか」を測定していると解釈できます。つまり、どちらの定義も「番組に CM を出稿した際に追加的なユニークリーチの増加分」を貢献度の計算に利用しているが、前提が異なるので貢献度が異なるということです。

実際に、番組 1 が追加的に獲得できるユニークリーチを計算してみましょう。番組 1 を選択することによって追加的に獲得できるユニークリーチは、他のどの番組に CM を出稿しているのかという前提によって異なります。

- どの番組にも CM を出稿していない場合、番組 1 による追加的なユニークリーチは $0.1 + 0.3 + 0 + 0.1 = 50\%$ (図 5.8 左上)
- 番組 2 に CM を出稿している場合、番組 1 による追加的なユニークリーチは $0.1 + 0 = 10\%$ (図 5.8 右上)
- 番組 3 に CM を出稿している場合、番組 1 による追加的なユニークリーチは $0.1 + 0.3 = 40\%$ (図 5.8 左下)
- 番組 2 と番組 3 に CM を出稿している場合、番組 1 による追加的なユニークリーチは 10% (図 5.8 右下)

同様の計算を番組 2 と番組 3 に対しても行い、結果を表 5.7 にまとめました。

5 章 離散最適化で広告出稿番組を選択する



■ 図 5.8: 番組 1 への CM 出稿で追加的に獲得できるユニークリーチ

▼ 表 5.7: 番組が追加的に獲得できるユニークリーチ (数値例)

番組の追加順	番組 1	番組 2	番組 3
1 → 2 → 3	0.5	0	0.2
1 → 3 → 2	0.5	0	0.2
2 → 1 → 3	0.1	0.4	0.2
2 → 3 → 1	0.1	0.4	0.2
3 → 1 → 2	0.4	0	0.3
3 → 2 → 1	0.1	0.3	0.3

番組の「追加順」に応じて、各番組による追加的なユニークリーチの増分が異なることがわかります。表を見ると、番組単体のユニークリーチを貢献度とする方法は「最初に番組 1 が選択される」という状況だけに注目し、番組固有のユニーク

リーチを貢献度とする方法は「最後に番組 1 に CM が出稿される」という状況だけに注目していることがわかります。ということは、すべての追加順を考慮して貢献度を計算することで、よりバランスのとれた貢献度の指標を作ることができそうです。

具体的な方法として、追加的に獲得できたユニークリーチをすべての追加順で平均することで、追加順の影響を打ち消すことにします。つまり、この貢献度の定義では、各番組の貢献度を

$$\begin{aligned}\phi_1 &= \frac{1}{6} (0.5 + 0.5 + 0.1 + 0.1 + 0.4 + 0.1) = \frac{1.7}{6} \approx 28.3\% \\ \phi_2 &= \frac{1}{6} (0 + 0 + 0.4 + 0.4 + 0 + 0.3) = \frac{1.1}{6} \approx 18.3\% \\ \phi_3 &= \frac{1}{6} (0.2 + 0.2 + 0.2 + 0.2 + 0.3 + 0.3) = \frac{1.4}{6} \approx 23.3\%\end{aligned}$$

で計算します。

この定義で計算された貢献度は、番組単体のユニークリーチを貢献度とする方法と番組固有のユニークリーチを貢献度とする方法の中間に位置しており、追加順の影響が打ち消されたよりバランスのとれた貢献度の指標になっています。もちろん、番組単体のユニークリーチと番組固有のユニークリーチがともに大きければ大きい貢献度を、ともに小さければ小さい貢献度を割り当てます。

さらに、この定義では貢献度の合計が合算のユニークリーチと一致していることも確認できます ($1.7/6 + 1.1/6 + 1.4/6 = 4.2/6 = 0.7$)。よって、「合算のユニークリーチ 70%のうち、番組 1 によって 28.3%、番組 2 によって 18.3%、番組 3 によって 23.3%のリーチが獲得されている」という解釈が可能な点も魅力です。

この貢献度の分解方法は協力ゲーム理論の Shapley 値の考え方に基づいているので、Shapley 値による貢献度の分解と呼ぶことにしましょう^{*11}。以降では、この Shapley 値による貢献度の分解に焦点を当てます。

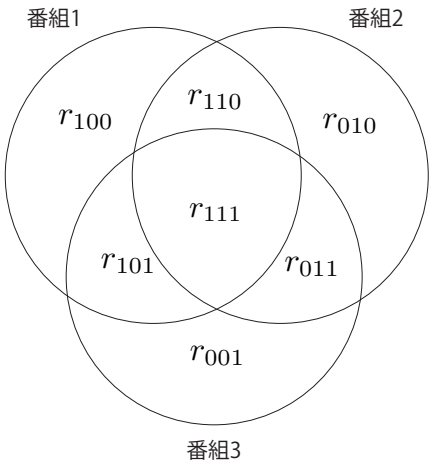
Shapley 値による貢献度の解釈

数値例による確認を終えたので、Shapley 値による貢献度の分解を一般化してみましょう。数値例と同様に 3 番組の状況を考えます。

図 5.9 では各領域に新しい記号を割り当てています。各番組の領域に該当するかどうかを $\{0, 1\}$ の添字で表していることに注意してください。例えば、番組 1 固有

* 11 協力ゲーム理論と Shapley 値についてより詳しく知りたい方は、岡田 (2021) を参照してください。データサイエンスとの関連として、Shapley 値は機械学習モデルの解釈手法にも応用されており、Lundberg and Lee (2017) で提案された SHAP (SHapley Additive exPlanations) という手法がよく知られています。SHAP を含む機械学習モデルの解釈手法に興味のある方は森下 (2021) をご確認ください。

5 章 離散最適化で広告出稿番組を選択する



■ 図 5.9: ベン図の各領域の表記

のユニークリーチは、番組 1 には該当するが番組 2 と番組 3 には該当しないので r_{100} です。同様にして、番組 1 と番組 2 だけの重複リーチ（番組 3 との重複部分は除く）は r_{110} 、番組 1 と番組 3 だけの重複リーチ（番組 2 との重複部分は除く）は r_{101} となります。さらに、すべての番組の重複リーチは r_{111} です。

この記法を使うと、Shapley 値による貢献度の計算は表 5.8 のように整理できます。

▼ 表 5.8: 番組が追加的に獲得できるユニークリーチ

番組の追加順	番組 1	番組 2	番組 3
1 → 2 → 3	$r_{100} + r_{110} + r_{101} + r_{111}$	$r_{010} + r_{011}$	r_{001}
1 → 3 → 2	$r_{100} + r_{110} + r_{101} + r_{111}$	r_{010}	$r_{001} + r_{011}$
2 → 1 → 3	$r_{100} + r_{101}$	$r_{010} + r_{110} + r_{011} + r_{111}$	r_{001}
2 → 3 → 1	r_{100}	$r_{010} + r_{110} + r_{011} + r_{111}$	$r_{001} + r_{101}$
3 → 1 → 2	$r_{100} + r_{110}$	r_{010}	$r_{001} + r_{101} + r_{011} + r_{111}$
3 → 2 → 1	r_{100}	$r_{010} + r_{110}$	$r_{001} + r_{101} + r_{011} + r_{111}$

表 5.8 から、番組 1 の Shapley 値の計算を確認すると以下になります。

$$\begin{aligned}
 \phi_1 &= \frac{1}{6}(r_{100} + r_{110} + r_{101} + r_{111}) \\
 &\quad + \frac{1}{6}(r_{100} + r_{110} + r_{101} + r_{111}) \\
 &\quad + \frac{1}{6}(r_{100} + r_{101}) \\
 &\quad + \frac{1}{6}(r_{100}) \\
 &\quad + \frac{1}{6}(r_{100} + r_{110}) \\
 &\quad + \frac{1}{6}(r_{100}) \\
 &= r_{100} && \text{(番組固有のリーチ)} \\
 &\quad + \frac{1}{2}(r_{110} + r_{101}) && \text{(2 番組の重複リーチ)} \\
 &\quad + \frac{1}{3}r_{111} && \text{(3 番組の重複リーチ)}
 \end{aligned}$$

ここから、Shapley 値は、2 番組の重複リーチは 2 等分、3 番組の重複リーチは 3 等分して各番組に貢献度として振り分けていると解釈できます。これは直感的な操作であり、Shapley 値は直感的にも解釈しやすい操作の結果として得られる指標であると言えます。

Shapley 値の計算量の克服

ここまでの議論で、Shapley 値で番組の貢献度をうまく分解できるだけでなく、Shapley 値は直感的にも解釈しやすい指標であることがわかりました。一方で、Shapley 値にも欠点があります。Shapley 値を計算するためには、まずベン図の各領域である番組固有のユニークリーチと重複リーチを計算しておいて、さらにすべての番組の追加順に対して追加的に獲得できるユニークリーチを計算する必要があります。

数値例では 3 番組だけが対象だったので、ベン図の領域は $2^3 - 1 = 7$ 個、追加順は $3! = 6$ 通りの計算で済みましたが、番組の数が増えると計算量が爆発的に増加してしまいます。例えば、10 番組の場合はベン図の領域は $2^{10} - 1 = 1023$ 個、追加順は $10! = 3628800$ 通りの計算が必要になります。

よって、計算量を削減するために、さらなるアプローチを考える必要があります。

5章 離散最適化で広告出稿番組を選択する

個人ごとの接触への分解

数値例による確認

ここまで、ユニークリーチとして集団全体のユニークリーチに注目していました。しかし、集団全体のユニークリーチを直接 Shapley 値で分解するには膨大な計算量が必要でした。

ところで、本編では集団全体のユニークリーチを個人ごとに分解することで計算量を削減する方法を考えました。同じように、今回も集団ではなく個人ごとの接触に焦点をあてることで計算量を削減する方法を考えてみましょう。

最初の3番組の数値例を例にとりて考えていきましょう。Shapley 値による各番組の貢献度は、番組固有のユニークリーチに重複部分を均等に配分したものを加えればよかったことを思い出すと、以下で計算できます。

$$\begin{aligned}\phi_1 &= (0.1) + \frac{1}{2}(0.3 + 0) + \frac{1}{3}(0.1) = \frac{1.7}{6} \approx 28.3\% \\ \phi_2 &= (0) + \frac{1}{2}(0.3 + 0) + \frac{1}{3}(0.1) = \frac{1.1}{6} \approx 18.3\% \\ \phi_3 &= (0.2) + \frac{1}{2}(0 + 0) + \frac{1}{3}(0.1) = \frac{1.4}{6} \approx 23.3\%\end{aligned}$$

この計算を起点に、個人ごとの接触に注目しましょう。表 5.9 に示すように、この集団としてのユニークリーチの裏側には、個人ごとの接触があります。

個人 1 から個人 10 までの 10 人の個人に対して、各番組に接触していれば 1、そうでなければ 0 と記載しています。表の設定は集団としてのユニークリーチと整合性をもっていることに注意してください。例えば、番組 1 には 10 人中 5 人が接触しておりユニークリーチは 50%、番組 2 には 10 人中 4 人が接触しておりユニークリーチは 40%、番組 3 には 10 人中 3 人が接触しておりユニークリーチは 30% となっています。

さて、Shapley 値の計算では、重複リーチは均等に分割して貢献度に加えていました。これを個人レベルで考えると、「複数の番組に接触している人は、貢献度を均等に分割して加える」という操作を行えば、同じように貢献度を均等に割り当てることができそうです。例えば、個人 1 は三つの番組に接触しているので、それぞれの番組に対して $1/3$ を貢献度として加えます。同様に、個人 2 は番組 1 と番組 2 の二つの番組に接触しているので、番組 1 と番組 2 にそれぞれ $1/2$ を貢献度として加えます。これを全個人で計算したものが表 5.10 です。

▼ 表 5.9: 個人ごとの各番組への接触状況

個人 i	番組 1	番組 2	番組 3
1	1	1	1
2	1	1	0
3	1	1	0
4	1	1	0
5	1	0	0
6	0	0	1
7	0	0	1
8	0	0	0
9	0	0	0
10	0	0	0
平均	5/10	4/10	3/10

▼ 表 5.10: 個人ごとの番組貢献度

個人 i	番組 1	番組 2	番組 3
1	1/3	1/3	1/3
2	1/2	1/2	0
3	1/2	1/2	0
4	1/2	1/2	0
5	1	0	0
6	0	0	1
7	0	0	1
8	0	0	0
9	0	0	0
10	0	0	0
平均	17/60	11/60	14/60

5 章 離散最適化で広告出稿番組を選択する

番組ごとの貢献度は各個人の貢献度を平均することで求めます。

$$\begin{aligned}\phi_1 &= \frac{1}{10} \left(\frac{1}{3} + \frac{1}{2} + \frac{1}{2} + \frac{1}{2} + 1 + 0 + 0 + 0 + 0 + 0 \right) = \frac{17}{60} \approx 28.3\% \\ \phi_2 &= \frac{1}{10} \left(\frac{1}{3} + \frac{1}{2} + \frac{1}{2} + \frac{1}{2} + 0 + 0 + 0 + 0 + 0 + 0 \right) = \frac{11}{60} \approx 18.3\% \\ \phi_3 &= \frac{1}{10} \left(\frac{1}{3} + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0 \right) = \frac{14}{60} \approx 23.3\%\end{aligned}$$

個人ごとの接触状況から計算した貢献度は、集団のユニークリーチを Shapley 値で分解した場合と一致していることが見てとれます。この計算はリーチの重複部分を個別に求めるような計算は不要で、「個人の総数 × 選択した番組数」個の個人の接触状況から計算でき、番組数が多くなっても現実的な計算量で貢献度を求めることができます。

個人ごとの貢献度計算の一般化

今後のために、上記の数値例で行った計算を一般化しておきましょう。個人 $i = 1, \dots, n$ が、選択した番組 $j \in \mathcal{J}$ の CM に接触したかどうかの実測値を $r_{i,j} \in \{0, 1\}$ とします。番組合算でのユニークリーチを $\bar{r}$ とすると、これは選択した番組のいずれかで CM に接触した個人の割合なので、

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\}$$

で計算できます。この番組合算のユニークリーチ $\bar{r}$ を番組ごとの貢献度 ϕ_j に分解したいというのが今回のモチベーションでした。

数値例で見たように、個人ごと接触を用いて個人 i の番組 j への貢献度 $\phi_{i,j}$ を計算し、それを平均することで番組の貢献度を計算します。複数の番組に接触した個人は、接触した番組の総数で割って各番組に対する貢献度を均等に分割します。なお、どの番組にも接触していない個人は分母が 0 になるので場合分けしておき、貢献度 0 とします。

$$\begin{aligned}\phi_{i,j} &= \begin{cases} r_{i,j} / \sum_{j \in \mathcal{J}} r_{i,j} & \text{if } \sum_{j \in \mathcal{J}} r_{i,j} \geq 1, \\ 0 & \text{if } \sum_{j \in \mathcal{J}} r_{i,j} = 0 \end{cases} \\ &= \frac{r_{i,j}}{\sum_{j \in \mathcal{J}} r_{i,j}} \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\} + 0 \times \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} = 0 \right\}\end{aligned}$$

$$= \frac{r_{i,j}}{\sum_{j \in \mathcal{J}} r_{i,j}} \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\}$$

この $\phi_{i,j}$ を全個人で平均したものを番組 j の貢献度とします。

$$\phi_j = \frac{1}{n} \sum_{i=1}^n \phi_{i,j}$$

以上で一般化が完了しました。念のため、番組ごとの貢献度の合計が合算のユニークリーチに一致することも確認しておきましょう。

$$\begin{aligned} \sum_{j \in \mathcal{J}} \phi_j &= \sum_{j \in \mathcal{J}} \frac{1}{n} \sum_{i=1}^n \phi_{i,j} \\ &= \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{J}} \phi_{i,j} \\ &= \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{J}} \frac{r_{i,j}}{\sum_{j \in \mathcal{J}} r_{i,j}} \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\} \\ &= \frac{1}{n} \sum_{i=1}^n \frac{\sum_{j \in \mathcal{J}} r_{i,j}}{\sum_{j \in \mathcal{J}} r_{i,j}} \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\} \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{1} \left\{ \sum_{j \in \mathcal{J}} r_{i,j} \geq 1 \right\} \\ &= \bar{r} \end{aligned}$$

この式変形の説明を以下に記載します。

- 1 行目の変形： ϕ_j の定義を代入
- 2 行目への変形：総和の順序を並べ替え
- 3 行目への変形： $\phi_{i,j}$ の定義を代入
- 4 行目への変形：総和の位置を分子に移動
- 5 行目への変形：分母分子で $\sum_{j \in \mathcal{J}} r_{i,j}$ が相殺されて 1 になる
- 6 行目への変形： $\bar{r}$ の計算方法そのもの

以上で、番組ごとの貢献度を足し合わせると、合算のユニークリーチと一致することが確認できました。

5章 離散最適化で広告出稿番組を選択する

個人ごとの接触確率への分解

Shapley 値による貢献度の近似

ここまでの議論で、個人 i の番組 j の CM 接触有無を表す $r_{i,j}$ を実測値として観測できている場合は、 $r_{i,j}$ を用いて効率的に貢献度を分解できるとわかりました。

過去キャンペーンのデータがある場合は $r_{i,j}$ が観測できるので問題なく貢献度を計算できますが、過去キャンペーンの評価ではなく、任意に組み合わせた番組群の合算のユニークリーチに対して貢献度の分解を行いたい場合は、 $r_{i,j}$ を実測値として観測できません。この設定のもとでは、CM への接触有無の $r_{i,j}$ ではなく、接触確率 $\pi_{i,j}$ を利用する必要があります。というのも、本編で見たように、番組には複数の CM が出稿されるので、その番組に CM を出稿することを決めたとしても、ある個人が実際に CM に接触するかどうかは確率的に定まると考えたほうが自然だからです。

ここまでの議論を接触確率に適用するために、個人 i の番組 j の CM 接触を実測値 $r_{i,j}$ ではなく確率変数 $R_{i,j}$ として扱いましょう。そして、番組 j の貢献度 ϕ_j の期待値を考えます。個人ごとの番組に対する貢献度 $\phi_{i,j}$ の定義に沿って期待値を式変形していくと、以下のように分解できます。

$$\begin{aligned}\mathbb{E}[\phi_j] &= \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n \phi_{i,j}\right] \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{E}[\phi_{i,j}] \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{E}\left[\left(\frac{R_{i,j}}{\sum_{j \in \mathcal{J}} R_{i,j}}\right) \mathbf{1}\left\{\sum_{j \in \mathcal{J}} R_{i,j} \geq 1\right\}\right]\end{aligned}$$

計算の簡略化のため、確率変数の積の期待値を期待値の積で近似^{*12}しましょう。すると、貢献度を二つのパートに分解できます。

*** 12** 確率変数 (X, Y) の積の期待値は

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] + \text{Cov}[X, Y] \neq \mathbb{E}[X]\mathbb{E}[Y]$$

であり、確率変数の積の期待値は、確率変数の期待値の積と一致しません。ここではあくまで近似を行っていることに注意してください。確率変数の期待値の積による近似は、テイラー展開による 1 階近似を行っていると同解釈できます。なお、2 階近似の場合は

$$\mathbb{E}[XY] \approx \mathbb{E}[X]\mathbb{E}[Y] + \text{Cov}[X, Y]$$

となり、正確な値と一致します。

$$\begin{aligned}\mathbb{E}[\phi_j] &\approx \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[\frac{R_{i,j}}{\sum_{j \in \mathcal{J}} R_{i,j}} \right] \mathbb{E} \left[\mathbf{1} \left\{ \sum_{j \in \mathcal{J}} R_{i,j} \geq 1 \right\} \right] \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[\frac{R_{i,j}}{\sum_{j \in \mathcal{J}} R_{i,j}} \right] \Pr \left(\sum_{j \in \mathcal{J}} R_{i,j} \geq 1 \right)\end{aligned}$$

まず、第一項は確率変数の割り算の期待値になっています。計算の簡単化のため、これも期待値の割り算で近似しましょう^{*13}。

$$\begin{aligned}\mathbb{E} \left[\frac{R_{i,j}}{\sum_{j \in \mathcal{J}} R_{i,j}} \right] &\approx \frac{\mathbb{E}[R_{i,j}]}{\mathbb{E} \left[\sum_{j \in \mathcal{J}} R_{i,j} \right]} \\ &= \frac{\pi_{i,j}}{\sum_{j \in \mathcal{J}} \pi_{i,j}}\end{aligned}$$

次に、第二項は個人 i が少なくとも一度はいずれかの番組に接触する確率なので、

$$\Pr \left(\sum_{j \in \mathcal{J}} R_{i,j} \geq 1 \right) = 1 - \prod_{j \in \mathcal{J}} (1 - \pi_{i,j})$$

で計算できます。

これらを元の式に戻すことで、Shapley 値による貢献度を以下で近似できることがわかります。

$$\mathbb{E}[\phi_j] \approx \frac{1}{n} \sum_{i=1}^n \left(\frac{\pi_{i,j}}{\sum_{j \in \mathcal{J}} \pi_{i,j}} \right) \left(1 - \prod_{j \in \mathcal{J}} (1 - \pi_{i,j}) \right)$$

ユニークリーチの貢献度分解手法の実装

それでは、ここまで検討した貢献度の分解手法を実装しましょう。本編で実装した `ReachEstimator` クラスを継承して、貢献度の分解手法を付け加えた `ContributionDecomposer` クラスを実装します。

*** 13** 確率変数 (X, Y) に対して、一般に $\mathbb{E}[X/Y] \neq \mathbb{E}[X]/\mathbb{E}[Y]$ であり、確率変数の割り算の期待値は、確率変数の期待値の割り算と一致しません。ここではあくまで近似を行っていることに注意してください。確率変数の期待値の割り算による近似は、テイラー展開による 1 階近似を行っていると同解釈できます。なお、2 階近似の場合は

$$\mathbb{E} \left[\frac{X}{Y} \right] \approx \frac{\mathbb{E}[X]}{\mathbb{E}[Y]} - \frac{1}{\mathbb{E}[Y]^2} \text{Cov}[X, Y] + \frac{\mathbb{E}[X]}{\mathbb{E}[Y]^3} \text{Var}[Y]$$

となります。2 階近似のほうがより正確ですが、今回は簡単のため 1 階近似で済ませることにします。

5 章 離散最適化で広告出稿番組を選択する

```

In[106]: class ContirbutionDecomposer(ProgramUniqueReachEstimator):
    """ユニークリーチの貢献度分解手法を実装したモデル"""

    def decompose_contribution(
        self,
        D: np.ndarray,
    ) -> pl.DataFrame:
        """貢献度を分解する

        Args:
            D (np.ndarray): 選択した番組群。1行m列を想定

        Returns:
            pl.DataFrame: 貢献度の分解結果
        """

        # 選択した番組群のインデックス
        j_indices = np.where(D)[1]
        # 選択した番組群の接触確率だけにデータを限定
        PI = self.PI[:, j_indices]

        # 番組単体のユニークリーチ
        r_single = PI.mean(0)
        # 選択した番組群の合算のユニークリーチ
        # floatにするため[0]で取り出す
        r_full = self.predict(D)[0]

        ## 貢献度1: 番組単体のユニークリーチによる貢献度 ##
        phi_single = r_single

        ## 貢献度2: 番組固有のユニークリーチによる貢献度 ##
        # 番組を一つ外した場合に
        # 合算のユニークリーチがどれだけ減少するかで測定
        phi_loo = np.zeros(len(j_indices))
        for i, j_index in enumerate(j_indices):
            D_ = D.copy()
            D_[0, j_index] = 0
            phi_loo[i] = r_full - self.predict(D_)[0]

        ## 貢献度3: Shapley値による貢献度 (1階近似) ##
        #  $1 - \prod_j (1 - \pi_{ij})$ を計算
        # PIを割る際にshapeが(n,1)になるように
        # keepdims=Trueを指定

```

```

r = 1 - (1 - PI).prod(1, keepdims=True)

#  $\pi_{ij} / \sum_j \pi_{ij}$  を計算
sum_pi = PI.sum(1, keepdims=True)
pi_devided_sum_pi = np.nan_to_num(
    PI / np.where(sum_pi == 0, np.nan, sum_pi),
    nan=0,
) # 分母が0の場合は0に置換

# 二つのパートを掛け合わせて平均をとることで貢献度を計算
phi_shapley = (pi_devided_sum_pi * r).mean(0)

# 貢献度をDataFrameにまとめて出力
return pl.DataFrame(
    {
        # インデックスは0始まりなので1を足す
        "j": j_indices + 1,
        "single": phi_single,
        "loo": phi_loo,
        "shapley": phi_shapley,
    }
)

```

貢献度の分解手法を `decompose_contribution` メソッドに実装しました。このメソッドでは、番組単体のユニークリーチによる貢献度 `single`、番組固有のユニークリーチによる貢献度 `loo`、Shapley 値による貢献度 `shapley` の三つを計算して、まとめて出力します。

実データに対する貢献度の計算

実際に、一つの企業が獲得するユニークリーチに対して貢献度を計算してみましょう。5 番組に CM を出稿している中で最もユニークリーチを多く獲得している企業を選択して、貢献度に分解します。

```

In[107]: # 5番組にCMを出稿している中で
# 最もユニークリーチを多く獲得している企業を選択
k_contribution = (
    df_company_reach_train.filter(pl.col.sum_d == 5)
    .filter(pl.col.r == pl.col.r.max())
    .select("k")
    .item()
)

```

5 章 離散最適化で広告出稿番組を選択する

```

)

# 選択した企業が出稿している番組群
D_contribution = (
    df_company_reach_train.filter(pl.col.k == k_contribution)
    .select(pl.selectors.starts_with("d_"))
    .to_numpy()
)

# 選択した企業の情報
df_company_reach_train.filter(
    pl.col.k == k_contribution
).select(
    "k", "r", "sum_d", "j_list"
)

```

Out[107]: shape: (1, 4)

	k	r	sum_d	j_list
	u32	f64	u32	list[u32]
	65	0.315	5	[26, 33, ...108]

企業 48 は 5 つの番組に CM を出稿し、合算のユニークリーチを 31.5% 獲得しています。それでは、企業 48 の合算のユニークリーチに対する各番組の貢献度を計算してみましょう。

```

In[108]: # 貢献度の計算
cd = ContributionDecomposer(PI_train)
df_contribution = cd.decompose_contribution(D=D_contribution)

# 結果を確認
df_contribution.sort("single", descending=True)

```

Out[108]: shape: (5, 4)

	j	single	loo	shapley
	i64	f64	f64	f64
	33	0.099	0.056	0.075
	61	0.097	0.046	0.068
	26	0.083	0.038	0.058
	108	0.079	0.033	0.055
	51	0.077	0.043	0.057

番組単体のユニークリーチ `single` を見ると、番組 33、番組 61、番組 26、番組 108、番組 51 の順に多いことがわかります。一方で、番組固有のユニークリーチ `loo` を見ると、番組 33、番組 61、番組 51、番組 26、番組 108 の順になっています。番組単体のユニークリーチでは 5 位だった番組 51 は、番組固有のユニークリーチでは 3 位になっており、番組単体のユニークリーチが少ないからといって必ずしも番組固有のユニークリーチも少ないわけではないことがわかります。

Shapley 値による貢献度 `shapley` を見ると番組 33、番組 61、番組 26、番組 51、番組 108 の順になっており、番組 51 は 4 位です。ここからも、Shapley 値による貢献度は、番組単体のユニークリーチと番組固有のユニークリーチの双方を考慮した指標になっていることが見てとれます。

最後に、Shapley 値による貢献度の合計が合算のユニークリーチの推定値と一致することを確認しておきます。

```
In[109]: # 合算のユニークリーチの推定値
r_pred_contribution = cd.predict(D_contribution)[0]

# 貢献度の合計を計算
sum_contribution = df_contribution.get_column("shapley").sum()

print(
    f"合算のユニークリーチの推定値: {r_pred_contribution:.3f}"
)
print(f"貢献度の合計: {sum_contribution:.3f}")
```

Out[109]: 合算のユニークリーチの推定値: 0.313
貢献度の合計: 0.313

5 章 離散最適化で広告出稿番組を選択する

両者が 31.3% で一致することを確認できました。よって、Shapley 値を用いることで、合算のユニークリーチを各番組の貢献度として分解できていることがわかります。

おわりに

本書では、データサイエンスの力を最大限に引き出し、ビジネスの現場で直面する課題を解決へと導くための一つのフレームワークをご紹介しました。具体的な分析手法そのものの詳細な解説というよりは、課題解決に向けてどのように思考し、アプローチしていくべきか、その「考え方」の核心をお伝えすることに心を砕きました。

本書を通じて、私が何よりも伝えたかったのは、データサイエンスを使ってビジネスに貢献することの楽しさです。私自身、REVISIO 株式会社に入社して間もない頃は、学んできたデータサイエンスの知識を、実際のビジネスにどう結び付ければよいのか分からず、試行錯誤を繰り返す日々でした。その過程で感じたもどかしさが、本書を執筆する原動力となりました。あの頃の私が知りたかったこと、そして今、同じように奮闘されているかもしれない読者のみなさんの背中を少しでも押せるような内容をお届けしたい一心で執筆しました。

この一冊が、データサイエンスをビジネスの成果へとつなげるための一助となれたなら、著者としてこれ以上の幸せはありません。

謝辞

本書の完成に至るまで、本当に多くの方々から温かいご支援と貴重なご助言を賜りました。この場をお借りして、心からの感謝を申し上げます。

技術評論社の高屋卓也さんには、本書の企画段階から出版に至るまで、並々ならぬお力添えをいただきました。前作『機械学習を解釈する技術』に続き二度目の執筆の機会をいただけたこと、2023 年初頭の企画から 2 年半という長い道のりを粘り強く伴走してくださったことに、深く感謝いたします。

伊藤貴史さん、高柳慎一さん、松尾佑太さん、本川哲也さん、吉田康仁さんには、貴重なお時間を割いてレビューにご協力いただきました。みなさんからの示唆に富むコメントの一つひとつのおかげで、本書の内容はより正確で、そして何よりも読者にとって分かりやすいものになりました。

REVISIO 株式会社代表取締役の郡谷康士さん、そして取締役であり共同創業者の河村嘉樹さんには、社内の貴重なデータ利用をご快諾いただきました。REVISIO はデータサイエンスの可能性を追求できる恵まれた環境です。刺激的な課題に満ちたこの場所を与えてくれたことに、心から感謝しています。

REVISIO に入社した当初から現在に至るまで、私と粘り強く議論し、データサイエンスの知識をビジネスの成果へとつなげる手助けをしてくれた山田慎也さんと東野晃大さん、ありがとうございました。ビジネス課題の一つひとつにお二人と取り組むことを通じて、本書で提示したフレームワークが洗練されていきました。

この場ではすべての方のお名前を挙げることは叶いませんが、REVISIO の社員一人ひとりに、心からの感謝を伝えたいと思います。日々の業務の中で交わされる、データ活用に関する活発な議論は、私にとって尽きることのない学びの泉です。みなさんの多様な視点や深い洞察にふれるたび、新たな発見があり、自身の知見が広がっていくのを感じています。

最後に、本業と執筆の両立を一番近くで支えてくれた、妻の千尋に感謝しています。

ここに挙げさせていただいた方々以外にも、数えきれないほど多くの方々から、温かい励ましや貴重なお言葉をいただきました。みなさまお一人おひとりのお力添えなくして、本書の完成はありえませんでした。あらためて、心より厚く御礼申し上げます。ほんとうにありがとうございました。

2025 年 5 月 森下光之助

参考文献

- Chen, Aiyou, Jim Koehler, Art Owen, Nicolas Remy, and Minghui Shi (2014) “Data enrichment for incremental reach estimation,” Technical report, Google Inc.
- Clegg, Matthew and Jiayu Peng (2022) “Generalized Mixture Overlap Model for Estimating Reach Surfaces,” Technical report, Technical Disclosure Commons.
- Danaher, Peter J. and Michael S. Smith (2011) “Modeling Multivariate Distributions Using Copulas: Applications in Marketing,” *Marketing Science*, Vol. 30, No. 1, pp. 4–21.
- Danaher, Peter Joseph (1991) “A Canonical Expansion Model for Multivariate Media Exposure Distributions: a Generalization of the Duplication of Viewing Law,” *Journal of Marketing Research*, Vol. 28, No. 3, pp. 361–367.
- Goerg, Georg M. (2014) “Estimating reach curves from one data point,” Technical report, Google Inc.
- Hansen, Bruce E. (2022) *Econometrics*: Princeton University Press.
- Hastie, T. J, R. J. Tibshirani, and J. H. (Jerome H.) Friedman (2009) *The elements of statistical learning : data mining, inference, and prediction*, Springer series in statistics: Springer, 2nd edition.
- Hoff, Peter D. (2010) *A first course in Bayesian statistical methods*, Springer texts in statistics: Springer.
- Kohavi, Ron, Diane Tang, and Ya Xu (2021) 『A/B テスト実践ガイド：真のデータドリブンへ至る信用できる実験とは』, 大杉直也訳, ドワンゴ, KADOKAWA (発売).
- Lambert, Ben (2018) *A student’s guide to Bayesian statistics*: SAGE.
- Lundberg, Scott M and Su-In Lee (2017) “A unified approach to interpreting model predictions,” *Advances in neural information processing systems*, Vol. 30.
- McElreath, Richard (2020) *Statistical rethinking : a Bayesian course with examples in R and Stan*, Texts in statistical science: CRC Press, 2nd edition.
- Meerschaert, Mark M. (2015) 『数理モデリング入門：ファイブ・ステップ法』, 佐藤一憲・梶原毅・佐々木徹・竹内康博・宮崎倫子・守田智訳, 共立出版.

- Provost, Foster and Tom Fawcett (2014) 『戦略的データサイエンス入門：ビジネスに活かすコンセプトとテクニック』, 竹田正和・古島敦・瀬戸山雅人他訳, オライリー・ジャパン, オーム社 (発売).
- Robert, Christian P. (2001) *The Bayesian choice : from decision-theoretic foundations to computational implementation*, Springer texts in statistics: Springer, 2nd edition.
- Shen, Xinyang, Yunwen Yang, Jim Koehler, and Lu Zhang (2019) “Conditional Independence Approximation to Cross-media Incremental Reach,” Technical report, Google Inc.
- Thakur, Abhishek (2021) 『Kaggle Grandmaster に学ぶ機械学習実践アプローチ』, 石原祥太郎訳, Compass data science, マイナビ出版.
- Vaughan, Daniel (2021) 『AI 技術を活かすためのスキル：データをビジネスの意思決定に繋げるために』, 長尾高弘・西内啓訳, オライリー・ジャパン, オーム社 (発売).
- van Wieringen, Wessel N (2015) “Lecture notes on ridge regression,” *arXiv preprint arXiv:1509.09169*.
- 安井翔太 (2020) 『効果検証入門：正しい比較のための因果推論/計量経済学の基礎』, 技術評論社.
- 安宅和人 (2024) 『イシューからはじめよ：知的生産の「シンプルな本質」』, 英治出版.
- 岡田章 (2021) 『ゲーム理論』, 有斐閣, 第3版.
- 河本薫 (2022) 『データドリブン思考：データ分析・AIを実務に活かす』, ダイヤモンド社.
- 久保川達也 (2017) 『現代数理統計学の基礎』, 第11号, 共立出版.
—— (2024) 『公式と例題で学ぶ統計学入門』, 共立出版.
- 古川一郎・守口剛・阿部誠 (2011) 『(新版) マーケティング・サイエンス入門: 市場対応の科学的マネジメント』, 有斐閣.
- 高柳慎一・長田怜士 (2023) 『評価指標入門：データサイエンスとビジネスをつなぐ架け橋』, 技術評論社.
- 上田雅夫・生田目崇 (2017) 『マーケティング・エンジニアリング入門』, 有斐閣アルマ, 第 Specialized 号, 有斐閣.
- 森下光之助 (2021) 『機械学習を解釈する技術：予測力と説明力を両立する実践テクニック』, 技術評論社.

- 中室牧子・津川友介 (2017) 『「原因と結果」の経済学：データから真実を見抜く思考法』, ダイヤモンド社.
- 馬場真哉 (2021) 『意思決定分析と予測の活用：基礎理論から Python 実装まで』, 講談社.
- 梅谷俊治 (2020) 『しっかり学ぶ数理最適化：モデルからアルゴリズムまで』, 講談社.
- 浜田宏 (2018) 『その問題、数理モデルが解決します：社会を解き明かす数理モデル入門』, ベレ出版.
- (2020) 『その問題、やっぱり数理モデルが解決します：データ時代を生き抜くための数理モデル入門』, ベレ出版.
- 富山吉孝・早川裕樹・齋藤慎一郎 (2024) 『Polars と pandas で学ぶデータ処理アイデアレシピ 55』, 講談社.
- 片平秀貴 (1987) 『マーケティング・サイエンス』, 東京大学出版会.
- 門脇大輔・阪田隆司・保坂桂佑・平松雄司 (2019) 『Kaggle で勝つデータ分析の技術』, 技術評論社.
- 齋藤優太 (2024) 『反実仮想機械学習：機械学習と因果推論の融合技術の理論と実践』, 技術評論社.
- 齋藤優太・安井翔太 (2021) 『施策デザインのための機械学習入門：データ分析技術のビジネス活用における正しい考え方』, 技術評論社.

索引

C

CM 接触回数 70

D

Data Generating Process 21

E

estimate x

extrapolation 23

L

Law of Iterated Expectations 231

Leave-One-Out Cross Validation 167

N

Negative Binomial Distribution 88

P

parquet 形式 216

S

Shapley 値 273

か

外挿 23

仮定 4

刈り取り型広告 43

完全な多重共線性 223

ガンマ関数 73

ガンマ分布 xi, 85

き

期待値 xiii

期待値の線形性 29

共分散 xiii

近似解 245

く

クリエイティブ戦略 113

繰り返し期待値の法則 232

グロスリーチ 44

け

決定係数 170

決定変数 7, 10

こ

貢献度 268

購買心理プロセス 42

し

指示関数 xiv

視聴率 49

集合 xiv

出稿金額 51

出稿量 51

純粋想起 44

条件付き期待値 xiii, 16–18

条件付き独立性の仮定	234
助成想起	44

す

推定値	x
数理最適化	xv
数理最適化問題	10
スポット	50

せ

製品認知率	115
制約条件	10
制約付き最大化問題	xv
説明変数	67
線形回帰モデル	20, 67

そ

想起	44
----	----

た

対数変換	68
タイム	48
多重共線性	223
単純集計	19

て

定数	10
データ生成過程	21

と

独立性の仮定	137
貪欲法	245

に

二項分布	xii
認知	44
認知カーブ	113
認知型広告	43

は

パーコスト	50
パラメータ	ix, 5

ひ

標本調査	46
------	----

ふ

負の二項分布	xiii, 88
フリークエンシー	44, 70
分散	xiii

へ

平均フリークエンシー	44
ベータ関数	129
ベータ二項分布	xii, 262
ベータ分布	x, 106, 129
ベルヌーイ分布	xi

ほ

ポアソン分布	xii, 72
補正係数	183

ま

マーケティングファネル	42
-------------	----

み

密集度	85
-----	----

め	
メディア戦略	113

も	
目的関数	10
目的変数	67

ゆ	
ユニークリーチ	44

よ	
要素	xiv

り	
リーチ $f+$	97
リーチカーブ	52
離散最適化問題	245
リッジ回帰	224

ろ	
ロジスティック関数	30
論理記号	xiv

著者プロフィール

森下光之助（もりした みつのすけ）

REVISIO 株式会社 執行役員 CDO データ・テクノロジー本部長

東京大学大学院にて経済学修士号を取得後、データサイエンティストとして活動。現在は REVISIO にてデータ戦略の策定・実行を統括。データサイエンスの知見を活かした実践的なデータ活用を推進している。REVISIO でのデータ基盤移行プロジェクトは Snowflake 社の「DATA DRIVERS AWARDS 2023」で最高賞を受賞。機械学習モデルの解釈性を扱った著書『機械学習を解釈する技術』は「IT エンジニア本大賞 2022」技術書部門ベスト 3 に選出された。

カバーデザイン ◆ 末吉亮（図工ファイブ）
図版作成 ◆ 青木健太郎（セメントミルク）
組版協力 ◆ 株式会社ウルス
担 当 ◆ 高屋卓也



ビジネス課題を解決する技術
数理モデルの力を引き出す3ステップフ
レームワーク

2025 年 7 月 10 日 初 版 第 1 刷発行

著 者 もりしたみつ の すけ 森下光之助

発行者 片岡 巖

発行所 株式会社技術評論社

東京都新宿区市谷左内町 21-13

電話 03-3513-6150 販売促進部

03-3513-6177 第5編集部

印刷／製本 港北メディアサービス株式会社

定価はカバーに表示してあります

本書の一部または全部を著作権法の定める範囲を超え、無断で複写、複製、転載、
テーパー化、ファイルに落とすことを禁じます。

© 2025 森下光之助

ISBN978-4-297-14992-5 C3055

Printed in Japan

■本書についての電話によるお問い合わせはご遠慮ください。質問等がございましたら、下記まで FAX または封書でお送りくださいますようお願いいたします。

〒162-0846

東京都新宿区市谷左内町 21-13

株式会社技術評論社 第5編集部

FAX : 03-3513-6173

「ビジネス課題を解決する技術」係

なお、本書の範囲を超える事柄についてのお問い合わせには一切応じられませんので、あらかじめご了承ください。

造本には細心の注意を払っておりますが、万一、乱丁（ページの乱れ）や落丁（ページの抜け）がございましたら、小社販売促進部までお送りください。送料小社負担にてお取り替えいたします。



本コンテンツは、Gihyo Digital Publishing において、
電子データとして再発行されたものです。

電子初版発行日 2025 年 06 月 27 日

最新版発行日 2025 年 06 月 27 日

電子版ISBN 978-4-297-14993-2